

Reti di calcolatori

Trufelli Elia

1.0 Introduzione alle reti di calcolatori

La **connessione fisica e logica** tra 2 o più dispositivi per la ricezione, trasmissione e condivisione di risorse viene detta “**rete di calcolatori**” (*computer network*). Questo concetto non va confuso con “sistema distribuito”, poiché quest’ultimo è un sistema atto all’espletamento di un determinato compito ed è basato su una rete di calcolatori. Un **esempio ben noto di sistema distribuito è il World Wide Web**. Una rete di computer è una rete di telecomunicazione, e come tale le sue performance sono influenzate da 4 caratteristiche:

- **Consegna:** il sistema deve consegnare le informazioni al **corretto destinatario**.
- **Precisione:** il sistema deve consegnare i dati **senza che vengano alterati**.
- **Tempestività:** dati che arrivano in ritardo potrebbero essere inutili.
- **Jitter:** **variazione di errore** con cui dati giungono al destinatario.

Un sistema di comunicazione viene identificato dei seguenti componenti:

- **messaggio**
- **mittente**
- **destinatario**
- **mezzo di trasmissione**
- **protocollo:** insieme di regole che governano la comunicazione tra i nodi che si scambiano il messaggio.

Il flusso di informazioni tra i nodi può essere di 3 tipi:

1. **Simplex:** il passaggio di informazioni è possibile **solo in una direzione**.
2. **Half-duplex:** un dispositivo può assumere **sia il ruolo di mittente che destinatario**, ma **non può farlo in contemporanea**.
3. **Full-duplex:** un dispositivo può fungere **contemporaneamente da mittente e destinatario**.

1.1 Applicazione delle reti di calcolatori

Le reti di calcolatori possono avere **applicazioni aziendali e domestiche**. Inoltre dagli anni 80 questa parte ha iniziato a **svilupparsi la telefonia mobile**.

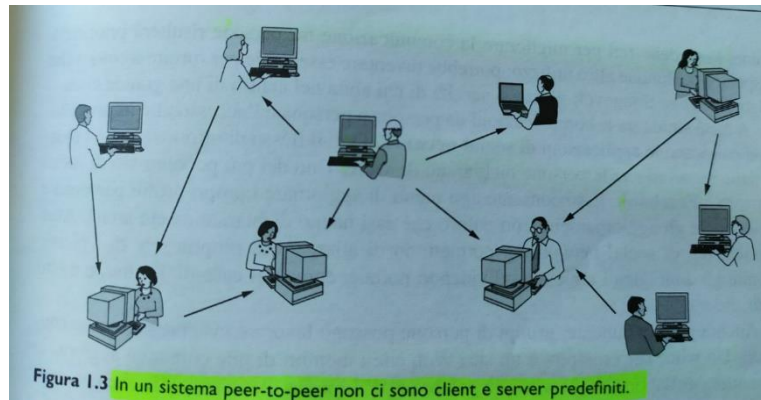
1.1.1 Modello Client-server

è il paradigma di comunicazione **più conosciuto all’interno delle reti**: in questo modello i **dati sono memorizzati in un computer ad alte prestazioni chiamato server**, che solitamente viene installato nel **nell’edificio centrale gestito dagli amministratori di sistema**, al contrario i **client** corrispondono a

macchine più semplici tramite le quali effettuano l'accesso ai dati remoti presenti nel server. L'implementazione più nota di questo modello è un'applicazione web in cui il server genera a partire dal suo database le pagine web in risposta a richiesta da parte del client che potrebbero, a loro volta, aggiornare il database stesso; il server può soddisfare le richieste di un gran numero di client contemporaneamente.

1.1.2 P2P – Peer to Peer

In questa forma di comunicazione gli individui che costituiscono un gruppo spontaneo sono in grado di comunicare tra di loro: all'interno del gruppo ognuno può comunicare senza che vi sia una divisione fra client e server.



Nei sistemi P2P ogni utente mantiene il proprio database il locale e fornisce un elenco di altri utenti a lui noti che fanno parte del sistema. Questo tipo di modello viene impiegato per la condivisione di musica e video. L'idea di base è quella di non localizzare in un solo punto il carico di lavoro, ma distribuirlo tra i partecipanti.

1.3 Applicazioni delle reti

Le reti permettono diverse applicazioni, che possono essere raggruppate in 5 macro-categorie:

- accesso ad informazioni remote
- comunicazione fra persone, in modo sincrono ed asincrono o addirittura multimediale
- intrattenimento interattivo (giochi)
- formazione online
- commercio elettronico

Le reti sono in continua evoluzione, sia dal punto di vista fisico/tecnologico che da quello logico/commerciale. Negli ultimi anni questo ha portato allo sviluppo di nuovi paradigmi, come il Grid-computing, il cloud e la virtualizzazione.

1.3.1 Grid computing

I sistemi grid sono soprattutto un'infrastruttura di calcolo distribuito, che permettono la condivisione coordinata di risorse all'interno di un'organizzazione virtuale. Questo concetto è nato dalla necessità di dover coordinare risorse condivise atte a risolvere in maniera dinamica problemi complessi con grosse quantità di dati; la condivisione non è limitata al software, ma prevede anche l'hardware. I

problemi trattati dal grid-computing sono di tipo **data-intensive**, per cui le applicazioni utilizzate hanno bisogno di accedere a **grandi quantità di dati geograficamente distribuiti** ed appunto il compito del grid è quello di far **collaborare tali applicazioni nel modo migliore**, attraverso **protocolli standard e aperti**, impiegando interfacce in grado di condividere una qualità del servizio non banale.

1.3.2 Cloud computing

Il cloud è **un'idea di marketing** che prevede di **considerare quelli che finora erano considerati beni (programmi, hardware..) come servizi**; essi vengono forniti attraverso internet da grossi **provider**. Questo sistema sta cambiando il modo dell'IT (*information technology*) di offrire servizi e la maniera di un utente di usufruire delle risorse di elaborazione da qualsiasi posto: il Cloud consente all'IT di rispondere alle opportunità di business con consegne on demand, che a lungo termine potrebbero essere economicamente efficienti ed agili. Un concetto chiave è quello di **servizio**: nel contesto dell'IT viene usato per descrivere **una forma di consegna o disponibilità di una risorsa**. Un modo per descrivere il cloud computing è quello di basarsi sui **modelli di distribuzione dei servizi**:

- **Software as a Service (SaaS)**: l'applicazione è disponibile su richiesta. Questo modello è il più comune.
- **Platform as a Service (PaaS)**: è una **piattaforma** disponibile su richiesta per lo **sviluppo, il testing, la distribuzione e la manutenzione continua delle applicazioni**.
- **Infrastructure as a Service (IaaS)**: è un ambiente IT che prevede un **infrastruttura completa per il sottoscrittore**. Questa infrastruttura viene fornita attraverso **macchine virtuali** in cui l'utente **mantiene il sistema operativo e le applicazioni installate**. Al momento dell'installazione esistono 3 modelli base: **Public Cloud**, che quella messa a disposizione attraverso internet per il pubblico ed è proprietà di un'organizzazione che offre tale tipo di servizio, **Private Cloud**, è disponibile **solo per un'organizzazione** ed è gestita da essa o da terze parti, **Community Cloud**, è un ambiente cloud pubblico nel quale sono **presenti elementi tipici di quello privato**.

A seconda del tipo di modello adottato, un **utente** e un **provider** di servizi hanno **differenti ruoli e responsabilità**: il **concetto di separazione** delle responsabilità di un abbonato da quella del fornitore di servizi è **fondamentale nel cloud computing**. Sottoscrivendo un **particolare modello di consegna dei servizi**, un **utente accetta di rinunciare ad un certo livello di accesso e di controllo** sulle risorse gestite dal fornitore di servizi. Ci possono essere funzionalità in più come ad esempio la **sicurezza**, la **tracciabilità** o **responsabilità** che **non sono esplicitamente fornite**.

1.3.3 Virtualizzazione

La virtualizzazione, benché non sia un concetto tipicamente delle reti di calcolatori, è un presupposto imprescindibile per fornire i servizi di tipo Cloud: è intesa come la capacità di **estrarre hardware e software su una sola macchina**.

Una macchina virtuale è un **contenitore software isolato**, in grado di eseguire propri sistemi operativi e applicazioni **come se fosse un computer fisico**. CPU, RAM, Disco rigido e porte di comunicazione (LAN, USB...) di un solo computer sono condivisi da macchine virtuali che hanno un proprio sistema operativo e svolgono applicazioni proprie.

La virtualizzazione è una tecnologia collaudata per risolvere **diverse problematiche**, come ad esempio il **sottoutilizzo dell'hardware**: possiamo dire che **consente di trasformare l'hardware in software**, ossia

virtualizzare le risorse fisiche della macchina. Più macchine virtuali condividono le risorse della macchina hardware senza interferire tra di loro, perciò è possibile eseguire contemporaneamente sistemi operativi diversi e applicazioni differenti su un singolo computer. Attraverso questo sistema è possibile quindi: allocare dinamicamente le risorse, riavvia automaticamente le applicazioni dopo una failure e utilizzare le risorse garantendo maggiore flessibilità.

1.4 Classificazione delle reti

1.4.1 Classificazione per estensione geografica

- **PAN:** Sostengono per un paio di metri nell'intorno dell'utilizzatore
- **LAN:** sostengono per un raggio di qualche centinaio di metri. Di solito sono quelle casalinghe ed i piccoli uffici.
- **MAN:** reti più estese, che coprono il territorio cittadino (televisioni via cavo americane).
- **WAN:** raggiungono diversi chilometri e coprono macroregioni o intere nazioni.
- **Wireless:** sono le reti che trasferiscono i dati attraverso mezzi fisici non guidati, come ad esempio le onde elettromagnetiche.
- **Home Network:** reti che distribuiscono segnali dentro casa (cablate e wireless).
- **Internetwork:** reti composte solitamente da pochi dispositivi virgola che interconnettono diverse reti tra di loro (ad esempio una WAN con una LAN). Si intende la connessione tra due o più reti.

1.4.2 Classificazione per tipologia di trasmissione

- **Broadcast:** I processi di comunicazione dei nodi in queste reti permettono la trasmissione dell'informazione di tipo punto-multipunto, ossia un nodo può comunicare la stessa informazione a differenti nodi contemporaneamente sfruttando lo stesso mezzo trasmissivo. I pacchetti inviati da uno degli host connessi sono ricevuti da tutti gli altri; attraverso il campo "indirizzo" all'interno del pacchetto viene individuato il destinatario. Se il pacchetto arriva all'host desiderato viene processato, altrimenti scartato.
- **Punto-Punto:** sono reti che collegano i nodi tra loro mediante un collegamento diretto. Si può pensare a queste reti come una versione hardware del paradigma P2P.

1.4.3 Classificazione per topologia

Questa classificazione permette di suddividere le reti in base alla struttura logica attraverso la quale sono interconnesse.

- **Bus:** un unico mezzo di trasmissione, chiamato bus, è usato per connettere diversi nodi della rete.
- **Anello:** i nodi della rete sono collegati attraverso link punto-punto fino a formare un anello logico.
- **Stella:** i partecipanti della rete hanno un coordinatore centrale a cui sono collegati tutti gli altri nodi. E' possibile ritirare questa tipologia con nodi limitrofi che a loro volta diventano centrali per una sottorete.

- **Gerarchica:** la rete ha la struttura di un **albero** con un **nodo radice** (di solito **rappresentato da un gateway**) e **gli altri nodi rappresentano le foglie**.
- **Magliata:** in questa tipologia **ogni nodo è connesso a ciascun'altro partecipante** con un **link punto-punto**.

1.5 Hardware di rete

1.5.1 Reti PAN

Le reti PAN permettono ai dispositivi di comunicare nello **spazio fisico alla portata di una persona**. Un classico esempio sono le reti wireless che connettono un pc alle sue periferiche (tastiera, mouse); fanno parte di questa categoria le reti wireless a corto raggio, **Bluetooth**. In questo modello si ha una gerarchia di tipo **master-slave**, dove di solito il **PC rappresenta il master** e gli altri componenti, quali **schermo, mouse e tastiera sono gli slave**. Le reti PAN possono anche essere implementate **con altre tecnologie a corto raggio, come gli RFID**. I protocolli adottati da questo tipo di rete sono differenti e possono essere:

- WiFi
- Bluetooth
- RFID
- HDMI
- USB
- ZigBee
- USB

1.5.2 Reti LAN

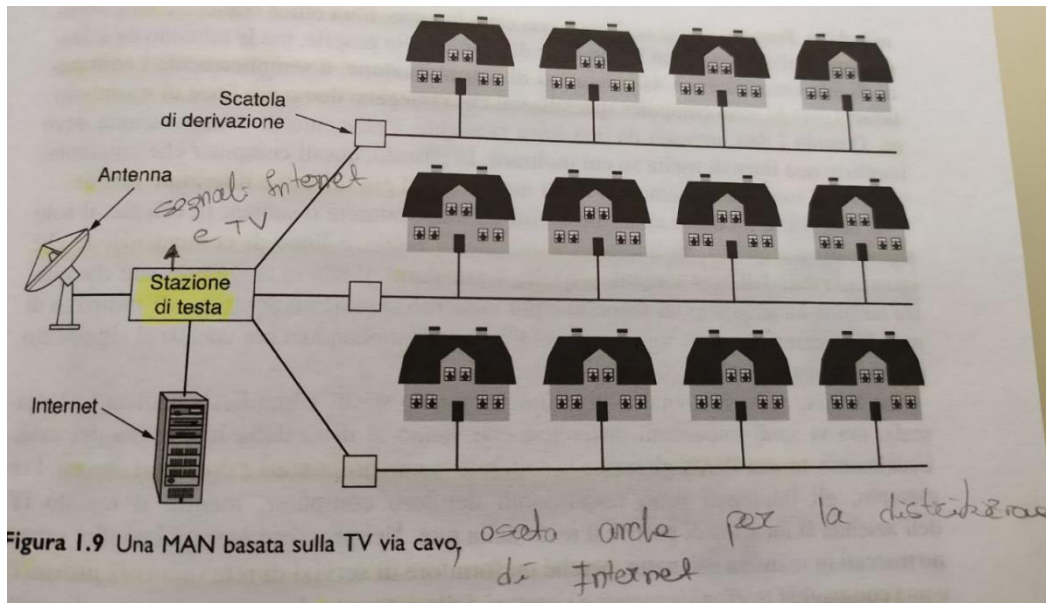
Una rete LAN è una **rete privata** che opera **all'interno** o nelle **vicinanze** di un **singolo edificio**. Sono usate per connettere PC e altri dispositivi in modo tale che questi possono condividere risorse. Le più diffuse sono le **LAN Wireless (IEEE 802.11)**, dove **ogni dispositivo connesso alla rete comunica con l'AP (Access Point), router wireless o base station**, che ha la funzione di **distribuire i pacchetti tra i PC** collegati in modo wireless e anche tra quelli in internet. Un'altra tipologia di LAN è quella **Ethernet (IEEE 802.3)** formata da connessioni **punto-punto**: ogni device si connette ad uno **switch**, attraverso una connessione punto-punto, che ha la funzione di **smistare i pacchetti tra i dispositivi connessi a lui**, selezionando il computer destinatario in base all'indirizzo scritto nel pacchetto. Quando si progetta una rete LAN bisogna tenere in considerazione due aspetti:

- **Wireless:** **convenienza e costi**
- **Cablato:** **sicurezza** maggiore

In entrambi i casi c'è bisogno di un meccanismo che regoli la trasmissione dei dati, poiché se **due o più host interagiscono contemporaneamente potrebbero verificarsi problemi**.

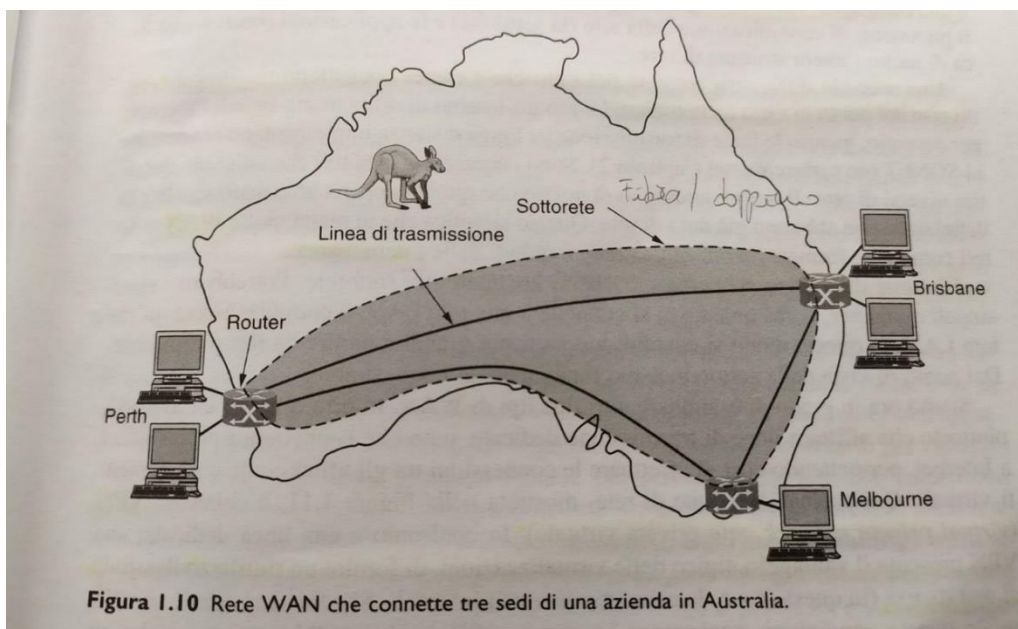
1.5.3 Reti MAN

Una **MAN** (Metropolitan Area Network) copre **un'intera città**: l'esempio più noto è la rete TV via cavo americana.



1.5.4 Reti WAN

Una **WAN** (Wide Area Network) copre un'area **geograficamente estesa**, spesso una **nazione o un continente**.



Il compito della sottorete è quello di **trasportare i messaggi da un host ad un altro**. Nella maggior parte dei casi la rete WAN è formata da **due componenti**: **linee di trasmissione** ed **elementi di commutazione**. Le linee di trasmissione **spostano i bit tra le macchine**, attraverso cavi di **rame, fibra ottica o radio**; gli elementi di commutazione (**commutatori** – switch) sono **computer specializzati che collegano due o più linee di trasmissione**.

La commutazione a pacchetto non è l'unico modo di trasferire le informazioni attraverso la rete. È possibile anche impostare un percorso unico attraverso il quale fluiranno tutti i pacchetti appartenenti a quel flusso. In questo caso abbiamo una rete a commutazione a circuito. In entrambe le tipologie di rete, i percorsi che i pacchetti seguiranno sono scelti in accordo a determinati algoritmi detti "algoritmi di routing".

1.6 Software di rete

Vengono analizzate le tecniche di strutturazione del software.

1.6.1 Gerarchie di protocolli

I protocolli rappresentano la definizione delle modalità di interazione fra host; sono caratterizzati dalla:

- **Sintassi:** insieme delle regole che la comunicazione deve seguire per essere valida
- **Semantica** appunta la giusta sequenza dei comandi in modo che le informazioni trasmesse abbiano senso.
- **Sincronizzazione:** gestione del tempo in maniera tale che non ci siano conflitti.

Gli standard che incontreremo possono essere:

- **De Facto:** nessuna norma gli ha regalato a priori che sono diventati tali per il loro vasto uso e la diffusione che hanno raggiunto.
- **De Jure:** un ente organizzazione ne ha dettato le regole a priori.

Organizzazione stratificata del software

Il software della rete deve essere strutturato in strati per gestire la complessità. ogni livello fornisce servizi ai livelli superiori nascondendo i dettagli interni (*information hiding*): questi livelli possono essere visti come macchine virtuali per quelli che stanno sopra. I livelli peer comunicano attraverso dei protocolli, mentre quelli contigui sono connessi tramite interfacce che definiscono i servizi e le operazioni che il livello inferiore rende disponibili per quello superiore. Livelli e protocolli costituiscono l'architettura di rete.

Gerarchie di protocollo

Sono le relazioni tra la comunicazione virtuale e reale. L'astrazione dei processi dei peer è un principio fondamentale del design della rete. I livelli più bassi possono essere implementati in hardware.

1.6.2 Progettazione dei livelli

Il punto chiave nel progettare una rete è l'affidabilità. Il meccanismo per intercettare errori nelle informazioni ricevute si serve di codici per l'individuazione dell'errore; le informazioni ricevute sbagliate possono essere ritrasmesse finché non è stata ricevuta correttamente, in questo caso si usano codici per la correzione degli errori. Un altro problema di affidabilità consiste nel trovare un percorso valido attraverso la rete: spesso ci sono dei percorsi da una sorgente ad un destinatario nelle reti grandi, in cui possono esistere link e router non funzionanti. Quest'argomento è chiamato routing (instradamento).

Addressing: meccanismo per identificare chi trasmette e chi riceve un particolare messaggio all'interno di una rete.

Esistono problemi legati anche alle varie tipologie di rete presenti: per esempio la differenza tra le reti del numero massimo di messaggi che possono essere trasmessi, in questo caso si utilizzeranno meccanismi per frazionare, trasmettere e quindi a scambiare messaggi. Quest'argomento è chiamato **internetworking**. Progetti di rete che continuano a funzionare bene, quando la rete diventa grande ed eterogenea sono detti **scalabili**.

Un altro problema è l'**allocazione delle risorse**: per funzionare bene le reti necessitano di meccanismi che dividono le risorse in modo che un host non interferisca troppo con gli altri. Ad esempio: molti progetti condividono dinamicamente la banda attraverso il *multiplexing statistico*. Un altro problema di allocazione riguarda come impedire che una sorgente veloce inondi di dati un ricevente lento: si utilizzano dei meccanismi di **controllo di flusso** per evitare la **congestione** della rete.

Molte reti devono fornire servizio sia di applicazioni che vogliono una consegna in tempo reale sia di applicazioni che richiedono una grande capacità di trasferimento: il meccanismo che concilia queste richieste è chiamato **qualità del servizio (QoS)**.

1.6.3 Classificazione dei servizi

- **Servizi orientati alla connessione:** vengono stabilite delle connessioni, si usa e infine si rilascia. Possono essere negoziati i parametri di connessione, come la massima dimensione di un messaggio o la qualità del servizio richiesta.
- **Servizi senza connessione:** ogni messaggio viene mandato indipendentemente dagli altri e deve conoscere la sorgente e la destinazione. Ha un'alta tolleranza ai fault, ovvero se qualche router si rompe, verrà intrapreso un percorso diverso.

Classificazione sulla base della qualità del servizio

Se un servizio è affidabile, i dati non vengono persi perché viene implementato un sistema basato su **ack**. Se un servizio è inaffidabile (privo di conferma) i dati potrebbero essere persi, come ad esempio avviene nei servizi datagram.

Relazione tra servizi e protocolli

Sono 2 concetti differenti: i **servizi** sono insiemi di operazioni primitive offerte da un livello a quello superiore, i **protocolli** sono insieme di regole che specificano il formato e la semantica dei messaggi scambiati tra peer dello stesso livello.

1.7 Modelli di riferimento

I modelli di riferimento più diffusi sono 2: **OSI** e **TCP/IP**. I protocolli associati al modello OSI sono ormai in **disuso** ma il modello è largamente impiegato. Invece il **TCP/IP** è poco utilizzabile ma i protocolli sono largamente impiegati.

1.7.1 ISO OSI

Si chiama modello di riferimento **ISO OSI** (*open system interconnection*) perché riguarda la connessione di sistemi aperti; è composto da **7 livelli**. Questo modello non è in sé un'architettura di rete, perché non specifica quali sono esattamente i servizi e i protocolli da usare in ciascun livello; si

limita a definire ciò che ogni livello deve fare, tuttavia ISO ha prodotto anche standard per ciascun livello.

Livello fisico

Si occupa della trasmissione dei bit sul canale di comunicazione. Questo livello deve assicurare che ogni bit trasmesso con valore uno sia ricevuto ancora con valore uno e non con valore zero. In questo livello vengono tenute in considerazione: la velocità trasmissiva, sincronizzazione, link e i flussi di dati.

Livello data link

Il compito principale del livello data link consiste nel far diventare una trasmissione grezza in una linea che appare priva di errori non rilevati: esegue questo compito mascherando gli errori reali in modo che il livello di rete non li veda. L'obiettivo è raggiunto forzando il trasmittente a suddividere i dati di ingresso in frame, trasmessi in maniera sequenziale: se il servizio è affidabile il ricevente conferma la corretta ricezione di ciascun frame restituendo un ack. In questo livello possono nascere dei problemi, come ad esempio il famoso collo di bottiglia, quindi si utilizzeranno dei meccanismi per regolare il traffico. Controllo di flusso e controllo di errori.

Livello di rete

Controlla il funzionamento della sottorete, in pratica si occupa di mandare i pacchetti tra i nodi della rete. Un problema chiave riguarda la modalità con cui pacchetti sono inoltrati dalla sorgente alla destinazione. In generale possiamo dire che la qualità del servizio offerto (ritardo, tempi di transito, jitter) è un problema a livello di rete. Indirizzamento logico e routing.

Livello di trasporto

La funzione essenziale del livello di trasporto è quella di accettare dati dal livello superiore, dividerli in unità più piccole (quando necessario), passarle al livello di rete e assicurarsi che tutti i pezzi arrivino correttamente a destinazione; il tutto eseguito con prestazioni efficienti. Nella pratica si occupa di mandare messaggi tra i processi della sorgente e della destinazione. Indirizzamento tra processi, segmentazione, controllo della connessione, controllo di flusso e degli errori.

Livello di sessione

Il livello di sessione permette ad utenti su computer diversi di stabilire tra loro una sessione. Le sessioni offrono diversi servizi, tra cui: controllo del dialogo, tenere traccia di quando è il turno di trasmettere e quando di ricevere, gestione dei token, evitare che le 2 parti tentino la stessa operazione critica nello stesso momento e sincronizzazione ovvero supervisionare una lunga trasmissione per consentire la sua ripresa nel punto in cui si è interrotta.

Livello di presentazione

A questo livello si occupa della sintassi e della semantica dell'informazione trasmessa. Effettua operazioni di: traduzione, cifratura e compressione.

Livello applicazione

Fornisce i servizi della rete all'utente finale: il livello applicazione comprende una varietà di protocolli: il più usato è HTTP, che sta alla base del World Wide Web. Esistono altri protocolli applicativi per trasferimento file, posta elettronica e news.

1.7.2 TCP/IP

Il modello di riferimento TCP/IP fu progettato con l'obiettivo di **collegare diverse reti fra loro** in modo più semplice e che la rete potesse **sopravvivere alla perdita dell'hardware di sottorete senza interrompere le conversazioni in corso**. A differenza del modello precedente in questo non sono presenti il livello di presentazione e sessione, ma sono **unificati il livello fisico e data link**.

Livello data link

Descrive che **cosa devono fare i collegamenti**, più che un vero e proprio livello nel senso usuale del termine è **un'interfaccia tra l'host e il mezzo trasmissivo**.

Livello internet

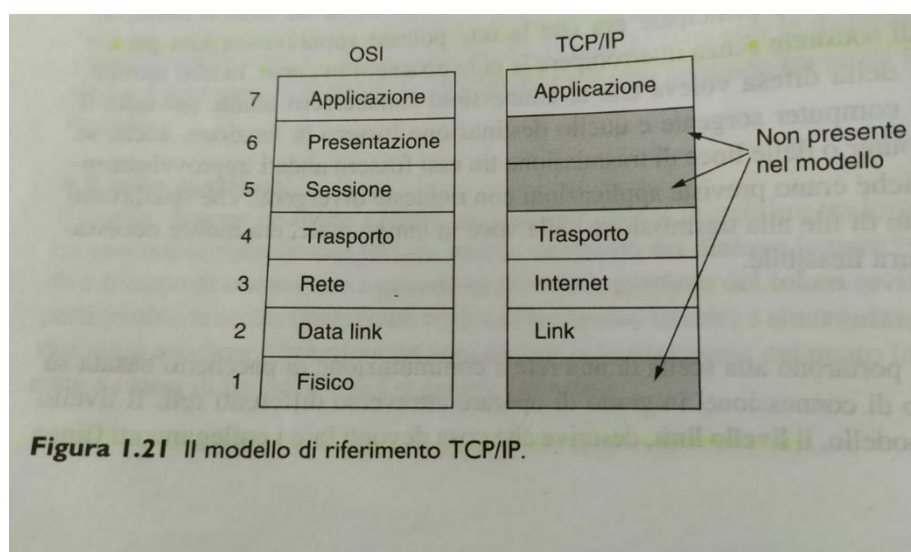
Il livello internet è il perno che tiene insieme l'intera struttura: ha il compito di **permettere agli host di inviare pacchetti su qualsiasi rete** per fare in modo che questi possono viaggiare indipendentemente verso ogni direzione. Definisce un formato ufficiale per i pacchetti e un protocollo chiamato **IP** (*internet protocol*) accompagnato da un protocollo di supporto chiamato **IMCP** (*internet control message protocol*).

Livello di trasporto

Questo livello consente la comunicazione tra peer degli host sorgente e destinazione. Sono stati definiti 2 protocolli di trasporto end to end: il primo, **TCP** (*transmission control protocol*) è un protocollo **affidabile** orientato alla connessione che **permette ad un flusso di byte emessi da un computer di raggiungere senza errori qualsiasi altro computer** sulla rete internet; gestisce anche il **controllo di flusso**. Il secondo protocollo di questo livello è **UDP** (*user datagram protocol*): un protocollo **inaffidabile** senza connessione per le **applicazioni che non vogliono la garanzia di ordinamento e il controllo di flusso**, viene largamente impiegato per le richieste client-server singole di tipo **request-reply** e dalle applicazioni dove la **consegna rapida è più importante dell'accuratezza**.

Livello applicazione

In questo livello ci sono le applicazioni che includono le funzioni di sessione e presentazione necessari. In questo livello si parla di: **SMTP, HTTP, DNS**.



1.7.3 Modello di riferimento

5	Applicazione
4	Trasporto
3	Rete
2	Link
1	Fisico

Noi prenderemo come modello di riferimento quello riportato nella tabella precedente. Nonostante i 2 modelli studiati in precedenza sembrano simili, hanno molte differenze. Nel modello OSI sono presenti **3 concetti essenziali**:

1. servizi
2. interfacce
3. protocolli

Il contributo maggiore di questo modello è la sua capacità di esplicitare la distinzione tra questi concetti: **ogni livello offre un servizio a quello che lo sovrasta** e **la definizione di tale servizio descrive ciò che fa il livello**, in pratica si va a **definire la semantica del livello**. Un aspetto fondamentale è che **un livello può usare i protocolli che preferisce**. Il modello TCP invece non faceva distinzione tra questi 3 concetti, e la conseguenza è che **nel modello OSI i protocolli sono nascosti meglio che nel TCP**. **OSI però è troppo complesso se ci sono scarse implementazioni**, mentre **TCP non è un modello generale ed era poco utile per descrivere le reti non basate su TCP/IP**.

Modalità di comunicazione (orientata o meno alla connessione)

Il modello **OSI** supporta **nel livello di rete entrambi i tipi di comunicazione**, ma **nel livello di trasporto (che è quello che conta, visto che è visibile agli utenti) supporta solo la comunicazione orientata alla connessione**. Il modello TCP al livello di rete **supporta solo la comunicazione orientata alla connessione**, ma entrambe nel livello di trasporto offrendo agli utenti una vera scelta.

Per queste ragioni si è scelto di usare un modello ibrido, prendendo il meglio da entrambi: usiamo **uno stack basato su OSI (senza sessione e presentazione)** e i **protocolli usati da TCP/IP**.

1.7.4 Critiche nei modelli

ISO/OSI

- **Scarso tempismo**: **pessima scelta dei tempi** per gli standard (apocalisse dei due elefanti).
- **Tecnologia scadente**: sessione e presentazione sono vuoti, gli altri sovraccarichi. **Difficile da implementare ed è inefficiente e incomprensibile**.
- **Implementazioni scadenti**
- **Incapacità politica**: all'inizio era del governo.

TCP/IP

Non distingue in modo chiaro i concetti di servizio, interfaccia e protocollo. E' poco generale ed inadatto a contesti differenti dal suo e il livello link non è un vero e proprio livello, ma un'interfaccia e questa distinzione è cruciale. Dovrebbe separare il livello fisico da quello link invece di accorparli: quello fisico ha a che fare con caratteristiche trasmissive (cavi di rame, ecc) mentre quello link deve delimitare inizio e fine dei frame.

1.8 Esempi di reti

Internet

Internet non è affatto una rete, ma una vasta raccolta di reti diverse che usano determinati protocolli e offrono certi servizi comuni. È un sistema inconsueto, che non ha un progettista e non è controllato da nessuno. Nasce come evoluzione di Arpanet. Per collegarsi a internet il computer deve connettersi ad un **ISP**, che fornisce all'utente l'accesso a internet: il collegamento viene effettuato attraverso la linea telefonica, che utilizzerà una linea **DSL** (*digital subscriber line*) che effettua la trasmissione di dati digitali sulla linea telefonica di casa. Il modem DSL svolge la funzione di conversione da pacchetti digitali ad analogici: per **modem** si intende qualunque dispositivo che converte bit digitali in segnali analogici. Un altro metodo utilizzato è quello di inviare segnali sul sistema della TV via cavo. I metodi di accesso menzionati in precedenza sono limitati dalla banda "dell'ultimo miglio" della trasmissione: posando fibre ottiche sino ad arrivare a casa, si otterrebbero accessi internet più veloci (da 10 a 100 Mbps). Questa configurazione è chiamata **FTTH** (*fiber to the home*). Anche la modalità wireless viene utilizzata per accedere ad internet (3G).

3G

I sistemi di terza generazione, o 3G, offrono servizi sia di chiamate vocali sia di trasmissione dati digitali a banda larga: fornisce tassi di trasmissione di almeno 2 Mbps per punti fermi o in movimento. L'**UMTS** (*universal mobile telecommunications system*) è il principale sistema 3G (14 Mbps in download e 6 Mbps in upload). La risorsa più scarsa di queste reti è lo spettro radio, poiché viene concesso dai governi attraverso gare d'appalto: per gestire le interferenze radio tra gli utenti, l'area di copertura viene divisa in celle. L'architettura di una rete cellulare è molto diversa da quella di internet: in primo luogo abbiamo l'*air interface*, che indica il protocollo di comunicazione radio usato tra il dispositivo e la stazione base cellulare (è basato sul CDMA). Il vantaggio della commutazione di circuito è che supporta più facilmente la qualità del servizio: si stabilisce in anticipo la connessione, in questo modo la sottorete può riservare risorse. Un'altra differenza fra le reti cellulari e quelle tradizionali è la mobilità: quando un utente esce dal raggio di una stazione base cellulare ed entra in quello di un'altra, il flusso di dati deve essere **reinstradato**. Questa tecnica prende il nome di **handover**.

2.0 Livello Fisico

Il livello fisico definisce gli aspetti elettrici, temporizzazione e le altre modalità con cui bit, sotto forma di segnali elettromagnetici, vengono spediti sui canali di comunicazione. Le caratteristiche dei vari tipi di canali che determinano le prestazioni sono:

- **Throughput**: capacità trasmissiva
- **Ritardo di trasmissione**

- **Tasso di errore**

2.1 Basi teoriche della comunicazione dati

Le informazioni possono essere trasmesse via cavo variando alcune proprietà fisiche, come la tensione o la corrente, nel tempo. I segnali che vengono inviati sul canale di comunicazione possono essere di due tipi:

1. **Segnali analogici:** si tratta di segnali continui nel tempo che possono assumere tutti gli infiniti valori della grandezza fisica osservabile (che si tratti di una tensione, una corrente ecc..) contenuti all'interno di un range.
2. **Segnali digitali:** si tratta di segnali discreti che possono assumere un numero limitato di valori in un range discreto.

Solitamente, nella comunicazione dei dati si utilizzano segnali analogici periodici e segnali digitali non periodici.

I segnali possono anche essere classificati in semplici e composti:

1. **Segnali semplici:** si tratta di un'onda sinusoidale che non può essere scomposta in segnali più semplici.
2. **Segnali composti:** si tratta di un segnale composto da più sinusoidi.

Quando si parla di segnali è bene tenere a mente alcune definizioni:

- **Frequenza:** variazione dell'onda nel tempo. Quando si ha una variazione repentina, ossia un cambiamento della curva dell'onda in un breve intervallo di tempo, la frequenza è alta. Se un segnale non subisce alcuna variazione nel corso del tempo la frequenza è zero, mentre se cambia istantaneamente è infinita. La frequenza è l'inverso del periodo.
- **Lunghezza d'onda:** è la distanza tra due creste o due gole consecutive dell'onda e si misura in velocità di propagazione / frequenza oppure velocità di propagazione * periodo.
- **Dominio della frequenza:** un'intera onda sinusoidale nel dominio della frequenza può essere rappresentata con un singolo picco nel dominio della frequenza, e questo torna particolarmente utile quando si ha a che fare con segnali composti, infatti nella comunicazione dei dati, segnali semplici a singola frequenza, sono del tutto inutili.

2.1.1 Analisi di Fourier

Fourier dimostrò che qualunque funzione sufficientemente regolare $g(t)$ con periodo T può essere ottenuta sommando un numero (anche infinito) di funzioni seno e coseno: questa composizione è chiamata serie di Fourier. Ogni segnale infatti è una combinazione di sinusoidi a diverse frequenze, ampiezze e fasi.

2.1.2 Segnali a banda limitata

Nessun mezzo di trasmissione è in grado di trasmettere segnali senza perdere parte dell'energia durante il processo: ciò produce un segnale in uscita diverso da quello inviato. Tre cause che portano a tale fenomeno sono:

1. **Attenuazione:** in uscita dal canale l'ampiezza del segnale viene diminuita: il segnale è lo stesso, ma viene attenuato. Per riprodurre il segnale originale bisogna amplificare quello ricevuto, utilizzando peer esempio degli switch
2. **Distorsione:** in uscita dal canale il segnale ha cambiato tipologia. La distorsione avviene poiché non tutte le frequenze viaggiano alla stessa velocità all'interno del canale.

3. **Rumore:** in uscita dal canale al segnale in ingresso vengono aggiunte delle componenti che non fanno parte del segnale trasmesso. Il rumore può essere dovuto sia a fonti esterne sia al mezzo stesso (per esempio, il rumore termico).

L'intervallo di frequenze trasmesse senza una forte attenuazione è chiamato banda, ma è possibile fornire una definizione di banda utilizzando come unità di misura il bit al secondo piuttosto che l'Hz: infatti la possiamo definire anche come la massima velocità di trasmissione su un canale. La banda passante è una proprietà fisica del mezzo di trasmissione e di solito dipende dalla costruzione, dallo spessore e dalla lunghezza del mezzo. I segnali che partono da 0 fino ad una frequenza massima prendono il nome di **banda base** (baseband), mentre i segnali che vengono traslati per occupare una gamma di frequenze più alta (come nel caso delle trasmissioni wireless), vengono detti segnali in **banda passante** (passband).

Nelle trasmissioni digitali l'obiettivo è ricevere un segnale in modo sufficientemente fedele da poter ricostruire la sequenza di bit inviata.

2.1.3 Velocità massima di trasmissione di un canale

Nyquist si accorse che anche un canale perfetto ha una capacità di trasmissione limitata e formulò un'equazione in grado di esprimere la velocità massima di trasmissione all'interno di un canale privo di rumore e con larghezza di banda limitata. Il teorema di Nyquist afferma che un segnale inviato su un canale con una certa larghezza di banda B può essere ricostruito correttamente se si prendono esattamente $2B$ campioni al secondo. Se il segnale è composto da V livelli discreti, il teorema di Nyquist afferma che:

$$\text{massimo tasso trasmissivo} = 2B \log_2 V \text{ bit/s}$$

Il teorema di Nyquist non teneva conto del fatto che *non esistono nella realtà canali ideali*, pertanto doveva essere apportata qualche modifica alla formula. Se sul canale è presente un rumore casuale, la situazione peggiora rapidamente, ma il rumore termico provocato dal movimento delle molecole del sistema non può essere rimosso. Il livello di rumore termico si misura facendo il rapporto tra la potenza del segnale e la potenza del rumore: questo rapporto viene chiamato **SNR** (*signal-to-noise ratio*). Solitamente, invece di indicare direttamente S/N , il rapporto segnale/rumore viene definito come $10 \log_{10} S/N$ e si misura in dB. Considerato questo fattore particolarmente rilevante, il **teorema di Shannon** afferma che il massimo tasso di invio dei dati in bit/s su un canale rumoroso la cui ampiezza di banda è B Hz e il cui rapporto segnale rumore è S/N può essere scritto come:

$$\text{massimo numero di bit/s} = B \log_2 (1 + S/N)$$

E definisce la massima capacità di un canale fisico.

Il prodotto tra la larghezza di banda e il ritardo trasmissivo definisce il numero di bit che riempiono il canale, ossia quale è il numero massimo di bit che possono essere trasmessi. Se immaginiamo il canale come un tubo, la larghezza di banda può essere assimilata alla sezione trasversale, mentre il ritardo alla lunghezza del tubo. Il volume del canale è il prodotto banda-ritardo. È importante non saturare completamente il canale: quando si riempie del tutto il volume significa che è in atto un attacco DOS.

2.2 Mezzi di trasmissione vincolati

Lo scopo del livello fisico è di permettere a bit grezzi di trasportare un flusso grezzo di bit da una macchina all'altra. Per realizzare la trasmissione è possibile utilizzare diversi tipi di mezzi fisici che si distinguono l'uno dall'altro per banda passante, ritardo, costo d'installazione e facilità di manutenzione. Si analizzano in prima battuta i mezzi di trasmissione vincolati, quali nastri magnetici, doppi, cavi coassiali e fibre ottiche.

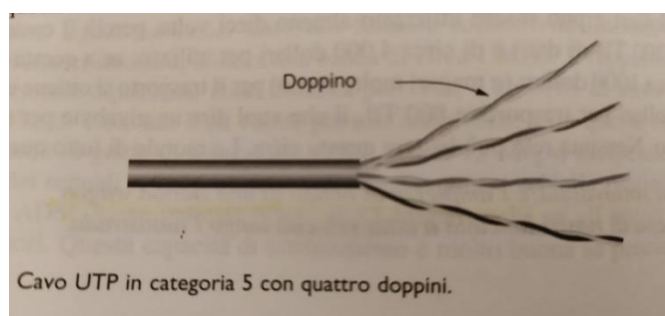
2.2.1 Supporti magnetici

I supporti magnetici, come nastri magnetici o DVD, servono per immagazzinare informazioni. Una volta che le informazioni sono state scritte sul dispositivo, possono essere trasportate ovunque fisicamente e lette utilizzando un'apposita unità installata nel computer di destinazione. I supporti magnetici sono molto economici e possono immagazzinare una grande quantità di dati, ma il ritardo è troppo alto per la maggior parte delle applicazioni.

2.2.2 Doppino

Il doppino è molto utilizzato, soprattutto nel sistema telefonico, poiché offre prestazioni discrete a basso costo. Un doppino si compone di due cavi in rame intrecciati fra di loro in una struttura ad elica. Tale tecnica viene adottata per permettere ai campi elettromagnetici generati dai due cavi di annullarsi a vicenda, in modo tale che non vadano ad interferire l'uno con l'altro: nel caso in cui i due cavi fossero stati disposti parallelamente sarebbero stati delle ottime antenne.

I cavi vengono poi schermati da una guaina di materiale isolante, per ridurre al minimo il rumore causato da fonti esterne. I doppi vengono suddivisi per categorie: le prime 6 categorie vengono definite **UTP** (*unshielded twisted pair*) perché i doppi vengono schermati solamente da una singola guaina protettiva (per esempio un doppino di cat. 5 si compone di 4 doppi uniti da un'unica guaina). Dalla categoria 7 ciascun doppino è schermato, oltre al materiale isolante che protegge l'intero fascio, e ciò permette di reagire ancora meglio alle interferenze e aumentare le prestazioni; più aumenta il numero di intrecci migliore è la qualità del segnale sulle lunghe distanze.

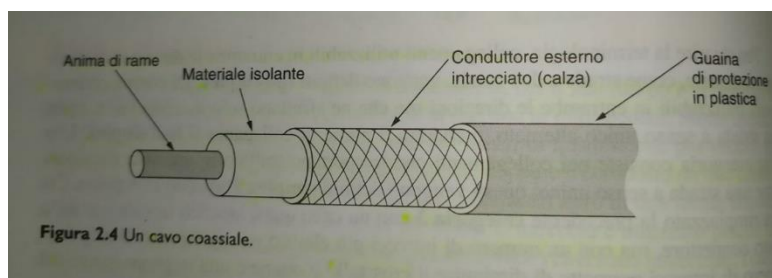


I doppi possono essere utilizzati per inviare sia segnali analogici sia segnali digitali. Standard differenti utilizzano i doppi in modo diverso, in modalità half-duplex o full-duplex.

2.2.3 Cavo coassiale

Un altro mezzo di trasmissione molto comune è il cavo coassiale. Un cavo coassiale è molto più schermato rispetto al doppino e questo gli consente di trasmettere il segnale per distanze maggiori e a velocità più elevate. Si compone di un'anima in rame, protetta da un rivestimento isolante, a sua volta circondato da un conduttore cilindrico (solitamente realizzato con una calza di conduttori sottili),

avvolto infine da una guaina protettiva in plastica. Questa doppia protezione fornisce eccellente immunità al rumore la sua costruzione permette di utilizzare molta banda.



Con l'avvento della fibra ottica il cavo coassiale non viene più utilizzato per coprire le lunghe distanze nel sistema telefonico ma viene piuttosto sfruttato nelle reti metropolitane e per la televisione via cavo.

2.2.5 Fibre ottiche

Le fibre ottiche vengono utilizzate per le trasmissioni a lunga distanza nelle dorsali di rete e le reti locali ad alta velocità. Un sistema di trasmissione ottico è formato da 3 componenti fondamentali: la sorgente luminosa, il mezzo trasmissivo e il rilevatore.

- **Sorgente luminosa:** viene posta ad un lato della fibra. Per convenzione si assume che la presenza di luce indica il valore 1 mentre la sua assenza indica il valore 0.
- **Rilevatore:** si colloca dall'altro capo della fibra rispetto alla sorgente luminosa e quando viene colpito dalla luce, genera un impulso elettrico.
- **Mezzo di trasmissione:** Il mezzo trasmissivo è una fibra di vetro sottile quanto un capello.

Si crea in questo modo un sistema di trasmissione unidirezionale, che, preso in ingresso un segnale elettrico lo converte e lo trasmette sotto forma di impulso luminoso, per poi riconvertirlo nuovamente in segnale elettrico all'estremità della fibra. Il sistema di trasmissione appena descritto è utile in quanto sfrutta un interessante principio della fisica che consente di non disperdere la luce. Nel momento in cui un segnale luminoso passa da un materiale ad un altro, per esempio dal silicio all'aria, si rifrange, ossia si curva, sul confine tra i due materiali.

L'angolo di rifrazione dipende dalle proprietà dei due materiali e dal loro indice di rifrazione: per gli angoli di incidenza che superano un particolare valore critico, la luce è rifratta indietro nel silicio senza disperdersi nell'aria (riflessione totale). Poiché tutti i raggi di luce che colpiscono il cavo con un angolo maggiore rispetto a quello critico vengono totalmente riflessi all'interno, la fibra può contenere molti raggi che rimbalzano ad angoli diversi a seconda della loro lunghezza d'onda. In questo caso ogni raggio ha una modalità diversa e una fibra di questo tipo viene detta fibra **multimodale**. Se invece il diametro della fibra viene ridotto fino a poche lunghezze d'onda della luce, la luce si propagerà in linea retta senza rimbalzare, si parla dunque di fibra **monomodale**, più costosa della multimodale e utilizzata soprattutto sulle lunghe distanze.

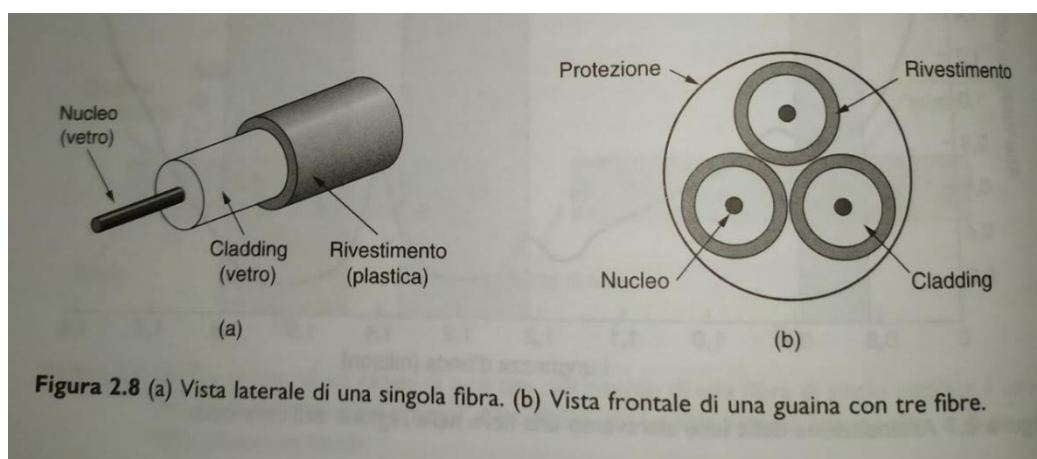
2.2.5.1 Trasmissione della luce attraverso la fibra

L'attenuazione della luce attraverso il vetro dipende dalla sua lunghezza d'onda, oltre che alle caratteristiche fisiche del vetro; è definita come il rapporto tra la potenza del segnale d'ingresso e quello di uscita. Le lunghezze d'onda più comuni per le comunicazioni ottiche sono tre e sono centrate rispettivamente a 0,85, 1,30 e 1,55 micron. Gli impulsi luminosi trasmessi attraverso la fibra si espandono nella lunghezza d'onda durante la propagazione.

2.2.5.2 Cavi in fibra ottica

I cavi in fibra ottica sono simili ai cavi coassiali, ma non sono avvolti dalla calza conduttrice. Al centro si trova un nucleo di vetro attraverso il quale si propaga la luce, circondato da un rivestimento di vetro, chiamato **cladding**, che presenta un indice di rifrazione più basso: ciò costringe la luce a rimanere nel nucleo e dunque ai dati di propagarsi solo in questa parte della fibra. Lo strato successivo è una sottile fodera di plastica che protegge il rivestimento. Solitamente le fibre vengono raggruppate in fasci protetti da una guaina più esterna. Le sorgenti luminose possono essere di due tipi: LED e laser. A lato ricevente viene posto un fotodiodo che produce un impulso elettrico nel momento in cui viene colpito dalla luce. Il tempo di risposta dei fotodiodi è limitato dalla velocità di trasmissione dei dati e dal rumore termico: l'impulso luminoso deve trasportare abbastanza energia da poter essere rilevato.

Elemento	LED	LASER SEMICONDUCTORE	A
Tasso di trasmissione dei dati	Bassa	Alta	
Tipo di fibra	Multimodale	Multimodale o monomodale	
Distanza	Breve	Lunga	
Durata	Lunga	Breve	
Sensibilità alla temperatura	Scarsa	Elevata	
Costo	Basso	Alto	



2.2.5.3 Confronto tra fibre ottiche e cavi in rame

FIBRE OTTICHE	CAVI IN RAME
Maggiore ampiezza di banda rispetto ai cavi in rame	Minore ampiezza di banda rispetto alla fibra ottica
A causa del basso livello di attenuazione, i ripetitori possono essere installati ogni 50 km	A causa di un livello di attenuazione più elevato, i ripetitori devono essere installati ogni 5 km
Non è influenzata da sorgenti elettriche né da agenti chimici, quindi può essere installata anche negli ambienti più inospitali	È influenzata dai campi elettromagnetici e dalle interruzioni della linea elettrica
Molto sottile e leggera	Cavi molto più pesanti
Non hanno perdite di luce, quindi è molto difficile intercettare i dati	I dati possono essere persi molto più facilmente

La fibra è una tecnologia meno nota, e ancora non esistono molti ingegneri capaci di maneggiarla agevolmente senza rovinarla, ma rappresenta il futuro delle comunicazioni dati.

2.3 Trasmissioni wireless

Per essere sempre online sono necessarie le comunicazioni wireless. Di seguito si farà una breve panoramica sulle onde elettromagnetiche utilizzate per le trasmissioni di questo tipo

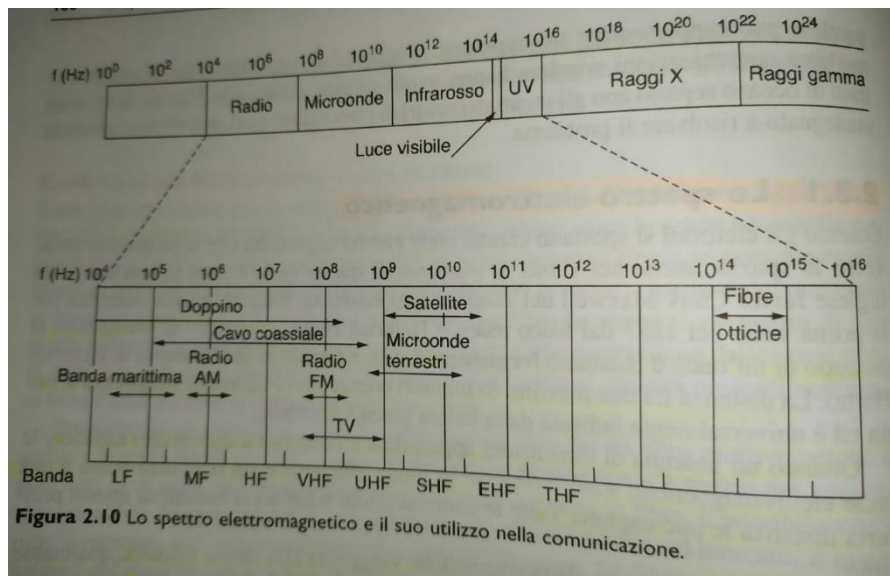
2.3.1 Lo spettro elettromagnetico

Gli elettroni spostandosi generano onde elettromagnetiche che si propagano in un mezzo oppure nel vuoto. Il numero di oscillazioni di un'onda nell'unità del tempo è detto frequenza e si misura in Hz. La distanza tra due massimi (o minimi) consecutivi invece è detta lunghezza d'onda e si indica con la lettera λ . Nel vuoto tutte le onde viaggiano alla stessa velocità, qualunque sia la loro frequenza. Questa velocità, chiamata velocità della luce, è pari a circa a $3 * 10^8$ m/s e nessun oggetto o segnale può muoversi più velocemente. La velocità della luce è legata alla frequenza e alla lunghezza d'onda dalla formula:

$$c = f * \lambda$$

Le porzioni dello specchio elettromagnetico indicate come radio, microonde, infrarosso e a luce visibile si possono utilizzare per trasmettere informazioni modulando ampiezza, frequenza o fase delle onde. Come già dimostrato da Shannon, la quantità d'informazione che un'onda elettromagnetica può trasportare dipende dall'energia ricevuta ed è proporzionale alla sua banda. La maggior parte delle trasmissioni utilizza una banda di frequenze ristretta per ottenere una maggiore ricezione, ma in certi casi si preferisce utilizzare la banda larga con due varianti:

- **Tecnica a spettro distribuito a frequenza variabile** (FHSS, *frequency hopping spread spectrum*): il trasmettitore salta da una frequenza all'altra centinaia di volte al secondo. Questo tipo di trasmissione viene utilizzata solitamente nelle comunicazioni militari perché rende difficile rilevare le trasmissioni ed è quasi impossibile da disturbare
- **Tecnica a spettro distribuito a sequenza diretta** (DSSS, *direct sequence spread spectrum*): utilizza una sequenza codificata per distribuire il segnale su una banda di frequenze molto più ampia ed è molto più efficiente nel permettere a più segnali di condividere le stesse bande di frequenza



Esiste poi una terza metodologia che fa uso di una banda ancora più larga, ossia **UWB** (ultra wideband, *banda ultra larga*) che permette di **trasmettere i dati tramite una serie di impulsi molto rapidi e in posizioni diverse**. Trasmette dunque a velocità elevatissime e tollera le interferenze più forti da parte di altri segnali a banda più stretta.

2.3.2 Trasmissioni radio

Le **onde radio sono semplici da generare**, inoltre possono **viaggiare per lunghe distanze e riescono ad attraversare facilmente gli edifici**, per questo motivo vengono utilizzate spesso sia nelle comunicazioni all'interno sia all'esterno. Hanno la **proprietà di essere omnidirezionali**, ossia di espandersi dalla sorgente in tutte le direzioni: ciò significa che l'antenna del trasmittente e del ricevente non deve essere necessariamente allineate.

Le proprietà delle onde radio dipendono dalla frequenza: alle **frequenze più alte le onde radio attraversano bene gli ostacoli**, ma la **potenza diminuisce di molto**, mano a mano che ci si allontana dalla sorgente, e il **segnale tende ad attenuarsi bruscamente all'aumentare della distanza**. Le onde radio **subiscono interferenze da parte di ogni sorta di dispositivo elettrico**. Questo comportamento implica che le onde radio possono percorrere lunghe distanze e l'**interferenza diventa un problema non banale**.

Nelle bande **VLF** (very low frequency), **LF** (low frequency) e **MF** (medium frequency) **le onde radio seguono la curvatura del pianeta**: le onde radio che attraversano queste bande passano **facilmente attraverso gli edifici** (per questo motivo le radio portatili funzionano al chiuso), mentre nel caso della comunicazione di dati il **problema principale di queste bande è la loro scarsa ampiezza di banda**. Nelle bande **HF** (high frequency) e **VHF** (very high frequency) **le onde radio vengono riflesse dalla ionosfera**. Le onde radio che attraversano queste bande vengono **utilizzate per le comunicazioni militari**.

2.3.3 Trasmissioni a microonde

Le **microonde viaggiano in linea retta a differenza delle onde radio**, quindi l'antenna del trasmittente e ricevente devono essere **accuratamente allineate le une con le altre**. Il vantaggio delle microonde è dato dal fatto che **l'energia viene concentrata in un piccolo raggio per mezzo di un'antenna parabolica**, così ottiene un **rapporto segnale/rumore molto più alto**. Dato il fatto che le microonde viaggiano in linea retta se le antenne sono troppo distanti le une dalle altre entra in gioco la **curvatura terrestre**, per

cui sono necessari dei ripetitori. Questo tipo di comunicazione viene utilizzata nelle comunicazioni telefoniche a lunga distanza, nella telefonia cellulare, nella televisione e in altre applicazioni, e sono così utilizzate poiché sono abbastanza economiche.

Politiche dello spettro elettromagnetico

Un approccio che viene spesso adottato consiste nel non assegnare le frequenze, in modo tale che ognuno possa trasmettere a piacere ma limitando la potenza utilizzata in modo da limitare la portata delle stazioni per evitare l'interferenza reciproca. La maggior parte dei governi ha deciso di riservare alcune bande di frequenza chiamate **ISM** (*industrial, scientific, medical*) per l'utilizzo senza licenze. Il vantaggio dell'utilizzo di queste bande è che sono gratis, mentre lo svantaggio è che sono spesso troppo affollate. Per limitare l'interferenza tra tutti i dispositivi non coordinati, tutti gli apparecchi nelle bande ISM limitano la frequenza trasmissiva ad un watt e utilizzano tecniche di trasmissione a spettro distribuito.

2.3.4 Trasmissioni a infrarossi

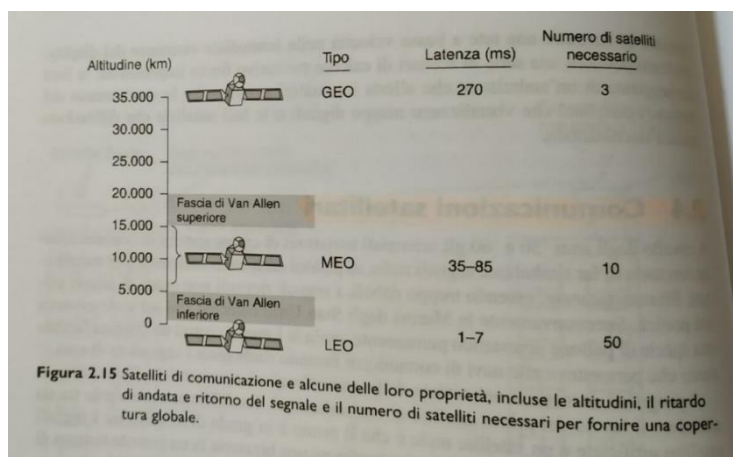
Vengono utilizzate per le comunicazioni a corto raggio (per esempio nei telecomandi). Le onde a infrarossi non riescono ad attraversare ostacoli solidi. È un sistema economico e facile da costruire.

2.3.5 Trasmissioni a onde luminose

Ogni edificio ha bisogno di un laser e di un rilevatore fotoelettrico e le onde viaggiano lungo una direzione.

2.4 Comunicazioni satellitari

Un satellite di comunicazione può essere immaginato come un grande ripetitore di microonde collocato nel cielo. Questo dispositivo contiene diversi transponder, ossia ricetrasmittitori satellitari: ognuno di questi ascolta una parte dello spettro, amplifica il segnale in ingresso e lo ritrasmette su un'altra frequenza per evitare interferenze con il segnale in arrivo. Attraverso questi sistemi è possibile manipolare o dirigere i flussi di dati all'interno della banda; l'informazione digitale può anche essere ricevuta dal satellite e ridistribuita. In base alla legge di Keplero vediamo che i satelliti su orbite basse scompaiono dalla vista rapidamente e ne servono molti per fornire una copertura continua. Un altro problema è rappresentato dalla presenza delle fasce di Van Allen: sono strati di particelle molto cariche intrappolate nel campo magnetico terrestre e se un satellite le attraversasse verrebbe distrutto. Questi fattori conducono alle 3 zone in cui satelliti possono essere collocati senza pericolo, mostrate nella figura sottostante.



Bent-pipe: modalità operativa attraverso la quale i raggi puntati verso il basso possono essere larghi, per coprire una considerevole frazione della superficie terrestre, oppure stretti e coprire un'area dal diametro di poche centinaia di chilometri.

2.4.1 Satelliti geostazionari - GEO

Questi satelliti sono collocati su orbite alte e hanno le seguenti caratteristiche:

Periodo $T = 24h$

Quantità minima necessaria $n = 3$

Altitudine = 35.000 Km

Al fine di evitare il caos totale nel cielo, l'ITU assegna la larghezza di banda alle applicazioni dei satelliti e l'allocazione degli slot orbitali. Le bande principali utilizzate dei satelliti sono le seguenti.

Banda	Downlink	Uplink	Larghezza della banda	Problemi
L	1,5 GHz	1,6 GHz	15 MHz	Banda stretta; affollamento
S	1,9 GHz	2,2 GHz	70 MHz	Banda stretta; affollamento
C	4,0 GHz	6,0 GHz	500 MHz	Interferenza terrestre
Ku	11 GHz	14 GHz	500 MHz	Pioggia
Ka	20 GHz	30 GHz	3.500 MHz	Pioggia; costo degli apparati

Figura 2.16 Le principali bande usate dai satelliti.

VSAT

Un recente sviluppo nel settore delle comunicazioni satellitari è rappresentato dalle microstazioni a basso costo chiamate **VSAT** (*very small aperture terminal*). Questi piccoli terminali possono generare all'incirca 1 watt di potenza: in *uplink* va bene per trasportare fino a 1 Mbps, mentre in *downlink* è generalmente di diversi Mbps (questa tecnologia è impiegata nelle televisioni satellitari). In molti sistemi VSAT le microstazioni non hanno abbastanza energia per comunicare direttamente, per questo motivo è necessario installare speciali stazioni terrestri, chiamate **hub** e dotate di grosse antenne che trasmettono il traffico attraverso le stazioni VSAT. In questa modalità operativa la stazione trasmittente oppure quella ricevente fanno uso di una grande antenna e di un potente amplificatore: il compromesso per avere stazioni terminali utente più economiche è quello di un maggiore ritardo di propagazione.

2.4.1.1 Proprietà dei satelliti di comunicazione

- Anche se i segnali trasmessi e ricevuti da un satellite viaggiassero a velocità della luce, il lungo viaggio di andata e ritorno introdurrebbe un sostanziale ritardo nei satelliti GEO. In generale i cavi sono più lenti, poiché i segnali elettromagnetici viaggiano più velocemente in aria che attraverso materiali solidi (collegamenti terrestri a microonde hanno un ritardo di propagazione pari a 3 $\mu s/km$, mentre le fibre ottiche intorno ai 5 $\mu s/km$).
- I satelliti sono mezzi di trasmissione broadcast per natura: inviare un messaggio a migliaia di stazioni poste nella stessa area del transponder costa quanto inviare un messaggio a una singola stazione.

- Dal punto di vista della sicurezza e della **privacy** i satelliti sono un vero disastro: **tutto può essere ascoltato da tutti**. Per proteggere le comunicazioni bisogna implementare sistemi di crittografia.
- **Il costo della trasmissione del messaggio è indipendente dalla distanza traversata**. Godono anche di un eccellente tasso di errore.

2.4.2 Satelliti su orbite medie - MEO

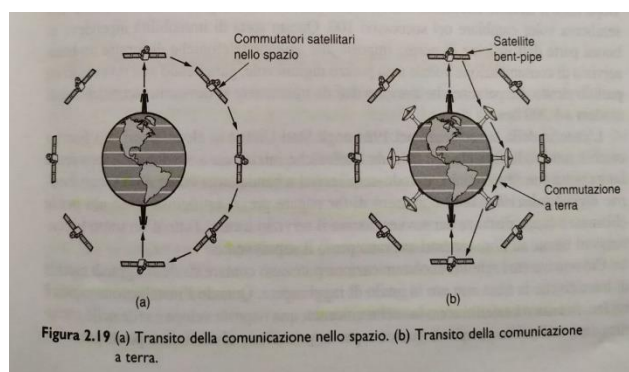
I satelliti **MEO** (*medium earth orbit*) si trovano ad altitudini molto più basse, comprese fra le **2 fasce di Van Allen**. Questi Satelliti si spostano lentamente lungo la longitudine impiegando **circa sei ore per compiere un giro intorno al pianeta**: coprono un'area più piccola e sono raggiungibili per mezzo di trasmettitori meno potenti. I circa **30 satelliti GPS (*global position system*)** che orbitano a circa **20.200 km di altezza** sono di questa tipologia.

2.4.3 Satelliti su orbite basse - LEO

Scendendo di altezza si incontrano i satelliti **LEO** (*low earth orbit*). Poiché si **spostano rapidamente**, **per realizzare un sistema completo bisogna utilizzarne molti**, d'altra parte vista la loro vicinanza le stazioni terrestri non hanno bisogno di molta energia, il **ritardo nelle comunicazioni è di pochi millisecondi** e anche il **costo complessivo è molto vantaggioso**.

Iridium

Un progetto di satelliti LEO è **Iridium**, che si trova a un'altitudine di **750 chilometri**, orbitando in orbite polari circolari, formando 6 collane intorno alla terra. Una proprietà interessante di Iridium è che la comunicazione tra clienti distanti avviene nello spazio, dove un satellite trasmette i dati al successivo: ogni satellite ne ha 4 vicini con cui è in grado di comunicare, di questi 2 nella stessa collana e 2 in collane adiacenti. I satelliti **reindirizzano la chiamata attraverso un'infrastruttura finché non viene mandato a terra all'utente chiamato**.



Globastar

Un progetto alternativo a Iridium è **Globastar**. Si basa su 48 satelliti LEO e utilizza un diverso schema di **commutazione**. Mentre Iridium ritrasmette le chiamate da satellite a satellite, attraverso una tecnica che richiede la presenza di sofisticate attrezzature, Globastar utilizza un modello tradizionale **bent-pipe** (Visibile nella figura sopra riportata). Ad esempio **se la chiamata ha origine al Polo Nord viene trasmessa sulla terra**, raccolta da una **grande stazione terrestre vicino al laboratorio di Babbo Natale** e dirottata attraverso una **rete terrestre alla stazione più vicino al destinatario**, che la riceve tramite un'altra connessione bent-pipe. Questo schema ha il grande vantaggio di **tenere la complessità a terra**, dove è più facile da gestire.

2.5 Modulazione digitale e multiplexing

I canali sia cablati che wireless trasportano **segnali analogici** e per inviare informazioni digitali dobbiamo usare questi segnali per **rappresentare i bit**. Il processo di conversione tra bit e i segnali che li rappresentano prende il nome di **modulazione digitale**. Esistono strategie che **convertono direttamente bit in segnali** per ottenere una **trasmissione in banda base**, nella quale i segnali occupano frequenze che vanno **da zero fino a un massimo** che dipende dal tasso di trasmissione. Altre strategie agiscono su **ampiezza, fase o frequenza** di un segnale di base (o portante) per il trasferimento dei dati, ottenendo così una **trasmissione in banda passante** nella quale il segnale **occupa una banda di frequenze attorno a quella del segnale importante**. I canali di trasmissione vengono condivisi di norma da più segnali; questa condivisione viene chiamata **multiplexing**.

2.5.1 Trasmissione in banda base

NRZ (*non return to zero*)

Questo tipo di approccio prevede l'uso di una **tensione positiva per rappresentare un 1** e **negativo per rappresentare uno 0**. Per una fibra ottica invece la **presenza di luce** potrebbe rappresentare un **1** e l'**assenza** uno **0**. NRZ sta a **significare che il segnale segue l'andamento dei dati**.

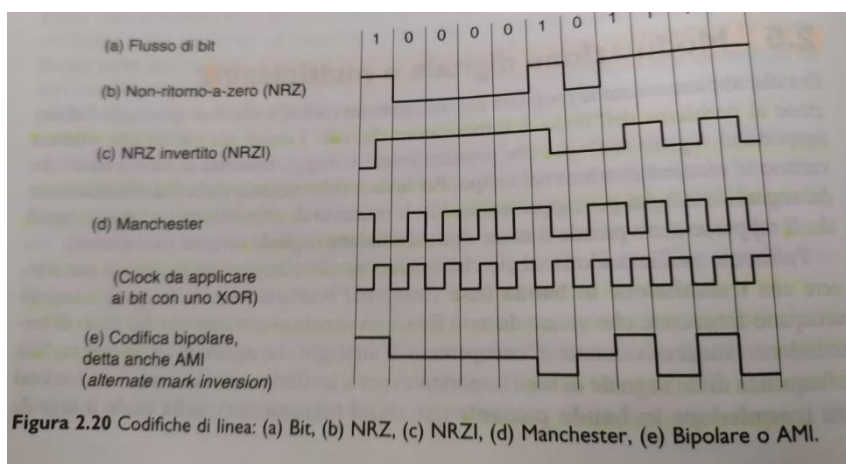


Figura 2.20 Codifiche di linea: (a) Bit, (b) NRZ, (c) NRZI, (d) Manchester, (e) Bipolare o AMI.

Una volta spedito, il segnale NRZ si **propaga lungo il cavo** e all'altro capo il ricevente lo **converte in bit campionandolo a intervalli regolari**. Il segnale ottenuto però **non sarà esattamente come quello originale**, ma **attenuato e distorto del canale e dal rumore** della stazione ricevente. Per la decodifica il ricevente assegna campioni ai simboli più vicini: in NRZ come detto prima tensione positiva corrisponde a **1**, zero viceversa.

Esistono strategie più complesse che possono convertire bit in segnali con conseguenti problematiche tecniche; questi bit vengono chiamati **line code** (*codifica di linea*). Di seguito verranno descritti alcuni line code che possono essere utili per l'efficienza di banda, clock recovery e bilanciamento in corrente continua.

2.5.1.1 Efficienza di banda

La banda spesso è una **risorsa limitata**, quindi vengono adottate **strategie per sfruttarla al meglio**. attraverso relazioni matematiche si arriva dimostrare che non possiamo fare andare NRZ più veloce a meno di non usare più banda. Una possibile strategia è quella di usare **più di 2 livelli di segnale**: se si usano **4 livelli di tensione**, per esempio, siamo in grado di spedire **2 bit per volta come unico**

simbolo. Questo approccio però funziona solo se i segnali al lato ricevente sono abbastanza forti da permettere di distinguere i 4 livelli.

Chiamiamo **symbol rate** il tasso con cui segnale cambia, per distinguerlo dal **bit rate** che equivale al symbol rate moltiplicato per il numero di bit in ogni simbolo.

2.5.1.2 Clock recovery

Per quanto riguarda tutte le strategie che codificano bit in simboli, per poter effettuare correttamente la decodifica il ricevente è tenuto a conoscere quando un simbolo termina e il successivo inizia. Dopo un po' diventa difficile distinguere bit a meno di non disporre di temporizzazione (clock) molto precisa. Clock precisi potrebbero essere utili, ma sono costosi. Una possibile alternativa è di spedire al destinatario un segnale di clock separato: aggiungere una connessione non è un grosso problema per il bus, ma è uno spreco di risorse per molte connessioni di rete visto che se avessimo una linea disponibile la potremmo usare per mandare dati.

Manchester encoding (codifica Manchester)

Questo tipo di codifica prevede di mandare il segnale di clock miscelato con i dati usando l'operazione logica di **XOR**, così da non dover usare una seconda connessione. Il clock effettua una trasmissione ad ogni bit, in questo modo procede al doppio del bit rate: quando viene applicato con uno XOR a un livello zero effettua una transizione dal basso verso l'alto che è semplicemente il clock, e questo rappresenta uno zero, quando invece viene combinato con un 1 si inverte ed effettua una transizione dall'alto verso il basso rappresentando un 1. L'aspetto negativo della codifica Manchester è che a causa del clock, richiede il doppio di banda rispetto a NRZ.

NRZI (no return to zero inverted)

In questo tipo di codifica viene codificato un 1 come una transizione e uno 0 come una situazione stazionaria. Lo standard USB, usato per connettere periferiche di calcolatori, usa NRZI. Purtroppo però lunghe sequenze di 0 sono ancora fonte di problemi e non possiamo trascurarle.

Scrambling

Questo approccio fa sembrare i dati come generati casualmente. In questo caso è altamente probabile che ci siano frequenti transizioni. Uno scrambler opera applicando, prima della trasmissione, l'operazione di XOR tra i dati e una sequenza di numeri pseudo casuali; in questo modo i dati sembreranno casuali tanto quanto la sequenza estratta. Il ricevente applicherà l'operazione di XOR ai dati in arrivo con la stessa sequenza pseudocasuale per recuperare i dati originali.

2.5.1.3 Segnali bilanciati

Segnali bilanciati presentino una tensione tanto positiva quanto negativa anche se su brevi periodi temporali: loro media è zero, il che significa che non hanno componenti in corrente continua. Un modo diretto di formulare un codice bilanciato è quello di usare 2 livelli di tensione per rappresentare un 1 (+1V e -1V) e 0V per rappresentare uno 0. Per spedire un 1 il trasmittente alterna tra i livelli +1V e -1V in maniera tale che questi si annullano in media: questo schema è chiamato **codifica bipolare**. La codifica bipolare aggiunge un livello di tensione per ottenere il bilanciamento.

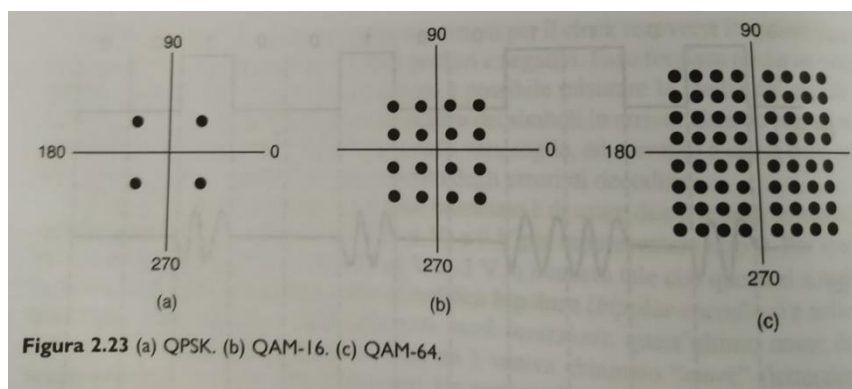
2.5.2 Trasmissione in banda passante

Nella trasmissione a banda passante viene usata una banda di frequenza per far passare il segnale. Possiamo prendere un segnale in banda base che occupa da 0 a B Hz e traslarlo fino ad occupare una

banda passante da S a $S + B$ Hz, senza cambiare le informazioni. Per elaborare un segnale al ricevente possiamo traslarlo indietro in banda base. Del segnale portante possiamo modulare ampiezza, frequenza o fase:

- **ASK** (*amplitude shift keying*): due diverse ampiezze sono usate per rappresentare 0 e 1.
- **FSK** (*frequency shift keying*): vengono usate due o più frequenze.
- **PSK** (*phase shift keying*): l'onda portante è traslata di 0 o 180 gradi all'inizio della trasmissione di ogni simbolo. In questo caso essendoci due fasi, il sistema viene chiamato BPSK.
- **QPSK** (*quadrature phase shift keying*): sistema più efficiente dal punto di vista della banda, che fa uso di 4 traslazioni: 45, 135, 225 e 315 gradi.

Per trasmettere più bit per simbolo possiamo combinare questi approcci e usare più livelli: solo una tra frequenza e fase può essere modulata ogni volta.



Questo tipo di diagramma prende il nome di **constellation diagram**: per ogni punto la fase è l'angolo tra la retta che lo congiunge con l'origine e l'asse delle ordinate, mentre l'ampiezza è la distanza dall'origine. Nella figura vediamo uno schema con una costellazione più densa, con sedici combinazioni di fase e ampiezza; prende il nome di **QAM-16**, in cui si possono trasmettere 4 bit per simbolo. Esiste una costellazione ancora più densa, con 64 combinazioni diverse in grado di codificare 6 bit per simbolo: **QAM-64**.

Queste costellazioni non mostrano come i bit siano assegnati ai simboli: al momento dell'assegnazione è necessario fare attenzione che bassi livelli di rumore in ricezione non portino troppi errori sui bit. Una soluzione è quella di associare bit ai simboli in maniera tale che i simboli adiacenti differiscano di un solo bit. Tale associazione è chiamata **Gray code**.

2.5.3 Multiplexing a divisione di frequenza

Per permettere a molti segnali di condividere lo stesso canale trasmissivo sono nate le tecniche di multiplexing.

FDM (*frequency division multiplexing*) sfrutta la trasmissione in banda passante per condividere un canale: divide lo spettro in bande di frequenza di cui ogni utente ha un uso esclusivo.

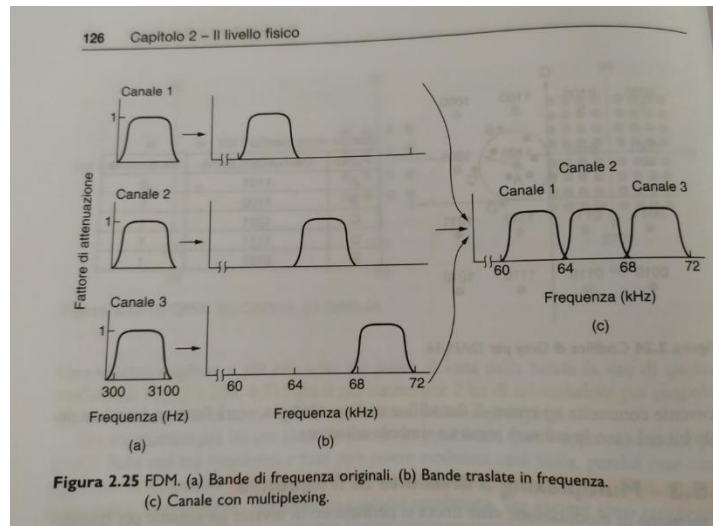


Figura 2.25 FDM. (a) Bande di frequenza originali. (b) Bande traslate in frequenza. (c) Canale con multiplexing.

L'eccesso di allocazione di banda prende il nome **banda di guardia** e permette di tenere i canali ben separati fra loro. Ci sono però delle sovrapposizioni tra i canali, perché i filtri nel mondo reale non hanno spigoli retti come nella teoria, e al giorno d'oggi si preferisce fare multiplexing nel tempo.

OFDM (*orthogonal frequency division multiplexing*) è una tecnica che non utilizza bande di guardia ma divide la banda del canale in molte sottoportanti che inviano dati in maniera indipendente; queste sottoportanti sono stipate una a ridosso dell'altra nel dominio della frequenza.

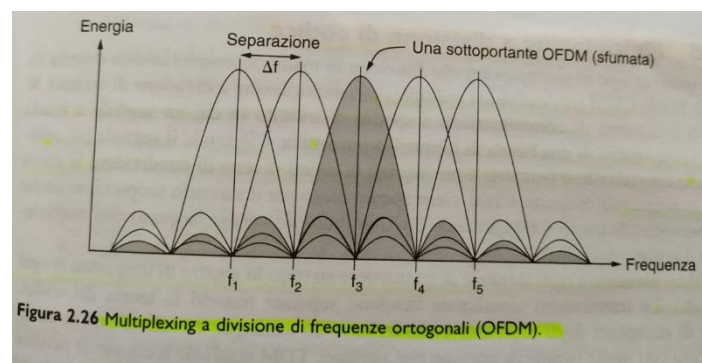
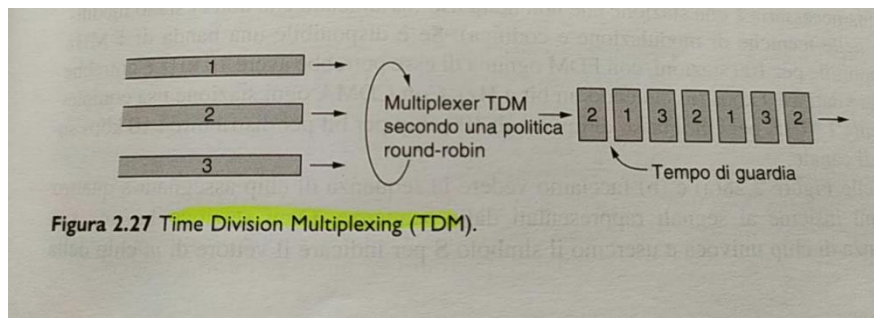


Figura 2.26 Multiplexing a divisione di frequenze ortogonali (OFDM).

La risposta in frequenza risulta essere zero al centro delle sottoportanti ad essa adiacenti, e può quindi essere campionata nella frequenza centrale senza interferenza da parte dei vicini. OFDM è usato per 802.11, TV via cavo e linee telefoniche.

2.5.4 Multiplexing a divisione di tempo

Un'alternativa a FDM è **TDM** (*time division multiplexing*): in questo caso gli utenti fanno i turni secondo una politica round-robin, e ognuno di loro, periodicamente, prende possesso della banda completa per un tempo limitato. I bit di ogni flusso in ingresso sono presi in un certo intervallo temporale (time-slot) e mandati in uscita come un flusso aggregato che procede alla somma delle velocità dei singoli flussi in ingresso. I flussi vengono sincronizzati nel tempo attraverso dei piccoli intervalli (tempi di guardia). TDM Viene spesso usato nelle reti telefoniche e cellulari.



2.5.5 Multiplexing a divisione di codice

CDM (*code division multiplexing*): si tratta di una forma di **comunicazione a spettro distribuito** in cui **un segnale a banda stretta viene sparso su una banda di frequenza più ampia**. Ciò rende il segnale più tollerante alle interferenze e **permette a più segnali gli utenti diversi di condividere la stessa banda di frequenza**. Siccome CDM viene spesso usato per il secondo scopo, è comunemente chiamato **CDMA** (*code division multiple access*), visto che **permette ogni stazione di trasmettere su tutto lo spettro di frequenze in ogni momento**.

2.5.6 Multiplexing a divisione della lunghezza d'onda

I canali in fibra ottica usano una **tecnica particolare chiamata WDM**. Un sistema WDM è **totalmente passivo**, a differenza delle altre tecniche. È basato su reticoli di diffrazione.

2.6 La rete telefonica pubblica commutata

PSTN (*public switched telephone network*) è la rete telefonica pubblica commutata, progettata per **trasmettere la voce umana in una forma più o meno comprensibile**. Esiste in questa infrastruttura un **fattore limitante, definito come "l'ultimo miglio"**, cioè il tratto di **cavo/doppino** tramite cui il cliente si connette.

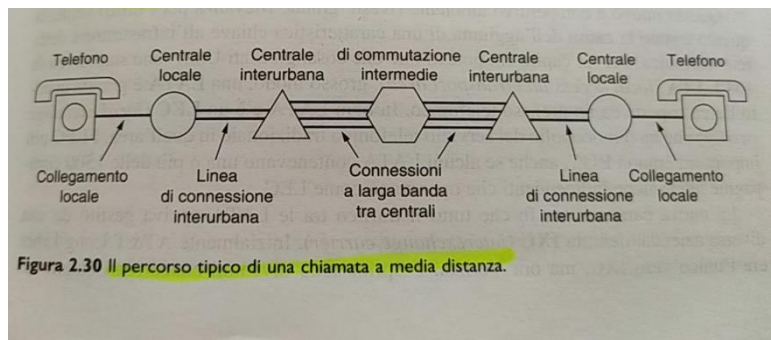
2.6.1 Struttura del sistema telefonico

Da ogni telefono partono **2 cavi di rame** che si collegano **direttamente alla centrale più vicina**, chiamata **centrale locale**. Le connessioni tramite doppino tra ogni telefono e le centrali sono chiamate **local loop**. Nella comunicazione telefonica ci possono essere vari scenari:

- Se i 2 utenti sono entrambi collegati ad una certa stazione centrale, verrà creata una connessione diretta tra i **2 collegamenti locali**.
- Se il telefono del chiamato è collegato ad un'altra centrale si attiva una procedura differente. Vengono impiegate le **linee di connessione interurbana** collegate a centri di commutazione chiamati centrali interurbane.

Le centrali interurbane **comunicano fra loro attraverso speciali linee a banda larga**.

Con l'avvento delle fibre ottiche, dell'elettronica digitale e dei computer, **tutte le linee e i commutatori sono diventati digitali** e **"l'ultimo miglio"** rappresenta l'ultimo tassello di tecnologia analogica del sistema.



Viene preferita la trasmissione digitale perché rende superfluo riprodurre in modo accurato una forma d'onda analogica dopo il passaggio attraverso molti amplificatori in una chiamata a lunga distanza; Oltre ad essere più affidabile è anche più economica e facile da gestire.

Analogico vs Digitale

All'interno dell'infrastruttura telefonica bisogna tenere in considerazione diversi fattori:

- Ricostruzione dei segnali
- Rapporto segnale-rumore
- Affidabilità del servizio
- Costo

La conversione viene effettuata dai modem e dai codec; i problemi che possono sorgere sono: attenuazione, distorsione (diverse frequenze che viaggiano a velocità diverse) e rumore.

2.6.3 Collegamenti locali: modem, ADSL e fibre

I modem telefonici analogici e ADSL soffrono delle limitazioni imposte da un ultimo miglio a volte datato: banda relativamente stretta, attenuazione, distorsione dei segnali e sensibilità al rumore. In alcuni casi l'ultimo miglio è stato modernizzato installando fibra ottica fino all'abitazione (o quasi).

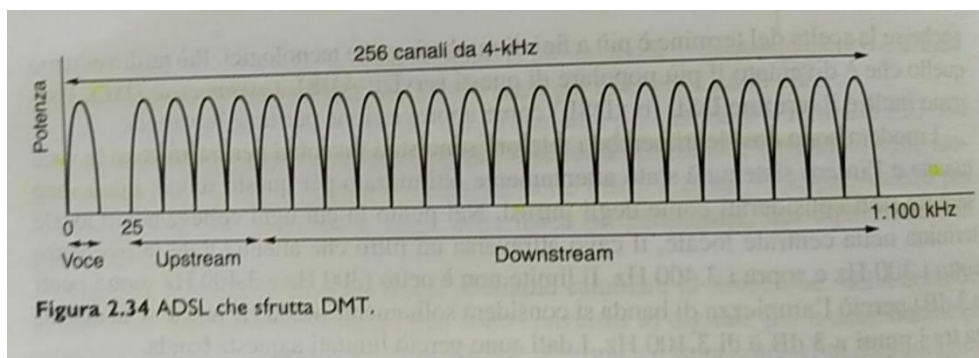
Modem Telefonici

Il modem è un dispositivo che converte un flusso di bit in un segnale analogico, in modo tale che può essere trasmesso sul canale. I modem telefonici sono usati per inviare bit tra 2 computer su una linea telefonica al posto della comunicazione vocale che normalmente ha luogo. La difficoltà principale è che una linea telefonica per la voce è limitata a 3100 Hz, ossia inferiore di almeno quattro ordini di grandezza di quella utilizzata per Ethernet o 802.11 (WiFi). Come abbiamo specificato nel paragrafo precedente un segnale sinusoidale viene modulato ed una delle sue caratteristiche (ampiezza, fase, frequenza) viene modificata secondo il segnale di informazione (segnale modulante) per produrre un segnale modulato. Nella pratica esistono molte varietà di modem: telefonici, DSL, cable, wireless. Da un punto di vista logico un modem si colloca tra il computer digitale e il sistema telefonico.

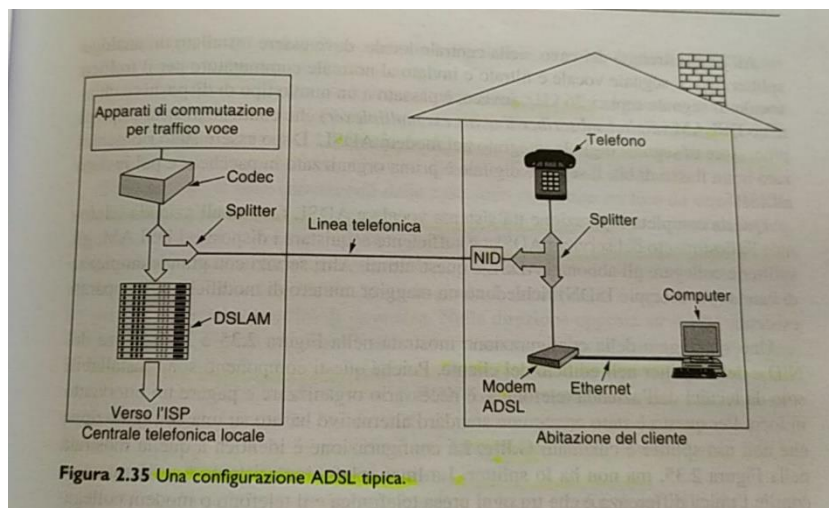
ADSL (*asymmetric digital subscribe line*)

I modem sono prestazionalmente lenti perché i telefoni sono stati inventati per trasmettere la voce umana e i dati sono sempre stati considerati come degli intrusi. Il meccanismo dietro ADSL è che, quando un cliente si abbona a questo servizio, la linea in ingresso viene collegata ad un diverso tipo di commutatore che, non usando un filtro per limitare la banda, rende disponibile l'intera capacità del

collegamento locale. Il **fornitore del servizio decide quanti canali utilizzare** per la trasmissione e quanti per la ricezione.



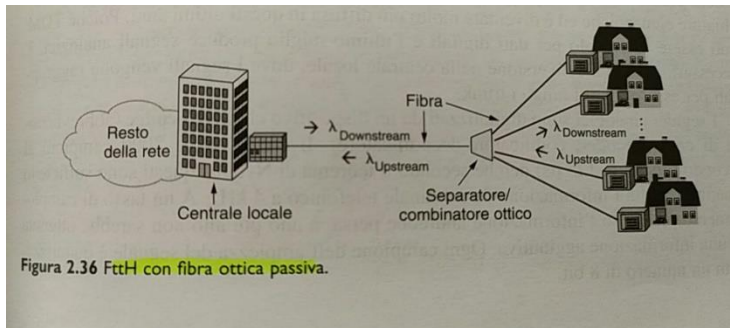
Una combinazione 50 e 50 è tecnicamente possibile, ma la maggior parte di questi assegna il canale utilizzato per scaricare i dati l'80-90% dell'ampiezza di banda, questo perché la maggior parte degli utenti preleva più di quanto trasmette. Questa soluzione fa nascere la "A" di asymmetric nella sigla.



La figura precedente mostra una tipica configurazione ADSL. In questo schema vediamo installato un **NID** (*network interface device*), ovvero una piccola scatola di plastica che segna la fine della proprietà dell'azienda telefonica e l'inizio di quella del cliente. Accanto a questo troviamo lo **splitter**, ossia un filtro analogico che divide i dati dalla banda 0-4000Hz utilizzata da POTS (vecchio servizio telefonico, che usa il canale 0). Il segnale POTS è inviato all'apparecchio telefonico o al fax, mentre i dati sono instradati verso un modem ADSL, che elabora i dati e implementa OFDM. All'altra estremità del cavo, nella centrale locale, deve essere installato un analogo splitter. Qui il segnale vocale è filtrato e inviato ad un normale commutatore per il traffico vocale, invece se supera i 26kHz è passato ad un dispositivo chiamato **DSLAM** (*digital subscribe line access multiplexer*), che contiene lo stesso tipo di processore di segnale digitale integrato nel modem ADSL. Il filtro per il telefono è un filtro passa basso, che elimina le frequenze sopra i 3400 Hz; il filtro per il modem a di ADSL è un passa alto virgola che elimina le frequenze sotto i 26 kHz.

FTTH – Fiber To The Home

L'ultimo miglio in rame attualmente in uso limita le prestazioni di ADSL e modem telefonici. Per poter offrire servizi migliori e più veloci vengono utilizzate le fibre ottiche: l'ultimo miglio di fibra è passivo, questo vuol dire che non è necessario disporre di apparati attivi (che consumano energia). La fibra semplicemente porta in segnali dall'abitazione alla centrale locale con conseguente riduzione dei costi e aumento dell'affidabilità.



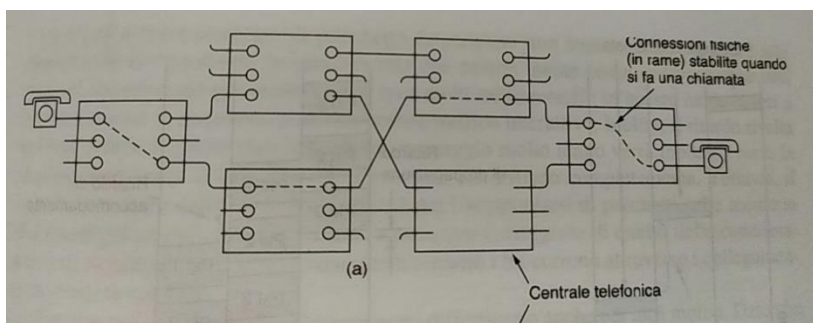
Per far condividere alle applicazioni la capacità della singola fibra che arriva alla centrale locale è necessario un protocollo. I messaggi provenienti dalle abitazioni non possono essere mandati contemporaneamente altrimenti potrebbero creare collisioni, inoltre non posso nemmeno ascoltare a vicenda le proprie trasmissioni. La soluzione offerta è che l'apparato di casa concordi con quello della centrale e locale gli intervalli di tempo da utilizzare per trasmettere.

2.6.4 Commutazione

Attualmente nelle reti si utilizzano 2 diverse tecniche di commutazione: la commutazione di circuito e quella di pacchetto. Il sistema telefonico tradizionale è basato su commutazione di circuito.

Commutazione di circuito

La tecnica della commutazione di circuito consiste nel creare un percorso fisico completo tra il telefono del chiamante e quello dell'utente chiamato. Quando una chiamata passa attraverso una centrale di commutazione, viene stabilita a livello concettuale una connessione fisica tra la linea di

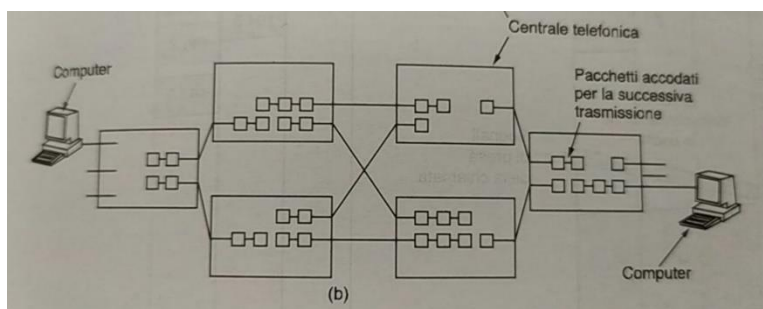


provenienza è una delle linee di uscita. Il percorso che viene creato dura fino al termine della chiamata. Una proprietà importante di questa tecnica è quella di stabilire un percorso da un punto ad un altro prima di iniziare a trasmettere i dati, il che potrebbe richiedere lunghi tempi di configurazione che

in alcuni contesti non sono accettabili. Analogamente l'unico ritardo per i dati è quello dovuto al tempo di propagazione e per lo stesso motivo non si corre il rischio di creare una congestione.

Commutazione di pacchetto

Con questa tecnologia i pacchetti vengono inviati non appena sono disponibili: diversamente che con



la commutazione di circuito non c'è alcuna necessità di creare un percorso dedicato in anticipo. È compito dei router usare la trasmissione di tipo store and forward per spedire ogni pacchetto verso la destinazione in maniera indipendente. Non essendoci un percorso prestabilito, i

pacchetti possono fare strade diverse a seconda delle condizioni della rete all'istante di invio e quindi potrebbe arrivare fuori ordine.

Le reti a commutazione di pacchetto impongono lo stretto limite superiore alla dimensione dei pacchetti, in maniera tale che nessun utente possa monopolizzare una linea di trasmissione. L'aspetto negativo è che in ritardo introdotto dallo store and forward per l'accumularsi di pacchetti nella memoria del router prima dell'inoltro al router successivo, è maggiore di quello della commutazione di circuito. Dato che non c'è prenotazione di banda, un pacchetto potrebbe dover aspettare prima di essere inoltrato; questo produce un ritardo di accodamento e congestione della rete, d'altra parte però non c'è più pericolo di avere un segnale di occupato. La commutazione di pacchetto non spreca banda ed è quindi più efficiente dal punto di vista del sistema: questo compromesso è cruciale per capire la differenza tra i 2 metodi perché dobbiamo scegliere un servizio garantito che spreca risorse e uno garantito che non ne spreca.

La commutazione di pacchetto è più resistente agli errori della comunicazione di circuito, poiché i pacchetti possono essere estradati aggirando i commutatori bloccati. L'ultima differenza riguarda l'algoritmo di addebito: con la commutazione di circuito, l'addebito dipende dal tempo e dalla distanza, con la commutazione di pacchetto, il tempo di connessione non è un problema (ma il volume del traffico a volte sì).

2.7 Il sistema telefonico mobile

Il sistema telefonico mobile è usato per comunicazioni a grande distanza sia vocali che dati. I telefoni mobili sono passati attraverso 3 diverse generazioni che sono:

- 1G – voce analogica
- 2G – voce digitale
- 3G – voce e dati digitali

1G

Sistemi di prima generazione, trasmettevano in modalità analogica ed erano in grado di gestire solo il traffico voce. La qualità della comunicazione offerta era limitata alla tipologia del segnale, come la scarsa qualità audio e le frequenti interruzioni.

2G

Passaggio alla comunicazione digitale ed è oggi lo standard di telefonia mobile con il maggior numero di utenti. L'uso del digitale ha sancito la nascita dei primi servizi di trasmissione dati, sotto forma di messaggi di testo (SMS), messaggi multimediali (MMS) e WAP, ossia lo standard che consentiva l'accesso ad appositi contenuti in internet su telefono.

3G

Tecnologia di terza generazione che ha l'obiettivo la realizzazione di un sistema mondiale di comunicazione mobile per il roaming. Tra questi abbiamo UMTS (universal mobile telecommunications system), tutt'ora il più impiegato. L'introduzione del protocollo W-CDMA, una particolare tecnologia ad accesso multiplo del canale, ha consentito ad UMTS di offrire un'ulteriore velocizzazione del trasferimento dati. Le prestazioni di questo sistema dipendono dai protocolli di trasmissione utilizzati.

3.0 Livello Data Link

Le proprietà fondamentali che un canale deve avere per essere simile ad un cavo è che i **bit vengano consegnati nell'esatto ordine in cui sono stati inviati**. Esaminare la natura degli errori, le loro cause e come possono essere rilevati e corretti sono **altre funzionalità che il data link deve prevedere**.

3.1 Progettazione del livello Data Link

Il data link fa uso del **servizio messo a disposizione dal livello fisico** per inviare e ricevere bit sui canali di comunicazione, fornendo differenti funzionalità: fornisce un'**interfaccia di servizio**, **gestisce errori di trasmissione e regola il flusso di dati**. Per raggiungere tali obiettivi prende i **pacchetti** e gli incapsula in **frame**, ogni frame ha: **header** (intestazione), **payload** (contenente il pacchetto) e **frame trailer** (sequenza di chiusura).

3.1.1 Servizi forniti al livello di rete

Lo scopo del data link è **fornire servizi al livello di rete**: trasferendo dati dal **livello di rete della macchina sorgente** a quella di destinazione. La sua progettazione dipende dal tipo di servizio che offre, che cambia da protocollo a protocollo:

- **Servizio senza conferma e senza connessione**: la macchina sorgente invia **frame indipendenti**, senza che la macchina ricevente dia una conferma dell'avvenuta ricezione. es: Ethernet, infatti questo servizio viene usato quando la **frequenza degli errori di trasmissione è molto bassa**. Se perdo un frame non viene rilevato.
- **Servizio con conferma senza connessione**: ciascun frame è **inviato individualmente** e ne viene data la conferma di ricezione, con la possibilità di rispedire il frame. Utile per i **canali di trasmissione poco affidabili** (es: WiFi). Questo servizio è considerato come un'ottimizzazione e su canali affidabili, come le fibre ottiche, è inutile.
- **Servizio con conferma orientato alla connessione**: le due macchine stabiliscono una **connessione prima del trasferimento dati**. Ogni frame è enumerato e il livello data link garantisce che **venga effettivamente ricevuto una sola volta e nell'ordine corretto**. Utile per **collegamenti lunghi e inaffidabili** (canali satellitari, linee telefoniche...). Terminato il trasferimento la **connessione viene rilasciata**.

Incapsulamento: aggiungere informazioni sui protocolli di un livello o servizio, al livello/servizio superiore.

3.1.2 Suddivisione in frame

Per erogare il servizio al livello di rete, il data link deve usare a sua volta il servizio che gli è fornito dal **livello fisico**; in tutto ciò non esiste la garanzia che il flusso di bit ricevuto dal livello data link sia privo di errori, per cui dovrà rilevarli e correggerli. Per questo il **flusso è suddiviso in una serie discreta di frame** e poi viene calcolato il **checksum**, ovvero il **valore a lunghezza fissa che verrà incluso nel frame**. Quando il frame arriva viene **ricalcolato il checksum**, se è differente c'è un errore.

Framing: suddividere il flusso di bit in frame.

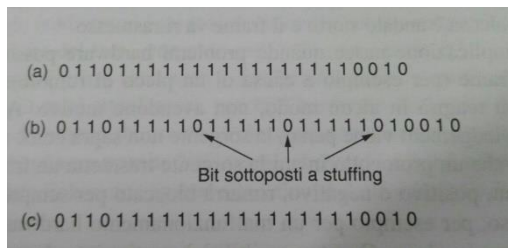
Esistono 4 tipologie di framing:

1. **Conteggio dei byte**: usa un campo nell'intestazione per **specificare il numero di byte nel frame**. Quando viene letto il numero dalla macchina di destinazione **sa quanti byte seguiranno e quindi saprà la fine del frame**. Questo conteggio può essere **alterato da un errore di**

trasmissione che andrà ad **alterare il bit di riferimento** rendendo impossibile trovare la posizione di partenza del frame successivo. Questo metodo non viene più utilizzato.

2. **Flag byte con byte stuffing**: vengono inseriti dei byte speciali all'inizio e alla fine del frame, solitamente è lo stesso byte (**flag byte**). Due flag byte consecutivi indicano la fine del frame e l'inizio del successivo; può accadere che il valore corrispondente al flag byte compaia dentro i dati. Per questo viene inserito un byte di escape (**ESC**) subito prima di ogni occorrenza accidentale del flag nei dati, distinguendo in maniera chiara i flag byte dei dati da quelli di controllo. I byte di escape verranno rimossi prima di passare al livello rete; questa tecnica è chiamata **byte stuffing**.

3. **Flag bit con bit stuffing**: operazione analoga alla precedente effettuata su bit, dove ogni frame inizia e finisce con una sequenza speciale di bit 01111110. Chi trasmette ogni volta che incontra 5 bit a 1 consecutivi inserisce uno 0 nel flusso di uscita (**bit stuffing**), chi riceve se



trova 5 bit a 1 seguiti da 0, elimina lo zero. Un effetto è che la **lunghezza del frame dipende dal contenuto dei dati che porta**.

4. **Violazioni della codifica a livello fisico**: viene usata una scorciatoia a livello fisico che consiste nell'usare alcuni segnali riservati per indicare l'inizio e la fine dei frame, effettuando una **coding violation**. Se si usano questi segnali è facile riconoscere l'inizio e la fine dei frame e non è necessario aggiungere dati.

3.1.2 Controllo degli errori

Il data link deve assicurarsi che al livello di rete della destinazione **arrivino tutti i frame**, e che arrivino **nell'ordine corretto**: tipicamente vengono inviati dei frame di controllo che contengono un **acknowledgement**, positivo o negativo. Questo sistema soffre della **perdita del frame di controllo**: se viene perso, la sorgente rimane **bloccata in attesa dell'ack**. Si introduce un timer entro il quale la destinazione deve rispondere alla sorgente; se ciò non avviene quest'ultima deve rinviare il frame originale rischiando però che la destinazione **lo accetti più volte creando ridondanze**. Per questo si assegna un numero di sequenza ai frame in uscita per **permettere alla destinazione di distinguere le ritrasmissioni dagli originali**. Il problema della gestione di timer e numeri di sequenza rappresenta una parte importante dei compiti del data link.

3.1.4 Controllo di flusso

Un altro aspetto da tenere in considerazione riguarda che cosa fare quando la **sorgente vuole trasmettere i frame più velocemente di quanto la destinazione sia in grado di accettarli**. Vengono utilizzate due politiche:

- **Controllo di flusso tramite feedback**: la destinazione rimanda alla sorgente delle informazioni per **darle il permesso di mandare altri dati** o per informarla del suo stato.

Controllo di flusso tramite limitazione del tasso di invio: il protocollo contiene al proprio interno un meccanismo che **limita il tasso al quale la sorgente può trasmettere i dati** (senza feedback).

Es di feedback: "Puoi mandarmi fino a n frame e poi non mandarmene altri finché non te lo dico io".

3.2 Rilevazione e correzione degli errori

Gli errori di trasmissione esistono e devono essere fronteggiati. Vengono usate due differenti strategie:

- **Codici a correzione d'errore:** includere informazioni ridondanti in numero sufficiente da permettere al destinatario di dedurre i dati realmente trasmessi. Impiegata nei canali con un'elevata probabilità di errore (Wireless).
- **Codice a rilevazione d'errore:** introdurre una ridondanza di informazioni sufficiente a permettere al destinatario di dedurre che c'è stato un errore. Impiegati nei canali affidabili (fibra ottica), dove è vantaggioso usare questi codici e ritrasmettere il blocco di dati dove c'era l'errore.

Esistono anche particolari tipi di errore: a volte è noto dove avviene un errore, magari perché il livello fisico ha ricevuto un segnale analogico molto diverso dal valore aspettato di 0 o 1 e in quel caso il bit è dichiarato perso. Questo modello prende il nome di **canale di cancellazione**.

3.2.1 Codici a correzione d'errore: codice di Hamming

Aggiunge una ridondanza alle informazioni trasmesse: un frame consiste di m bit di dati e r bit di ridondanza. In un **codice a blocco** gli r bit sono calcolati unicamente in funzione degli m bit di dati, come se questi ultimi fossero usati come indici di una tabella per trovare quelli di controllo. In un **codice sistematico** gli m bit sono trasmessi insieme a quelli di controllo piuttosto che essere prima codificati. In un **codice lineare** gli r bit sono una funzione lineare degli m bit di informazione quali l'OR esclusivo (XOR).

$n = m + r$ dove n è la lunghezza totale del blocco.

Parola del codice: unità di n bit, con bit di dati e di controllo.

Code rate: rapporto di bit non ridondanti m/n

Date due sequenze di bit è possibile calcolare la **distanza di Hamming**, ovvero il numero di bit differenti: si esegue XOR delle due parole del codice e si conta il numero di bit pari a 1 nel risultato. Questa distanza è molto importante perché ci dice che se le parole sono ad una distanza d una dall'altra, saranno necessari d errori sui singoli bit per convertire una sequenza nell'altra.

La rilevazione e correzione degli errori di una codifica dipendono dalla distanza di hamming: per trovare in modo affidabile d errori è necessaria una codifica con distanza $d+1$, in quanto garantisce che d errori sui singoli bit non possano cambiare una parola di codice valida con un'altra anch'essa valida. Analogamente per correggere d errori è necessaria una codifica con distanza $2d+1$.

3.2.2 Codici a rilevazione di errore

Le codifiche a correzione di errore sono usate per le trasmissioni wireless, poiché senza di questi sarebbe molto difficile trasmettere su queste reti. Invece i codici a rilevazione sono usati nelle trasmissioni con un basso tasso d'errore. Esistono 3 diversi codici a rilevazione d'errore:

- **Parità:** un singolo bit di parità viene aggiunto in coda ai dati. Questo bit viene scelto in modo che il numero di 1 nella parola di codice sia dispari(o pari). Questa tipologia è in grado di rilevare solo errori su singoli bit. Per migliorare la protezione offerta contro gli errori a burst si usa l'**interleaving**, una tecnica che calcola il bit di parità per ognuna delle n colonne e si

trasmettono tutti i bit di dati come k righe inviate dall'alto al basso, a all'interno di ogni riga da sx a dx. All'ultima riga si inviano i bit di parità.

- **Checksum:** è strettamente collegato al bit di parità; un gruppo di bit di parità è un esempio di checksum. Il checksum è solitamente posizionato alla fine del messaggio come complemento ad uno dei risultati della somma dei dati, in modo tale che gli errori possano essere rilevati sommando l'intera parola contenete sia i dati sia il checksum. Però visto che è solo una somma non fornisce una protezione forte.
- **CRC** (*cyclic redundancy check*) *codifica polinomiale*: si basa sul fatto di trattare la sequenza di bit come polinomi a coefficienti che possono assumere solo i valori 0 e 1. Quando si usa una codifica polinomiale, sorgente e destinazione devono mettersi d'accordo su un **polinomio generatore** $G(x)$. Per poter calcolare il checksum di un frame di m bit corrispondente al polinomio $M(x)$, il frame deve essere più lungo del polinomio generatore. L'algoritmo è il seguente:

Sia r il grado di $G(x)$, si aggiungono r bit con valore 0 dopo la parte di ordine più basso del frame, così che al suo interno ci sono $m + r$ bit e corrisponda al polinomio $M(x)$. Dividere la sequenza corrispondente a $M(x)$ per la sequenza corrispondente a $G(x)$ usando la **divisione modulo 2**. Sottrarre il resto della sequenza corrispondente a $M(x)$ usando la sottrazione in modulo 2. Il risultato è il frame con checksum pronto per la trasmissione. Chiamiamo il polinomio $T(x)$. Per avere un buon polinomio generatore bisogna avere che il termine di grado più alto e quello di grado più basso sono 1. Il polinomio $M(x)$ deve essere più lungo di $G(x)$.

3.3 Protocolli data-link elementari

I protocolli data-link elementari sono:

- Simplex utopistico
- Stop-and-wait per canale privo di errori
- Stop-and-wait per un canale soggetto ad un rumore

3.3.1 Protocollo simplex utopistico

Non esiste nella realtà, solo a scopo informativo. Non si preoccupa che qualcosa vada storto, tutte situazioni utopistiche ideali. *Simplex*, vuol dire che la comunicazione avviene in una sola direzione; i livelli di rete, sia della sorgente che della destinazione sono sempre pronti a ricevere/inviare pacchetti e non bisogna tener conto dei tempi di elaborazione dei pacchetti, inoltre i buffer sono infinito (no intasamento). Il canale di comunicazione (livello fisico) è privo di errori.

Il funzionamento è molto semplice: il mittente invia in continuazione frame al destinatario, senza subire ritardi o errori, ed il destinatario riceve tutto quello spedito dal mittente. Non c'è il controllo di flusso né controllo degli errori, per questo è **utopistico**.

3.3.2 Protocolli simplex stop-and-wait per un canale privo di errori

Prevenire che la sorgente intasi il destinatario: quando il buffer del destinatario è saturo il mittente si ferma ed aspetta un feedback per ricominciare. Questo frame si chiama **dummy**, in pratica è un frame di feedback che serve a "risvegliare" il mittente. Quindi il mittente invia un frame al destinatario e attende il feedback (**stop-and-wait**): in questo tipo di comunicazione di solito i frame viaggiano in

maniera bidirezionale, quindi si può usare un canale half-duplex che permette l'alternanza mittente-destinatario. Qui c'è solo il controllo di flusso, no controllo di errori, perché questo protocollo non prevede errori nel canale di comunicazione. La destinazione invia solo frame di dummy, per cui il mittente non controlla il frame perché sarebbe inutile.

3.3.3 Protocollo simplex stop-and-wait per un canale soggetto a rumore

Questo protocollo prevede una situazione reale, introducendo sia il controllo di flusso che quello di errori. I frame possono essere danneggiati durante la trasmissione e la destinazione può accorgersi della presenza o meno di errori controllando il checksum. Ci può essere una situazione in cui il frame è danneggiato ma il checksum risulta corretto, non si può rilevare l'errore; per questo si introduce un timer, avviandolo nel momento in cui viene spedito il frame, se l'ack non viene ricevuto prima dello scadere del tempo, allora la sorgente ritrasmette il frame. Questo procedimento soffre di limitazioni perché l'ack potrebbe perdersi, oppure ci sono dei ritardi nel canale e quindi ci troviamo con due frame uguali (ridondanza dei dati): si introduce nell'intestazione dei frame un numero di sequenza che permette di distinguere i frame originali dai duplicati.

Quanti bit ci servono per rappresentare il numero in sequenza?

Bastano 2 bit, 0-1, che si alternano l'un l'altro. Quando arriva un frame con un numero in sequenza corretto viene accettato dalla destinazione e quest'ultima invia l'ack, poi il numero di sequenza che è atteso dalla sorgente viene incrementato in aritmetica modulo 2 (da 0 a 1 e viceversa). La sorgente memorizza il numero di sequenza dal frame successivo da inviare, la destinazione memorizza il numero di sequenza che si aspetta di ricevere. Il timer non deve essere né troppo lungo né troppo corto: se è troppo corto la sorgente invia frame duplicati in continuazioni diminuendo le prestazioni, invece se è troppo lungo gli ack impiegheranno troppo tempo per arrivare alla sorgente che rimarrà in attesa (bloccata). I frame danneggiati o duplicati non vengono passati a livello di rete ma fanno in modo che venga generato l'ack dell'ultimo frame corretto per segnalare alla sorgente di passare al frame successivo o ritrasmettere quello danneggiato.

ARQ (*automatic repeat request*): protocolli in cui la sorgente aspetta un ack positivo dalla destinazione.

3.4 Protocolli a finestra scorrevole

Nei protocolli elementari la comunicazione avviene seguendo un'unica direzione, mittente-destinatario, però nella realtà si vuole trasmettere in maniera bidirezionale, ovvero in contemporanea, quindi avremmo bisogno di una comunicazione full-duplex. Si può ottenere in due modi:

1. 2 canali simplex, uno che trasmette da mittente a destinatario per i dati, uno che trasmette da destinatario a mittente per gli ack. Problema: la banda del canale di ritorno è sprecata, perché gli ack sono piccoli visto che non trasportano dati, per cui c'è uno spreco di banda.
2. 1 canale per entrambe le direzioni di comunicazione e basterà guardare l'intestazione del frame per capire se si tratta di un dato o un ack. Un ulteriore miglioramento a questo modello può essere effettuato andando a inserire un ack in coda ad un dato: il mittente invia un frame, il destinatario risponde con un ack che viene accodato ad un nuovo frame (ovvero al frame successivo). Questa tecnica è definita piggybacking, servita per migliorare l'uso della banda disponibile. All'interno del frame metti il campo ack settato a 1 (pochi bit per ack), così da evitare di costruire dei frame solo per gli ack.

Si useranno i protocolli a finestra scorrevole perché sono **bidirezionali**. La sorgente mantiene la finestra di invio, ovvero traccia l'insieme dei numeri di sequenza corrispondenti ai frame che può inviare, invece la finestra di ricezione a lato destinatario serve per tracciare il frame che la destinazione può accettare. Dato che i frame della finestra di invio potrebbero **perdersi**, la sorgente deve **tenere memoria dei frame** per un eventuale trasmissione futura. Quando un nuovo pacchetto arriva dal livello di rete gli viene assegnato il **numero di sequenza** successivo in ordine crescente e il **limite superiore** della finestra della sorgente è **incrementato di 1**, quando invece arriva un ack si incrementa di 1 il limite inferiore; invece nella finestra di ricezione quando viene ricevuto un frame con un numero di sequenza uguale al limite inferiore, il frame **viene passato a livello di rete e la finestra ruotata di una posizione**.

3.4.1 Protocollo a finestra scorrevole a 1 bit

Questo protocollo utilizza delle **finestre scorrevoli che hanno dimensione di 1 bit**, quindi la dimensione max della finestra del mittente e destinatario hanno dimensione 1. Il mittente può inviare un unico frame alla volta e il destinatario dopo che ha ricevuto il frame può inviare l'ack. Si riduce ad uno **stop-and-wait**. Solo dopo l'ack il mittente può inviare il **successivo frame**. In questo caso il campo di ack all'interno del frame contiene il numero dell'ultimo frame ricevuto correttamente; se il numero concorda con il numero di sequenza che la sorgente vuole trasmettere (il ricevuto è uguale a quello trasmesso) significa che il **buffer del mittente può prendere il successivo frame dal livello di rete**.

3.4.2 Protocollo a finestra scorrevole go-back-n

Si tiene in considerazione il tempo con cui il frame arriva a destinazione. Le finestre hanno **dimensione >1 (n byte)**: la sorgente può mandare fino ad **n frame** contemporaneamente prima di bloccarsi per ricevere gli ack dalla destinazione. Bisogna stare **attenti alla dimensione della finestra**: se n è abbastanza grande significa che la sorgente non si bloccherà mai, gli ack arrivano **prima che la finestra si riempie del tutto**. Può capitare che i frame **subiscano errori** e allora bisogna gestire quelli persi (a livello di rete **l'ordine dei frame è fondamentale**): in questo protocollo abbiamo che la destinazione **scarta tutti i frame successivi all'errore** e non viene mandato l'ack, quindi la sorgente va in **time-out** e non ricevendo l'ack dei frame scartati gli ritrasmette in ordine. Il problema è **se la frequenza degli errori è alta allora si ha un grande spreco di banda** perché ogni volta bisogna ritrasmettere i frame successivi a quello afflitto da errore.

3.4.3 Protocollo a finestra scorrevole con ripetizione selettiva

Questo protocollo colma le lacune del precedente: quando si ha un frame con errore questo viene scartato ma quelli dopo vengono memorizzati e immagazzinati in un buffer. Nel momento in cui la sorgente va in time-out perché non ha ricevuto l'ack nell' n -esimo frame afflitto da errore, allora **solo il frame danneggiato viene ritrasmesso**. Successivamente vengono trasmessi a livello di rete anche tutti i frame memorizzati nel buffer in sequenza. Se la finestra è molto grande anche il buffer **deve essere grande**: le performance dipendono dalla **grandezza della finestra**. In questo protocollo per aumentare le prestazioni si usano i **nack** (senza aspettare che scade il time-out della sorgente), nel momento in cui viene riscontrato l'errore: ovvero si **controlla il checksum del frame ricevuto** e si fa partire subito il **nack**. La destinazione avvisa subito la sorgente che c'è un errore, risparmiando banda.

3.5 Protocolli data-link

Vedremo: HDLC, PPP. Utilizzati per comunicazioni punto-punto.

3.5.1 HDLC

HDLC (*high level data link control*) prevede l'utilizzo di 3 diverse tipologie di stazioni:

1. **Stazione primaria:** si occupa di controllare tutte le operazioni che vengono effettuate sui collegamenti e invia anche dei comandi alle stazioni secondarie.
2. **Stazione secondaria:** sono sotto controllo della stazione primaria e aspettano un comando dalla essa e lo eseguono (invia risposte alla primaria).
3. **Stazioni combinate:** i ruoli di primaria e secondaria non sono definiti. Una stazione secondaria può diventare primaria e viceversa, ovvero può sia inviare comandi che inviare risposte.

La comunicazione tra queste stazioni può essere di due tipi:

1. **Sbilanciata:** prevede un modello master-slave, ovvero una stazione primaria e tutte le stazioni secondarie che dipendono dalla principale.
2. **Bilanciata:** consente di avere stazioni combinate che comunicano fra loro, ognuna comunica con le altre senza appellarsi alla stazione primaria.

Entrambi i modelli supportano la comunicazione *half-duplex* che *full-duplex*. La modalità di trasferimento dei frame può avvenire in 3 diversi modi:

1. **Normal-response-mode:** prevede configurazione sbilanciata, solo la stazione primaria può iniziare la comunicazione e le stazioni secondarie devono solo stare in attesa di comandi (attesa attiva).
2. **Asynchronous-balance-mode:** si ha una configurazione bilanciata ed è quella più usata. Non c'è overhead perché non c'è polling: le stazioni possono iniziare a trasmettere quando ne hanno la necessità, senza chiedere permessi.
3. **Asynchronous-response-mode:** poco usata, prevede una configurazione sbilanciata però le secondarie possono iniziare la comunicazione senza chiedere il permesso alla stazione primaria.

Struttura dei frame in HDLC

Viene usato il *bit-stuffing* per il framing e si ha il campo *address* che viene usato per link *punto-multipunto*, quindi questo campo va ad identificare la stazione secondaria che invia o che riceve i frame. Si ha il campo *control*, in cui viene inserito l'ack e il numero di sequenza. 1 bit di *pool*, settato a uno quando la stazione primaria invia il comando e si aspetta una risposta, 1 bit di *final*, settato a uno quando la stazione secondaria risponde al comando della primaria. Possiamo avere tre diversi tipi di frame:

1. **Frame di informazione:** contengono il numero in sequenza dei frame trasmessi
2. **Frame supervisori:** usati per il controllo di flusso degli errori, non hanno dati all'interno. Questo frame porta l'ack.

3. **Frame non numerati**: usati per **connessione/disconnessione**. Più in generale dei messaggi di servizio (manutenzione, stato della connessione...)

Poi nel **frame** abbiamo altri due campi: **data**, che **contiene i dati** e questo campo lo troviamo solo nei frame di informazione e quelli non numerati, **checksum**, che utilizza un **codice CRC a 16 bit**. Il protocollo **HDLC** si svolge in 3 fasi:

1. **Inizializzazione**
2. **Trasferimento dei frame**
3. **Disconnessione**

3.5.2 PPP – **Point to point protocol**

Utilizzato su internet per trasferire dei datagrammi multi protocollo su dei collegamenti punto-punto (instaurare connessioni punto-punto su internet). Bisogna tenere in considerazione **3 caratteristiche**:

1. A differenza di HDLC per i frame, anziché il bit-stuffing usa il byte-stuffing, e quindi usando questo metodo di framing si può andare a **vedere dove termina un frame e ne inizia un altro** (anche con il bit-stuffing si può fare).
2. Utilizza un **protocollo interno, LCP** (*link control protocol*), che **serve per gestire la connessione di rete e anche la disconnessione**. Permette anche di fare dei test su una linea e negoziare le opzioni di collegamento tra sorgente e destinatario. I pacchetti LCP sono **incapsulati nel payload** del frame PPP, i campi di LCP sono: **codice**, definire il tipo di pacchetto LCP (3 tipi), **ID** che contiene un valore per fare il match fra richiesta e risposta (endpoint), **lunghezza e informazioni**. I pacchetti LCP possono essere di 3 tipologie:
 - **Configurazione**: definire il **set-up iniziale della connessione**. Possono essere di 4 tipologie: *CONFIGURE_REQUEST*, *CONFIGURE-ACK*, *CONFIGURE-NACK*, *CONFIGURE-REJECT*.
 - **Terminazione dei link**: servono per disconnettere la connessione quando finisce la comunicazione. Sono di due tipi: *TERMINATE-REQUEST* e *TERMINATE-ACK*.
 - **Monitoraggio e debugging**: servono per monitorare il collegamento, si controllano gli errori nel canale. Sono di 4 tipi: *CODE-REJECT*, *PROTOCOL-REJECT*, *EECHO-REPLY*, *DISCARD-REQUEST*.
3. Utilizza il protocollo **NCP** (*network control protocol*), che viene usato per **negoziare opzioni relative al livello di rete**.

Struttura del frame PPP

- **Flag**: byte-stuffing
- **Address**: è sempre impostato a 11111111, indica che tutte le stazioni devono accettare il frame (broadcast). Evita l'assegnamento degli indirizzi, **senza che inserisci l'indirizzo specifico**.
- **Control**: ha un valore di default 00000011, che indica **un frame senza numero**.
- **Protocol**: indica il pacchetto nel campo payload (LCP/NCP). **Protocollo utilizzato nel pacchetto incapsulato nel PPP**.

- **Info:** pacchetto che deriva dal livello di rete. Può avere una lunghezza variabile (max 1500byte).
- **Checksum:** 2 o 4 byte, usa il CRC a 32bit.

Il protocollo HDLC fornisce delle trasmissioni affidabili usando la finestra scorrevole (ack e time-out) invece il PPP fornisce trasmissioni affidabili anche per canali soggetti a rumore. Questo è possibile perché PPP utilizza dei protocolli di autenticazione che permettono di identificare l'utente autorizzato ad accedere a determinate risorse. I due protocolli di autenticazione sono:

- **PAP (password authentication protocol):** l'autenticazione avviene in due passi; l'utente invia in chiaro l'ID e la password al sistema, e quest'ultimo controlla la validità dei dati inseriti. Se sono corretti da accesso all'utente alle risorse, altrimenti interrompe la connessione. Il problema è che invia le credenziali in chiaro.
- **CHAP (challenge handshake authentication protocol):** si tratta di un protocollo con handshake a tre vie, e consiste in 3 passi. Il sistema va a generare un challenge (pochi byte) e lo invia all'utente, quest'ultimo esegue una funzione che prende come parametri il valore di challenge e la password dell'utente e genera un risultato. Questo risultato viene inviato al sistema che applicherà la stessa funzione dell'utente (il sistema conosce la password dell'utente) al challenge e la password, se i due risultati combaciano allora ci può essere una connessione, altrimenti no. Il valore di challenge è semirandomico.

4.0 Sottolivello MAC

Ora si parla dei protocolli utilizzati nelle reti di tipo broadcast. In queste reti più persone tentano di accedere simultaneamente allo stesso canale condiviso, quindi il punto focale è capire chi ha il diritto di trasmettere al fine di prevenire le collisioni. Per questo vengono implementati dei protocolli ad accesso multiplo per allocare l'uso del canale. Il sottolivello MAC è molto importante per le wireless LAN, perché si servono spesso di questo tipo di comunicazione, a differenza delle WAN che usano connessioni punto-punto. Il sottolivello MAC è la parte inferiore del data-link (media access control) e serve per accedere e comunicare con il livello fisico; invece si ha anche un altro sottolivello del data-link chiamato LLC (logical link control) che serve a comunicare con il livello di rete.

4.1 Allocazione del canale

In una comunicazione broadcast si hanno più trasmettitori e più ricevitori in ascolto su un canale fisico, e bisogna fare in modo che tutti loro siano sincronizzati per evitare collisioni: in uno stesso istante tutti i nodi potrebbero mandare frame contemporaneamente generando collisioni. Questi frame vengono danneggiati e bisognerà ritrasmetterli, portando ad un notevole spreco di banda. Le caratteristiche ideali di un protocollo ad accesso multiplo dovrebbero essere:

- Se c'è un solo nodo che deve inviare i frame, lo deve fare al massimo rate (occupare tutta la banda possibile).
- Se si hanno n nodi che devono trasmettere lo devono fare n/r bit per sec. Dove r è il rate.
- Il protocollo deve essere decentralizzato (se c'è un failure non crasha tutto).
- Deve essere semplice da implementare.

Andremo a vedere tre diverse categorie di protocolli ad accesso multiplo:

1. **Partizionamento del canale**
2. **Accesso casuale**
3. **Accesso controllato**

4.1.1 Allocazione statica del canale

Sono i protocolli a partizionamento del canale, infatti se si hanno n utenti la larghezza di banda viene divisa equamente fra gli utenti così ognuno ha la propria porzione di banda, evitando interferenze. Questo tipo di partizionamento viene detto **FDMA** (*frequency division multiple access*). I problemi sorgono quando:

- Gli utenti trasmettono una grande quantità di informazioni.
- Quando il traffico è irregolare.

Lo spettro è diviso in n regioni, naturalmente se ci sono meno di n utenti che trasmettono si ha uno spreco di banda, invece se più di n utenti vogliono trasmettere non possono farlo e dovranno aspettare. Molto spesso parte del canale non viene usata e non si riesce a trasmettere. Scarse prestazioni. Un'altra tipologia di questi protocolli è il **TDMA** (*time division multiple access*) e si utilizza l'intera larghezza di banda da ogni utente e la banda è divisa in slot temporali dove ognuno di loro deve aspettare il suo turno per trasmettere e non potrà trasmettere più del tempo previsto dallo slot. Nel momento in cui l'utente non ha niente da trasmettere si ha un grande spreco di banda, e quindi si hanno scarse prestazioni.

Esiste un altro metodo detto **CDMA** (*code division multiple access*), in questo caso il canale porta tutte le trasmissioni simultaneamente e a differenza degli altri 2 questo si basa su dei codici convoluzionali: ad ogni stazione viene assegnato un codice e questa convolve i propri dati con il codice associato ad essa. Bisogna tenere in considerazione alcune operazioni di teoria della codifica: questi codici sono anche chiamati **Chip** (sequenza di n numeri, dove n è il numero delle stazioni), se questo chip viene moltiplicato per uno scalare ogni elemento della sequenza viene moltiplicato per quel numero, altrimenti se ho 2 sequenza uguali e le moltiplico sommando i risultati ottenuti si ottiene n , invece se moltiplico due sequenze diverse e sommo i risultati ottengo zero. Questo chip è una specie di firma per riconoscere il dato inviato dalla stazione.

4.1.2 Allocazione dinamica del canale

Bisogna tenere in considerazione 5 punti:

1. **Traffico indipendente:** n stazioni indipendenti e ognuna ha un utente che genera frame per la trasmissione.
2. **Canale singolo:** 1 solo canale per tutte le comunicazioni (trasmettere/ricevere).
3. **Collisioni osservabili:** se vengono inviati 2 frame contemporaneamente si sovrappongono e il segnale risultante sarà incomprensibile.
4. **Tempo continuo o diviso:** è continuo quando la trasmissione può incominciare in qualsiasi momento, diviso la stazione devono attendere l'inizio del proprio slot per trasmettere.

5. **Rilevamento di portante e non rilevamento di portante:** se si rileva la portante le stazioni possono capire se il canale è in uso prima di occuparlo, invece se non si rileva la portante le stazioni non possono mettersi in ascolto nel canale e trasmettono lo stesso, procedendo con la trasmissione senza ascoltarlo (solo alla fine sanno se la trasmissione ha avuto successo).

4.2 Protocolli ad accesso multiplo

4.2.1 ALOHA

ALOHA puro

I frame inviati hanno tutti la stessa lunghezza. L'idea alla base è che quando una stazione deve trasmettere un frame lo fa senza curarsi degli altri utenti: ogni volta che si hanno dati da inviare si inviano. È un problema! Si creano tante collisioni. Nel momento in cui ho una collisione se il frame va perso va ritrasmesso e il trasmettitore attende un tempo randomico, altrimenti continueranno a crearsi collisioni. Si spreca tantissima banda, si usa il canale al 18%. Però le ritrasmissioni non sono infinite, si ha un max di 15 ritrasmissioni e se si arriva al limite la stazione smette di ritrasmettere il frame. Quando si trasmette un frame si deve sempre avere un ack di conferma, e il tempo d'attesa della sorgente è 2 volte il tempo di propagazione (dove il tempo di propagazione è il tempo necessario affinché il frame arrivi dalla sorgente fino alla stazione più lontana in assoluto. Corrisponde al diametro della rete, aspetta il tempo necessari nel caso pessimo: il frame è trasmesso alla stazione più lontana.

Slotted ALOHA

Anche in questo caso i pacchetti sono tutti della stessa lunghezza L , e il tempo è diviso in intervalli discreti (L/r secondi). Le trasmissioni possono iniziare solo all'inizio dello slot temporale: nel momento in cui una stazione vuole inviare un frame deve attendere l'inizio del nuovo slot temporale. Se non abbiamo collisioni si può procedere alla prossima trasmissione, ma se c'è stata una collisione il pacchetto verrà ritrasmesso con probabilità p : dati n nodi la probabilità che un nodo ritrasmetta è p , la probabilità che gli altri nodi siano in silenzio è $(1-p)^{(n-1)}$, quindi si ha che un nodo generico possa trasmettere è $np \cdot (1-p)^{(n-1)}$. I nodi devono essere sincronizzati fra loro per accordarsi sui limiti degli intervalli (sapere quando inizia uno o l'altro). L'utilizzo del canale è del 37%. I vantaggi sono: se abbiamo solo 1 nodo che deve trasmettere frame lo può fare al massimo rate e un altro vantaggio è che è semplice da implementare, dove non abbiamo bisogno di una stazione primaria che controlla il canale. Gli svantaggi arrivano quando ci sono più stazioni che vogliono trasmettere, creando tante ritrasmissioni.

4.2.2 Protocolli ad accesso multiplo con rilevamento della portante

Sono quei protocolli che si mettono in ascolto del canale e vengono detti protocolli "carrier sense". CSMA (*carrier sense multiple access*). Esistono 3 tipi di questi protocolli che si distinguono per il metodo di attesa:

1. **CSMA Non persistente:** la stazione quando deve trasmettere si mette in ascolto del canale e se il canale è in idle può trasmettere, altrimenti riprova la ritrasmissione dopo aver aspettato un tempo di back-off. Le stazioni hanno una politica democratica, ovvero se trovano il canale occupato aspettano il loro turno. Il vantaggio è che il canale è utilizzato al meglio, però se abbiamo un tempo di back-off molto grande la banda viene sprecata (il mezzo rimane in idle e non si trasmette niente). Questi ritardi casuali (*back-off*) riduce la probabilità che si creano collisioni.

2. **CSMA 1-persistente**: se una stazione ha dei dati da trasmettere si mette in ascolto del canale e vede se è occupato; se è libero trasmette i frame tranquillamente ma se è occupato si mette in ascolto perenne del canale finché non si libera. Quando il canale si libera trasmette con probabilità 1, e quindi si risolve il problema del non persistente (il canale non rimane mai in idle per un tot di tempo). In questo caso lo svantaggio è che le stazioni sono “egoiste”, creando delle collisioni quando 2 o più stazioni vogliono trasmettere.
3. **CSMA P-persistente**: cerca di unire il meglio di tutti due. In questo caso, abbiamo che il tempo è diviso in slot, solitamente di lunghezza pari al massimo ritardo di propagazione. Se il mezzo è libero la stazione trasmette con probabilità P , oppure aspetta uno slot di tempo con probabilità $1-P$, rimandando la trasmissione all'intervallo successivo. Se quell'intervallo è libero trasmette, altrimenti rimanda allo slot successivo; questo processo si ripete finché il frame non è stato trasmesso correttamente. Le collisioni sono ridotte (come nel non persistente) e il tempo di idle è ridotto (come nel 1-persistente).

CSMA/CD

CD sta per *collision detection*, perché sia il CSMA persistente che quello non persistente sono un miglioramento dell'ALOHA, però può capitare che 2 stazioni contemporaneamente vedono il canale libero e trasmettono nello stesso momento, generando collisioni, per questo motivo è stato creato il CSMA/CD che permette di rilevare la collisione (molto velocemente), interrompere le trasmissioni migliorando sia l'utilizzo della banda che il ritardo. Questo protocollo viene utilizzato in Ethernet, più in generale nelle LAN con tipologia bus.

Domanda d'esame: “Come si fa ad ascoltare il canale?”

Si deve guardare la potenza. Ci accorgiamo che sta avvenendo una collisione quando si ha un picco di potenza. La potenza osservata è $>$ della potenza trasmessa + la riflessione attenuata del proprio segnale. Un altro modo è andare a vedere gli hub: se si ha un input simultaneo su due porte significa che c'è una collisione e quindi l'hub segnalerà questo evento a tutte le stazioni.

Il CSMA/CD utilizza come metodo di persistenza il 1-persistente oppure il non persistente. Con 1-persistente riesce ad occupare il canale per il 50%, invece il throughput con il non persistente arriva al 90%. Nel caso pessimo per rilevare una collisione serve il doppio del massimo ritardo di propagazione.

Quando viene rilevata una collisione, cosa succede?

1. Si annulla la trasmissione.
2. Si trasmette un segnale di jam di 48 bit, che serve a notificare a tutte le altre stazioni che è avvenuta una collisione.
3. Ora si aspetta un tempo random (ottimale per ritrasmettere un frame).
4. Il segnale viene ritrasmesso.

Il tempo randomico viene determinato dall'algoritmo di backoff esponenziale:

1. Viene impostato lo slot temporale a 2 volte il ritardo massimo di propagazione, sommato al tempo che serve ad inviare il segnale di jam.
2. Dopo la k -esima collisione, viene selezionato un numero randomico tra 0 e $(2^k - 1)$.

3. -Si aspetta per un tempo pari al numero random per il tempo dello slot.
4. Si possono fare fino a 16 tentativi, $k \leq 10$ per non aumentare troppo il range di valori.

Con $k \leq 10$ la probabilità che le stazioni ritrasmettano frame allo stesso istante è molto bassa. L'algoritmo lavora in modalità LIFO, predilige le stazioni che hanno poche collisioni. I vantaggi sono che è facile da implementare, decentralizzato, se c'è poco traffico il ritardo è basso e se c'è un unico nodo che deve trasmettere può farlo con tutta la banda. Gli aspetti negativi sono dati dal fatto che vengono predilette le stazioni con poche collisioni, poiché le altre soffrono di starvation; se ho m nodi potrebbero non trasmettere ad un rate di r/m

4.2.3 protocolli ad accesso controllato

- **Protocolli a prenotazione:** abbiamo che le stazioni fanno a turno per trasmettere e si devono prenotare sul proprio mini-slot temporale. Ogni stazione ne ha uno. Tutte le altre ascoltano l'intervallo di prenotazione, quindi sanno quando devono trasmettere le altre e in che ordine. Questo metodo serve per prevenire le collisioni.
- **Polling:** può essere centralizzato e distribuito. Il centralizzato ha una stazione primaria e n stazioni secondarie, dove queste ultime fanno a turno per trasmettere sul mezzo e possono parlare solo con il master (non fra di loro); la stazione primaria fa polling sulle secondarie, stando sempre in ascolto se devono inviare dati (il polling fa consumare molta energia). Il polling distribuito non ha un modello master-slave, e viene mantenuta una lista con le stazioni che hanno priorità maggiore.
- **Token passing:** viene passato un token (messaggio breve), che rappresenta il permesso di inviare dei dati. Chi ha il token può trasmettere. Se la stazione non ha frame da trasmettere, passerà il token ad altri. Gli stati in cui può essere una stazione sono due: ascolto e trasmissione.

4.2.3 Dove sono utilizzati i protocolli ad accesso multiplo

Vengono utilizzati per:

- Canali wireless
- Canali satellitari
- LAN

Domanda esame: "Il livello 2 prevede l'indirizzamento?"

"Sì, esistono i MAC address"

Gli indirizzi MAC sono indirizzi a 6 byte, ciò significa che possiamo ottenere 2^{48} indirizzi possibili. L'indirizzo MAC viene scritto permanentemente all'interno della ROM, nel momento in cui l'adattatore viene costruito ed integrato nella memoria (non esistono 2 indirizzi MAC uguali: IEEE supervisiona l'attribuzione degli indirizzi MAC). I primi 3 byte vanno ad identificare il produttore della scheda, gli ultimi tre byte contengono l'indirizzo vero e proprio. Possono essere indirizzi broadcast, quindi contenere tutti 1: la scelta di questi indirizzi è effettuata nel momento in cui si vuole inviare il frame a tutti. Se si ha che il bit meno significativo del primo byte è zero significa che si tratta di un indirizzo ordinario, se invece è un 1 si tratta di un indirizzo di gruppo; questo ci fa capire che l'indirizzamento MAC supporta sia la comunicazione broadcast che multicast.

4.3 Ethernet

Ethernet appartiene allo standard 802.3 e ne esistono 2 tipologie:

1. Ethernet classica: risolve il problema dell'accesso multiplo con le tecniche discusse in precedenza.
2. Ethernet commutata: vengono utilizzati dei commutatori (switch) per connettere diversi pc fra loro.

Ethernet oggi è la rete di calcolatori più utilizzata nel mondo, e rappresenta per le LAN ciò che internet rappresenta per l'internet working. Di solito viene usata una tipologia di rete a stella (master-slave) con codifica Manchester e utilizza il protocollo CSMA/CD 1-persistente. Esistono varie categorie di Ethernet che si distinguono in base al data rate (Mbit/sec):

1. Standard ethernet (classica)

1°versione Standard: chiamata 10base5, invia dati a 10Mbit/sec. Si aveva un singolo lungo cavo attraverso il quale erano collegati tutti gli altri computer. Venne sostituita da un ethernet più flessibile. Il cavo era lungo 500m.

2°versione thin ethernet: chiamata 10base2, più economica della prima versione e più facile da installare, ma il difetto è dato dal fatto che il cavo può arrivare fino a 185m per ogni segmento (quindi ad ogni 185m bisognava mettere un ripetitore).

10baseT: utilizzava dei twisted pair.

10baseF: utilizzava cavi a fibra ottica.

2. Ethernet commutata:

Fast ethernet: sostituisce l'ethernet classica togliendo l'unico lungo cavo, andando verso un altro tipo di cablaggio, dove ogni stazione aveva un proprio cavo dedicato. Tutti questi cavi confluivano in un unico hub centrale. L'hub connette tutti i cavi collegati come se fossero saldati insieme (un unico cavo). Questo sistema non aumenta la capacità della rete. Questa tipologia può combinare la standard ethernet con la fast ethernet sulla stessa rete attraverso uno switch, ciò vuol dire che è in grado di operare con gli switch. Della fast ethernet ne esistono vari tipi:

- 100baseT4: si hanno 4 doppini telefonici di categoria 3 che si possono estendere fino a 100 m. Ne vengono usati 4 per raggiungere la velocità di 100Mbit/sec. Viene scelta perché ogni "ufficio" disponeva di almeno 4 doppini telefonici di categoria 3 collegati ad una centralina. Lo svantaggio è la capacità di trasportare 100Mbit/sec per 100m.
- 100baseTX: utilizza 2 doppini telefonici di categoria 5, che poteva arrivare facilmente a 100m trasmettendo 100Mbit/sec.
- 100baseFX: utilizza 2 fibre ottiche che riescono ad estendersi fino ad arrivare a due chilometri.

La 100baseTX e 100baseFX possono trasmettere sia in modalità half duplex che full duplex; se si trasmette in full-duplex si può raddoppiare i Mbit/sec che può inviare sul canale perché si può trasmettere e ricevere contemporaneamente.

Gigabit ethernet: vengono sostituiti **gli hub con degli switch**, contenenti una scheda **hardware di collegamento per le schede di rete**. Lo switch è facile da mantenere, perché possiamo inserire o rimuovere stazioni semplicemente **staccando e riattaccando cavi**; quindi ogni volta che si ha un malfunzionamento basta vedere **qual è la stazione che lo genera e staccarla**. Gli switch devono far **uscire i frame** verso le giuste porte di destinazione e quindi devono capire **quali porte corrispondono a quali indirizzi**; in più dobbiamo pensare a che cosa può accadere nel momento in cui più di una stazione o più di una porta vogliono spedire frame contemporaneamente. Nel caso degli **hub** **tutte le stazioni sono all'interno dello stesso dominio di collisione**, dove il dominio di collisione identifica l'insieme dei nodi che concorrono per accedere al mezzo trasmissivo, quindi per evitare le collisioni si deve usare il **CSMA/CD**; invece nel caso degli **switch** ogni **porta ha il suo dominio di collisione** separato quindi si può fare a meno del CSMA/CD (se la trasmissione è full-duplex, perché in questo modo sia la stazione che la porta possono spedire il frame senza preoccuparsi degli altri). In generale la gigabit ethernet opera in full-duplex.

- **1000baseSX:** utilizza la fibra multimodale di 550m.
- **1000baseLX:** utilizza una fibra di 5000m o mono o multimodale.
- **1000baseCX:** utilizza due coppie di STP (doppini telefonici schermati) di 25m.
- **1000baseT:** 4 coppie di UTP di categoria 5 che arrivano a 100m.

Ten Gigabit ethernet: invia dati a 10Gbit/sec, in questo caso le distanze vanno dai 300m ai 40Km e si usa solo la fibra ottica; il collegamento **supporta solo il full-duplex (non si usa mai il CSMA/CD)**

4.3.2 Protocollo del sottolivello MAC di ethernet classica

Il formato utilizzato per spedire i frame è il seguente:

- **PREAMBOLO:** 8byte, contiene la sequenza di bit 0101010 con l'eccezione dell'ultimo byte dove gli ultimi 2 bit sono 11, e infatti questo ultimo byte è chiamato delimitatore di inizio frame.
- **2 INDIRIZZI:** il primo è **quello di destinazione di 6 byte**, se il **primo bit trasmesso è zero si tratta di indirizzi ordinari** invece se è 1 si tratta di un gruppo di indirizzi. Se l'indirizzo di destinazione è composto da tutti 1 significa che è riservato per trasmissioni broadcast. Il secondo è l'indirizzo d'origine di 6 byte che corrisponde all'indirizzo MAC. L'idea di base si può rivolgere ad un'altra solo se gli fornisce il giusto numero di 48bit (indirizzo MAC corretto).
- **TYPE:** 2byte, viene usato per indicare al ricevente cosa deve fare con il frame. **Si occupa del multiplexing.**
- **DATA:** può arrivare **fino a 1500byte** (max dimensione del frame ethernet), all'interno di questo campo si ha un frame che va da 0 a 1500byte. In generale **non** bisogna avere un campo dati da **0byte perché può creare problemi**, quindi ethernet richiede che i frame validi abbiano una dimensione **minima di 64byte**.
- **PAD:** **riempire il frame fino alla dimensione minima.**

- **CHECKSUM:** CRC a 32 bit, si occupa di rilevare gli errori di trasmissione.

4.4 LAN Wireless

Sono sempre più diffuse (case, aeroporti, luoghi pubblici...), utilizzate per collegare ad internet pc, smartphone e altri dispositivi di rete. Lo standard per la LAN wireless più diffuso è 802.11 (WiFi).

4.4.1 Architettura e stack di protocolli di 802.11

Per quanto riguarda l'architettura possiamo utilizzare 2 differenti modalità:

1. **Modalità ad infrastruttura:** ogni client è associato ad un AP, ovvero ad un access point. Questo access point è connesso ad un'altra rete: i client comunicano attraverso gli AP per comunicare fra loro. I pacchetti vengono spediti all' AP che smisterà tra i vari client. Oltre all'AP abbiamo diverse componenti: *stazione*, *basic service set* e *extended service set*. Il basic service set è una rete wireless con l'AP e i client, invece l'extended service set è l'insieme di due o più AP connesse alla stessa rete cablata (è una sorta di sistema di distribuzione).
2. **Modalità ad hoc:** non esiste nessun AP, dove i client comunicano direttamente fra loro spendendosi i frame l'un l'altro. Di solito si preferisce usare la modalità ad infrastruttura.

Nel livello fisico si hanno tutti i protocolli che ti permettono di modulare il segnale, e nel sottolivello MAC determinano come è allocato il canale. Si ha che gli AP devono occuparsi di gestire la trasmissione (chi deve trasmettere). Sopra il sottolivello MAC c'è il livello LLC (*logical link control*) che ha il compito di nascondere le differenze tra i vari 802, quindi di tutti gli standard 80 (ethernet, wimax...) vengono resi indistinguibili a livello di rete: ossia quando siamo nel livello di rete non ci interessa dei vari dettagli dei livelli sottostanti.

4.4.2 Livello fisico di 802.11

Lo standard WiFi originale 802.11 si divide in due sottogruppi:

1. **802.11b:** la velocità di funzionamento è quasi sempre di 11Mbit/sec, per l'accesso al canale viene utilizzato il protocollo CSMA/CA (limitare le collisioni) permettendo di limitare le interferenze sul canale e massimizza la possibilità che i pacchetti possano raggiungere la destinazione in modo corretto. Lo svantaggio è che vengono effettuate tante operazioni di controllo, saturando la banda diminuendo le prestazioni (arrivando a 6/7 Mbit/sec.) Questa tecnica trasmette nella banda di frequenza di 2,4 GHz.
2. **802.11a:** permette velocità di trasmissione fino a 54Mbit/sec e trasmette nelle bande dei 5GHz. Questo standard non ha riscosso successo per via dei costi elevati, e anche se la banda di frequenze di 5GHz è meno affollata del 802.11b, si ha una maggiore difficoltà nell'attraversare rumori e barriere (frequenza alta).
3. **802.11g:** combinazione fra le due. Opera nella banda di 2,4GHz e offre gli stessi tassi trasmissivi (vel di trasmissione) di 802.11a (54Mbit/sec).
4. **802.11n:** progettato per aumentare l'efficienza dell'802.11g. L'obiettivo è raggiungere un tasso trasmissivo di 100Mbit/sec, andando a raddoppiare i canali da 20 MHz a 40MHz, e vennero usate 4 antenne in grado di trasmettere 4 flussi di informazione contemporaneamente.

Poiché i segnali dei flussi possono interferire a lato ricevitore, si separano utilizzando la tecnica **MIMO** (*multiple input multiple output*) (protocollo di 802.11).

4.4.3 Protocollo del sottolivello MAC di 802.11

Il protocollo del sottolivello MAC ci serve per andare a **gestire l'allocazione del canale**. Bisogna tenere in considerazione:

- Le interfacce radio usate da 802.11, sono quasi sempre half-duplex, a **differenza di ethernet** che predilige la trasmissione full-duplex. Significa che non **possiamo trasmettere dei segnali e contemporaneamente controllare i picchi di rumore** (collisione); bisogna fare una cosa alla volta. Il segnale ricevuto può essere molto più debole del segnale che è stato inviato, quindi rende difficile in controllo di errori.
- Il rilevamento delle collisioni che avevamo con ethernet (CSMA/CD) non funziona, ma dobbiamo utilizzare un altro protocollo, il **CSMA/CA**.

CSMA/CA

Questo protocollo è **molto simile al CSMA/CD**: si ha che la stazione che vuole spedire un frame **inizializza un tempo di back-off** casuale e **non aspetta che avvenga la collisione**, quindi il frame parte dopo il tempo di back-off. Il numero di slot di back-off è scelto in un **intervallo che va da 0 a 15**: la stazione sorgente aspetta finché il canale è libero, per controllare la disponibilità **verifica che non ci sia segnale per un certo periodo di tempo**. Se due o più stazioni sono pronte a trasmettere fanno partire il back-off e ha il diritto di trasmettere la stazione con valore (back off inferiore) più basso: in attesa della stazione che sta occupando il canale, l'altra che è **pronta a trasmettere mette in pausa il proprio back-off**, e lo farà ripartire **quando la stazione in possesso del canale terminerà la trasmissione**. Se **non si ha un ack significa che il frame non è arrivato a destinazione**, quindi il mittente va a raddoppiare il periodo di back-off e prova di nuovo fintanto che **o il frame è arrivato correttamente, oppure si è raggiunto il numero max di ritrasmissioni**. In generale il CSMA/CA **non prevede la rilevazione degli errori** (collisioni nel canale), ma punta sulla prevenzione: per vedere se c'è un errore, abbiamo a disposizione solo l'ack di ritorno.

Problematiche di 802.11

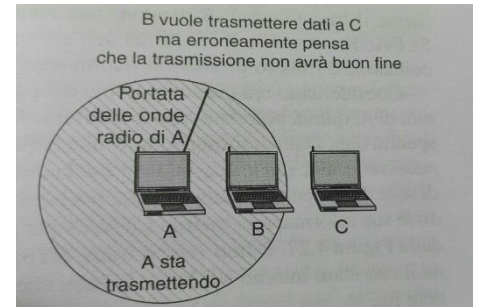
Abbiamo che le stazioni possono avere dei raggi d'azione differenti, questo perché se utilizziamo un cavo sai con certezza che tutte le stazioni collegate al cavo, possono identificarsi a vicenda, invece la complessità di propagazione elettromagnetica non ci consente di fare ciò, perché sono molto più inaffidabili dei cavi cablati; per questo motivo possono sorgere del terminale nascosto e del terminale esposto.

- Problema del terminale nascosto**: **non tutte le stazioni si trovano nella stessa portata radio**



abbiamo che le trasmissioni che avvengono in una parte della cella possono non essere ricevute in qualche punto della stessa cella. In pratica abbiamo più stazioni che si trovano nello stesso raggio d'azione, la trasmissione avviene attraverso onde radio; può capitare che **una stazione A non si accorge che una stazione B sta inviando segnale a C** (perché non lo sente), allora **A pensa che può iniziare la trasmissione portando ad una collisione**.

- **Problema del terminale esposto:** in questo caso si ha che una stazione **B vuole inviare dati a C** ma prima di farlo si mette in ascolto del canale; mettendosi in ascolto **sente una trasmissione** quindi si blocca decidendo di non trasmettere, però questa trasmissione che è effettuata dalla stazione A potrebbe non essere rivolta a C, ma ad un'altra stazione (D, fuori portata di B). Quindi si va a **perdere un'occasione di trasmissione**.



Per cercare di risolvere questi problemi si vanno a rilevare le trasmissioni **sia dal punto di vista fisico che dal punto di vista virtuale**. Per quanto riguarda la rilevazione fisica, questa permette di controllare il canale wireless per vedere se sta passando un segnale valido; invece con la rilevazione virtuale ogni stazione va ad appuntarsi se delle altre stazioni stanno occupando il canale e queste informazioni vengono mantenute all'interno di una **NAV (network allocation vector)**.

Le reti Wireless sono in generale **inaffidabili poiché soggette a rumore** e hanno la tendenza ad **interferire con altre apparecchiature**; sia l'**ack** che le ritrasmissioni sono **poco utili se la probabilità di consegna dei frame è scarsa**. Per aumentare la possibilità del numero di trasmissioni si va a **diminuire il tasso di trasmissione**, ossia si fa in modo che vengono spediti **pochi frame alla volta**, oppure si possono spedire dei frame più corti. I frame possono essere spezzettati in pezzi più piccolo, dove ognuno di questi pezzetti **avrà il suo checksum**. Un'altra cosa da tenere in considerazione è la durata della batteria, quindi per risparmiare energia bisogna adottare tecniche di risparmio energetico e il meccanismo alla base è il **beacon frame**: non sono altro che trasmissioni periodiche inviate all'AP per visionare i dispositivi inattivi per un po' di tempo e se non sono necessari mettersi in stand-by.

Qualità del servizio

È un punto molto **importante**, per esempio quando abbiamo un traffico **VOIP che compete con applicazioni del traffico Peer to peer** (file sharing), dobbiamo fare in modo che il traffico VOIP abbia una priorità più elevata. Questa qualità del servizio si può raggiungere estendendo il **CSMA/CA** con intervalli tra i vari frame: una volta spedito il frame dobbiamo attendere un intervallo di tempo **prima che un'altra stazione possa spedire un altro frame**, per verificare che il canale non sia più utilizzato; questo intervallo che intercorre fra l'invio di un frame e un altro viene chiamato **DIFS**. Esiste un'altra tipologia di intervallo, chiamato **SIFS**, utilizzato per permettere a coloro che partecipano ad uno scambio di messaggi di poter partire per primi (ack, controllo,...). Abbiamo l'**AIFS**, che prevede di **inviare frame nell'intervallo di tempo che hanno una certa priorità**; **EIFS** invece viene **usato dalle stazioni che hanno ricevuto un frame difettoso per segnalare che si è verificato un problema**.

4.4.4 Struttura del frame 802.11

Anche in questo caso abbiamo **3 differenti tipi di frame**:

1. **Dati**
2. **Controllo**
3. **Gestione**

Ci focalizzeremo solo sul quello dati, perché gli altri 2 sono molto simili. Quindi la struttura del frame dati è la seguente:

- **FRAME CONTROL**: questo campo ha 11 sottocampi. *Type*, distinzione fra frame dati/controllo/gestione, *Subtype*, contiene ack, rts, cts, ecc., *To* e *From*, indicano se il frame sta andando o venendo dalla rete connessa con l'AP.
- **DURATION**: indica per quanto tempo il frame e il corrispettivo ack occuperanno il canale.
- **3 INDIRIZZI**: indirizzo 1 è l'indirizzo del ricevente, indirizzo 2 è l'indirizzo della sorgente, invece il terzo corrisponde all'indirizzo MAC dell'interfaccia router della sottorete. Grazie all'ultimo indirizzo un AP può tradurre l'indirizzo su reti molto diverse fra loro. Esiste anche un quarto indirizzo, usato per reti ad hoc.
- **SEQUENCE**: numerare i frame per rilevare eventuali duplicati.
- **DATA**: contiene il payload che può arrivare fino a 2312byte, ma di solito non supera i 1500byte.
- **CHECK SEQUENCE**: CRC a 32 bit.

Interferenze

- *Cocanale*: in questo caso ogni client e l'AP competono per parlare
- *Canale adiacente*: i client e l'AP sui canali sovrapposti parlano gli uni sugli altri. Questa è la peggiore perché non appena un AP prova a comunicare con i propri client la trasmissione è confusa a causa di altri client che parlano in contemporanea (continuo reinvio dei pacchetti). Si risolve mantenendo una differenza tra i segnali di 20 dB.
- *Non WiFi*: avviene quando dispositivi che non appartengono allo standard 802.11 competono per accedere al mezzo. Può capitare quando dispositivi wireless usano le stesse frequenze del WiFi.

4.6 Bluetooth

È un Wireless PAN, e a differenza del WiFi copre un'area molto limitata. Utilizza una topologia **ad hoc**, e comprende un supporto ai dispositivi voce e dati, il tutto con un basso consumo di energia. Il bluetooth nasce per permettere il **collegamento tra vari dispositivi utilizzando un sistema radio wireless che fosse a basso costo, bassa potenza e a portata ridotta**. Il gruppo 802.11 si concentrò sullo sviluppo di reti Wireless a corte distanze usate per il networking di dispositivi portatili e mobili: bluetooth (802.15), BAN (body area network), PAN ottica. Bluetooth opera sulla banda ISM alla frequenza di 2,4 GHz: è la stessa banda del WiFi, quindi ci possono essere interferenze; per prima cosa andiamo ad analizzare l'architettura.

4.6.1 Architettura del bluetooth

La topologia della rete è ad hoc scattered, dove coesistono tante piccole reti ognuna formata da qualche terminale. L'unità base del sistema bluetooth è la PICONET, che è composta da un solo nodo master e da massimo 7 slave attivi entro un raggio di 10m. I nodi possono trovarsi in **4 stati differenti all'interno della rete**:

1. **Master**
2. **Slave**
3. **Stand-by**

4. *Parked-hold*

Oltre ai 7 nodi slave attivi presenti in una piconet, si possono avere fino a 255 nodi sospesi (parked-hold state); questi nodi sono stati messi in questo stato dal master in modo da attivare il risparmio energetico e ridurre l'utilizzo delle batterie. Quando un dispositivo è nello stato di sospensione può rispondere solo ad una richiesta di attivazione o un segnale di beacon; gli slave rispondono soltanto alle direttive del master, dove quest'ultimo opera in modalità round-robin se gli slave fanno parte ancora della rete. In ogni momento uno slave potrebbe diventare un master, chiaramente deve essere l'unico master presente all'interno della struttura. Più piconet interconnesse fra loro danno vita ad una SCATTERNET.

4.6.2 Applicazioni bluetooth

Il bluetooth viene utilizzato per diverse applicazioni e per ognuna di queste si ha un differente stack di protocolli; al momento esistono 25 applicazioni chiamate profili, per esempio alcuni profili sono destinati all'audio/video, altre servono per connettere tastiere/mouse ai pc. In generale possono essere utilizzati due stack di protocolli, uno che serve per il trasferimento dei file e l'altro per la gestione dei flussi di comunicazione in tempo reale.

4.6.3 Stack di protocolli bluetooth

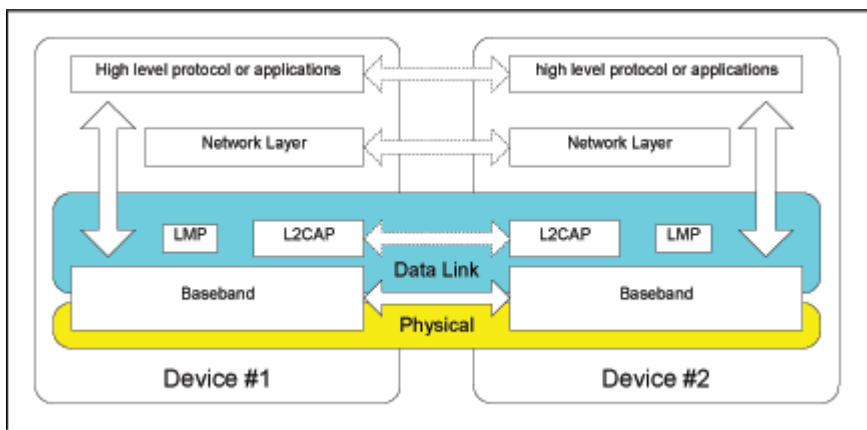
Bluetooth segue un modello a strati: nel livello più basso si ha il livello radio, che si occupa della trasmissione radio gestendo la modulazione dei segnali. Sopra abbiamo il livello link control (baseband) e lo possiamo ricondurre con il sottolivello MAC, anche se include elementi del livello fisico: serve per gestire il modo in cui il master controlla gli intervalli temporali e il raggruppamento degli intervalli in frame. Poi si ha il link manager che serve per gestire la creazione dei canali logici tra dispositivi, si occupa del pairing (accoppiamento dei dispositivi) e la qualità del servizio. Sopra

questo livello si ha una linea di separazione che marca l'interfaccia con il controller dove sopra di essa si ha il protocollo L2CAP, che suddivide in frame i messaggi di lunghezza variabile (segmentazione e riassettaggio dei pacchetti grandi). Il protocollo RFCOM serve per emulare le porte seriali standard. Infine si hanno le applicazioni dei livelli superiori. Gli unici protocolli obbligatori per tutti gli stack bluetooth sono:

- **LMP**: serve per settare e controllare i link radio tra due dispositivi.
- **L2CAP**: utilizzato per fare il multiplexing di più connessioni tra due dispositivi. Si occupa anche della segmentazione e riassettaggio dei frame.
- **SDP**: permette ad un dispositivo di trovare i servizi offerti da altri dispositivi.

4.6.4 Utilizzo del bluetooth

Il bluetooth viene utilizzato per il trasferimento dei file, quindi un dispositivo bluetooth può entrare nel file system del dispositivo associato (manipolare oggetti, ecc.), può accedere ad una LAN,



utilizzando lo stack di protocolli **TCP/IP basato su PPP**, può essere utilizzato per connettere **cuffie wireless** oppure per telefoni cordless.

4.6.5 Livello radio del bluetooth

Si occupa di spostare i bit da master a slave: opera nella banda ISM di 2,4 GHz nella portata di 10m. Abbiamo che la banda è divisa in 79 canali di 1MHz ciascuno, per poter coesistere con le altre reti che utilizzano la stessa banda delle frequenze viene utilizzata una tecnica chiamata **frequency hopping spread spectrum** (spettro distribuito a frequenza variabile) dove possiamo avere fino a 1600 salti (hop) al secondo su una slot temporale. Tutti i nodi della PICONET cambiano frequenza in modo **pseudorandomico** seguendo gli ordini del master, si salta in continuazione per **mantenere alta la sicurezza e minimizzare le interferenze**. La lunghezza del frame può essere di 1, 3 o 5 slot.

4.6.6 Livello link del bluetooth

Il livello link control è **simile al sottolivello MAC**, e trasforma il flusso di bit grezzi in frame: questi frame possono occupare 1,3 o 5 slot e solitamente un frame da 5 slot è molto più efficiente di quello con una slot perché si riescono a spedire più dati. Il protocollo link manager serve per **creare i canali logici che vengono chiamati link**: questi link servono per trasportare i frame dal master agli slave che sono stati scoperti tramite la **procedura del discovery** (procedura per scoprire quali dispositivi fanno parte della rete), dopo segue il **pairing** (aggancio ed accoppiamento) per assicurarsi che possono comunicare l'un l'altro, di solito questo pairing si fa richiedendo agli utenti di confermare che tutti e due gli apparecchi mostrino la chiave d'accesso, oppure la chiave viene vista su un apparecchio e **immetterla nel secondo dispositivo**. Una volta completata la procedura viene generato il link, che può essere di due tipologie:

- **Link SCO:** **synchronous connection oriented**, utilizzato per dati **real-time (telefono)** e viene allocato uno slot per ogni direzione di comunicazione. In una piconet si possono avere fino a 3 SCO attivi nello stesso momento. In questo caso i **frame non vengono mai ritrasmessi**.
- **Link ACL:** **asynchronous connectionless**, utilizzato per i dati commutati a pacchetto che sono **disponibili ad intervalli irregolari**. In questo caso parliamo di **link punti-multipunto** tra il master e gli slave all'interno della piconet: si ha **solo 1 ACL all'interno della rete**. I frame possono essere persi, quindi è necessaria la ritrasmissione.

Domanda d'esame: **“Da che cosa deriva il nome del bluetooth?”**

“Deriva da un re vichingo che unificò la Danimarca e la Norvegia. Le aziende che brevettarono il bluetooth erano: IBM, Intel, Nokia e Toshiba.”

4.7 Zigbee

E' una wireless PAN, a topologia mesh o a stella. Viene utilizzato soprattutto in ambiente industriale (automazione, domotica). Lo Zigbee opera sulle frequenze libere (bande radio) a 2,4 GHz. I dispositivi utilizzati all'interno della rete sono:

- **Coordinatore:** è il dispositivo più importante, in lui si **concentrano tutti i dati**. Funge da **gateway verso un PC**; infatti opera come un bridge fra le reti.
- **Zigbee router:** funge da intermediario che va ad **inoltrare i dati agli altri nodi della rete**.

- **Zigbee end-device:** contengono **solamente delle funzionalità per poter comunicare con il nodo coordinatore**; possono essere di due tipi: **sensori e attuatori**. Questa tipologia di nodo può essere messo in sleep per molto tempo e può essere svegliata a un segnale **di alive**.

La topologia della rete è ad hoc, ovvero i dispositivi comunicano fra loro **senza l'utilizzo di access point**.

4.7.1 Protocolli di Zigbee

I protocolli Zigbee supportano:

- **reti abilitate a beacon**, dove i router Zigbee inviano periodicamente dei pacchetti di beacon per confermare la loro presenza agli altri dispositivi (confermare che sono ancora presenti nella rete); se invece questi nodi rimangono inattivi **vengono messi in risparmio energetico**. I nodi Zigbee possono risvegliarsi in 30ms (anche meno, poca latenza), quindi sono molto reattivi a differenza dei nodi del bluetooth (3s per il risveglio).
- **reti non abilitate a beacon**, dove Zigbee utilizza il **CSMA/CA** per l'accesso al canale e in questo caso a differenza delle precedenti gli Zigbee router **devono avere molta energia**, perché i loro ricevitori sono sempre attivi.

4.8 RFID (*radio frequency identification*)

Si tratta di una tecnologia per **l'identificazione e memorizzazione automatica di informazioni inerenti ad oggetti, animali e qualsiasi altra cosa**. Si parla di etichette RFID, che hanno le **dimensioni di un francobollo** e possono essere applicate a **qualsiasi oggetto**. Esistono due tipologie di etichette:

1. **Tag Passivi:** non richiedono energia, perché per funzionare hanno bisogno di energia che è fornita sotto forma **di onde radio dai lettori RFID**. Hanno una capacità di memorizzazione molto bassa, con dei range di lettura molto corti. Di solito sono delle etichette *wrtite-once / read-many* (scritti una volta, letti più volte) oppure *read-only*, dove l'ID dell'etichetta non può più essere cambiato. Costano molto poco.
2. **Tag Attivi:** esiste una sorgente di energia nell'etichetta (alimentati da batteria). La **capacità di memorizzazione è molto alta (512 Kbyte)**, il range di lettura è molto più ampio. Possono essere **riscritti più volte**, il loro **costo è molto maggiore** rispetto agli altri.

Nell'infrastruttura RFID esistono anche dei lettori, che hanno diverse funzioni:

- Fornire energia alle etichette, come nel caso dei tag passivi.
- Leggere fino a 300 etichette al secondo.
- Scansionatori, estrapolare le informazioni contenute all'interno delle etichette.

Possono essere posizionati **in punti specifici, ma anche mobili**. Le applicazioni più comuni sono: nei processi di manifattura, logistica, tracking system, vendite (inventario), applicazioni di sicurezza (antitaccheggio).

Vantaggi degli RFID:

- Il tag non ha bisogno del contatto visivo per essere letto (come nel codice a barre)
- Più prodotti possono essere letti contemporaneamente in una singola scansione.

- All'interno dei tag si hanno raggruppati molti dati.
- I tag passivi possono durare all'infinito, mentre i tag attivi possono essere letti a grandi distanze.
- Gli RFID possono essere combinati con la tecnologia del codice a barre.

4.8.1 comunicazione RFID

L'host gestisce il lettore e invia dei comandi, viene inviato un segnale radio al lettore attraverso un'antenna, fino a che dall'antenna non raggiunge le etichette. Una volta che l'etichetta riceve il segnale radio, lo va a modulare e lo rimanda indietro all'antenna: questo segnale modulato viene inoltrato dall'antenna al lettore che decodifica i dati e poi invia i risultati all'applicazione host. L'antenna è accoppiata in maniera induttiva attraverso campi elettromagnetici (per i tag passivi), invece l'accoppiamento viene effettuato tramite propagazione (per i tag attivi).

Interferenze

Possiamo avere due tipologie di interferenze: una tra due lettori di tag RFID, l'altra tra un lettore e un tag. Abbiamo che deve essere risolto il problema del terminale nascosto e quello del terminale esposto perché si ha a che fare con trasmissioni wireless: si ha che diverse etichette possono essere all'interno dello stesso raggio di lettura; può capitare che due lettori cerchino di leggere in contemporanea la stessa etichetta, e quest'ultima non sa a quale lettore rispondere. Per risolvere queste interferenze si assegnano diverse frequenze operative ai lettori. Inoltre per risolvere le interferenze fra lettori e tag si assegnano diversi time-slot, uno per ciascuna operazione, invece se nelle stesso momento si verificano entrambe le interferenze, prima si allocano differenti time-slot e poi si pensa alla frequenza.

4.9 Commutazione a livello Data Link

“Perché non è possibile costruire un'unica grande LAN?”

Perché si hanno dei fattori limitanti: il traffico supportato da una rete è limitato, quindi con un'unica grande LAN tutte le stazioni dovrebbero condividere la larghezza di banda con un grosso dominio di collisione, con un grande tempo di vulnerabilità; si dovranno gestire distanze fisiche notevoli (cavi lunghi chilometri), con un gran quantitativo di dispositivi sincronizzati fra loro.

L'unica soluzione è dividere la LAN, in reti LAN più piccole e collegarle fra loro con dei bridge, che permettono di unire le varie parti della rete aumentando la distanza fisica ricoperta.

Adesso andremo a studiare tutti i dispositivi usati in questa configurazione, dal livello fisico al livello di rete.

4.9.1 Hub

Gli hub sono dispositivi a livello fisico, e sono essenzialmente di ripetitori che operano a livello di bit. Sono formati da linee di input collegate elettricamente. I bit che arrivano sulle linee in input, vengono trasmessi alle altre interfacce; se arrivano dei frame nello stesso momento collidono fra loro. Tutte le linee che si collegano all'hub devono operare alla stessa velocità e solitamente non amplificano i segnali di ingresso, si occupano solo a trasmetterli. Gli hub possono essere disposti in gerarchie. Ogni LAN che viene connessa è chiamata segmento LAN.

Vantaggi:

- Semplici ed economici.
- I vari segmenti della LAN continuano a rimanere operativi anche in caso di malfunzionamento, grazie alla progettazione gerarchica.
- Estendono la distanza massima tra una coppia di nodi.

Svantaggi:

- Non vanno ad isolare i domini di collisione: i singoli domini di collisione non aumentano l'efficienza massima. Il throughput della rete non aumenta ma rimane lo stesso per ogni segmento.
- Si hanno dei limiti per quanto riguarda il numero di dispositivi che si possono agganciare all'hub.
- Dispositivi a velocità differenti non riescono a comunicare fra loro. Per esempio non si possono connettere fra loro dei diversi tipi di Ethernet.

Ora mai gli hub sono diventati obsoleti.

4.9.2 Bridge

I bridge sono dispositivi del livello data link, a cui vengono agganciati stazioni e hub: se la tecnologia LAN è Ethernet i bridge sono conosciuti come *Switch Ethernet*. Infatti i bridge operano sui frame ethernet andando ad esaminare l'intestazione del frame e lo inoltrano basandosi sulla destinazione. Convogliano il pacchetto su una porta specifica che sa dove è contenuto l'indirizzo di destinazione.

Caratteristiche dei bridge

Le stazioni collegate ad una stessa porta appartengono allo stesso dominio di collisione, che è diverso da quello delle altre porte. Ad un bridge possono essere collegati diversi tipi di cavo: fibra ottica, doppino telefonico, ecc... . All'interno dei bridge viene mantenuta una tabella di hash (tabelle di filtraggio) contenente le possibili destinazioni con le porte associate ad esse: nel momento in cui si collegano i bridge per la prima volta, le tabelle saranno vuote, significa che i bridge non conoscono le varie destinazioni, per questo si applica l'algoritmo di *flooding*, praticamente si ha che ogni frame in entrata viene fatto uscire su tutte le porte a cui il bridge è connesso, ad eccezione da quella da cui è arrivato; attraverso questo algoritmo i bridge imparano le destinazioni e i frame verranno posti sulla porta corretta senza dover essere spediti in tutte le porte.

Quando viene ricevuto il frame il bridge memorizza la posizione del mittente; quindi ogni entry della tabella di filtering è composto da:

- Indirizzo LAN del nodo
- Interfaccia del bridge
- Time-stamp: è utile per gestire le topologie dinamiche. Ogni volta che viene aggiunto un elemento alla tabella di hash si tiene conto del tempo d'arrivo.

Periodicamente all'interno del bridge vengono eliminati tutti gli elementi vecchi di qualche minuto.

Vantaggi:

- Isolamento dei domini di collisione, aumentando il throughput.
- Si possono connettere diversi tipi di Ethernet.
Sono trasparenti, ovvero tutti i dettagli implementativi vengono nascosti agli utenti e agli apparati di rete.
- L'interconnessione può essere con o senza dorsale. È sempre preferibile usare quella con dorsale (perché quella senza dorsale se un nodo cade, failure, viene compromesso l'intero sistema e il traffico tra i due nodi va a coinvolgere anche i nodi intermedi).

I bridge guardano solamente l'indirizzo MAC per decidere su che porta indirizzare un frame; possono iniziare ad inoltrarlo non appena vanno a leggere il campo destinazione, riducendo la latenza, perché non bisogna aspettare che arrivi tutto il frame ma solo il campo destinazione, ed il numero di frame che il bridge deve mantenere nella sua memoria interna. Questo tipo di inoltro viene chiamato **cut-throught-switching**. Il pacchetto arriva dal livello di rete, poi da lì passa al livello MAC, qui acquisisce l'header ethernet e infine viene passato al livello fisico, spedito in un cavo raccolto dal bridge per poi essere inoltrato alla stazione corretta.

I bridge fanno uso dell'algoritmo di apprendimento, che viene utilizzato nel momento in cui non si ha più bisogno di fare il flooding, poiché il bridge ha imparato la porta di destinazione. Guardando l'indirizzo sorgente i bridge sanno anche quali stazioni sono accessibili su quali porte. Se un bridge vede un frame sulla porta 3, che arriva dalla stazione C, il bridge sa che C deve essere raggiungibile attraverso la porta 3, andando a segnare questa informazione nella tabella di hash; da qui in poi tutti i frame successivi indirizzati a C che arrivano al bridge su qualsiasi altra porta vengono inoltrati attraverso la porta 3.

4.9.3 Bridge con spanning tree

Per aumentare l'affidabilità potremmo pensare di creare collegamenti ridondanti tra i bridge: in questo modo anche se fosse tagliato un collegamento, le due stazioni potrebbero continuare a parlare fra loro. Se si introduce questa ridondanza si creano dei problemi, per via dei cicli generati nella topologia della rete; i loop avvengono nel momento in cui si ha una rete molto complessa, composta da tanti segmenti e colleghiamo i segmenti alle stesse porte, perché a questo punto le tabelle di filtering non riescono ad individuare quali sono i corretti ingressi e uscite delle porte e ciò genera un loop di frame che si propaga all'infinito all'interno della rete. La soluzione a questo ciclo è data dall'algoritmo dello **spanning tree**: quest'algoritmo fa un modo che alla topologia fisica della rete si vada a sovrapporre un albero di copertura, che raggiunge ogni bridge della rete: questo albero di copertura potremmo pensarlo come un grafo aciclico, poiché c'è un unico percorso da ogni sorgente ad ogni destinazione. L'algoritmo funziona nel seguente modo:

1. Assegnare un ID univoco ad ogni bridge. Questo ID si basa sul proprio indirizzo MAC.
2. I bridge vanno a scegliere il bridge con ID più basso, che diventerà il bridge radice (root bridge). Per identificarlo è scambiare dei messaggi di configurazione.
3. Costruzione di un albero con i percorsi più brevi dalla radice ad ogni bridge. Solitamente si utilizza la metrica dell'hop (salti): a parità di hop, viene scelto l'ID con valore minore. Il percorso più breve, dalla radice all'ultimo bridge viene detto **root-path-cost**

4. Ogni volta i bridge ricordano il percorso più breve verso la radice, andando a spegnere tutte le porte che non fanno parte del tragitto più breve.

4.9.4 Router

I router sono i dispositivi a livello di rete: nel momento in cui un pacchetto raggiunge un router viene eliminato l'header e il trailer (coda), e la parte rimanente ossia il payload viene passato al software di instradamento (routing) che va a vedere quale è la linea di input sulla quale fare uscire il determinato frame; questa informazione viene presa nell'intestazione.

Sia i router che i bridge sono dispositivi memorizza e inoltra, ovvero vanno a memorizzare informazioni che poi inoltreranno. I router mantengono le tabelle di routing e implementano algoritmi di routing, mentre i bridge mantengono delle tabelle di filtering e implementano lo spanning, l'algoritmo di apprendimento e il filtraggio.

Le operazioni dei bridge sono più semplici di quelle dei router, però hanno topologie ristrette e infatti si ha bisogno dello spanning tree per evitare i cicli, mentre i router le topologie possono essere le più svariate e per evitare i cicli vengono usati dei time-to-live aggiunti al pacchetto che sono dei contatori che indicano il livello di vita del pacchetto all'interno della rete. I bridge non offrono protezione dalle tempeste broadcast, mentre i router tramite il firewall riescono a contrastarle. I bridge vengono utilizzati quando si hanno pochi host, mentre i router quando si hanno migliaia di host.

4.9.5 Gateway

Gateway di trasporto

Sono dispositivi a livello di trasporto che connettono due host che usano protocolli di trasporto orientati alle connessioni differenti. Possono copiare i pacchetti provenienti da una connessione all'altra, modificando il formato secondo necessità.

Gateway applicazione

Sono dispositivi a livello applicazione che possono tradurre i messaggi da un formato all'altro.

4.10 VLAN

Virtual LAN, sono utilizzate per una questione di sicurezza, in questo caso una LAN può ospitare web server e altri host, destinati ad utilizzo pubblico. Un secondo problema che vanno a risolvere è il carico della rete, infatti evita che alcuni host si prendono tutta la connessione di rete. Le VLAN nascono per soddisfare i clienti che richiedevano maggiore flessibilità, quindi i produttori di dispositivi di rete iniziarono a pensare ad un metodo per ricablare gli edifici via software. Si basano su appositi switch e l'amministratore di rete stabilisce quante VLAN devono esserci oltre che quali host collegare a ciascuna VLAN e quali nomi assegnarli.

5.0 Livello di rete

Si occupa del trasporto dei pacchetti lungo tutto il percorso dall'origine alla destinazione, che può essere raggiunta attraverso diversi salti tra router intermedi. Per raggiungere tale obiettivo il livello di rete deve conoscere la topologia della rete (insieme dei router e dei collegamenti) e poi scegliere i percorsi appropriati all'interno della rete stessa. Si deve occupare di gestire i problemi che insorgono quando sorgente e destinazione si trovano su reti diverse; nel livello di rete non si parla più di frame ma di pacchetti.

5.1 Problematiche nella progettazione del livello di rete

5.1.1 Store and forward

Il pacchetto trasmesso da un host viene inoltrato al router più vicino attraverso la sua LAN; quindi il pacchetto viene memorizzato all'interno di quel router *finché non arriva nella sua interezza per verificare il checksum*. Quando è arrivato del tutto viene inoltrato al router successivo, che si trova lungo il percorso, e tutta questa procedura si ripete finché il pacchetto non arriva all'host di destinazione. Questo meccanismo è chiamata *commutazione del pacchetto store and forward*.

A livello di rete si *parla di host*, che hanno almeno il livello 4.

5.1.2 Servizi forniti a livello di trasporto

Il livello di rete *fornisce servizi* al livello superiore, che è il livello di *trasporto*. I servizi del livello di rete devono essere progettati seguendo i seguenti punti:

- Questi *servizi non devono essere legati alla tecnologia dei router*.
- Al livello di *trasporto* devono essere nascosti i *dettagli dei router* (numero, tipo...)
- Gli indirizzi di rete disponibili a livello di trasporto dovrebbero utilizzare uno *schema di numerazione uniforme*.

Il livello di trasporto dovrebbe vedere i router come dei *dispositivi che spostano pacchetti da una sorgente ad un'altra, senza pensare a tutto ciò che avviene nel percorso intermedio*.

5.1.3 Implementazione del servizio senza connessione

Se il servizio che viene offerto dal livello di *rete è senza connessione*, i pacchetti vengono inoltrati nella rete individualmente uno dall'altro, e si parla di *datagrammi* (pacchetti instradati indipendentemente l'uno dall'altro): la rete viene chiamata *rete datagram*. Se il servizio offerto è orientato alla connessione, prima di inviare i pacchetti si stabilisce un *percorso che collega il router sorgente al router destinazione*; in questo caso si parla di *connessione a circuito virtuale* e queste reti sono dette *reti a circuito virtuale*.

In queste due tipologie di reti, i router mantengono una *tabella di routing, che ci indica dove devono essere inviati i pacchetti a ogni possibile destinazione*. Ogni entry della tabella ha la destinazione e la linea di trasmissione che si può utilizzare: sia i percorsi che le tabelle si calcolano attraverso gli algoritmi di routing. Il protocollo IP, che sta alla base di internet, è l'esempio più diffuso di servizio non orientato alla connessione: in questo caso i pacchetti *non usano sempre lo stesso cammino* per raggiungere la destinazione, ciò comporta che i pacchetti arrivano in ordine sparso quindi c'è *bisogno di servizi per lo smistamento e riordino*.

5.1.4 Implementazione di servizi orientati alla connessione

In questo caso si ha bisogno di un circuito virtuale che evita di scegliere una nuova strada per ogni pacchetto inviato: prima di stabilire una connessione fra sorgente e destinazione, si *imposta tutto il percorso che devono seguire i pacchetti*, e questo percorso viene *archiviato nelle tabelle di routing*. In questa tipologia di servizio i pacchetti contengono un ID che indica il circuito virtuale di appartenenza; anche in questo caso l'esempio è **MPLS** (*multi protocol label switching*), questo

protocollo viene sempre più utilizzato perché garantisce la qualità del servizio e gestisce al meglio il traffico.

5.1.5 Confronto tra reti a circuito virtuale e reti datagram

Le reti a circuito virtuale abbiamo bisogno di una fase di configurazione iniziale per impostare il circuito; questa fase consuma sia tempo che risorse, però è facile capire che cosa si deve fare con il pacchetto. I router non fanno altro che andare a vedere a quale circuito virtuale appartiene il pacchetto per poi instradarlo.

Nelle reti datagram non si ha nessuna configurazione iniziale, per cui è più veloce, ma si ha bisogno di una procedura di ricerca più complicata: si ha bisogno dell'indirizzo completo sia della sorgente che della destinazione.

Domanda d'esame: “Qual'è il servizio di rete più veloce?”

“Le reti datagram”

Un altro problema è la quantità dello spazio richiesto nelle tabelle di routing: nelle reti datagram si ha bisogno di una voce della tabella per ogni possibile destinazione, invece nelle reti a circuito virtuale si ha una voce per ogni circuito virtuale, per cui si occupa meno spazio nelle reti a circuito virtuale.

I circuiti virtuali danno garanzie sulla qualità del servizio, perché tutte le risorse (banda, buffer...) possono essere riservati in anticipo, riuscendo ad evitare più facilmente congestioni. Invece nelle reti datagram è molto più difficile evitare la congestione del traffico.

Nel momento in cui ho un malfunzionamento del router, nelle reti a circuito virtuale se un router subisce un malfunzionamento tutto il percorso cade (disconnessione del percorso), invece nelle reti datagram non si ha nessun effetto perché i pacchetti verranno instradati su un altro router.

Le informazioni sullo stato della connessione nelle reti a circuito virtuale si ha bisogno di uno spazio nella tabella di routing per lo stato della connessione, viceversa per le reti datagram.

5.2 Algoritmi di routing

Il compito del livello di rete è inoltrare i pacchetti dalla sorgente alla destinazione; nella maggior parte delle reti i pacchetti compiono degli hops (salti). Gli algoritmi di routing sono la parte del software del livello di rete che devono scegliere su quali linea in uscita vanno inoltrati i pacchetti in arrivo: se la rete è di tipo datagram, questa decisione deve essere ripetuta per ogni pacchetto di dati in arrivo visto che il percorso migliore potrebbe variare nel corso del tempo, invece per le reti a circuito virtuale le decisioni vengono prese nel momento in cui si imposta il circuito virtuale (routing di sessione: il percorso rimane valido per tutta la sessione utente).

Routing: si occupa di riempire e aggiornare le tabelle di forwarding.

Forwarding: gestisce i pacchetti in arrivo, cercando all'interno delle tabelle di routing la linea di trasmissione per l'inoltro del pacchetto.

Bisogna cercare di raggiungere delle determinate caratteristiche per creare l'algoritmo di routing perfetto:

- Ottimizzazione.

- **Stabilità:** un algoritmo si dice stabile nel momento in cui raggiunge l'equilibrio, ovvero le tabelle di routing non variano più.
- **Semplicità.**
- **Robustezza:** far fronte ai cambiamenti topologici e del traffico.

Gli algoritmi di routing si dividono in 2 categorie:

- **Algoritmi non adattativi (statici):** non basano le proprie decisioni sulla stima del traffico, sulla topologia corrente, visto che i percorsi sono calcolati in anticipo e memorizzati nei router nel momento della connessione. L'altro è detto **routing statico**. Questi algoritmi non sono reattivi agli errori, questo routing è adottato quando si ha ben chiaro quello che si deve fare.
- **Algoritmi adattativi (dinamici):** cambiano le decisioni a seconda dell'andamento della topologia e del traffico. Si distinguono fra loro per la fonte da cui traggono informazioni che può essere un router adiacente oppure tutti i router della rete. Si distinguono nel momento in cui vanno a modificare i percorsi, ovvero ogni dt secondi oppure quando cambia la topologia. Si distinguono anche in base alla metrica utilizzata per scegliere il percorso ottimo.

5.2.1 Principio di ottimalità

Questo principio afferma che: se abbiamo un router intermedio j , sul percorso ottimale che collega il router i al router k , allora anche il percorso da j a k e il percorso da i a j sono ottimali.

La diretta conseguenza del principio di ottimalità è che la serie di percorsi ottimali che collegano tutte le sorgenti ad una data destinazione forma una struttura ad albero dove il nodo principale è costituito dalla destinazione. Spesso c'è conflitto tra la fairness e l'ottimalità: se c'è una non c'è l'altra.

5.2.2 Algoritmi statici

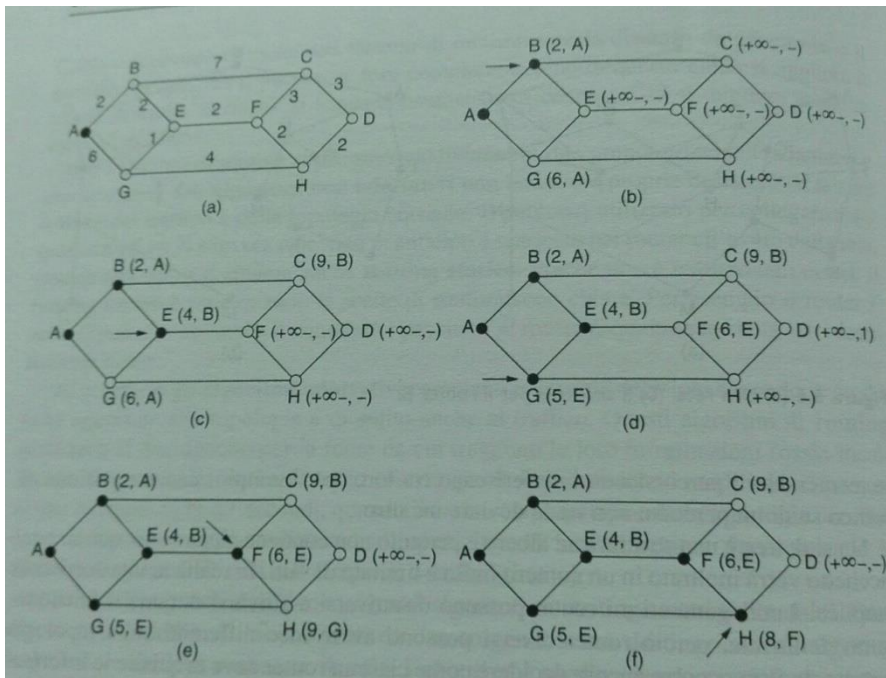
1 - Algoritmo di cammino minimo

L'idea alla base è di costruire un grafo della rete dove ogni nodo rappresenta un router e ogni arco una linea di comunicazione: nel momento in cui si va a scegliere il percorso fra una coppia di router l'algoritmo prende il cammino minimo che collega i due nodi. Per cammino minimo può essere misurato secondo diverse metriche:

- Numero di hop (salti)
- Basarsi sulla distanza geografica espressa in Km
- Ritardi di trasmissione
- Ecc...

L'algoritmo usato è quello di **Dijkstra**, che permette di trovare i cammini minimi tra la sorgente e tutte le possibili destinazioni della rete: ogni nodo ha un'etichetta, racchiusa tra parentesi, dove viene riportata la distanza dal nodo origine lungo il percorso migliore. Le distanze devono essere non negative, quindi il grafo ha solo pesi positivi, perché ci si basa su grandezze fisiche di natura positiva. Dato che all'inizio non si conosce la distanza fra nessun nodo della rete, tutti i nodi sono etichettati con $+\infty$. Le etichette possono essere provvisorie o permanenti, e all'inizio saranno provvisorie visto che dovranno aggiornarsi.

1. Il nodo sorgente è **contrassegnato come nodo permanente**.
2. Si esaminano tutti i nodi adiacenti al sorgente, andando ad etichettarli con la **distanza dal nodo sorgente**. Ogni volta che si etichetta un nodo si prende nota del nodo da cui è partita l'analisi.



3. Si analizzano i nodi adiacenti, andando a vedere le etichette per rendere permanente il nodo che ha la distanza minore dal nodo sorgente. Questo nuovo nodo permanente diventa il **working node**, ossia il nuovo nodo da esaminare.

4. Da qui si usa la **ricorsione sul working node**.

2 - Algoritmo di flooding

La scelta è locale, ossia abbiamo che i router effettuano la **sceita del percorso migliore in base alla conoscenza locale della rete e non totale**. Ogni pacchetto in arrivo viene **inviato su tutte le linee in uscita**, tranne su quella da cui proviene: questo meccanismo fa in modo che si vengano a generare tanto duplicati, dando luogo a problemi di traffico. Per mitigare questo problema potrebbero essere aggiunti dei contatori di salti nell'intestazione di pacchetti e decrementare il contatore ad ogni salto, in modo tale che **quando si raggiunge lo zero il pacchetto viene scartato**; di solito al contatore è assegnato il valore dell'intero diametro della rete.

Esiste una variante, il **flooding selettivo**, che viene impiegato in **operazioni militari perché si tratta di un algoritmo robusto**: si **accerta che il pacchetto viene consegnato a tutti i nodi della rete e richiede poco tempo per configurarlo** (deve conoscere solo i propri vicini e inviare a tutti il pacchetto in arrivo).

5.2.3 Algoritmi dinamici

Sono più efficienti di quelli statici e per questo sono i più utilizzati. Trovano il cammino minimo su tutta la topologia della rete.

1 - Routing basato sul vettore delle distanze

Questo algoritmo di routing fa in modo che ciascun router **conservi una tabella, chiamata vettore**, in cui viene inserita la migliore distanza conosciuta per ogni destinazione e il collegamento che conduce a quest'ultima. **Le tabelle vengono continuamente aggiornate** scambiando informazioni con i vicini, e alla fine ogni router sa qual è il **collegamento migliore per raggiungere qualsiasi destinazione**. Questo algoritmo è chiamato anche algoritmo di **Bellman-Ford**, che prevede a differenza di Dijkstra **pesi negativi**.

Domanda d'esame. "Perchè ci sono dei pesi negativi?"

Di solito la distanza è misurata in hop. La debolezza di questo algoritmo è che ha una **convergenza lenta**: reagisce bene e rapidamente alle buone notizie (ritardi da valori alti a piccoli), ma reagisce troppo lentamente con situazioni svenienti (da valori piccoli a valori alti di ritardo); un altro problema è la **divergenza** che si verifica nel momento in cui un router invia un pacchetto di aggiornamento ad un nodo vicino che subisce una failure, perciò il pacchetto gira all'infinito generando il problema del conteggio all'infinito. Per ovviare al problema si inizializza un timer che ha un valore massimo pari al diametro della rete: il contatore prende il nome di **time-to-live**, che si trova nell'intestazione del pacchetto IP. Gli altri metodi sono: usare dei timer di **hold-down**, che permettono ai router di rifiutare qualsiasi cambiamento apportato ai percorsi per un certo periodo di tempo, usare la regola **spleet-horizon** che afferma che un router non può inviare informazioni al router da cui le ha prese, usare la spleet-horizon-rule con **poison reverse** che prevede che se un router viene a conoscenza di un percorso irraggiungibile attraverso un'interfaccia lo deve far sapere attraverso la stessa interfaccia. Esistono 2 algoritmi di routing che utilizzano il vettore delle distanze:

- **Reep**: routing information protocol
- **Eigrp**: protocollo utilizzato da CISCO

Questo algoritmo è facile da implementare che da mantenere; inoltre richiede bassi requisiti di risorse.

2 – Routing basato sullo stato del collegamento

Questo algoritmo nasce per andare a migliorare il precedente, perché prima si impiegava troppo tempo per raggiungere la convergenza dopo che si cambiava la topologia della rete. Questo nuovo algoritmo ha due protocolli:

- **IS-IS**
- **OS-PF**

Struttura dell'algoritmo:

1. **Scoprire i propri vicini e i relativi indirizzi di rete**: per scoprire i vicini il router invia un pacchetto di "hello" su ogni collegamento punto-punto, il router che riceve hello risponde con il proprio nome; questi nomi devono essere unici a livello globale.
2. **Misurare la distanza da ogni vicino** (oppure si usa un'altra metrica): se la rete è molto estesa come metrica si prende il ritardo dei collegamenti in modo tale da favorire quelli con un ritardo minore. Per determinare il ritardo viene inviato un pacchetto di "echo", e l'altra parte deve rispondere immediatamente, in questo modo si misura il tempo impiegato per andare e tornare. Oppure nei collegamenti si possono usare metriche di distanza o di costo.
3. **Costruire un pacchetto che contenga tutte le informazioni raccolte**: bisogna avere tutti i dati necessari: ID del trasmittente, numero in sequenza, age e una lista di vicini. Viene riportato anche il ritardo misurato per ogni vicino. Questi pacchetti possono essere costruiti periodicamente oppure quando si verifica un evento particolare.
4. **Inviare il pacchetto a tutti gli altri router e ricevere i loro pacchetti**: è la parte più delicata perché i pacchetti devono essere distribuiti in modo rapido e affidabile. Se due router mantengono informazioni diverse sulla topologia della rete (uno dei due non è aggiornato), si potrebbero creare cicli, oppure punti irraggiungibili. Viene utilizzato il **flooding**. Il numero in

sequenza all'interno del pacchetto serve per tenere sotto controllo il flusso di dati e viene incrementato per ogni nuovo pacchetto inviato, quindi nel momento in cui arriva un pacchetto al router, questo va a confrontare i dati ricevuti con quelli in suo possesso: se il pacchetto è nuovo lo inoltra con il flooding, se è un duplicato viene scartato, se arriva un pacchetto con un numero in sequenza minore rispetto a quello visto fino a quel momento anche in questo caso si scarta il pacchetto perché è obsoleto. Però il flooding crea problemi: numeri di sequenza ripetitivi, un router se si dovesse bloccare perde traccia del suo numero di sequenza oppure il numero in sequenza potrebbe danneggiarsi. Per risolvere tali problemi bisogna includere l'età del pacchetto (age), nel momento in cui raggiunge lo zero le informazioni che provengono da quel router vengono scartate.

5. **Elaborare il percorso più breve verso gli altri router:** eseguire localmente l'algoritmo di Dijkstra verso tutte le possibili destinazioni.

Questo algoritmo viene utilizzato più spesso dell'altro, perché non ha problemi di convergenza

OS-PF: *open shortest path forwarding protocol*

È un protocollo molto potente, infatti è il più utilizzato anche se crea molto traffico, perché anche lui di base utilizza il flooding. Utilizza un routing basato sullo stato dei collegamenti e per questo motivo può bilanciare il carico; per gestire questo protocollo in maniera congrua si suddivide la rete in aree che contengono un sottoinsieme di router.

Tutto il sistema viene partizionato in sistemi autonomi, ed in questi sistemi ci sono le aree di routing: si utilizza un routing multicast.

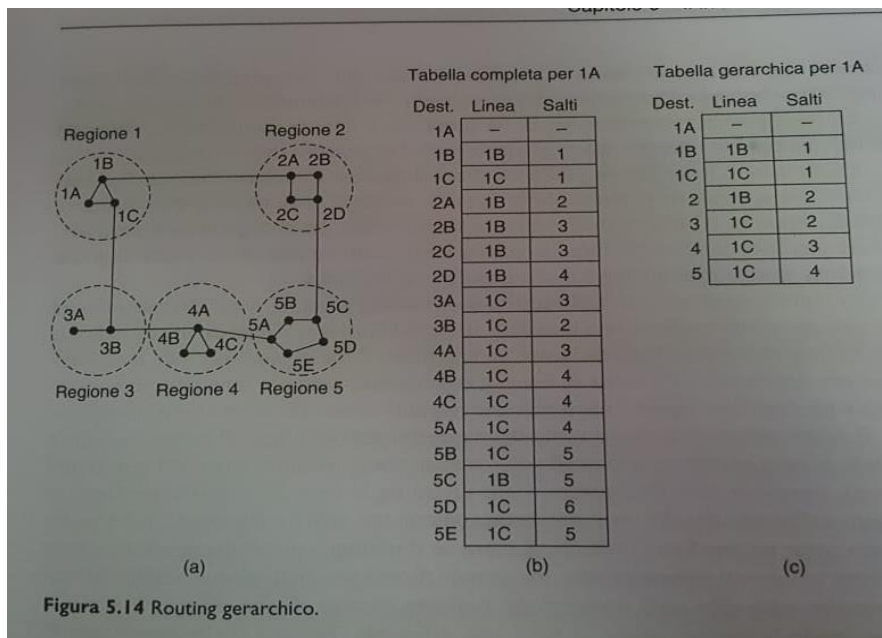
Struttura dei messaggi OS-PF

Abbiamo 5 diversi tipi di pacchetti OS-PF:

1. **hello:** permette di testare se un nodo è raggiungibile. Viene inviato ai router nel momento in cui vengono accesi; infatti i pacchetti OS-PF contengono una lista dei router raggiungibili.
2. **Link-state-advertisement (LSA):** fornisce informazioni sulla topologia della rete. Può essere utilizzato per ricavare il percorso più breve.
3. **Link-status-request (LSR):** questo pacchetto viene inviato ad un router per richiedere un aggiornamento sullo stato.
4. **Link-status-update(LSU):** risposta al LSR.
5. **Link-status-acknowledgement:** inviati per indicare che è stato ricevuto correttamente LSU. Se non si riceve questo ack vuol dire che c'è un problema nel collegamento.

5.2.4 Routing gerarchico

Quando si utilizza il routing gerarchico i router sono divisi in regioni: ogni router conosce tutti i dettagli dei pacchetti che si trovano all'interno dell'area, però non sa nulla della struttura interna delle altre regioni; questo fa sì che un router non è costretto a conoscere la topologia delle altre aree. La decisione di suddividere la rete in regioni, porta alle creazioni di tabelle di routing più piccole ma con percorsi più lunghi.



Se la rete diventa molto grande bisogna domandarci quanti sono i livelli della gerarchia. Il numero ottimale di livelli per una rete con N router è pari a $\log(N)$.

5.2.5 Routing Broadcast

Il routing broadcast è utile nelle applicazioni in cui gli host devono inviare messaggi a tutti. Un metodo di trasmissione broadcast è il flooding, che **permette di spedire il pacchetto a tutte le destinazioni**: in questo modo si spreca banda e obbliga la sorgente ad avere una lista di tutte le destinazioni. Una soluzione migliore è quella di adottare il **routing multi-destinazione**: in questo caso **ogni pacchetto contiene una lista con tutte le destinazioni** e nel momento in cui il router riceve un pacchetto va a **controllare tutte le destinazioni** per determinare le **linee di trasmissione richieste**, invece quando va ad inviare il pacchetto a più linee di trasmissione **mantiene nel pacchetto solamente le destinazioni della linea corrispondente**. In questo modo la **banda è usata in modo più efficiente**.

Reverse path forwarding

In questo caso se un pacchetto arriva attraverso la linea usata solitamente per inviare i pacchetti alla sorgente, probabilmente quello sarà un pacchetto che ha seguito il percorso ottimale e si tratterà della prima copia di quel pacchetto. Una volta appurato questo il router **inoltra la copia del pacchetto su tutte le linee a parte in quella da cui è arrivato**; in caso contrario il pacchetto viene scartato perché si tratta di un duplicato. È efficiente e facile da implementare.

5.2.6 Routing Multicast

Il routing multicast è utilizzato in quelle applicazioni in cui è **necessario mandare messaggi solamente ad un gruppo, significativamente piccolo se paragonato all'intera rete**: sia le soluzioni broadcast che punto-punto sono inutili per questa tipologia. Si ha il bisogno di gestire i gruppi di utenti al meglio.

Gli algoritmi utilizzati per il routing multicast si basano sulla riduzione dello spanning tree, per tagliare i collegamenti che non portano ai membri del gruppo. Possiamo ridurre lo spanning tree in 2 modi:

- Se si utilizza il **link state routing** come algoritmo, **ogni router conosce l'intera topologia della rete** e si rimuovono tutti i router che non appartengono al gruppo.
- Quando si utilizza il routing basato sul vettore delle distanze la riduzione avviene in questo modo: si usa il **reverse path forwarding**, se **un router vede che un host non appartiene al gruppo invia un messaggio di prune** qualora i router ricevano un messaggio multicast, ma non hanno host che appartengono a quel gruppo, il tutto ricorsivamente. Uno svantaggio è che i router svolgono **molti calcoli; non è il più appropriato con reti di grandi dimensioni**: se la rete

ha n gruppi, dove ogni gruppo ha m nodi, si devono memorizzare m spanning tree ridotti, quindi $n*m$ alberi.

5.2.7 Routing per host mobili

Questo routing viene utilizzato per mandare pacchetti ad host mobili: ora si ha il problema di capire dove si trova l'host mobile, quindi per farlo tutti gli host mobili hanno una *home location* (LAN di appartenenza) che non cambia mai; questi host hanno un indirizzo sulla LAN di appartenenza permanente che serve per determinare la home location. Il routing mobile consente di inviare pacchetti ad host mobili utilizzando gli **home address**.

Per prima cosa l'host mobile deve indicare dove si trova ad un altro host della LAN di appartenenza, e questo secondo host si chiama *home agent*. Nel momento in cui l'home agent conosce dove si trova l'host mobile, può inviargli i pacchetti.

5.3 Algoritmi per il controllo della congestione

La congestione si verifica quando nella rete sono presenti troppi pacchetti che portano al verificarsi di ritardi e alla perdita; il tutto causa un calo delle prestazioni. Il fenomeno della congestione si verifica all'interno della rete e sarà quindi compito del livello di rete capire come agire sui pacchetti in eccesso; il livello di rete e quello di trasporto collaborano per gestire la congestione, perché il modo più ovvio per limitare il numero di pacchetti che accedono alla rete è proprio ridurre il carico che il livello trasporto immette nella rete stessa.

Controllo di flusso: ha a che fare con il traffico tra il singolo trasmittente e un dato ricevente, il suo compito è evitare che una sorgente veloce trasmetta continuamente una quantità di dati maggiore di quella che il ricevente è in grado di assorbire.

Controllo della congestione: serve per garantire che la rete sia in grado di trasportare il traffico immesso. Si tratta di un problema globale che coinvolge il comportamento di tutti gli host e router.

5.3.1 Approcci al controllo della congestione

Se si verifica una congestione significa che il carico è maggiore di quello che può essere gestito dalle risorse di rete; per ridurre tale carico possono essere adottate due soluzioni differenti:

- Aumentare il numero di risorse.
- Diminuire il carico

Esistono delle metodologie che permettono di prevenire la congestione, mentre alcune agiscono nel momento in cui abbiamo già la congestione della rete: partendo dai metodi per il controllo della congestione più lenti (preventivi) possiamo dire che il modo più semplice per evitare le congestioni è costruire delle reti che si adattano al traffico che la rete può supportare. Gli amministratori di rete vanno ad aggiornare i collegamenti più usati, in modo tale che agiscono in base alle mutazioni della rete: si basano su quelle che sono delle stime a lungo termine del traffico. Questo metodo è chiamato **provisioning**.

Il secondo metodo è il **traffic-aware routing**, che si basa sul cambiamento di traffico che può avvenire durante la giornata (la notte il traffico è più basso..). Si cerca di alleggerire il carico quando alcuni collegamenti sono invece pesantemente utilizzati.

5.3.2 Traffic-aware routing

Il traffic-aware routing permette di **tener conto del carico della rete** quando si vanno a calcolare i percorsi in modo tale da dirottare il traffico lontano da quelli che sono i punti più trafficati che saranno i primi a subire congestioni. Per raggiungere questo obiettivo bisogna **fare in modo che i pesi dei cammini siano una funzione della banda e del ritardo di propagazione** (sono fissi, non cambiano nel tempo) più funzione del ritardo misurato (che è variabile nel tempo); tenendo conto di questi parametri si avrà che i percorsi con peso minore favoriranno i cammini con meno carico, poiché tutto il resto rimane invariato. In questo metodo c'è un problema: il traffico viene dirottato nei cammini meno trafficati, ma dopo un po' di tempo questi percorsi diventeranno i più trafficati, quindi si avrà uno cambio di linea dove il traffico verrà instradato in un altro percorso; questo avviene in continuazione. Si ha quindi un instradamento irregolare: una soluzione è di spostare il traffico lentamente.

5.3.3 Controllo di ammissione

È una tecnica molto utilizzata per evitare le congestioni, in pratica implica che **non si può stabilire una connessione se la rete non è in grado di supportare un traffico aggiuntivo**; il problema sta nel capire quando questa connessione può portare ad una congestione. Per poter applicare il controllo di ammissione i circuiti devono conoscere il traffico.

Questo metodo non va ad aumentare la capacità della rete, ma va a diminuire il traffico in base alla sua forma anche se varia in continuazione nel tempo: i picchi di traffico sono degli avvertimenti per una futura congestione. **Due modelli** che servono per cattura la forma del traffico per poi gestire la congestione sono il **leaky e token bucket**.

5.3.4 Limitazioni del traffico

in questo caso **si ha che la rete continua ad operare fino a poco prima che si verifichi la congestione**; nel momento in cui sta per avvenire la congestione si **avvertono i mittenti di ridurre** la trasmissione per iniziare a rallentare il tasso trasmissivo; questi avvertimenti avvengono mediante **feedback**.

La prima cosa che si deve fare è determinare quando sta per sopraggiungere la congestione: per avere questo dato si potrebbe o **controllare l'utilizzo dei collegamenti in uscita oppure lo stato del buffer dei pacchetti in coda** all'interno del router, oppure il **numero di pacchetti che vengono persi**. Quella più utile è **vedere lo stato del buffer**, perché nel caso dei pacchetti persi se ce ne sono tanti vuol dire che la rete è già congestionata, invece se si va a vedere l'uso medio delle linee in uscita è poco obbiettiva (un 50% potrebbe essere poco per un determinato traffico ma tanto per un altro tipo di traffico).

Nel momento in cui si verifica la congestione **vedremo che il buffer si riempie più velocemente** e bisogna tenere una stima del ritardo di accodamento dei pacchetti. La stima si fa con:

$$d_{\text{nuovo}} = \alpha * d_{\text{vecchio}} + (1-\alpha)*s$$

d_nuovo è la **nuova stima del ritardo di accodamento**

alfa è un parametro che determina la **velocità con cui il router dimentica la statistica recente**

d_vecchio è la stima del **ritardo di accodamento calcolata al passo precedente**

s è la **lunghezza della coda del buffer**

Questa formula prende il nome di **EWMA**, *exponentially weighted moving average* (media mobile esponenziale ponderata). Una volta determinata la stima andiamo ad avvisare le sorgenti di diminuire l'invio di pacchetti.

Esistono vari tipi di feedback:

- **Choke packet:** questo metodo consiste nella selezione da parte di un router di un pacchetto congestionato e l'invio all'host sorgente di un choke packet, in quest'ultimo è contenuta la destinazione trovata nel pacchetto congestionato. Comunque il choke packet è un pacchetto che viene inviato all'interno di una rete già congestionata, per questo motivo i router dovrebbero inviargli solo quando è necessario. Nel momento in cui la sorgente riceve il choke packet riduce il traffico verso la destinazione che era stata specificata nel pacchetto.
- **Notifica esplicita di congestione (ECN):** in questo caso invece di generare pacchetti aggiuntivi per avvertire della congestione il router etichetta i pacchetti inoltra nella rete (setta un bit a 1 nell'intestazione) per segnalare che sta avvenendo una congestione, in questo modo la destinazione basta che controlla il bit relativo alla congestione che si trova nell'intestazione e avviserà la sorgente di diminuire il traffico. Usato in internet.
- **Choke packet hop-by-hop:** viene utilizzato su lunghe distanze o ad alte velocità. Si ha che quando avviene la congestione su lunghe distanze avremmo un lungo ritardo a causa della lenta diffusione del segnale; con il choke packet tradizionale ci vuole troppo e lo stesso vale per ECN, quindi questo nuovo metodo ha effetto su tutti i salti: nel momento in cui il pacchetto raggiunge un router quest'ultimo riduce il flusso diretto ad un altro router, e così via. Ad ogni hop si dà sollievo immediato ad un router (diminuire il flusso router by router), fino a che il choke packet raggiunge la destinazione dove si ha un vero e proprio rallentamento del flusso: l'effetto è alleviare la congestione nel punto dove si forma, a discapito di un maggior utilizzo del buffer di trasmissione al passo precedente.

5.3.5 Load shedding

Il load shedding (eliminazione del carico) viene utilizzato quando nessuno dei metodi visti in precedenza riesce ad eliminare la congestione; si vanno ad eliminare dei pacchetti. Bisogna scegliere i pacchetti da eliminare, a seconda delle applicazioni: per il trasferimento dei file è più importante un pacchetto vecchio rispetto ad uno nuovo, mentre nei dati multimediali è vero il contrario.

Random early-detection

Anche conosciuto come **RED**, è un metodo per eliminare pacchetti nel momento in cui c'è la congestione. L'unica indicazione che abbiamo è vedere che si stanno perdendo un gran numero di pacchetti e dobbiamo fare in modo che i router vadano a scartare i pacchetti prima che la situazione degeneri: si utilizza il RED che serve per stabilire quando si è nel momento giusto per iniziare a scartare i pacchetti. I router mantengono una media mobile delle lunghezze delle code che viene comparata a due soglie, una minima e una massima, nel momento in cui la dimensione media della coda è minore della soglia minima allora non viene marcato nessun pacchetto, invece quando la dimensione è maggiore della massima vengono marcati tutti i pacchetti in arrivo; quando la media mobile su una linea supera una soglia di guardia quella linea è da considerarsi congestionata e una frazione dei pacchetti viene scartata in modo casuale; in questo modo è più probabile che vengano scartati i pacchetti che provengono dalle sorgenti più veloci. La sorgente si accorge che sono stati scartati dei pacchetti, perché non riceve ack, e quindi va a rallentare il suo traffico in uscita. Questo

algoritmo va a migliorare le prestazioni della rete anche se richiede una fase di inizializzazione complessa; infatti il migliore è ECN.

5.4 Qualità del servizio

Alcune applicazioni hanno bisogno di garanzie di prestazioni migliori rispetto a quelle che riescono a fornire gli algoritmi visti fino adesso, come per esempio tra queste applicazioni troviamo quelle **multimediali** che hanno bisogno di **una capacità minima e richiedono un ritardo massimo**. Dobbiamo capire come fornire la qualità del servizio richiesta da queste applicazioni: la soluzione più semplice è costruire una **rete che abbia una capacità sufficiente per gestire qualsiasi traffico (overprovisioning)**, però questa procedura si basa su delle stime e quindi è **poco affidabile**. Per garantire la qualità del servizio bisogna rispettare 4 punti:

1. **Di che cosa le applicazioni hanno bisogno** dalla rete.
2. **Regolare il traffico** che entra in rete.
3. **Prenotare le risorse dei router** per garantire le prestazioni.
4. **Capire se la rete può accettare più traffico**.

5.4.1 Requisiti delle applicazioni

Ogni flusso di pacchetti che va da una sorgente ad una destinazione ha bisogno di soddisfare 4 parametri:

- **Affidabilità:** Le diverse applicazioni avranno dei requisiti più o meno rigidi riguardo l'affidabilità, per esempio applicazioni quali: **posta elettronica, file sharing, accesso web e terminale remoto** hanno bisogno che **tutti i bit vengano consegnati correttamente** (prevede ritrasmissioni); invece per applicazioni **audio/video** si tollerano perdite di dati.
- **Ritardo:** Per le applicazioni che trasferiscono **file, posta elettronica e video** non sono sensibili al ritardo, a differenza delle **applicazioni interattive** sono molto più sensibili al ritardo; chiaramente le più restrittive al ritardo sono le applicazioni di **telefonia e videoconferenze**.
- **Jitter:** Il jitter è la **variazione del ritardo**, quindi **posta elettronica, file sharing e accesso web** non sono sensibili al jitter, mentre il **log-in da remoto e soprattutto audio/video** sì. Per ridurre il Jitter si può usare un **buffer a lato ricevente**.
- **Banda:** **posta elettronica, audio e log-in da remoto** ne richiedono poca, mentre **file sharing e video** ne richiedono tanta.

Questi parametri vanno a determinare la qualità del servizio richiesta dal flusso.

5.4.2 Traffic shaping

Prima di fornire delle garanzie sulla qualità del servizio la rete deve sapere qual'è il traffico da garantire, anche se quest'ultimo è irregolare con dei picchi; chiaramente ciò ne aumenta la difficoltà di gestione. Vengono usati meccanismi di **traffic shaping** per **regolare il traffico** irregolare: **fare in modo che le applicazioni possono trasmettere il traffico secondo le proprie necessità**, inclusi i picchi, ma in qualche modo si va a regolare il traffico, ovvero succede che l'utente e la rete si mettono d'accordo su **che forma dare al traffico e finché l'utente si attiene all'accordo** vengono inviati solo i **pacchetti che rientrano nell'accordo stesso**. In questo modo si **riduce la congestione**, però la rete **deve**

controllare periodicamente che l'utente rispetti i patti iniziali, perché se così non fosse andrà a **scartare i pacchetti di troppo**. Questo meccanismo di controllo si chiama **traffic policing**. Entrambi i meccanismi, traffic shaping e traffic policing, sono importanti per applicazioni real-time perché queste hanno restringenti requisiti di qualità di servizio.

Leaky e Token bucket

Per capire il **Leaky bucket** si considera l'analogia con un **secchio bucato sul fondo**: si può introdurre una certa quantità d'acqua e acqua uscirà con velocità costante. **Se il secchio si riempie l'acqua aggiuntiva verrà persa**; questa idea del secchio viene applicata ai pacchetti che rappresentano l'acqua. Gli host sono collegati alla rete tramite un'interfaccia che contiene un leaky bucket: se vogliamo immettere un pacchetto nella rete **dobbiamo poter inserire dei pacchetti nel secchio**, ma se il secchio è pieno dei pacchetti andranno persi. Per quanto riguarda il Token bucket immaginiamo l'interfaccia di rete sempre come un secchio che si riempie, però a differenza del Leaky **è il secchio proprio a riempirsi con rate costante** (visto che nel Leaky erano i pacchetti ad essere trasmessi a rate costante): per inviare un pacchetto bisogna **prelevare il token dal secchio, che poi verrà distrutto**. Nel momento in cui si hanno dei pacchetti in coda e non si hanno token sufficienti all'interno del secchio, i pacchetti che riescono **prelevano i token disponibili**, mentre gli altri devono aspettare che vengano generati nuovi token. Una volta che si è trasmesso al massimo rate, quando il secchio si svuota i pacchetti andranno a prelevare i token ad un **rate pari alla generazione dei token stessi**.

Entrambi gli algoritmi vanno a spalmare il traffico nel corso del tempo, vista la difficoltà nel gestirlo tutto insieme.

5.4.3 Scheduling dei pacchetti

per avere delle buone prestazioni la cosa migliore da fare è andare a **prenotare risorse sufficienti** lungo tutto il percorso che porta il **pacchetto dalla sorgente alla destinazione**; quindi bisogna andare ad **impostare un circuito virtuale** che i pacchetti appartenenti ad un determinato flusso di dati devono seguire. Gli algoritmi di scheduling dei pacchetti vanno ad **allocare le risorse dei router tra i pacchetti di un flusso**. Si possono prenotare 3 diversi tipi di risorse:

- **Banda**: **non andare a sovraccaricare le linee** in output.
- **Spazio nei buffer**: se non è disponibile spazio nel buffer il **pacchetto viene scartato**, per cui si **dedicano dei buffer** per un flusso specifico (real-time...).
- **Cicli di CPU**: i router **possono elaborare un certo numero di pacchetti al secondo**, quindi bisogna **accertarsi che la CPU non si sovraccarichi** ma al contempo i **pacchetti devono essere elaborati velocemente**.

Algoritmo FIFO

In questo algoritmo ogni router mantiene nel **buffer i pacchetti in coda** per ogni linea di uscita finché i pacchetti non possono essere inviati, e nel momento dell'inoltro verranno **prelevati in base all'ordine di arrivo**. Se i router ha la **coda del buffer piena** (c'è solo una coda), **allora si perdono dei pacchetti**; questo algoritmo è **molto semplice da implementare**, ma **non fornisce una buona qualità di servizio**. Non raccomandato per applicazioni audio/video, e inoltre non dà priorità ai vari tipi di traffico.

Accodamento con priorità

Si hanno 4 code di traffico all'interno del router, e ciascuna di esse è etichettata come coda ad alta priorità, media priorità, normale priorità o bassa priorità: chiaramente verranno elaborati prima i pacchetti con priorità più elevata, ignorando gli altri. Il problema è la starvation.

Customer queuing

È un accodamento equo, perché viene usato il metodo Round-Robin: si hanno 16 code, dove ognuna di esse ha un determinato traffico, si inizia svuotando la prima coda e poi in ordine RR si procede fino all'ultima. Questo metodo va ad evitare la starvation, ma non dà garanzie sul ritardo: poco raccomandato per audio/video.

Accodamento equo ponderato (WFQ)

Questo algoritmo permette di non dare la stessa priorità a tutti i flussi, ma permette di gestirla al meglio soprattutto per i flussi con maggiori esigenze; viene impiegato molto nella pratica, usando due metodi per cestinare i pacchetti in eccesso:

- Early dropping, scarto dei pacchetti anticipato. I pacchetti vengono eliminati non appena si raggiunge la soglia di congestione.
- Aggressive dropping, avviene quando si eliminano pacchetti che provengono da flussi aggressivi (flussi che occupano tutto lo spazio nel buffer).

Class Based WFQ

WFQ basato sulle classi: invece di basare l'accodamento sui flussi lo basa sulle classi, definite in precedenza (256 classi di traffico). In questo caso non si ha garanzia di ritardo ma garanzia di banda, e se abbiamo del traffico che non appartiene a nessuna classe questo viene inserito all'interno della classe di default. Poco raccomandato per audio/video, ma è il migliore degli algoritmi visti fino adesso.

LLQ

Permette ai dati sensibili al ritardo di avere un trattamento preferenziale e di essere inviati prima di altri pacchetti.

5.4.4 Controllo di ammissione

L'utente deve verificare che un flusso di pacchetti rispecchi i parametri della qualità del servizio, quindi va a decidere se accettare o rifiutare il flusso in arrivo in base alle capacità della rete; se accetta il flusso deve riservargli le risorse necessarie, per cui su tutti i router del percorso vanno allocate le risorse necessarie per trasmettere il flusso. Se le risorse non vengono prenotate può succedere che i router lungo il percorso si congestionano: per cui si divide il traffico uniformemente nei router per non causare congestioni. I flussi devono essere descritti accuratamente per poter negoziare i parametri per gestirli al meglio; si vanno a definire le **specifiche di flusso**. In queste specifiche vengono inserite delle informazioni riguardanti il flusso, poi vengono propagate nei router che modificheranno i parametri all'interno della specifica a seconda di quante risorse si hanno a disposizione, andando a stabilire se ridurre il flusso o meno. I parametri sono:

1. Tasso di token bucket: indica il massimo tasso di trasmissione della sorgente.
2. Dimensione del token bucket: picco massimo che può essere trasmesso in poco tempo.

3. Tasso di picco: indica la velocità di trasmissione massima, che non deve mai essere superata dal trasmittente.
4. Dimensione minima del pacchetto.
5. Dimensione massima del pacchetto.

5.4.5 Servizi integrati

I servizi integrati nascono per gestire i flussi di pacchetti, affrontando il problema di fornire la qualità del servizio alle applicazioni gestendo le risorse all'interno dei router. Per gestire i flussi i servizi integrati si basano su:

- **Classificazione**: si stila una tabella con la tipologia di flussi che possono entrare in rete. Viene identificato un flusso particolare utilizzando gli indirizzi IP della sorgente e della destinazione, i numeri di porta UDP e TCP di sorgente e destinatario e il numero di protocollo IP.
- **Scheduling**: una volta che i flussi sono stati classificati vanno gestiti. Si fa in modo che vengono allocate le risorse necessarie: i flussi possono essere mappati in classi, in modo tale da gestire flussi simili allo stesso modo.
- **Controllo di ammissione**: per fornire garanzie a ciascun flusso si effettua il controllo di ammissione ad ogni hop, per assicurarsi che tali risorse siano disponibili. Il controllo delle risorse avviene attraverso un protocollo.

RSVP

Resource reservation protocol, è il protocollo usato per controllare la disponibilità delle risorse all'interno della rete. Si occupa delle prenotazioni delle risorse, e permette a più sorgenti di trasmettere a diversi gruppi di destinatari e questi ultimi possono cambiare canale liberamente. Le prenotazioni sono unidirezionali, se sono richieste delle prenotazioni bidirezionali allora dobbiamo stabilire due RSVP, una per ogni direzione. Il protocollo utilizza il routing multicast, quindi nel momento in cui si vuole trasmettere ad un gruppo, la sorgente inserirà l'indirizzo del gruppo nei suoi pacchetti, inviandoli attraverso una struttura spanning-tree. Questo protocollo viene usato per eliminare le congestioni e ottimizzare l'uso della banda: per far fronte al primo punto i destinatari inviano un messaggio di prenotazione alla sorgente propagato attraverso il reverse path forwarding, per cui ad ogni hop il router tiene conto di quanta banda riservare a seconda di quanto è riportato all'interno della prenotazione e se non dispone di banda sufficiente allora si comunica il fallimento. È un protocollo *soft state*, ovvero le prenotazioni scadono se non refreshate, inoltre si adatta ai cambiamenti topologici della rete.

5.4.6 Servizi differenziati

Nascono perché i servizi integrati offrono una buona qualità del servizio ma hanno un lato negativo, perché dobbiamo impostare preventivamente ogni flusso, e naturalmente in sistemi con milioni di flussi sorgono dei problemi, inoltre all'interno dei router viene mantenuto uno stato interno per ogni flusso per cui se un router si rompe perdo tutti i dati dei flussi memorizzati al suo interno. Per questo nascono i servizi differenziati che mirano ad un approccio locale dei router, basando la qualità del servizio sulle classi (e non sui flussi): vengono offerti da un insieme di router che vanno a costituire un vero e proprio dominio amministrativo, si occupano di definire le classi di servizio e le regole per inoltrare i pacchetti appartenenti ad una data classe. I punti focali sono:

- **Classificazione:** viene classificato in base alle classi. I router di confine hanno l'obbligo di classificare e marcare i pacchetti, così come ha deciso il gestore, e sono proprio loro a marcare e classificare i pacchetti. Mentre i router in uscita si occupa di togliere la marcatura al pacchetto.
- **Marcatura DSCP:** i pacchetti vengono marcati utilizzando il DSCP (differential service code point), che viene inserito nel campo differential service all'interno dell'intestazione del pacchetto IP. Il DSCP indica a quale classe appartiene il pacchetto.
- **Comportamento per hop:** le classi vengono definite tramite un comportamento per hop. Significa che il pacchetto viene descritto ad ogni salto, e non su tutta la rete.

I servizi differenziati sono i più usati, anche in internet, e permette di assicurare le risorse ad ogni classe per ogni salto. Ad ogni salto abbiamo 4 comportamenti:

1. **Expedited forwarding:** inoltro accelerato, in questo comportamento sono disponibili 2 classi di servizio ed i pacchetti sono marcati come appartenenti a queste 2 classi di servizio: regolare e accelerata. La maggior parte del traffico dovrà essere considerato come regolare, mentre solo una piccola parte è marcata come accelerata; i pacchetti accelerati devono attraversare la rete come se quelli regolari non esistessero. Utile per applicazioni VoIP.
2. **Assured forwarding:** in questo caso si hanno 4 classi di priorità. Classe oro, argento, bronzo e una senza nome (senza garanzia di qualità di servizio); oltre alle 4 classi si hanno 3 tre classi di eliminazione che servono per i pacchetti soggetti a congestione: bassa, media e alta. La sorgente genera dei pacchetti, che vengono classificati in una delle 4 classi di priorità dal router posto all'ingresso della rete, poi si determina la classe di scarto per ogni pacchetto utilizzando un token bucket. Si vanno a combinare per ogni pacchetto la classe di priorità e quella di scarto, poi si usa un algoritmo di scheduling che va ad assegnare il peso in base alla priorità (maggiore a quella oro), facendo in modo di dare a tutti la banda. Se all'interno di una classe di priorità si ha una combinazione con una classe di scarto alta, i pacchetti si eliminano con l'algoritmo RED.

5.5 Internet working

Con internetwork si intende il collegamento tra due o più reti, naturalmente si possono collegare fra loro reti molto differenti (Ethernet – reti satellitari). Le reti possono differire per diverse caratteristiche.

5.5.1 Differenze tra le reti

Quando i pacchetti vengono inviati dalla sorgente alla destinazione si possono presentare dei problemi, perché la sorgente deve essere in grado di definire l'indirizzo della destinazione, infatti ci dobbiamo chiedere se la sorgente e la destinazione si trovano su reti differenti, ad esempio Ethernet e WiMax.

Elemento	Alcune possibilità
Servizio offerto	Orientato alla connessione oppure no
Indirizzamento	Diverse dimensioni, lineare o gerarchico
Trasmissione broadcast	Presente o assente (come pure il multicast)
Dimensione del pacchetto	Ogni rete ha un suo valore massimo
Ordinamento	Consegna ordinata oppure no
Qualità del servizio	Presente o assente, molti tipi diversi
Affidabilità	Vari livelli di perdita delle informazioni
Sicurezza	Regole di riservatezza, cifratura dei dati e altro
Parametri	Scadenze diverse, specifiche di flusso e altro
Contabilizzazione dei costi	Per tempo di connessione, pacchetto, byte o nessuna

figura 5.38 Alcuni dei molti modi in cui le reti possono essere diverse.

5.5.2 Come connettere le reti

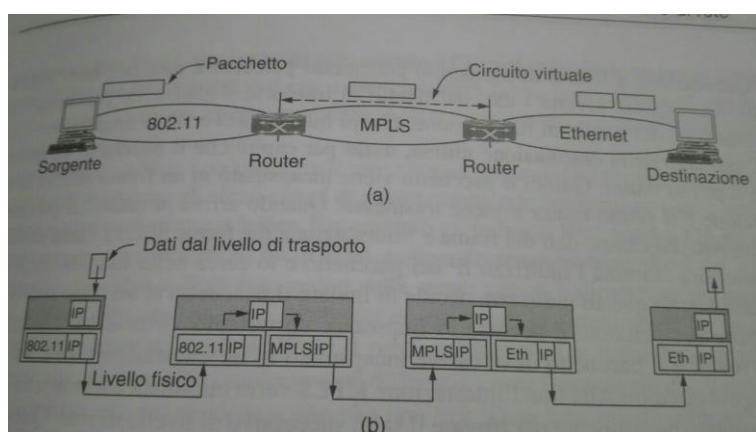
Per connettere le reti possiamo pensare a due scelte plausibili:

1. Costruire dei dispositivi che vanno a convertire i pacchetti da un tipo di rete ad un altro.
2. Costruire un livello comune al di sopra delle reti diverse.

Il protocollo IP fornisce un formato universale ai pacchetti, che viene riconosciuto da tutti i router su tutte le reti; proprio da qui si comincia a capire come connettere fra loro reti diverse.

Prendiamo in esame il seguente esempio: abbiamo un internetwork composta da una rete 802.11, MPLS e Ethernet, quindi una macchina sorgente che si trova sulla rete 802.11 vuole inviare un pacchetto ad un host che si trova in Ethernet, però in mezzo ci sta MPLS. Il problema sta nel fatto che 802.11 fornisce un servizio non orientato alla connessione, mentre MPLS sì, quindi il pacchetto ha bisogno di un circuito virtuale che deve essere impostato; nel momento in cui arriva in Ethernet può capitare che abbia una dimensione che esce dagli standard, per cui viene suddiviso in frammenti

spediti separatamente che quando raggiungono la destinazione vengono riassemblati. Il percorso che il pacchetto segue si basa su un'instestazione comune a tutti i pacchetti, IP, e poi si aggiungono le varie intestazioni dei diversi protocolli a secondo della rete che sta attraversando. Come si può vedere dall'immagine vengono impiegati i router per estrarre dal frame l'indirizzo di rete del pacchetto, al contrario di quello che avverrebbe con gli switch, dove l'intero frame viene trasportato sulla base dell'indirizzo MAC. I



router in grado di gestire reti multiple sono chiamati **router multi-protocollo**.

5.5.3 Tunneling

Connettersi sue reti diverse molto difficile, ma esiste un caso particolare che è più facile da gestire: ossia il caso in cui una sorgente e una destinazione di trovano sullo stesso tipo di rete, ma sono separati da una rete differente. La soluzione a questo problema è il **tunneling**: permette di creare un percorso che si estende da un router multi-protocollo ad un altro.

Esempio:

Si ha un host che costruisce un pacchetto contenente l'indirizzo IPv6 della destinazione; lo invia al router multi-protocollo che collega la sua rete (IPv6 anch'essa) alla internet IPv4 che si trova in mezzo alle due. Il router prende il pacchetto IPv6 e lo incapsula nell'intestazione di un pacchetto IPv4, infine lo invia ad un altro router multi-protocollo connesso alla rete IPv6 della destinazione; quando arriva si estrae il pacchetti IPv6 contenuto nell'intestazione e mandato a destinazione.

L'unico svantaggio del tunneling è che solo la sorgente e la destinazione possono vedere il pacchetto, ma comunque questa tecnica è molto usata nelle VPN.

5.5.4 Routing tra reti distinte

Le reti possono utilizzare algoritmi di routing differenti, quindi gli operatori di rete devono capire le politiche adottate da ogni rete; bisogna tenere in considerazione che ogni rete è gestita in modo indipendente, per questo si chiamano **AS** (*autonomon system*)

5.5.5 Frammentazione dei pacchetti

Ogni rete supporta una dimensione massima dei pacchetti, che dipende da diversi fattori: hardware, sistema operativo, protocolli usati, ecc. I progettisti non hanno libero arbitrio per la scelta della dimensione. Il problema si ha quando un pacchetto grande viaggia su una rete che prevede pacchetti molto più piccoli: per prima cosa si potrebbe evitare di spedire il pacchetto, ma è una soluzione poco pratica e limitativa. Si può risolvere il problema dividendo i pacchetti in frammenti, che verranno inviati separatamente come normali pacchetti a livello di rete; il **riassemblaggio** non è poi così intuitivo. Vengono adottate due strategia per la ricostruzione dei pacchetti:

- Quando un router riceve un pacchetto troppo grande lo divide in frammenti che verranno indirizzati tutti allo stesso router in uscita. In questo router i pacchetti verranno riassemblati così le reti successive saranno ignare della frammentazione avvenuta in precedenza; si parla di frammentazione trasparente. Ci sono problemi: il router in uscita deve sapere quando ha ricevuto tutti i frammenti, i percorsi dei frammenti sono vincolati (introducendo un calo delle prestazioni) e il tempo di elaborazione per memorizzare nel buffer i frammenti è molto alto.
- Una volta che il pacchetto viene suddiviso dal router tutti i frammenti passano attraverso gli altri router intermedi come se fossero il pacchetto originale, e poi la ricostruzione viene effettuata dalla destinazione. Il vantaggio è che i router devono fare molto meno lavoro. I frammenti devono essere numerati per poter ricostruire la sequenza corretta e possono essere memorizzati nel buffer di destinazione (anche non in ordine); quando il pacchetto passa dentro una rete più piccola può essere frammentato di nuovo. Però si ha un overhead maggiore e viene perso un intero pacchetto anche se solo un solo frammento non arriva a destinazione.

È meglio liberarsi della frammentazione nella rete e spostarla a livello di host: questo processo prende il nome di **Path-MTU-Discovery** (identificazione della massima unità trasferibile sul percorso). Nel momento in cui viene inviato un pacchetto IP nell'intestazione viene settato a 1 il campo "don't

fragment”: significa che durante il percorso nella rete il pacchetto non può essere frammentato e se il router riceve un pacchetto troppo grande va a generare un messaggio di errore che invia alla sorgente. Quando la sorgente riceve l’errore, frammenta il pacchetto in modo tale che i frammenti possono essere gestiti dal router; se durante il percorso si incontrerà un router con un MTU ancora più piccola, questo processo verrà iterato. Lo svantaggio è dato dal ritardo iniziale, ossia il tempo di elaborazione della frammentazione, però è il migliore.

5.6 Livello di rete in Internet

Sono stati elencati i principi alla base del livello di rete in internet:

- Assicurarsi del **corretto funzionamento**: fare in modo che il progetto finito sia quello definitivo.
- Mantenerlo **semplice**: tutte le funzionalità non necessarie devono essere eliminate.
- **Scelte chiare**.
- Sfruttare la **modularità**: i protocolli di ciascun livello devono essere indipendenti.
- **Etereogenerità**: gestire reti di grandi dimensioni e diverse fra loro.
- **Evitare parametri statici**.
- **Non puntare alla perfezione**.
- Essere **rigorosi nella trasmissione** e **tolleranti nella ricezione**.
- **Scalabilità**: distribuire le risorse il più possibile.
- **Costi e prestazioni**.

Internet può essere vista come un insieme di reti interconnesse fra loro (AS); non si ha una vera e propria struttura ma si hanno molte dorsali alle quali ci si collega se si vuole raggiungere il resto di internet. Attaccati alle dorsali ci sono gli ISP, che forniscono l’accesso a internet; il protocollo a livello di rete utilizzato, che tiene insieme tutta la struttura è l’IP: permette di trasportare i datagrammi senza dover tener conto della posizione delle macchine. La comunicazione in internet funziona in questo modo: si ha che il livello di trasporto prende i flussi di dati e li divide in modo tale da passarli al livello di rete, per poi essere inviato sotto forma di pacchetti IP. Di solito i pacchetti IP possono raggiungere i 64 Kbyte, però di solito non superano i 1500 Byte perché così vengono inseriti in frame Ethernet.

5.6.1 Protocollo IPv4

Datagramma: sono i pacchetti inviati all’interno delle reti datagram, ovvero quelle che seguono un percorso arbitrario tra i router. I datagrammi non hanno bisogno di ack.

Il datagramma IPv4 è un indirizzo a 32bit costituito da:

- **Intestazione**: fissa a 20Byte:
 1. **VERSION**, tiene traccia della versione del protocollo (in questo caso si la versione 4).
 2. **THL**, indica la lunghezza dell’intestazione.

3. **SERVIZI DIFFERENZIATI**, utilizzato per distinguere le classi di servizio utilizzate. 6 bit.
4. **TIME-TO-LIVE**, indica il tempo di vita del pacchetto (255 sec). Quando è zero il pacchetto viene scartato, così non viaggia all'infinito.
5. **DON'T FRAGMENT**.
6. **PROTOCOL**, processo di trasporto in attesa dei dati (TCP, UDP).
7. **HEADER-CHECKSUM**, trasporta informazioni vitali e ha bisogno del checksum per rilevare errori. Deve essere calcolato ad ogni salto.
8. **INDIRIZZO SORGENTE**.
9. **INDIRIZZO DESTINAZIONE**.
10. **TIME-STAMP**, indica la data e ora di elaborazione del pacchetto.

- **Payload**: corpo del pacchetto (max 6Mbyte).

4Bytes	Ver	IHL	TOS.	Total length	
4Bytes	Identification			Flag	Fragment offset
4Bytes	TTL		Protocol	Checksum	
4Bytes	32 bits Source Address				
4Bytes	32 bits Destination Address				
	IP Options				Padding

5.6.2 Indirizzi IP

Gli indirizzi IP sono lunghi 32bit, sia lato sorgente che destinazione, e non si riferiscono ad un host specifico (inteso come macchina) ma alla scheda di rete: nel momento in cui un host è collegato a due reti differenti, deve avere 2 indirizzi IP.

Prefissi

Al contrario degli indirizzi Ethernet quelli IP sono gerarchici, quindi si ha una parte dell'indirizzo che può essere di lunghezza variabile ed è dedicata alla rete, mentre gli ultimi bit è dedicata agli host: quando più host appartengono ad una stesa rete, l'indirizzo di rete è uguale per tutti, ciò significa che una rete corrisponde ad un blocco di indirizzi IP continui, chiamato prefisso.

Gli indirizzi di rete vengono scritti in notazione decimale contata (es: 192.168.8.1), dove ognuno dei 4 byte è rappresentato da un numero che va da 0 a 255; gli indirizzi includono l'indirizzo dell'host e l'indirizzo della rete di appartenenza, per cui con un solo indirizzo si può trovare un dispositivo sulla rete. La lunghezza della parte dedicata alla rete viene scritta dopo uno slash '/', alla fine dell'indirizzo IP.

Maschera di sottorete – Subnet Mask

La subnet mask è una maschera binaria che va ad indicare la lunghezza del prefisso; per esempio se si ha l'indirizzo 255.255.255.255 la sua maschera di sottorete è pari a otto 1 per ogni 255 (in totale

32 1). La subnet mask ci permette di verificare dove finisce la parte di sottorete ed inizia quella dedicata all'host.

I vantaggi dell'indirizzamento gerarchico sono:

- I router inoltrano i pacchetti basandosi solo sulla parte che riguarda l'indirizzo.
- Solo quando i pacchetti arrivano a destinazione vengono inoltrati all'host corretto, mantenendo le tabelle di routing molto più piccole. Permette una buona scalabilità della rete.

Presenta però 2 svantaggi:

- L'indirizzo IP di un host dipende da dove quest'ultimo si trova all'interno della rete: ad esempio se un indirizzo ethernet può essere utilizzato in ogni parte del mondo, quello IP appartiene ad una rete specifica.
- La gerarchia va a sprecare indirizzi, dato che gli IPv4 stanno scarseggiando questo svantaggio ha un certo spessore.

Sottoreti

Le sottoreti nascono quando abbiamo reti grandi da gestire, quindi la soluzione adottata è quella di suddividere il blocco di indirizzi che appartengono alla stessa rete in più parti, per poterle utilizzare internamente come se fossero reti multiple, ma presentandole all'esterno come se fossero un'unica rete: questa procedura si chiama **subnetting**. Il prefisso viene suddiviso in più parti (perché è lui che indica la lunghezza della parte di rete); quando il pacchetto arriva al router principale bisogna sapere a quale pacchetto inviarlo: per cui il router guarda l'indirizzo di destinazione e la sottorete di appartenenza.

Indirizzamento per classi

Inizialmente gli indirizzi IP erano basati sull'indirizzamento per classi, ossia gli indirizzi IP venivano suddivisi in 5 categorie: A, B, C, D, E dove le classi A, B e C venivano usate per gli indirizzi degli host, mentre la classe D per il multicast e la classe E per usi speciali. I primi 8 bit nella subnet mask nella classe A rappresentavano sempre la rete, mentre i restanti erano riservati agli host; già da questo primo caso vediamo che la situazione non è bilanciata al meglio; si ha un ridotto numero di reti, ma un gran numero di host, per cui si possono avere 128 reti differenti e 16 milioni di host possibili (nella realtà il numero è troppo alto). La classe B considera i primi due ottetti per la rete e gli altri per gli host: si arriva a 16.384 reti possibili e sui 65 mila host. La classe C assegna i primi tre ottetti alla rete e solamente l'ultimo agli host, quindi si ha reti molto piccole, ma tante, con pochi host per ciascuna rete (max 255 host); questi indirizzi, sia B che C portano ad uno sbilanciamento. Per ovviare a questo problema nasce **CIDR** (classless interdomain routing), che va a diminuire la dimensione delle tabelle di routing; si può applicare lo stesso meccanismo delle subnetting viste in precedenza. I router che si trovano in posizioni differenti possono riconoscere un indirizzo IP come appartenente a prefissi di dimensione diversa: un router può trattare uno stesso indirizzo IP come parte di un blocco, per esempio che contiene 2 alla 10 indirizzi, mentre un altro router potrebbe trattarlo come parte di un blocco più grande (a seconda delle esigenze). Supponiamo di avere un blocco di 8192 indirizzi IP a partire da 194.24.0.0, una certa università necessita di 2048 indirizzi, quindi gli verranno assegnati indirizzi che vanno da 194.24.0.0 a 194.24.7.255; c'è anche una maschera che è 255.255.248.0 (la rete è 194.24.0 e la restante parte dell'indirizzo è assegnata agli host); un'altra università richiede 4096 indirizzi alla quale verranno assegnati indirizzi che partono da 194.24.8.0 e così via. Ad ognuna viene assegnato il blocco di indirizzi necessario con la relativa maschera. In pratica CIDR lavora in

questo modo: quando arriva un pacchetto si esamina la tabella di routing per **determinare se la destinazione si trova all'interno del prefisso**, e se abbiamo più voci con lunghezza del prefisso differenti che corrispondono fra loro, viene scelta **la voce con il prefisso più lungo**.

Indirizzi speciali

L'indirizzo IP **0.0.0.0** viene utilizzato dagli host al momento del boot, e significa **o questa rete o questo host specifico**; infatti tutto gli indirizzi che hanno 0 come numero di rete, si riferiscono alla rete corrente. L'indirizzo **1.1.1.1** viene usato per indicare tutti gli host di una rete: permette la trasmissione broadcast sulla rete locale. Gli indirizzi espressi nella forma **127.x.x.x** vengono usati per **fare controlli all'interno dell'host (loopback)**; i pacchetti diretti a questo indirizzo non sono indirizzati nel cavo ma sono elaborati localmente e trattati come pacchetti in arrivo.

NAT

Gli indirizzi IP sono scarsi, per questo si sono create delle tecniche per “riciclarli”: vengono assegnati indirizzi IP agli host in modo dinamico, quando ci si connette e una volta che la sessione è terminata verranno **riassegnati a nuovi host che si conatteranno in futuro**. Questa strategia funziona bene quando i pc non sono sempre connessi, ma non in ambienti aziendali dove devono essere sempre attivi: nasce la **NAT (network address translation)**, che si pone come **risolutore del problema**. Assegna ad ogni azienda/casa un **singolo indirizzo IP per il traffico di internet all'interno della rete locale**; nel momento in cui un pacchetto deve lasciare la rete locale per dirigersi verso l'ISP, viene seguita la **traduzione dell'unico indirizzo IP** usato nella rete locale **nell'unico indirizzo pubblico**. Vengono riservati degli **indirizzi** assegnati ad una casa/azienda, **che non potranno mai comparire su internet**, fra questi l'intervallo più usato è quello che va da **10.0.0.0 a 10.255.255.255/8** perché permette di **gestire il maggior numero di host**. All'interno di un'azienda, **un host ha un unico indirizzo, espresso nella forma 10.x.y.z** e quando il pacchetto lascia la rete locale **passa attraverso un apparato NAT**, che converte l'IP interno nel vero IP assegnato all'azienda; di solito alla **NAT si abbina un firewall per proteggere la rete locale**. Dobbiamo vedere qual'è l'indirizzo di destinazione del pacchetto: **dato che la maggior parte di pacchetti IP** trasporta al suo interno **TCP o UDP**, ed entrambi contengono **sia la porta sorgente che quella di destinazione**, quando viene consegnato il pacchetto si va a vedere la porta sorgente che indica dove devono essere consegnati i pacchetti in arrivo, mentre quella di destinazione determina chi deve ricevere i pacchetti. **La NAT viola la gerarchia del modello IP**, perché in questo modo **IP non è in grado di identificare in modo univoco un singolo host**, inoltre i pacchetti in entrata non possono essere accettati se non dopo quelli in uscita e la NAT **trasforma internet in una rete orientata la connessione** perché va a mantenere delle informazioni relative alla connessione (IP privato, porta TCP).

5.6.3 IPv6

Nasce perché gli **IPv4 stanno esaurendo** e l'unica soluzione è adottare indirizzi più grandi; infatti gli IPv6 sono **indirizzi a 128 bit**, però è difficile da implementare perché si differenzia da IPv4 e per questo non sono compatibili: ciò che proponeva di fare IPv6 era riuscire a **supportare miliardi di host**, **ridurre la dimensione delle tabelle di routing**, semplificare i protocolli (elaborare i pacchetti più velocemente), fornire una **sicurezza maggiore**, prestare **più attenzione alle applicazioni real-time**, **evolversi nel tempo**, **mantenere una continuità con IPv4**, permettere agli host di **sposarsi senza cambiare indirizzi** e aiutare il **multicasting**.

L'intestazione del pacchetto IP è stata ridotta a **7 campi**, migliorando il ritardo e il throughput; vengono supportate meglio le opzioni, incrementando la velocità di elaborazione: **La sicurezza fornita**

da IPv6 è maggiore, in quanto sia la sicurezza stessa che la privacy diventano importanti, inoltre si dà maggiore attenzione alla qualità del servizio. In questa versione non si ha più la modalità broadcast, solo multicast.

Intestazione IPv6

- VERSION: 6
- SERVIZI DIFFERENZIATI: distinguere le classi di servizio dei pacchetti. Usato nell'architettura servizi differenziati. Vengono usati i bit di ordine più basso per segnalare congestioni in atto.
- FLOW LABEL: consente alla sorgente e alla destinazione di marcare un gruppo di pacchetti che appartiene alla stessa classe, quindi trattate allo stesso modo.
- PAYLOAD LENGHT: quanti byte seguono l'intestazione.
- NEXT HEADER: indica il gestore del protocollo di trasporto (TCP, UDP) al quale va passato il pacchetto.
- HOP LIMIT: usato per impedire ai pacchetti di vivere per sempre all'interno della rete.
- INDIRIZZO SORGENTE 16byte
- INDIRIZZO DESTINAZIONE: 16byte

Per gli indirizzi IPv6 si ha una nuova notazione: 8 gruppi di 4 cifre esadecimali separati da due punti, 8000:0000:0000:0000:0123:4567:89AB:CDEF. Spesso ci possono essere numerosi zero consecutivi sono state utilizzate tre ottimizzazioni:

- Gli zeri iniziali di un gruppo possono essere omessi.
- Se uno o più gruppi contengono 16 bit a zero, possono essere sostituiti da una coppia di due punti.
- Permette di scrivere in notazione decimale dopo una coppia di due punti.

Il campo IHL è stato eliminato perché l'intestazione di IPv6 ha una lunghezza fissa, così come PROTOCOL; il più importante campo che è stato tolto è il CHECKSUM, perché diminuiva le prestazioni, visto che le reti di oggi sono affidabili e il controllo sono fatti nel livello data-link e di trasporto.

Intestazione estesa

Aggiunta perché alcuni campi di IPv4 vengono utilizzati, ed essa aggiunge informazioni aggiuntive: ne esistono 6 di queste intestazioni e ognuna di loro è opzionale.

5.6.4 Protocolli di controllo in internet

ICMP (*internet control message protocol*)

Questo protocollo nel momento in cui avviene qualcosa di imprevisto in internet comunica l'evento, e spesso viene usato per fare dei test.

ARP (*address resolution protocol*)

Questo protocollo controlla le operazioni di conversione da indirizzi IP a indirizzi MAC. Le schede ethernet non capiscono gli indirizzi internet e quindi ci dobbiamo chiedere in che modo gli indirizzi IP vengono associati a quelli data-link. Supponiamo che host01 trasmette attraverso la rete ethernet un pacchetto broadcast, e chiede chi è il proprietario di un certo IP; ogni macchina controlla il proprio indirizzo IP se corrisponde con quella richiesto da host01, naturalmente risponderà solo l'host che ha quel determinato indirizzo. Ora l'host01 scopre che quell'indirizzo IP è assegnato ad un certo host con un suo indirizzo MAC: il protocollo ARP entra in gioco qui per associare i due tipi di indirizzi, rendendo tutto più semplice evitando l'uso di file di configurazione.

DHCP (*Dinamic host configuration protocol*)

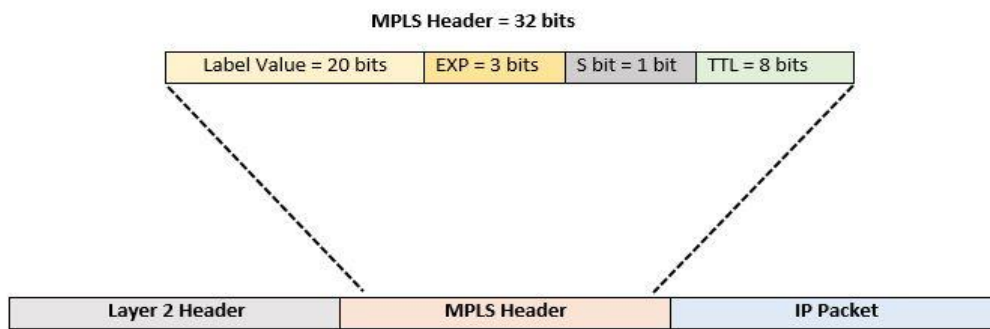
Sia Arp che altri protocolli assumono che gli host siano configurati con alcune info di base, per esempio l'indirizzo IP. Il DHCP viene usato per estrarre le informazioni contenute all'interno degli host: ogni rete deve avere un server DHCP responsabile della configurazione. Quando un pc viene avviato contiene al suo interno un indirizzo MAC nella sua scheda di rete, però non possiede un indirizzo IP ancora, quindi invierà sulla rete una richiesta broadcast attraverso un pacchetto chiamato DHCP_DISCOVER per richiedere l'indirizzo IP; questo pacchetto raggiungerà il server DHCP e a quel punto per quel determinato host verrà allocato un IP libero che viene inviato attraverso un pacchetto DHCP_OFFER. Si potrebbe verificare un problema perché si attinge ad una riserva comune di indirizzi IP e si deve stare attenti alla durata dell'allocazione (quanto tempo un IP è riservato ad una macchina); una volta terminata la sessione l'host deve restituire l'IP, altrimenti tale indirizzo verrà perso. Si adotta una tecnica chiamata **leasing**, che prevede di assegnare un indirizzo IP solo per un certo periodo di tempo.

5.6.5 Commutazione di etichetta MPLS (*multi protocol label switching*)

MPLS inizia ad essere sempre più utilizzata in internet: è una tecnologia per reti IP che permette di instradare flussi di traffico multi-protocollo tra nodo origine e destinazione tramite l'utilizzo di identificativi tra coppie di router adiacenti.

Davanti ad ogni pacchetto viene aggiunta un'etichetta, quindi i router inoltrano i dati in base a questa label aggiunta in precedenza, anziché tener conto dell'indirizzo di destinazione. Questa etichetta funge da indice associato ad una tabella interna, che serve per determinare la linea di output: l'inoltro può essere effettuato molto più velocemente perché basta eseguire la ricerca di questa etichetta. MPLS offre quindi un routing flessibile e veloce, improntato alla qualità del servizio.

Problema: capire dove posizionare l'etichetta, perché MPLS funziona come un circuito virtuale, ma sappiamo che internet offre un servizio non orientato alla connessione, per cui non sono strutture compatibili. Nell'intestazione IP non si hanno dei campi a disposizione per la numerazione dei circuiti virtuali, quindi si è aggiunta un'intestazione MPLS prima dell'intestazione IP.



Il formato del frame è composto dalle intestazioni PPP a livello 2, MPLS (tra 2 e 3), IP del livello 3 e infine TCP.

L'intestazione MPLS è lunga 4 byte ed ha 4 campi:

- **LABEL**: contiene l'indice.
- **EXP**: quality of service, indica la classe di servizio usata.
- **S**: accatastamento di più etichette.
- **TTL**: indica per quante volte ancora il pacchetto deve essere inoltrato.

Questo protocollo si trova fra il livello 2 e il livello 3. Le intestazioni MPLS non fanno parte del pacchetto a livello di rete né del frame a livello data-link, ma dipende comunque da entrambi; questa cosa rende possibile la costruzione dei switch MPLS in grado di inoltrare pacchetti IP e non (a seconda da quello che si presenta), ed è proprio da questa caratteristica che prende il nome di multiprotocollo. Quando un pacchetto ha l'etichetta MPLS raggiunge un router (chiamato LSR label switching router) l'etichetta viene usata come indice in una tabella per determinare la linea di trasmissione e la successiva etichetta da utilizzare, infatti ad ogni router si riassegna l'etichetta che indica la prossima linea di uscita (le etichette vengono assegnate ad ogni salto).

Dato che molti host e anche molti router non comprendono le etichette MPLS, bisogna capire quando bisogna attaccarle ai pacchetti. Ciò avviene quando i pacchetti IP raggiungono il confine della rete MPLS, dove si trova un router **LER** (*label edge router*) che guarda l'indirizzo della destinazione IP e tutti gli altri campi per vedere qual'è il percorso MPLS che il pacchetto deve seguire all'interno della rete ed una volta capito attacca l'etichetta in cima al pacchetto. Nel momento in cui raggiunge il bordo (ovvero l'altro confine), l'etichetta visto che ha svolto il suo compito può essere rimossa; ora il pacchetto IP che non ha più l'etichetta viene instradato verso un'altra rete.

È possibile raggruppare sotto una singola etichetta dei flussi di pacchetti che appartengono alla stessa classe di servizio, quindi devono essere trattati allo stesso modo; questa metodologia è chiamata **FEC** (*forwarding equivalence class*).

Campo 'S' dell'etichetta

Permette l'accatastamento di più etichette, se si hanno molti pacchetti con etichette diverse che vogliono seguire un percorso comune verso la destinazione, grazie a questo campo invece di settare un percorso specifico per ogni etichetta possiamo configurare solo un percorso a commutazione di

etichetta. Quando i pacchetti arrivano all'inizio del percorso si va a creare una pila di etichette, si segue il percorso descritto in precedenza e alla fine si rimuovono le etichette una dietro l'altra, e grazie al bit *s* si capisce quante etichette dovranno essere rimosse (*s* è 1 per l'etichetta più in basso, 0 per le altre).

5.6.6 BGP – Protocollo di routing per gateway esterni

È stato concepito per risolvere problemi economici, politici e di sicurezza nel routing di sistemi autonomi. I vincoli possono essere configurati manualmente nei router che sono collegati attraverso connessioni TCP; è un protocollo di tipo *distance-vector* e tiene traccia dell'intero cammino verso ogni destinazione. Periodicamente i router vanno ad inviare ai vicini il percorso che stanno utilizzando per una data destinazione, a ciascuno di essi viene assegnata una metrica e infine verrà scelto quello con distanza più breve.

6.0 Livello trasporto

Il livello di trasporto si basa sul livello di rete per fornire il trasporto di dati da un processo che si trova sulla macchina sorgente ad uno che si trova sulla macchina di destinazione, con un livello di affidabilità indipendente dalle reti in uso. Questo livello deve fornire un servizio efficace, efficiente e affidabile ai suoi utenti, dove per utenti facciamo riferimento ai processi del livello applicazione; utilizza i servizi forniti dal livello di rete e il codice del livello di trasporto è implementato sugli host utenti, mentre quello del livello di rete viene seguito per la maggior parte all'interno dei router. Il livello di trasporto va a fornire un canale di comunicazione per dati di tipo end-to-end, tra i processi che si trovano in diversi host e l'infrastruttura sottostante è trasparente, ciò significa che gli host comunicano fra loro come se fossero direttamente collegati.

6.1 Servizio di trasporto

6.1.1 Servizi forniti ai livelli superiori

L'obiettivo del livello di trasporto è fornire un servizio di trasmissione dei dati affidabile al livello superiore, che è il livello applicazione, per raggiungere il suo fine utilizza i servizi offerti dal livello di rete. Colui che svolge il lavoro, hardware o software che sia, all'interno del livello di trasporto viene chiamato **entità di trasporto**, che si può trovare all'interno del kernel del sistema operativo o in un processo utente separato. Così come nel livello di rete anche nel livello di trasporto si hanno servizi orientati alla connessione e non, che sono molto simili al livello precedente: per esempio il servizio di trasporto orientato alla connessione prevede di instaurare connessioni in 3 fasi:

1. Impostazione della connessione.
2. Trasferimento dei dati.
3. Rilascio della connessione.

Dato che i due sono così simili ci poniamo la domanda il perché esistono entrambi i livelli (rete e trasporto): il codice del livello di trasporto è implementato interamente sulle macchine, mentre quello del livello di rete è eseguito sui router, ciò significa che gli utenti non hanno controllo sul livello di rete. La soluzione è inserire un livello superiore a quello di rete per migliorare la qualità del servizio. In pratica il servizio offerto dal livello di trasporto è più affidabile da quello offerto dal livello di rete, inoltre le primitive del livello di trasporto sono implementate come chiamate a procedure a libreria per renderle indipendenti dalle primitive del servizio di rete. Il livello di trasporto svolge anche la

funzione di isolare i livelli superiori dalle imperfezioni della rete: potremmo vedere questo livello come un separatore, tra i livelli sottostanti e quelli superiori, quindi i 4 livelli inferiori sono i fornitori del servizio di trasporto, mentre quelli superiori sono gli utenti del servizio di trasporto.

6.1.2 Primitive del livello di trasporto

Per fare in modo che gli utenti possano accedere al servizio, il livello di trasporto deve fornire un'interfaccia, ossia delle funzionalità per programmi applicativi (ogni servizio di trasporto ha la sua interfaccia). Il servizio di rete è stato pensato per modellare il servizio offerto da reti reali, però quest'ultime sono piene di imperfezioni ed è di per sé un servizio inaffidabile; invece il servizio di trasporto orientato alla connessione è affidabile, quindi il suo scopo è fornire un servizio affidabile basandosi però su reti inaffidabili.

Una seconda differenza fra il servizio offerto dai due livelli, è che quello di rete viene utilizzato solo dalle identità di trasporto e solo pochi utenti possono scrivere nelle entità di trasporto e potranno poi avere accesso diretto ai servizi di rete (SOLO pochi utenti), invece le primitive del livello di trasporto sono accessibili a molti programmi e quindi devono essere facili da utilizzare e intuitive.

Ci sono 5 primitive base nel servizio di trasporto:

- **LISTEN:** è una chiamata bloccante, e permette al server di bloccarsi fino a che un processo non richiede di connettersi.
- **CONNECT:** effettuare una richiesta di connessione attraverso un segmento di REQUEST si tenta di stabilire una connessione con il server. Quando arriva la REQUEST l'entità di trasporto controlla se il server è bloccato sulla LISTEN, e se così fosse quest'ultimo verrà sbloccato e invierà una CONNECTION ACCEPTED.
- **SEND:** inviare dati nel momento in cui la connessione fra host e server è stata stabilita.
- **RECEIVE:** chiamata bloccante dove ciascun lato possono bloccarsi in attesa di dati.
- **DISCONNECT:** quando la connessione non è più necessaria deve essere rilasciata. SI hanno 2 tipi di disconnessione: asimmetrica, prevede che ogni utente invii una DISCONNECT nello stesso istante, simmetrica, ogni direzione è separata l'un l'altra e la disconnessione avviene solo quando si hanno entrambi i messaggi di DISCONNECT.

6.1.3 Socket di Berkeley

Sono un gruppo di primitive di trasporto, utilizzate dalla socket per TCP. Le primitive sono simili alle precedenti, ma offrono una maggiore flessibilità:

- **SOCKET:** permette di creare un nuovo endpoint di comunicazione e allocare lo spazio corrispondente all'interno dell'entità di trasporto. Non hanno ancora l'indirizzo di rete.
- **BIND:** assegnare indirizzo di rete ad una socket.
- **LISTEN:** annuncia la capacità di accettare connessioni. Fornisce anche la dimensione della coda (quanti client possono connettersi contemporaneamente). Questa LISTEN non è bloccante.
- **ACCEPTED:** chiamata bloccante eseguita dal server. Blocca il chiamante finché non arriva un tentativo di connessione.

- CONNECT: effettuata dal client, che tenta di stabilire una connessione con il server.
- SEND: inviare dati sulla connessione
- RECEIVE: ricevere dati sulla connessione
- CLOSE: rilasciare la connessione. Avviene solo in modo simmetrico.

6.2 Elementi dei protocolli di trasporto

Il servizio di trasporto è implementato dai protocolli di trasporto, ed è utilizzato dall'entità di trasporto che si trova sulla macchina sorgente e da quella che si trova sulla destinazione. I compiti fondamentali di questi protocolli sono:

- Controllo di flusso
- Controllo degli errori
- Ordinamento

Assomigliano molto ai protocolli data-link, però cambiano gli ambienti in cui si opera, per cui ci sono sostanziali differenze tra i due tipi

Protocolli di trasporto	Protocolli data-link
Il canale fisico è sostituito dall'intera rete.	Due router comunicano attraverso un mezzo fisico. Può essere vincolato e non.
Deve essere effettuato l'indirizzamento esplicito delle destinazioni.	Nei collegamenti punto-punto i router non specificano l'altro router con in quale vogliono comunicare.
Stabilire una connessione è più complicato (sez. 6.2.2).	Stabilire una connessione è semplice: perché l'altra estremità è sempre presente. Se non si riceve un ack, può essere rispedito.
La capacità di memorizzazione della rete: il pacchetto potrebbe nascondersi in qualche angolo della rete e riapparire dopo tanto tempo, e potrebbe arrivare un duplicato.	I pacchetti vengono spediti lungo un'unica linea trasmissiva, quindi si può perdere ma non può nascondersi o rimbalzare nella rete.
Buffering: numero variabile di connessioni, quindi non può essere adottato il metodo del data-link, nel dedicare spazio nel buffer per ogni linea.	Buffering: un buffer dedicato all'unica linea trasmissiva

6.2.1 Indirizzamento

Quando un processo a livello applicazione o un utente vuole stabilire una connessione con un'applicazione remota, la prima cosa da fare è specificare l'applicazione alla quale connettersi. Si vanno a definire indirizzi di trasporto, sui quali i processi rimangono in ascolto in attesa di richieste di connessione; in internet questi endpoint (punti terminali) prendono il nome di **porte TSAP** (*transport service access point*), si riferisce ad un endpoint specifico nel livello di trasporto. Gli equivalenti nel livello di rete sono gli NSAP, come ad esempio gli indirizzi IP.

Gli indirizzi TSAP funzionano bene per un numero di servizi ridotto che non varia mai, ma spesso i processi utente vogliono comunicare con degli altri processi che possiedono o un indirizzo TSAP a priori o che esiste per un determinato periodo di tempo. Per gestire queste situazioni si usa un processo speciale, chiamato **port-mapper**.

6.2.2 Stabilire una connessione

Per stabilire una connessione non basta effettuare una connection request da parte del client, e di attendere una connection accepted da parte del server, ma il problema principale è dato dal fatto che la rete potrebbe perdere, corrompere e ritardare i pacchetti.

Problema dei duplicati in ritardo

Bisogna stabilire attraverso algoritmi delle connessioni affidabili, per evitare il problema dei duplicati in ritardo: i duplicati vengono interpretati come se fossero pacchetti nuovi. Un modo per risolvere il problema è utilizzare gli indirizzi di trasporto ogni volta che si stabilisce una connessione, però rende difficile la connessione ad un processo; un'altra possibilità è assegnare ad ogni connessione un identificatore: numero sequenziale che viene incrementato ad ogni nuova connessione, per cui ogni volta si va a controllare la tabella all'interno dell'entità di trasporto, per vedere se quella connessione era già stata rilasciata, però ha il difetto che bisogna mantenere una tabella con informazioni storiche riguardante tutte le connessioni stabilite in precedenza. Un altro metodo è eliminare i pacchetti più vecchi: si possono utilizzare 3 opzioni:

- Le reti vengono progettate soggette a restrizioni.
- Inserito un contatore di salti ad ogni pacchetto.
- Applicare un time-stamp ad ogni pacchetto.

Non è semplice da implementare. Non dobbiamo solo garantire che i pacchetti siano eliminati dalla rete, ma anche i loro ack devono essere rimossi; questo metodo è ottimo per risolvere il problema iniziale.

Metodo di Thomlison

Algoritmo più utilizzato per limitare il tempo di vita dei pacchetti in internet. La sorgente deve etichettare i segmenti con dei numeri di sequenza che non saranno riutilizzati entro un tempo t , quindi si t che il numero di pacchetti generati al secondo determinano la dimensione dei numeri in sequenza, in questo modo un solo pacchetto con un dato numero di sequenza può essere sospeso in un certo istante. Dato che t , in una rete complessa, non deve essere molto grande questa strategia non è conveniente, per cui Thomlison decise di andare ad inserire all'interno di ogni host un orologio, niente altro che un contatore binario che si incrementa ad intervalli regolari. Usando l'orologio si riuscì a capire che si poteva effettuare una distinzione fra duplicati e non: ma anche qui sorge un problema

perché normalmente non si ha modo di sapere se un segmento di connection request che contiene il numero di sequenza iniziale sia o meno il duplicato di una connessione recente. Per risolvere quest'ultimo problema si utilizza un handshake a tre vie, dove si richiede la verifica sia dalla sorgente che dalla destinazione, che la richiesta non sia un duplicato: l'host1 sceglie un numero di sequenza X e invia un segmento di connection request insieme ad X alla destinazione, quest'ultimo risponde con un segmento di ack per confermare X e gli manderà anche il suo numero di sequenza Y. Per inviare poi i dati la sorgente utilizza X e l'ack utilizzerà Y. Si potrebbero verificare dei problemi nel caso in cui ci sono pacchetti duplicati o ritardati:

- La richiesta di connessione duplicata arriva a destinazione e quest'ultima manda il segmento di ack per chiedere se effettivamente la sorgente sta cercando di stabilire una nuova connessione. La sorgente rifiuta, e la destinazione si accorge che era una richiesta di connessione in ritardo e si interrompe la comunicazione.
- Quando all'interno della rete si ha una richiesta di connessione ritardata e anche un ack ritardato: si inizia come il precedente, ma quando la destinazione risponde alla richiesta di connessione utilizza Y e nel momento in cui viene confermata la connessione, visto che anche l'ack è in ritardo, si avrà un altro numero di sequenza, per cui l'ack sarà in ritardo.

6.2.3 rilascio della connessione

Esistono 2 modi per terminare una connessione:

- *Simmetrico*: ogni connessione deve inviare una richiesta di disconnect, dove entrambi i segmenti attendono la richiesta dell'altro per poter terminare la connessione. Questo metodo funziona bene quando si ha una quantità fissa di dati da inviare e sappiamo quando gli abbiamo inviati, però si può verificare il problema dei due eserciti che è un problema senza soluzione.
- *Asimmetrico*: es telefono. Può provocare una perdita improvvisa dei dati. Si ha bisogno di un protocollo più articolato.

Problema dei due eserciti

Abbiamo due eserciti, bianco e blu: quello blu si trova a monte, e in mezzo ai due c'è una valle con l'esercito bianco. L'esercito blu è in maggioranza numerica, ma è diviso in due parti che si vogliono accordare sull'attacco; l'unico modo per farlo, visto che sono disposti nei due fianchi opposti, devono mandare un messaggero a valle, dove però c'è l'esercito bianco. Non si ha mai la certezza che l'altra parte dell'esercito blu attacchi effettivamente il nemico: non esiste soluzione, nemmeno con un handshake a 4 vie.

L'unica soluzione accettabile per la disconnessione è far decidere indipendentemente ogni partecipante, non spetta all'entità di trasporto.

6.2.4 Controllo degli errori e controllo di flusso

Il livello di trasporto adotta le stesse soluzioni, per i controlli, usate nel livello data-link: nonostante la loro somiglianza, bisogna comunque distinguerli l'un l'altro, perché il livello di funzionalità è differente.

Controllo degli errori

Il checksum del data-link protegge il frame nel singolo collegamento, mentre quello del livello di trasporto protegge il segmento mentre attraversa l'intera rete: in questo caso è un controllo end-to-

end, che è molto diverso da quelli che si ha sul singolo collegamento. I checksum del data-link proteggono i pacchetti mentre passano sul collegamento, ma non all'interno dei router, quindi possono risultare errati anche se sul singolo collegamento sono corretti. I controlli nel livello trasporto sono essenziali per la correttezza, mentre quelli nel livello data-link sono più importanti per le prestazioni.

Controllo di flusso

I protocolli di trasporto utilizzano delle finestre scorrevoli grandi, quindi bisogna analizzare il problema di memorizzare i dati nel buffer. Un host può avere molte connessioni, e ciascuna di queste è trattata separatamente per cui abbiamo bisogno di una grande quantità di buffer; questi sono indispensabili sia a lato mittente che destinatario, perché quelli del mittente contengono tutti i segmenti che sono stati trasmessi senza aver ricevuto l'ack, invece il destinatario mantiene un singolo buffer condiviso da tutte le connessioni: quando un segmento tenta di acquisire un nuovo buffer si controlla la disponibilità. Anche se il destinatario perde il segmento, non ci sono problemi perché la sorgente è pronta a ritrasmettere, il tutto però spreca risorse. Di solito i buffer sono gestiti in modo dinamico, prevedendo una finestra scorrevole di dimensione variabile.

6.2.5 Multiplexing

Torna utile in due situazioni:

- Quando un host ha a disposizione un solo indirizzo di rete e tutte le connessioni di trasporto del computer (le porte) devono utilizzare quello stesso indirizzo perché è l'unico disponibile. In questo caso quando arriva un segmento si deve stabilire a quale processo passarlo, e questo è chiamato **upward-multiplexing**.
- Quando un host può utilizzare diversi percorsi di rete e un utente ha bisogno o di più affidabilità o di più ampiezza di banda, si utilizza una connessione che distribuisce il traffico sui diversi percorsi di rete, attraverso la politica Round-Robin. Questo viene chiamato **downward-multiplexing**.

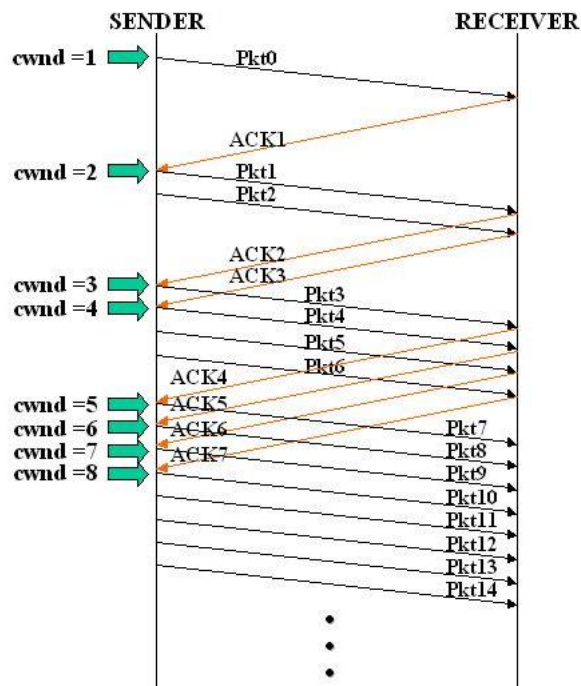
6.3 Controllo di congestione

Il controllo della congestione è gestito sia dal livello di rete che dal livello di trasporto: se le entità di trasporto inviano troppi pacchetti nella rete, quest'ultima si congestionerà. Proprio la perdita dei pacchetti, lo scadere del timeout e la ritrasmissione dei pacchetti sono sintomi di congestione; la sorgente deve essere in grado di reagire, diminuendo il tasso di trasmissione. Poiché le finestre scorrevoli non sono sufficienti, perché ci aiutano solo per il controllo di flusso, quindi si va a variare dinamicamente la dimensione della finestra scorrevole, a seconda del numero dei pacchetti da inviare. La dimensione della finestra (**CWND**) è dinamica.

Esistono diversi algoritmi che regolano la dimensione di questa finestra, il più importante è il *slow-start*.

Slow-start

Il mittente mantiene due finestre, sia quella del ricevente sia quella di congestione: ogni volta che si riceve una ack, la finestra di congestione può aumentare di un segmento, ciò implica che quando viene ricevuto un ack si trasmettono due nuovi segmenti, e nel momento in cui si ricevono gli ack dei segmenti appena inviati si trasmettono 4 nuovi segmenti, e così via. L'andamento di questo algoritmo è esponenziale.



Vista la sua crescita esponenziale in rete ci saranno troppi pacchetti che verranno spediti troppo velocemente: si avrà congestione. È stata introdotta una soglia per il collegamento che viene chiamata **slow-start-threshold**, all'inizio ha un valore fisso e il protocollo continua a far crescere la finestra fino ad un time-out oppure fino a che non si supera questa soglia; ogni volta che si rileva la perdita di un pacchetto la soglia di threshold viene dimezzata, e poi si ricomincia da capo. In questo modo si ha rispetto a prima, un andamento lineare: per finestre grandi vediamo dei notevoli miglioramenti nelle prestazioni.

6.4 Protocolli di trasporto di internet: UDP

Nel livello di trasporto di internet abbiamo due protocolli principali: TCP e UDP. UDP è il protocollo non orientato alla connessione dove il suo compito è quello di spedire pacchetti tra le applicazioni. Il TCP è orientato alla connessione e si occupa un po' di tutto.

6.4.1 Introduzione ad UDP

UDP (*user datagram protocol*), offre un modo per inviare datagrammi senza stabilire una connessione: trasmette dei segmenti costituiti da un'intestazione di 8 byte, seguita dal payload. Nell'intestazione c'è il campo **source port** (porta sorgente) e **destination port**, la porta sorgente è utile quando si deve inviare una risposta al mittente, invece destination port serve per inviare un messaggio alla destinazione. Abbiamo anche il campo **UDP length** (lunghezza) che include sia l'intestazione di 8byte sia i dati: in generale non si superano i 1500byte; infine si ha il checksum, inserito per aumentare l'affidabilità.

UDP fornisce un'interfaccia al protocollo IP, permette il demultiplexing di più processi e rileva gli errori end-to-end. Spesso può essere utile in alcune comunicazioni client-server: il client si aspetta

una risposta rapida da parte del server (le risposte di entrambe le parti sono brevi e veloci). Un'applicazione di esempio è il protocollo DNS.

UDP è interessante per la velocità, ma è inaffidabile perché è orientato alla connessione. Non c'è bisogno di un setup iniziale. Non viene utilizzato il controllo di congestione, e potremmo perdere tanti pacchetti.

6.4.2 Protocolli di trasporto real-time

UDP è impiegato anche nelle applicazioni multimediali real-time: si sviluppò un protocollo per queste applicazioni, chiamato **RTP** (*real time transport protocol*), che fu il primo protocollo per il trasporto di dati audio/video. RTP è basato quindi su UDP, e viene seguito nello spazio utente. Le funzioni svolte da RTP sono essenzialmente 2:

1) Trasporto di dati audio/video

Un'applicazione multimediale è un insieme di flussi audio/video e la funzione di base di RTP è eseguire il multiplexing dei dati real-time in un unico flusso di pacchetti UDP. Poiché RTP si appoggia su UDP non si ha la certezza che questi pacchetti giungano integri a destinazione; ad ogni pacchetto in un flusso viene assegnato un numero che viene incrementato di uno rispetto al suo predecessore, aiutandoci a capire se mancano pacchetti. Non possiamo ritrasmettere pacchetti mancanti, perché potrebbero essere consegnati in ritardo; per questo non si usano né ack e né ritrasmissioni, ma se manca un pacchetto nel caso di audio si approssima il valore mancante con l'interpolazione, invece nei video si approssima tra i fotogrammi adiacenti. Si aggiunge il time-stamp nei pacchetti che consente alla sorgente di associarlo al primo campione di ogni pacchetto, in questo modo la destinazione può gestire un buffer e riprodurre il campione nell'istante in cui è richiesto.

Il segmento RTP è strutturato nel seguente modo:

1. VERSION: versione 2.
2. P: indica che il pacchetto è riempito fino ad un multiplo di 4 byte.
3. X: indica che è stata aggiunta un'intestazione estesa.
4. CC: indica in numero di sorgenti attive presenti.
5. M: contrassegno specifico dell'applicazione (prima parola dell'audio/ prima immagine de video).
6. PAYLOAD TYPE: algoritmo di codifica utilizzato.
7. SEQUENCE NUMBER: contatore ad ogni invio di pacchetto.
8. TIME STAMP.
9. IDENTIFICATORE DI SINCRONIZZAZIONE DI FLUSSO: specifica il flusso di appartenenza del pacchetto. Utile per il multiplexing e demultiplexing.

RTP è accompagnato al protocollo **RTCP** (*real time transport control protocol*) che ha 3 funzioni:

- Fornisce un feedback alla sorgente del flusso sui ritardi, feedback, larghezza di banda, ecc..
- Sincronizzazione tra flussi. Flussi diversi usano clock differenti e questo protocollo mantiene la sincronia tra i vari clock.

- Fornire un metodo di denominazione delle sorgenti in ASCII.

2) Riproduzione delle applicazioni multimediali

I pacchetti possono arrivare in ordine sparso al destinatario, e questa variazione del ritardo è chiamata **Jitter**, perché dal lato sorgente sono inviati correttamente. Anche solo un piccolo Jitter crea disagi. Per ridurre il jitter si usa un buffer a lato destinatario, dove vengono immagazzinati i pacchetti prima di essere riprodotti: il più delle volte per lo streaming vengono usati buffer di 10 secondi per assicurarsi di ricevere pacchetti in tempo. In pratica vengono immagazzinati, riordinati e poi riprodotti un po' alla volta.

6.5 Protocolli di trasporto in internet: TCP

Si è visto che UDP è un protocollo semplice, con utilizzi importanti, ma per la maggior parte delle applicazioni in internet è necessaria l'affidabilità della consegna, che lui non riesce a dare. Per questo abbiamo bisogno di TCP, che è il principale motore di internet.

6.5.1 Introduzione a TCP

TCP (*transmission control protocol*), è stato progettato per fornire flusso di byte affidabile end-to-end su una internetwork inaffidabile; quindi si adatta dinamicamente alle proprietà della rete che costituiscono internet e per garantire prestazioni buone anche in caso di svariati tipi di errore. Ogni computer che supporta il protocollo TCP contiene un'entità di trasporto TCP, che può essere un processo utente o altro: questa va a gestire i flussi TCP e si interfaccia con il livello IP. Il livello IP non prevede gli errori, e quindi sarà compito del TCP spedire dati velocemente e in modo sicuro, con l'opzione di riordinarli nel caso in cui i datagrammi arrivano in ordine sparso. TCP fornisce l'affidabilità che IP non riesce a dare.

Le caratteristiche principali di TCP sono:

- Orientato alla connessione: si ha un handshake a 3 vie.
- Affidabile.
- Le sequenze sono ordinate a lato destinatario.
- Controllo di flusso.
- Gestione dei buffer su entrambe le parti.
- Comunicazione full-duplex.

6.5.2 Modello di servizio di TCP

Il servizio TCP è ottenuto con la creazione di punti terminali, chiamati **socket**. Ogni socket possiede un indirizzo composto sia dall'IP dell'host che dall'indirizzo locale dell'host, ovvero la porta: ogni endpoint è caratterizzato da indirizzo IP e porta. Per stabilire una connessione bisogna collegare le socket della sorgente e del destinatario; si può usare una singola socket per più connessioni contemporaneamente.

I numeri di porta minori di 1024 possono essere utilizzati solo da utenti privilegiati e sono riservati per servizi standard; prendono il nome di **well known port**.

Porta	Protocollo	Utilizzo
20 e 21	FTP	Trasferimento di file. La 20 è usata per i dati, la 21 per il controllo
25	SMTP	Posta elettronica
80	HTTP	WWW
110	POP3	Accesso remoto alla posta elettronica.
143	IMAP	Accesso remoto alla posta elettronica
443	HTTPS	Accesso sicuro al WWW

All'avvio del sistema operativo, si potrebbe associare un demone per ciascun protocollo alla sua porta associata: se si avviasse un demone per ciascun protocollo consumeremo molta memoria, quindi si avvia un singolo demone, chiamato **inetd**, che viene associato a più porte e attende la prima connessione in ingresso; quando c'è la nuova connessione questo inetd genera un nuovo processo ed esegue il demone appropriato.

TCP non supporta né il multicast né il broadcast, solo connessioni punto-punto full-duplex.

Una connessione TCP è un flusso di dati, non di messaggi (i limiti dei messaggi non vengono conservati): quando un'applicazione passa dati al TCP, quest'ultimo decide se inviare subito i dati, oppure metterli dentro un buffer. Per forzare l'uscita dei dati all'interno del buffer si utilizza un flag di PUSH all'interno dei pacchetti per non ritardare la connessione.

TCP va a etichettare alcuni dati come dati urgenti, ovvero quei dati ad alta priorità che devono essere subito elaborati: questa etichettatura viene effettuata tramite il flag URGENT.

6.5.3 Il protocollo TCP

Ogni byte in una connessione TCP ha il proprio numero di sequenza a 32 bit e l'entità TCP sia di invio che di ricezione, si scambiano dati sotto forma di segmenti: un segmento TCP ha un'intestazione fissa di 20byte, seguita da zero o più byte di dati, sarà compito del software TCP di decidere la dimensione dei segmenti che tiene conto di due fattori:

1. Ogni segmento, compresa l'intestazione, deve essere contenuto nel payload IP.
2. Ogni collegamento ha una maximun transfert unit. In generale sono 1500byte.

Il protocollo di base che utilizzano le entità TCP è quello a finestra scorrevole a dimensione dinamica; per cui quando la sorgente fa partire un segmento fa partire un timer e quando il segmento arriva a destinazione, l'entità TCP ricevente rimanda un segmento che contiene un numero di ack uguale al numero di sequenza successivo che prevede di ricevere.

6.5.4 Intestazione del segmento TCP

Ogni segmento inizia con l'intestazione fissa a 20 byte, che può essere seguita da alcune opzioni; dopo troviamo il payload, che può essere zero (es. ack). I campi dell'intestazione sono:

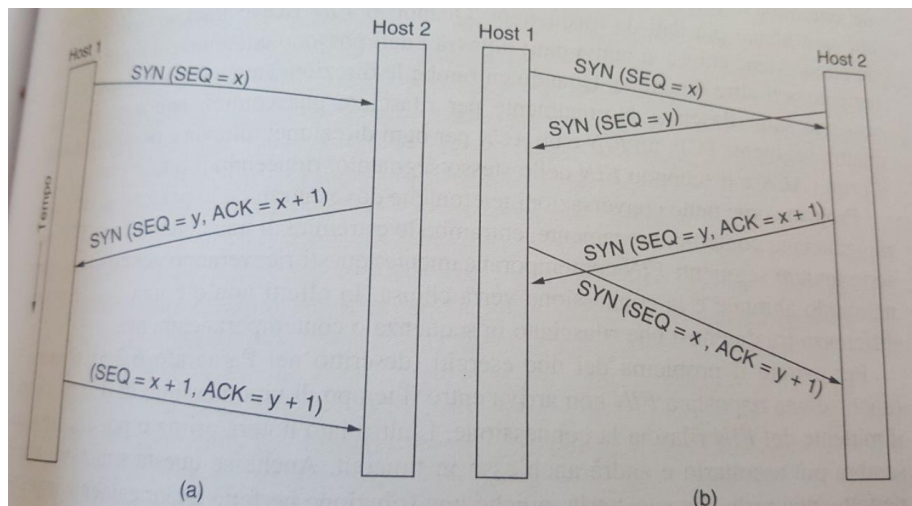
1. SOURCE PORT
2. DESTINATION PORT: identifica, insieme al source port, gli endpoint.
3. SEQUENCE NUMBER
4. ACK NUMBER: specifica il byte successivo previsto e non l'ultimo byte ricevuto.
5. TCP HEADER LENGTH: necessaria perché il campo opzioni ha lunghezza variabile.
6. 8 flag su 1 bit: URGE, ACK per indicare che ack number è valido (se è zero vuol dire che il segmento non contiene un ack), PSH per indicare i dati push (evitare di archiviare i dati nel buffer, ma inviarli subito, RST usato per impostare una connessione che ha subito un malfunzionamento o per rifiutare un segmento non valido, SYN usato per stabilire le connessioni, FIN rilasciare la connessione (gli ultimi 3 non sono stati fatti).
7. WINDOW SIZE
8. CHECKSUM: effettua un controllo dell'intestazione dei dati e di una pseudointestazione. La pseudointestazione si usa per controllare i pacchetti non consegnati correttamente.
9. OPTION: informazioni aggiuntive. Una delle più usate è quella che specifica il maximum segment size che si è disposti ad accettare; utilizzare segmenti più grossi è più efficiente di quelli piccoli, ma host con poche capacità non potrebbero essere in grado di gestirli.

6.5.5 Instaurazione delle connessioni TCP

Le connessioni TCP vengono stabilite tramite l'handshake a 3 vie: per stabilire la connessione un lato attende passivamente una connessione in ingresso eseguendo le primitive LISTEN e ACCEPTED, l'altro lato esegue la primitiva CONNECT specificando IP e porta alla quale vuole connettersi e specifica la dimensione massima del segmento.

Quando viene eseguita la CONNECT viene inviato un segmento TCP con il bit SIN=1 e ACK=0, nel momento in cui arriva a destinazione l'entità di trasporto controlla se un processo ha eseguito la LISTEN sulla porta indicata, se questo non è avvenuto RST=1 e si rifiuta la connessione, altrimenti si decide se accettare o meno la connessione; nel caso di accettazione viene restituito al mittente ACK=1. Il numero di sequenza iniziale dovrebbe variare lentamente invece di essere un numero costante, per proteggere pacchetti in ritardo e duplicati.

La vulnerabilità dell'handshake a 3 vie è che il processo in ascolto deve ricordarsi il proprio numero di sequenza quando risponde con il segmento SIN.



6.5.6 Rilascio della connessione TCP

Per rilasciare la connessione TCP, si considerano due connessioni simplex:

1. Entrambe le parti inviano un segmento TCP con $\text{FIN}=1$.
2. Quando FIN riceve l'ack la direzione viene chiusa a nuovi dati, mentre fino a che l'altra direzione non ha ricevuto l'ack può continuare a fluire.
3. Per ridurre la connessione possiamo associare al FIN un ACK, così riduciamo il tempo.

La connessione deve essere chiusa da entrambe le parti; si usano dei timer per il rilascio se l'ACK non arriva in tempo, viene rilasciata la connessione da una delle due parti, e l'altra si renderà conto che la connessione è stata terminata perché non riceverà più risposte.

6.5.7 Finestra scorrevole di TCP

Quando la finestra è zero il mittente non può inviare segmenti, tranne che in due casi:

1. Quando dobbiamo inviare dati urgenti.
2. Oppure per inviare un segmento di 1byte che funge da trigger di controllo per fare in modo che il ricevente annunci il successivo byte atteso e anche la dimensione della finestra (evitare i deadlock).

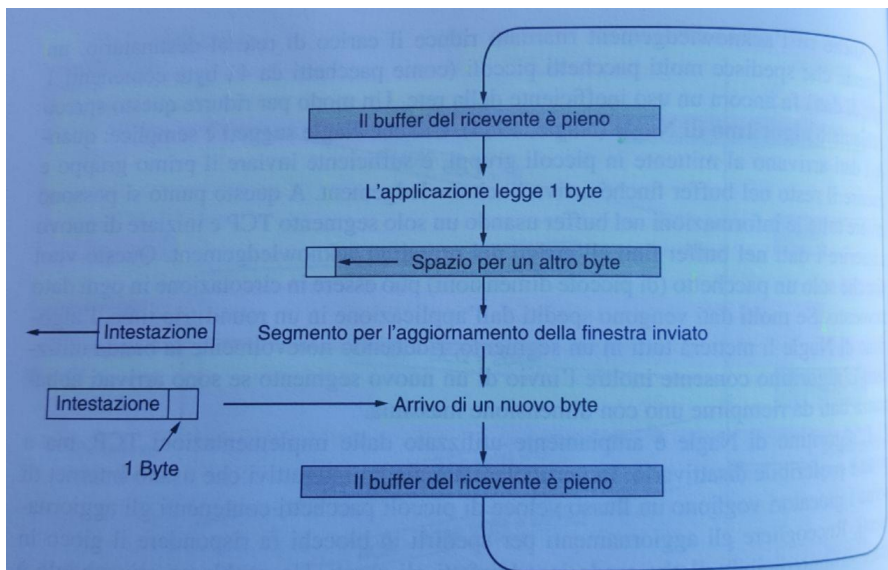
I mittenti non devono trasmettere i dati appena li ricevono dall'applicazione, e i riceventi non hanno l'obbligo di inviare immediatamente gli ack appena ricevono i dati: questa libertà serve per migliorare le prestazioni.

Consideriamo una connessione ad un terminale remoto, che utilizza il protocollo SSH che reagisce ad ogni pressione dei tasti sulla tastiera; quando arriva il carattere digitato all'entità TCP sorgente, viene creato un segmento TCP di 21byte che viene passato al livello IP, in modo tale che vengo costruito un datagramma di 41byte. A questo punto il lato ricevente TCP invia l'ack di 40byte, la finestra si aggiorna spostandola di un byte a destra e poi viene inviato l'aggiornamento della tabella. Con tutti questi passaggi si sono stati usati 162byte di banda, e se la banda è scarsa questo metodo non conviene.

Quello che viene fatto è ritardare gli ack e gli aggiornamenti di 500ms, andando a dimezzare l'uso della banda e il numero di pacchetti inviati: anche in questo caso se il mittente spedisce pacchetti

molto piccoli si utilizza male la banda a disposizione. Un metodo per ridurre lo spreco è l'algoritmo di **Nagle**: quando arrivano dati al mittente in piccoli gruppi, si invia solo il primo gruppo e tutto il resto viene inserito in un buffer, finché non arriva l'ack; dopodiché si inviano tutte le informazioni dentro il buffer utilizzando un singolo segmento TCP, e poi si ripete il ciclo. Questo permette di far girare nella rete solo un pacchetto di piccole dimensioni, e possiamo inviare un segmento solo quando abbiamo il buffer pieno.

Un altro problema che può verificarsi è **silly window syndrome**, ovvero nel momento in cui i dati vengono passati dal livello applicazioni al TCP in blocchi grandi però a lato ricevente abbiamo applicazioni che leggono 1byte per volta. Il buffer a livello mittente sarà sempre pieno, e verrà inviato 1byte alla volta



La soluzione al problema è l'algoritmo di **Clark**, che impedisce al ricevente di inviare l'aggiornamento della finestra per un solo byte ed è obbligato ad attendere la disponibilità di una certa quantità di spazio, ossia solo quando è in grado di gestire la dimensione massima del segmento, oppure quando il buffer è vuoto per metà. Il mittente può aiutare il destinatario inviando pacchetti piccoli.

I due algoritmi visti sono complementari: quello di Nagle risolve il problema delle applicazioni a lato mittente che consegnano i dati 1byte per volta, mentre Clark risolve il problema dell'applicazione a lato ricevente che legge 1byte per volta. Quello che si vuole raggiungere è che il mittente non invii segmenti piccoli, e il ricevente non gli richieda a sua volta.

6.5.9 Gestione dei timer TCP

TCP utilizza più timer per svolgere il proprio lavoro e uno dei timer più importanti è il timeout di ritrasmissione (**RTO**). Se RTO è troppo corto si hanno congestioni inutili nella rete, se troppo grande i ritardi di ritrasmissione è troppo elevato e quindi le performance diminuiscono; la media e la varianza della distribuzione dell'arrivo degli ack possono cambiare sensibilmente a causa della congestione che va e che viene.

Per stabilire il tempo di RTO si usa un algoritmo dinamico, detto algoritmo di **Jacobson** che adatta l'intervallo di timeout in base a misurazioni fatte sulla rete per verificarne le prestazioni. Per ogni connessione TCP viene mantenuta una variabile chiamata **SRTT** (*smoothed round trip time*) che rappresenta la stima migliore del tempo impiegato dal pacchetto nel raggiungere la destinazione e poi

ricesegnare l'ack. Se l'ack torna indietro prima della scadenza del timer, TCP misura quanto tempo è stato richiesto, e si aggiorna il valore $SRTT = \alpha * SRTT + (1 - \alpha) * R$, α indica quanto velocemente vengono dimenticati i vecchi valori di SRTT (7/8) mentre R misura il tempo richiesto. Questa formula è chiamata EWNA.

Questa formula non dava dei valori giusti, quando ad esempio la varianza aumentava e per risolvere il problema il valore di round trip time doveva essere filtrato nei tempi. Bisogna aggiungere la variabile RTTVAR.

$$RTTVAR = \beta * RTTVAR + (1 - \beta) * |SRTT - R|$$

$$RTO = SRTT + 4 * RTTVAR$$

TCP implementa anche il **timer di persistenza**, utilizzato per evitare i deadlock, e il timer di **keep alive** che viene utilizzato per le connessioni inattive per molto tempo.

7.0 Il livello applicazione

Il livello applicazione è il livello in cui si trovano le applicazioni, infatti tutti i livelli inferiori visti si occupano del trasporto affidabile delle informazioni, ma non si occupano minimamente degli utenti. Abbiamo bisogno di protocolli per permettere all'applicazione di funzionare; queste verranno eseguite su terminali e le architetture più usate sono quelle client-server e peer to peer.

Esempi di applicazioni: posta elettronica, VoIP, messaggistica istantanea.

7.1 DNS – Domain Name System

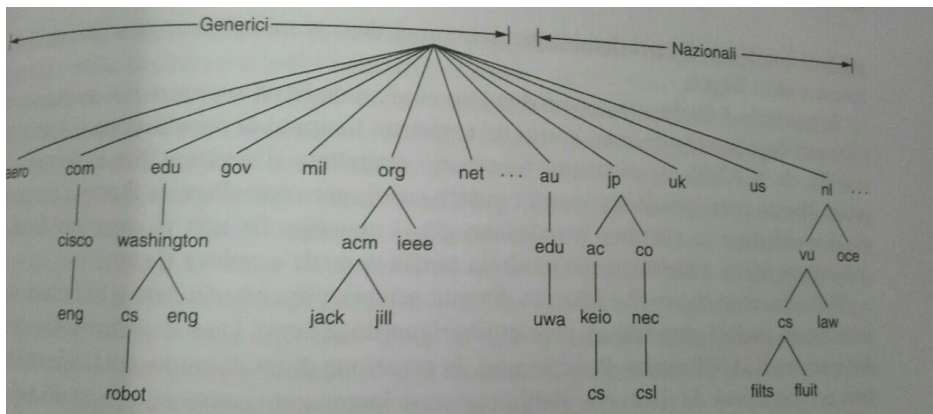
Molto spesso gli utenti non si ricordano gli IP degli host, ed è proprio per questo motivo che sono stati introdotti dei nomi ad alto livello che possono essere letti agevolmente dagli utenti e anche per separare i nomi dei computer dai rispettivi indirizzi: ad esempio nel momento in cui un'azienda sposta il server su una macchina con IP diverso, il nome dell'azienda rimane lo stesso (leggibile dall'utente).

La rete sa interpretare solo gli indirizzi IP, per cui si ha bisogno di un protocollo che converte i nomi ad alto livello in indirizzi IP: questo prende nome di DNS (*domain name system*). Si basa su un sistema di denominazione gerarchico, incentrato su domini, e su un database distribuito.

Per associare un nome ad un indirizzo IP l'applicazione invoca una procedura di libreria detta **resolver**, e gli passa il nome come parametro. A questo punto il resolver passa un pacchetto UDP che contiene la richiesta al DNS locale che cerca il nome dentro la tabella e restituisce l'IP associato a quel determinato nome; a sua volta questo indirizzo viene passato al resolver nuovamente. Tramite l'IP si può instaurare una connessione TCP.

7.1.1 Lo spazio dei nomi DNS

Si tratta di un indirizzamento gerarchico (come gli indirizzi postali): internet è divisa in 250 domini di **primo livello**, e ciascuno di questi copre molti host, dove ogni dominio ha i suoi sottodomini e così via fino a comporre una struttura ad albero.



7.1.2 Record delle risorse di dominio

Ad ogni dominio che può contenere uno o più host, può essere associato un insieme di record delle risorse che vanno a formare il database DNS. Quando il resolver va a convertire il nome in IP interrogando il server DNS (name server) ottiene i record delle risorse associate ai nomi di dominio: la funzione vera e propria del DNS è associare i nomi di dominio ai record delle risorse.

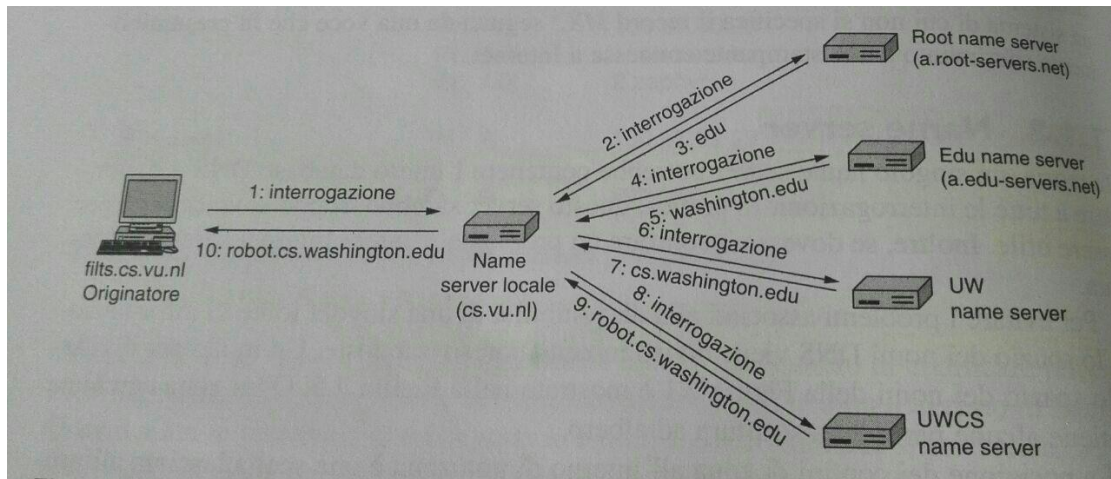
I record delle risorse sono una quintupla che sono:

1. **Nome di dominio:** dominio a cui si riferisce il record.
2. **Time to leave:** usato per discriminare le informazioni volatili e non. Per le informazioni stabili il time to leave ha un valore elevato, mentre per quelle di breve durata sarà basso.
3. **Classe:** per le informazioni di internet questo campo sarà sempre 'IN'.
4. **Tipo:** specifica il tipo di record. Questi record possono essere: SOA, che fornisce il nome della fonte primaria di informazioni, A, è il più importante perché contiene l'IPv4 di un host, MX, specifica il nome dell'host che accetta la poste elettronica del dominio specificato in ingresso, NS, specifica il name server.
5. **Valore:** può essere un numero, un dominio o una stringa ASCII.

7.1.3 Name Server

Lo spazio dei nomi DNS è diviso in zone non sovrapposte. Ognuna di queste zone è associato ad uno o più **name server**, dove per name server si intende il server DNS locale: in pratica sono degli host che mantengono il database per la zona. Le zone di solito hanno un name server primario che ottiene le informazioni o da un file su disco oppure da i name server secondari: il processo di ricerca di un nome è chiamato **risoluzione del nome**.

Quando il resolver ha un'interrogazione su un nome di dominio passa l'interrogazione ad uno dei name server locali: se il dominio richiesto è dentro il name server locale viene restituito immediatamente il record, chiamato **record autoritativo** perché viene direttamente fornito dall'autorità che lo gestisce ed è sempre corretto. Esistono anche i **cached record**, che potrebbero avere all'interno informazioni non aggiornate e quindi non essere sempre corretti. Se il record che ci interessa non si trova localmente il name server inizierà una ricerca a ritroso nel sistema.



Per prima cosa si va ad interrogare il **root name server**, che contiene la maggior parte delle informazioni sui domini di primo livello e grazie a lui si ottiene la prima parte del dominio. MA questo punto si va a scorrere l'albero da dx a sx, passando da name server a name server finché non si ottiene l'IP desiderato.

Possiamo avere due differenti tipi di interrogazione ai server:

- **Ricorsiva:** il client chiede al server di occuparsi di tutto il lavoro. Non vengono fornite delle risposte parziali ma solo totali.
- **Iterative:** non mirano a continuare l'interrogazione ricorsivamente sul server locale, ma viene restituita una risposta parziale e poi dice al client dover cercare dopo (vedi figura precedente).

Caching

Tutte le risposte anche quelle parziali sono archiviate dentro una cache, ciò permette di aumentare le prestazioni. Però potrebbero non essere aggiornate, per cui queste informazioni non devono rimanere a lungo dentro le cache (si usa il time to leave); il caching viene fatto all'interno del client.

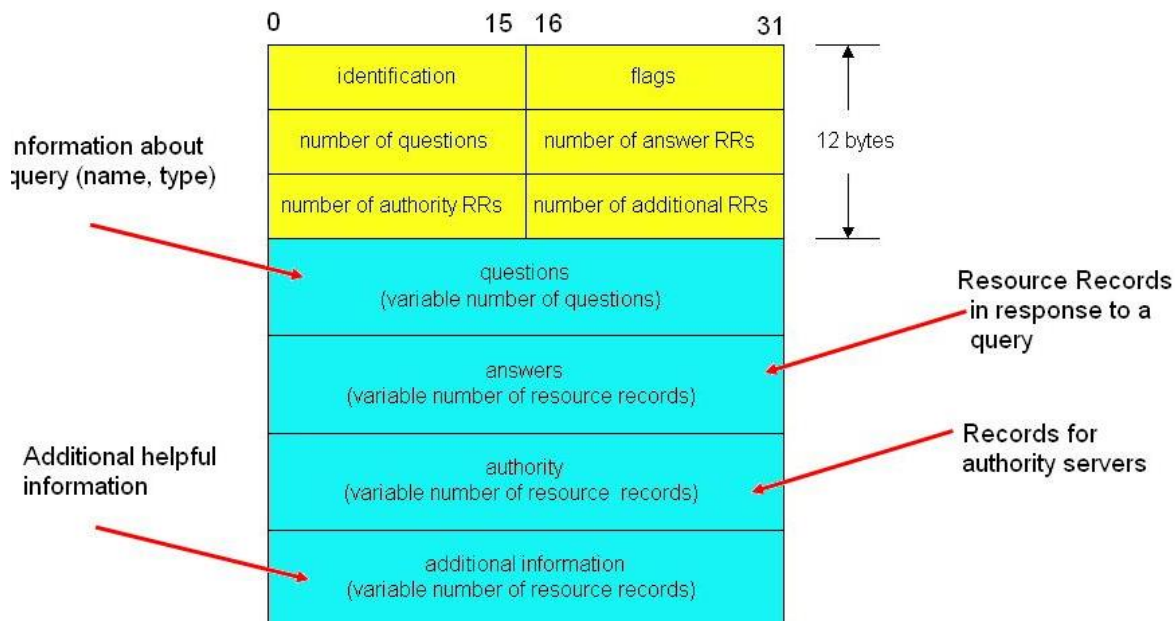
Protocollo DNS

Il compito principale è fornire affidabilità e buone prestazioni, in quanto è stato progettato per essere molto robusto. Poiché si tratta di un protocollo leggero viene posto sopra UDP, in questo modo le interrogazioni possono essere effettuate molto velocemente senza overhead; però le interrogazioni che superano i 512byte utilizzano TCP, altrimenti la consegna diventerebbe problematica.

Campi del protocollo DNS

- Identification: 16bit che identifica sia le richieste che le risposte
- flags: 4bit, indica se il pacchetto è una richiesta oppure una risposta
- opcode: indica il tipo di messaggio trasportato
- AA: 1bit, indica se la risposta è autoritativa
- TC: 1bit, indica il troncamento
- RD: 1bit, si vuole la ricorsione

- RA: 1bit, indica se la ricorsione è supportata dal server
- QDCOUNT: indica quante domande ci sono nella sezione domande
- ANCOUNT: indica quante risposte ci sono nella sezione risposte
- NSCOUNT: indica il numero di record delle risorse nella sezione autoritativa
- ARCOUNT: indica il numero di record delle risorse nella cache



7.2 Posta elettronica

Prime applicazioni nate in internet e la comunicazione con la posta elettronica è asincrona; ovvero i messaggi vengono inviati e letti quando vuole l'utente. Gratuita, veloce e facile da usare.

7.2.1 Architettura e servizi

Si hanno due sottosistemi:

- **User agent**: che consente all'utente di leggere, rispondere, inoltrare, comporre e salvare messaggi. Es. Outlook, MozillaMail...
- **E-mail server**: detto message transfer agent, che rappresentano il nucleo dell'intera infrastruttura e permettono di spostare i messaggi dalla sorgente alla destinazione. Normalmente sono dei processi di sistema e vanno a spostare le mail da sorgente a destinatario utilizzando il protocollo SMTP (*simple mail transfer protocol*). Implementano anche le mail list, dove le copie identiche di un messaggio vengono consegnate ad ogni iscritto alla lista.

Sopra il message transfer agent si hanno le **mail box** che memorizzano le mail ricevute dall'utente e vengono mantenute dal mail server.

Il messaggio della mail è composto dal **envelop** e dal contenuto vero e proprio. L'envelop incapsula il messaggio e contiene tutte le informazioni per il trasporto: indirizzo di destinazione, priorità, ecc. Quindi le mail server utilizzano l'envelop per il routing, invece il messaggio all'interno è composto in due parti:

- *Intestazioni*: contiene le informazioni necessarie all'agent per i controlli.
- *Contenuto*

7.2.3 Formato dei messaggi di internet

I messaggi che vengono inviati dallo user agent devono essere in un formato standard che possa essere gestito dal mail server. I campi dell'intestazione correlati al trasporto del messaggio che costituiscono l'involucro sono:

- *To*: indirizzi DNS del destinatario principale.
- *Cc*: destinatari secondari.
- *From*: chi ha scritto il messaggio
- *Sender*: chi ha spedito il messaggio
- *Received*: indica che il messaggio è stato ricevuto dal message transfer.

7.2.4 Protocollo SMTP

SMTP è un protocollo utilizzato per inviare mail in internet; per svolgere questa operazione si deve stabilire una connessione TCP sulla porta 25, per trasferire il messaggio dalla sorgente alla destinazione. SMTP è un protocollo vecchio e sia le intestazioni che il corpo vengono trattati come codici ASCII a 7bit.

Una volta che è stata stabilita la connessione il server è pronto ad accettare i messaggi di posta e il client annuncia chi sta inviando il messaggio e a chi è destinato; se l'indirizzo DNS è corretto allora il server comunica al client che può inviare il messaggio, e una volta arrivato a destinazione viene inviato una ack. Poiché TCP offre un flusso di byte affidabile non abbiamo bisogno di checksum e la connessione viene poi rilasciata. Utilizza delle connessioni persistenti, ovvero viene sfruttata sempre la stessa connessione TCP per inviare messaggi finché se ne hanno.

Gli svantaggi di SMTP sono:

- Non include l'autenticazione; nel campo from può esserci una qualunque destinazione (utile per spam).
- Trasferisce messaggi in ASCII e non messaggi binari: non si possono inviare messaggi eccetto che in lingua inglese (non sono supportati i vari alfabeti e allegati).
- Invia messaggi in chiaro, senza cifratura.

Trasferimento dei messaggi

Per sapere qual è il server di posta elettronica da contattare si consulta il DNS e in particolar modo si guarda il campo mx che c'è nel record delle risorse.

Protocollo MIME (*multi purpose internet mail extension*)

È stato introdotto per poter mandare messaggi di lingue internazionali e per inviare tutti quei file che non contengono testo (immagini, video..). Abbiamo due tipi di intestazioni estese fondamentali che sono:

- **Content type:** indica il tipo e il formato del contenuto. Serve all'user agent di destinazione per capire come elaborare correttamente il corpo del messaggio.
- **Content transfer encoding:** utilizzato per la conversione del corpo del messaggio nel formato originale. Ci dà delle informazioni su quello che è contenuto nell'involucro.
- **MIME version:** indica la versione del protocollo MIME.
- **Content description:** serve al destinatario per capire se vale la pena o meno decodificare un messaggio stesso.

7.2.5 Consegna finale

Adesso dobbiamo trasferire una copia del messaggio che è arrivata al mail server e deve essere trasferita al destinatario. Questi protocolli che ci permettono di accedere alla casella di posta elettronica sono:

- POP3 *post office protocol*
- IMAP *internet mail access protocol*

POP3

L'user agent apre una connessione TCP sulla porta 110 con il mail server. Quando viene stabilita questa connessione si hanno 3 fasi:

1. *Autorizzazione:* fa in modo che l'user agent possa trasmettere sia l'username che la password, così che possa accedere alle risorse del server.
2. *Transazione:* l'user agent prende i messaggi di interesse e marca quelli da cancellare.
3. *Aggiornamento:* il client effettua un quit, e il mail server elimina i messaggi marcati in precedenza.

POP3 lavora in modalità scarica e cancella, dunque le mail vengono scaricate in locale e cancellati immediatamente dal server (quelle marcate).

IMAP

Cerca di migliorare POP3; in IMAP il mail server sta in ascolto sulla porta 143 e nasce per risolvere il problema di gestione degli spazi. Viene introdotta la modalità di lettura delle mail direttamente da internet, piuttosto che in locale; inoltre supporta dei meccanismi per leggere tutti o parte dei messaggi. esistono vari comandi IMAP:

- LOGIN: Autenticare l'utente sul server
- SELCET: selezionare una cartella
- EXAMINE: selezionare una cartella in sola lettura
- CREATE: creare una cartella
- DELETE: cancellare una cartella
- RENAME: rinominare una cartella

- LIST: elencare le cartelle disponibili
- STATUS: controllare lo stato di una cartella
- LOGOUT: scollegare e chiudere la sessione

Webmail

La web mail è l'alternativa più popolare ai precedenti metodi; in questa architettura il provider gestisce un server di posta elettronica, accettando i messaggi degli utenti usando SMTP sulla porta 25. Solo lo user agent sarà diverso: invece di essere un programma indipendente è un'interfaccia utente che viene fornita tramite una pagina web; questo significa che gli utenti potranno utilizzare qualsiasi browser per accedere alla loro e-mail. Se la procedura di autenticazione a successo, server trova la casella di posta dall'utente e crea una pagina web andando a visualizzare la casella di posta dell'utente; questa pagina verrà inviato al browser per la visualizzazione. I messaggi verranno inviati dal mail server al browser di destinazione attraverso HTTP, I mittenti manderanno i messaggi ai mail server attraverso il browser utilizzando http.

7.2.6 SSL, TLS e STARTTLS

SSL e TLS Forniscono entrambi un modo di criptare un canale di comunicazione tra 2 computer ovvero l'host e il server. TLS è il successore di SSL, ed entrambi i termini sono usati in modo interscambiabile. STARTLS È un modo di prendere una connessione esistente non sicura e renderla sicura usando SSL/TLS.

SSL (*secure socket layer*)

È un protocollo progettato per consentire alle applicazioni di trasmettere informazioni in modo sicuro e protetto. Le applicazioni che utilizzano i certificati SSL sono in grado di gestire l'invio e la ricezione di chiavi di protezione e di criptare/decriptare le informazioni trasmesse utilizzando le stesse chiavi. Alcune applicazioni sono già in grado di ricevere connessioni tramite l'utilizzo di 'SSL, tra queste troviamo i web browser come Internet Explorer e Firefox, programmi di gestione della posta come Outlook, Mozilla Thunderbird, Apple Mail app, e programmi SFTP (*Secure File Transfer Protocol*). Per stabilire una connessione sicura tramite SSL, è necessario che l'applicazione abbia una chiave di protezione, che deve essere assegnata da un'Authority preposta che la rilascerà sotto forma di certificato. Quando navighiamo con una connessione che si basa su **protocolli SSL** lo capiamo già dall'indirizzo sulla barra di navigazione, dove vedremo un lucchetto e anziché *http://* troveremo *https://* proprio per specificare che si sta effettuando una connessione sicura che fa uso di **certificati SSL o TLS**. La garanzia fornita agli utenti dai **protocolli SSL** sull'autenticazione del dominio del sito cui si collega e sulla reale identità dell'azienda collegata a quel dominio.

STARTTLS

StartTLS viene utilizzato come estensione del protocollo, principalmente nella comunicazione tramite e-mail per i protocolli SMTP, IMAP e POP. Un fattore che complica la situazione è che alcuni software usano in modo scorretto il termine TLS quando dovrebbero usare STARTTLS: alcune vecchie versioni di Thunderbird usavano TLS per dire di forzare l'uso di STARTTLS cercando di migliorare la connessione, oppure fallisci se non veniva supportata e "TLS if available" per dire di usare STARTTLS se il server lo supporta, altrimenti usare una connessione non sicura. Questa situazione è problematica quando si deve configurare un numero di porta per ogni protocollo.

per aggiungere sicurezza ad alcuni protocolli esistenti (IMAP, POP..) si decise di aggiungere il criptaggio SSL/TLS come livello sottostante al protocollo esistente: per specificare che il software deve comunicare con la versione criptata SSL/TLS veniva utilizzato un numero di porta diverso per ogni protocollo:

- IMAP usa la porta 143, ma la versione criptata usa la porta 993
- POP usa la porta 110, ma la versione criptata usa la porta 995
- SMTP usa la porta 25, ma la versione che usa la porta 465

Un certo punto si decise che avere 2 porte per ogni protocollo era troppo costoso, e che invece si doveva avere una porta che all'inizio era plaintext (testo in chiaro), Ma permetteva il client di passare ad una connessione criptata. STARTTLS Fu creato proprio per questo. Però c'erano già alcuni problemi perché esistevano dei software che usavano i numeri di porta alternati con connessioni SSL/TLS, e visto che questi possono vivere a lungo non si potevano disabilitare le porte criptate fino a quando non è stato aggiornato tutto il software.

Furono giunti dei meccanismi a ogni protocollo per dire ai client il protocollo plaintext supportava l'aggiornamento a SSL/TLS e che non dovevano cercare di loggarsi senza fare l'aggiornamento. Questo creò 2 situazioni spiacevoli:

1. alcuni software ignoravano la notifica “login disabilitato fino all'aggiornamento” che cercavano comunque di loggarsi inviando user name e password in chiaro. Anche se il server rifiutava il login, i dettagli erano già stati mandati in chiaro su internet.
2. Altri software vedevano la notifica ma non aggiornavano la connessione automaticamente e quindi riportava un errore di login l'utente, causando confusione su cosa stava effettivamente andando storto.

Entrambi questi problemi erano significativi per quanto riguarda la compatibilità con i client esistenti quindi la maggior parte degli amministratori di sistema continuava ad usare connessioni in chiaro su un numero di porta, e le connessioni criptate su un numero di porta separato.

Questo è diventato lo standard de facto, molti siti ora disabilitano l'IMAP semplice (porta 143) e quello POP (porta 110) così gli utenti devono usare per forza connessioni criptate. Questo rimuove completamente STARTTLS come opzione per le connessioni IMAP/POP.

SMTP come eccezione

L'unica eccezione è SMTP, poiché la maggior parte dei software e-mail (user agent) usavano SMTP sulla porta 25 per inviare messaggi al mail server. Però questo sistema in origine era progettato solo per il trasferimento per cui per l'invio fu definita la porta 587. Anche se questa porta non obbliga a STARTTLS io sto utilizzo divenne popolare quando si realizzò che il criptaggio SSL/TLS delle comunicazioni tra client e server era una questione importante per la sicurezza, per la privacy e quando vennero definite delle estensioni the criptaggio per SMTP. Quindi poco dopo la definizione della porta 465, fu revocata con l'aspettativa che i client sarebbero passati ad usare STARTTLS sulla porta 587.

Il risultato è che nella maggior parte dei casi, sistemi che offrono l'invio di messaggi sulla porta 587 richiedono ai client di usare STARTTLS per migliorare la connessione e richiedono anche username password per l'autenticazione. Un altro beneficio è che allontanando gli utenti dall'uso della porta 25 per l'invio di e-mail, gli ISP possono bloccare le connessioni utenti alla porta 25 the computer che

erano sorgente di spam significativa per via dei virus. Sfortunatamente però lo svantaggio di cambiare i numeri di porta perché diversi client mail supportavano solo SSL/TLS sulla porta 465 e non STARTTLS sulla porta 587: i client molto spesso vivono a lungo e quindi rimuovere la porta 465 non era un'opzione per molti siti senza infastidire i vari utenti.

7.3 World Wide Web

Il World Wide Web è un'infrastruttura per l'accesso a documenti collegati, sparsi su milioni di macchine in internet.

7.3.1 Panoramica dell'architettura

Il web per definizione una vasta raccolta mondiale di documenti o pagine web: ogni pagina può contenere collegamenti ad altre pagine situati ovunque nel mondo (*link*). Queste pagine vengono visualizzate con un programma chiamato browser ad esempio Chrome, Firefox eccetera; il **browser** per le va la pagina richiesta, interpreta il testo e i comandi di formattazione e quindi va a visualizzare la pagina correttamente formattata sullo schermo. Il contenuto può essere di diversi tipi: testo, immagini oppure video e programmi. Un pezzo di testo, icona, immagine o altro collegati ad un'altra pagina è chiamato **collegamento ipertestuale**.

Le pagine possono trovarsi sulla stessa macchina oppure su macchine differenti sparse lungo il globo: l'utente non può saperlo perché il recupero della pagina è svolto unicamente dal browser. Il funzionamento del web è il seguente: ogni pagina viene presa inviando una richiesta a uno o più server che rispondono inviando i contenuti della pagina, il protocollo di domanda e risposta per catturare le pagine è un semplice protocollo basato sul testo che funziona su TCP, **HTTP** (*hypertext transfer protocol*). Se il documento è sempre uguale siamo in presenza di una **pagina statica**, mentre se è generato su richiesta da un programma o contiene del codice eseguibile è chiamato **pagina dinamica**; questa si presenta in modo diverso ogni volta che viene visualizzata.

Il lato client

Per operare correttamente il web deve implementare un meccanismo per denominare e individuare le pagine, in pratica doveva rispondere a 3 domande:

1. Come si chiama la pagina
2. dov'è situata la pagina
3. com'è possibile accedere alla pagina

la soluzione scelta identifica le pagine in modo da risolvere tutti e 3 i quesiti contemporaneamente: ad ogni pagina è assegnato un **URL** (*uniform resource locator*) che serve come nome mondiale per la pagina. Gli URL sono composti da 3 parti: il protocollo, il nome DNS della macchina su cui è situata la pagina e un percorso che indica in modo univoco la pagina specifica; Tuttavia l'interpretazione del percorso è compito del server.

Quando un utente fa clic un collegamento ipertestuale il browser esegui una serie di passaggi per recuperare la pagina:

1. il browser determina l'URL
2. il browser richiede al DNS l'indirizzo IP del server
3. il DNS con l'effettivo indirizzo

4. il browser esegui una connessione TCP sulla porta 80
5. invia una richiesta HTTP per la pagina
6. il server invia la pagina come risposta http
7. se la pagina include URL necessari alla visualizzazione il browser prende gli altri URL usando lo stesso procedimento
8. il browser visualizza la pagina
9. la connessione viene rilasciata

Questo degli URL è uno schema aperto, nel senso che consente ai browser di utilizzare più protocolli per raggiungere tipi diversi di risorse.

Nome	Utilizzato per	Esempio
http	Iper testo (HTML)	http://www.ee.uwa.edu/~rob/
https	Iper testo con sicurezza	https://www.bank.com/accounts/
ftp	Trasferimento file con FTP	ftp://ftp.cs.vu.nl/pub/minix/README
ci	File locale	file:///usr/suzanne/prog.c

L'utilizzo crescente del web ha evidenziato una debolezza nello schema degli URL: un URL fa riferimento ad un host specifico, ma a volte è utile riferirsi a una pagina senza dover dire contemporaneamente dove sia. Per risolvere questo problema gli URL sono stati generalizzati in URI (*uniform resource identifier*): gli URL dicono dov'è situata una risorsa, mentre gli URI dicono il nome della risorsa, non dove si trova; questi sono chiamati **URN** (*uniform resource name*). Gli URL sono usati per allocare e trovare le risorse, mentre gli URN sono usati per l'identificazione.

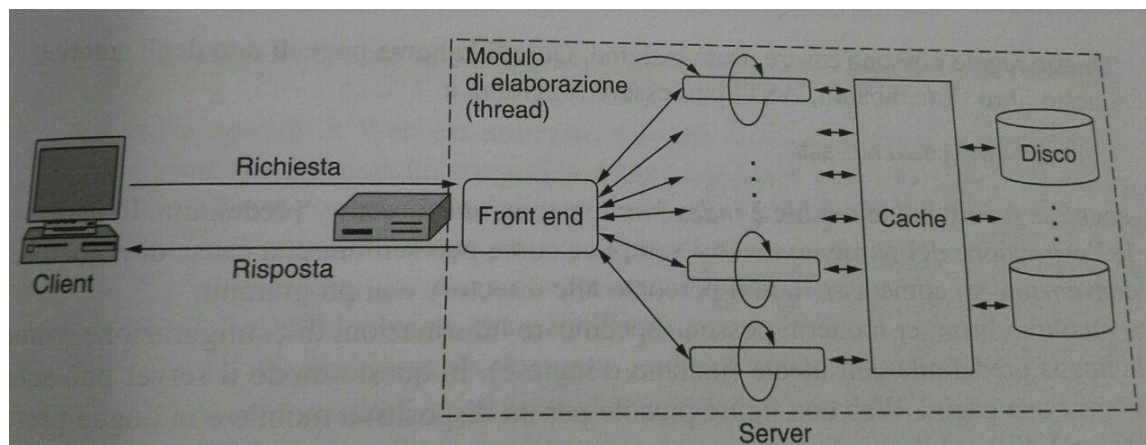
Il lato server

Il server web è un vero e proprio server e come tale viene fornito il nome di un file da cercare che dovrà restituire. I passaggi eseguiti dal server sono i seguenti:

1. accettare una connessione TCP da un browser
2. ottenere il percorso della pagina
3. ottenere il file del disco
4. inviare il contenuto del file al client
5. rilasciare la connessione

Per i contenuti dinamici il terzo passo viene sostituito dall'esecuzione di un programma che andrà a restituire i contenuti. Un problema di questo semplice schema è che l'accesso al disco è spesso il collo di bottiglia per cui la lettura è molto lenta rispetto l'esecuzione di un programma e gli stessi file possono essere letti ripetutamente, inoltre può essere elaborata solo una richiesta alla volta e i file possono essere grandi e quindi bloccare le altre richieste mentre vengono trasferiti. Un miglioramento a queste problematiche consiste nel mantenere all'interno di una cache gli *n* file utilizzati più di recente o un certo numero di gigabyte: anche se l'utilizzo effettivo di questo sistema richiede un'elevata quantità di memoria e un tempo di elaborazione extra controllare la cache il gioco vale la

candela. Invece per ovviare il problema di servire una singola richiesta nel server viene implementato un approccio **multithread**.



Quando arriva una richiesta il front end l'accetta e costruisce un piccolo record che la descrive, passerà quindi questo record a uno dei moduli di elaborazione per l'erogazione del servizio. Il modulo di elaborazione controlla per prima cosa la cache per vedere se contiene il file: se non è presente avvia un'operazione su disco per la lettura e quando il file viene recuperato verrà poi inserito nella cache e consegnato al client, altrimenti aggiorna il record per includervi un puntatore al file. Il vantaggio è che mentre uno più moduli di elaborazione sono bloccati in attesa del completamento di un'operazione su disco o di rete, altri moduli possono lavorare su altre richieste.

I cookie

I server non possono usare l'indirizzo IP per tenere traccia degli utenti per ovviare a questo problema vengono utilizzati cookie: quando un client richiede una pagina web il server può fornire informazioni aggiuntive sotto forma di cookie insieme alla pagina richiesta. Un cookie è un piccolo file o stringa che i server possono associare ad un browser; quest'ultimi vanno ad archiviare i cookie in un'apposita directory sul disco rigido del client. Questi cookie però potrebbero teoricamente contenere un virus visto che sono trattati come dati.

Un cookie può contenere fino a 5 campi:

- Dominio: indica da dove proviene il cookie.
- Path: ovvero il percorso nella struttura delle directory del server.
- Contenuto: che assume la forma nome = valore dove queste stringhe rappresentano il contenuto del cookie.
- Scadenza: specifica quando scade il cookie. Se questo campo è assente il browser scarcerà il cookie alla sua chiusura e un cookie di questo tipo ed è definito **cookie non persistente**. Se vengono fornite invece una data e un'ora dico chi è **persistente**.
- Sicuro: può essere impostato per indicare che il browser può restituire i cookie un server solo usando un sistema di trasporto sicuro SSL/TLS.

Il browser prima di inviare una richiesta di una pagina di qualche sito web, controlla nella directory dei cookie se sono presenti cookie inseriti dal dominio a cui si rivolge la richiesta: in questo caso tutti i cookie inseriti di tale dominio vengono inclusi nel messaggio di richiesta.

Un impiego più controverso dei cookie è quello di tracciare il comportamento online degli utenti, questo permette agli operatori di siti web di capire come gli utenti navigano nel loro sito. Il punto focale è che gli utenti tipicamente non sono consci che la loro attività in quel momento viene tracciata. Per questa ragione i cookie che tracciano gli utenti attraverso i siti sono considerati da molti come spyware. Per mantenere la privacy degli utenti molti browser permettono di bloccare i cookie provenienti da terze parti e questo permette di prevenire il tracciamento di più siti web. Naturalmente non possiamo eliminare del tutto i cookie altrimenti molti siti web non funzionerebbero bene.

7.3.2 HTTP – hypertext transfer protocol

HTTP è un protocollo a livello applicativo basato sul modello di domanda e risposta, normalmente implementato su TCP: specifica quali messaggi i client possono inviare ai server e quali risposte vengono restituite. Viene usato per il trasferimento di risorse web (pagine, file, immagini...). Anche se il meccanismo si basa su domanda e risposta tra client e server, è possibile che sistemi aggiuntivi nel mezzo partecipino allo scambio di messaggi: questi possono essere proxy, cache web, gateway o altri sistemi. Le intestazioni delle domande e delle risposte sono formulate in ASCII.

Uno dei vantaggi di http è che si appoggia ad una connessione TCP e per questo né i browser e né i server devono preoccuparsi di messaggi lunghi, affidabilità o controllo di congestione, perché ci pensa il TCP.

HTTP 1.1

In HTTP 1.1 dopo aver stabilito la connessione, veniva inviata una singola richiesta e restituita una singola risposta e a questo punto la connessione TCP veniva rilasciata. Successivamente fu migliorato questo aspetto permettendo a più transazioni di avvenire su una connessione TCP singola e persistente. Possiamo dire che HTTP 1.1 supporta le **connessioni persistenti**, dove attraverso questa tecnologia è possibile inviare più richieste ed ottenere più risposte; questa strategia è detta **riutilizzo della connessione**.

Sono state apportati anche altri miglioramenti:

- La consegna di pagine generate dinamicamente è stata migliorata supportando la codifica di trasferimento a pezzi, che permette di inviare una risposta prima che si conosca la sua lunghezza totale.
- Altre aggiunte permettono a più domini di essere serviti
- da un singolo indirizzo IP, permettendo un uso più efficiente dello spazio di indirizzamento.

Nella prima versione di http, 1.0, il client doveva stabilire una nuova connessione TCP separata per ogni risorsa da procurare, e questo creava una perdita di tempo: ogni connessione TCP richiede un tempo di andata e di ritorno per essere stabilita e inoltre all'inizio di questa connessione si usa la procedura slow start per aumentare il throughput e la conseguenza di questo periodo di "riscaldamento" è che tante connessioni TCP brevi hanno tempo più lungo di una singola lunga. Inoltre la continua chiusura e apertura di connessioni TCP per ogni risorsa aumenta i ritardi di accesso e l'overhead.

In http 1.1 le risposte devono arrivare in ordine così come le richieste, per cui se ci sono problemi di connessione potremmo avere seri problemi con la richiesta http: questo problema prende il nome di blocking head offline. Un esempio pratico è che se il primo pacchetto di risposta viene perso tutti gli altri rimangono bloccati.

Sempre la v1.1 introduce il pipeline che permette al cliente di inviare parecchie richieste http sulla stessa connessione, comunque sia le risposte devono sempre arrivare in ordine. Invece nella versione successiva 2.0 si risolve il problema descritto in precedenza con il multiplexing delle richieste sulla singola connessione così il client può effettuare richieste multiple senza aspettare che le precedenti siano complete e possono arrivare in qualsiasi ordine.

HTTP 2

Questa nuova versione di http riduci l'impatto della latenza delle applicazioni web e utilizza il TLS default ammortizzando i costi per l'intera applicazione. Inoltre si vanno migliorare le performance dei siti web diminuendo la terza per migliorare la velocità di caricamento delle pagine nei vari web browser:

- **compressione dei dati degli header http e codifica binari:** Nella prima versione di http i dati dell'header venivano inviati in chiaro, invece adesso vengono compressi secondo uno standard che usa la codifica di Huffman. per cui questi dati presenti nell'header vengono inviati in binario anziché in ASCII, riducendo la dimensione del file; mi casi in cui gli header non compressi entriamo in un singolo pacchetto TCP, la compressione non aiuta. Sia la compressione sia la codifica binaria rendono difficile il debug rispetto al testo in chiaro.
- **server push:** Questa funzionalità permette al server di inviare dati al browser senza dover aspettare che quest'ultimo li richiede prima. Questo è utile perché normalmente dobbiamo aspettare che il browser faccia il parsing dei file .html e trovi i link necessari prima che queste risorse possono essere inviate. Effettuare il push aiuta anche ad assicurarsi un miglior uso della banda all'inizio della connessione
- **richieste in parallelo**
- **si risolve il problema del blocking head offline**

Quello che c'è di nuovo in questa versione è il modo in cui i dati vengono messi nel frame e trasportati tra client e server: i miglioramenti di performance vengono dal multiplexing delle richieste e delle risposte per evitare il problema del blocking head offline, dalla compressione del header e dall'assegnazione di priorità alle richieste. I siti web che sono efficienti minimizzare il numero di richieste necessarie per indirizzare un'intera pagina. Http 2 Permetta server di effettuare il push del contenuto cioè di rispondere attraverso dati la più richiesta del client tutto questo senza l'overhead di un ciclo di richiesta aggiuntivo.

7.3.3 FTP e TFTP

FTP (*file transfer protocol*), È un protocollo per la trasmissione di dati tra host basato su TCP e un architettura client-server. Fu progettato per permettere agli utenti di trasferire file in modo efficiente e affidabile da un host ad un altro su internet: il protocollo utilizza connessioni TCP distinte per trasferire i dati e per controllare i trasferimenti, inoltre richiede un'autenticazione del client attraverso una coppia account-password oppure il server può essere configurato per accettare le connessioni anonime (senza restrizioni). Gli obiettivi principali di FTP sono:

- Promuovere la condivisione di file
- incoraggiare l'uso diretto o implicito di computer remoti
- risolvere l'incompatibilità tra differenti sistemi di stoccaggio file tra host

- trasferire dati in maniera affidabile ed efficiente
- fornire la replicazione di dati, che permette un backup di dati su larga scala

L'accesso ad un file system remoto avviene in 3 step:

1. il client FTP setta una connessione TCP con un server FTP remoto
2. vengono inviati username e password come comandi FTP
3. I file possono essere trasferiti da/al server

A differenza del protocollo HTTP, FTP utilizza 2 connessioni separate di tipo TCP per gestire i dati (non persistente) e il controllo (persistente). Entrambe le connessioni possono essere aperte in 2 modalità differenti:

- **Attiva:** la connessione di controllo via iniziata dal client mentre la connessione per i dati viene iniziata dal server.
- **Passiva:** entrambe le connessioni vengono iniziate dal client.

Dato che FTP trasmette in chiaro le credenziali e ogni comunicazione che avviene all'interno dell'infrastruttura, e visto che non dispone di meccanismi di autenticazione del server presso il client, il protocollo è spesso reso sicuro utilizzando un sottostrato SSL/TLS; questa variante è chiamata FTPS.

TFTP (RFC 783)

È un protocollo di trasferimento file a livello applicativo, sfrutta le funzionalità di base del FTP. E' stato progettato per UDP e non possiede meccanismi di autenticazione o cifratura. Viene impiegato per trasferire i file tra i vari processi, il tutto con un overhead minimo (poiché non ha sicurezza): questo lo rende anche facile da implementare e molto piccolo di dimensione. Questo protocollo prevede 5 messaggi:

- richiesta di lettura
- richiesta di scrittura
- dati
- ACK
- errore

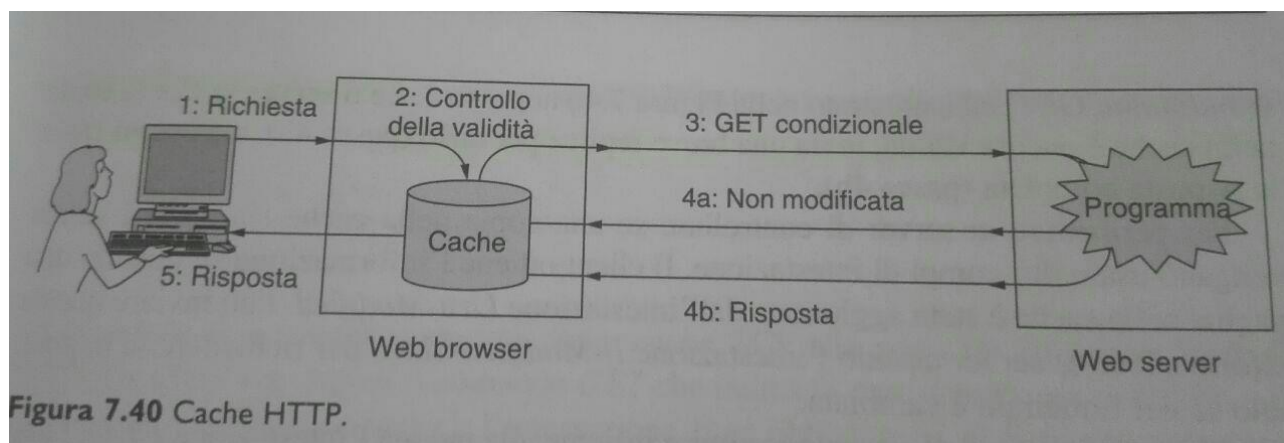
Per gestire i pacchetti persi il mittente, che può essere sia client che il server, utilizza un timeout e la ritrasmissione. Inoltre bisogna prevedere i pacchetti di dati duplicati che devono essere conosciuti, quindi ignorati, e infine ritrasmettere l'ACK: il mittente non dovrebbe inviare di nuovo un pacchetto di dati in risposta ad un ACK duplicato.

7.3.4 Caching

Il web caching è un meccanismo per accelerare il download di un file sul web: la procedura di memorizzare in una cache delle pagine recuperate per farne un uso successivo viene chiamata **caching**. Il vantaggio è che quando una pagina è memorizzata nella cache, non è necessario ripetere il trasferimento; proprio come viene computer viene mantenuta localmente nella cache del server una

copia del contenuto remoto precedentemente procurato per accedervi in futuro, tutto ciò per migliorare l'efficienza di banda e la reattività delle esperienze di navigazione.

Le pagine di solito sono memorizzate sul disco, in modo che siano riutilizzabili quando il browser viene eseguito in un momento successivo.



Il problema difficile da risolvere è come determinare che la copia di una pagina precedentemente memorizzata nella cache sia la stessa che verrebbe recuperata nuovamente; questa decisione non può essere presa solo sulla base del URL, perché quest'ultimo potrebbe dare una pagina che visualizza le ultime notizie. Il procedimento per affrontare questo problema è quello mostrato nella figura precedente: la prima strategia, dopo la richiesta, consiste nella validazione della pagina, ovvero si consulta la cache se ha una copia della pagina richiesta che è ancora valida. Per stabilire questa validità la cache va effettuare una richiesta **GET condizionale** al server, e se quest'ultimo sa che la copia è ancora valida invia una breve replica (non modificata) altrimenti invia la risposta completa.

Proxy trasparente

In generale le richieste http possono essere instradate lungo una serie di cache, e l'uso di una cache esterna al browser è chiamata proxy caching. Questo sistema aiuta ad inoltrare le interrogazioni alle destinazioni appropriate se avviene un cache miss; la destinazione di inoltro potrebbe essere un'altra cache server o il web server corrispondente. Si hanno 2 vantaggi:

1. viene eliminato l'overhead del rinvio della richiesta dal client al web server.
2. può essere raggiunto il controllo massimo della rete concentrando tutti gli accessi sul proxy server e monitorarli.

7.4 Streaming audio e video

Con la crescita sempre maggiore di Internet, cresce sempre di più anche l'esigenza di trasferire audio e video in tempo reale tra utenti. Queste ultime applicazioni differiscono da quelle Web poiché sono poco tolleranti al ritardo, quindi i dati devono essere obbligatoriamente inviati a tassi fissati affinché l'utente possa essere soddisfatto del servizio. Per l'esecuzione di musica o film sulla rete rimane cruciale la variazione sul ritardo, chiamata **jitter**, che deve essere mascherata dall'esecutore, altrimenti la qualità audio/video sarà pietosa. Se un video in streaming fosse trasmesso a scatti a causa del ritardo che impiegano alcuni pacchetti ad essere trasmessi, l'utente non si divertirà. Invece il caricamento delle pagine Web può richiedere anche del tempo, purché limitato, e non creare nessun problema. Il traffico audio e video è aumentato così tanto in Internet sia perché i computer sono sempre più potenti, sia perché è stata resa disponibile una banda enorme su Internet. Le compagnie telefoniche approfittarono di ciò, sfruttando Internet per trasportare il traffico audio e rendere meno

costose le chiamate, fu così che nacque voice over IP (VoIP) e la telefonia in Internet. Assumiamo che sia disponibile abbastanza banda per trasmettere tutto il traffico, ora il problema da affrontare è quello del ritardo in rete quando si parla di applicazioni di streaming e di teleconferenza. Si procede per gradi, analizzando dapprima come avviene lo streaming di video registrati, come nel caso dei video su YouTube, per poi passare all'analisi dello streaming in tempo reale, come quello delle dirette radiofoniche, infine si analizzano le videoconferenze interattive e le chiamate su Internet come Skype.

7.4.1 Audio digitale

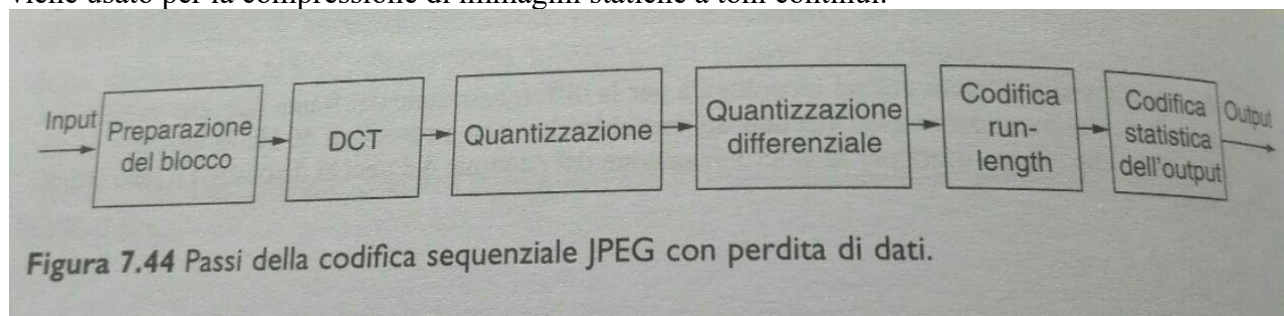
Un'onda audio è un'onda acustica monodimensionale: nel momento in cui la voce entra in contatto con un microfono, viene generato un segnale elettrico, che rappresenta l'ampiezza del suono in funzione del tempo. L'orecchio umano è capace di percepire il suono contenuto nel range delle frequenze 20 e 20 000 Hz, ma l'orecchio ascolta in modo logaritmico, da 0 a 120 dB. Poiché un'onda acustica è un'onda continua, dunque rappresenta un segnale analogico, per poter essere elaborato ha bisogno di essere convertito in un segnale digitale, per mezzo di un convertitore da analogico a digitale chiamato AD. Esistono molti esempi di suoni campionati, fra cui i CD e il telefono. Essenzialmente sono due le operazioni che devono essere effettuate per trasformare un segnale da analogico a digitale e da digitale ad analogico: la codifica e la decodifica. È naturale pensare che per il processo di codifica avremo bisogno di un hardware più costoso e di conseguenza più lento poiché è un processo che avviene una volta sola, e il documento multimediale viene salvato sul server multimediale, mentre la decodifica viene effettuata da molti utenti diversi. La compressione audio viene effettuata per non occupare tutta la banda disponibile e favorire anche altri tipi di traffico, e lo standard maggiormente utilizzato per file audio è MP3, mentre MPEG consente sia la compressione audio sia quella video. Una volta che i dati vengono convertiti, potremo avere una perdita di alcuni bit, in questo caso la codifica viene detta **lossy**, mentre se tutti i bit vengono trasferiti con successo si ha una codifica **lossless**, ossia senza perdita. Durante la compressione audio non è necessario rendere reversibile il processo di codifica/decodifica, ma è accettabile che sia leggermente diverso; inoltre il campionamento può essere effettuato usando un canale, *mono*, o 2, *stereo*.

7.4.2 Video digitale

Un video è un insieme di fotogrammi chiamati in gergo frame. Ciascun frame è formato da un insieme di pixel, dove ognuno di questi rappresenta un bit di informazione sul colore. L'occhio umano è meno sensibile alle variazioni rispetto all'orecchio, per cui anche se qualche frame viene perso non viene percepito alla vista. Naturalmente inviare frame più grandi su Internet significa usare più banda, che non sempre è disponibile. Per questo motivo anche per inviare video su Internet è necessaria la compressione.

JPEG

JPEG è abbastanza complicato, e la compressione ha più o meno lo stesso costo della decompressione; viene usato per la compressione di immagini statiche a toni continui.



MPEG

MPEG è asimmetrico, quindi la compressione è molto più costosa della decompressione. All'inizio MPEG veniva utilizzato per salvare film all'interno dei CD, poi per i DVD. MPEG comprime sia l'audio che i video, ma i codificatori audio e video lavorano in modo indipendente, e vi è un problema su come i due flussi vengono sincronizzati in uscita. La soluzione sta nell'avere un orologio che fornisce dei marcatori temporali per entrambi i flussi. L'output di MPEG è composto da 3 tipi di frame:

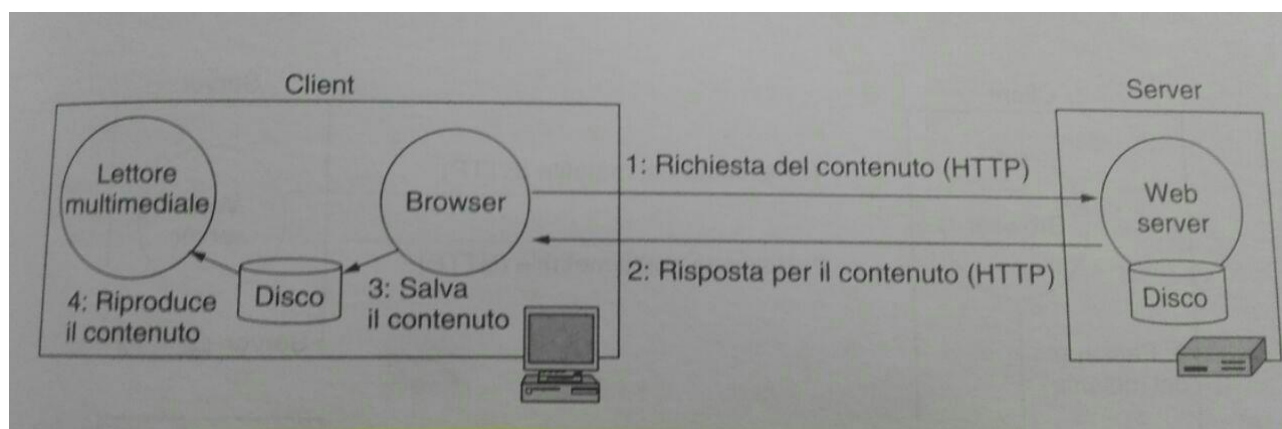
1. **I-frame** (intracodificati) → immagini statiche compresse e auto-contenute. Sono delle semplici immagini statiche ed è necessario che compaiano 1 o 2 volte al secondo. Codifica indipendente del frame: questo perché lo standard non può essere utilizzato per una trasmissione multicast, se si verificasse un errore su un frame non si potrebbe effettuare la decodifica.
2. **P-frame** (predittivi) → si codifica la differenza relativa all'I-frame, blocco per blocco.
3. **B-frame** (bidirezionali) → differenze blocco per blocco con il frame precedente e quello successivo. Si codifica la differenza relativa al frame interpolato. Questa libertà del blocco di trovarsi, nel frame precedente e in quello successivo, permette una compensazione del movimento migliorata.

Sfrutta la similarità tra frame consecutivi, però è un tipo di codifica complessa.

7.4.3 Streaming dei contenuti registrati

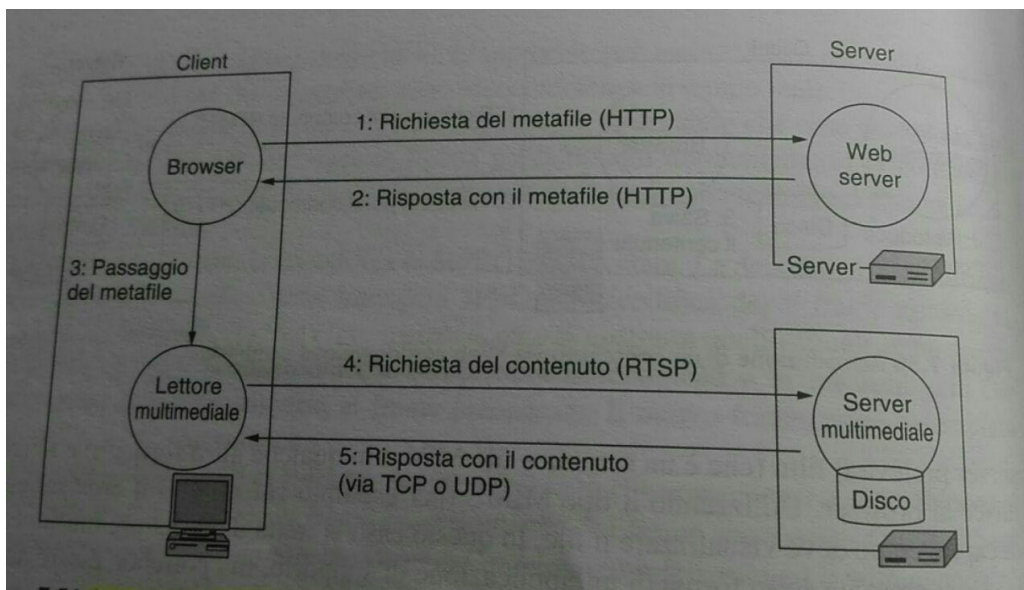
Questo streaming comprende dei contenuti già archiviati in file (*stored media*), e l'esempio più comune è quello di guardare video su internet; è una forma di **video on demand VoD**.

Riproduzione di un contenuto via web con download semplice



Il problema di questa architettura è che l'intero video deve essere trasmesso sulla rete prima dell'inizio della riproduzione, questo problema riguarda anche l'audio. Chiaramente non vi sarà nessun problema di rete in tempo reale perché si tratta semplicemente di un download. Usare HTTP significa avere limitazioni di interattività.

Streaming usando un server web e un server multimediale



La pagina collegata al film non è il vero e proprio file ma si tratta di un **metafile**, ovvero un file molto piccolo il cui scopo è quello di assegnare al film da vedere. Al passo 3 si avvia il lettore multimediale che legge i file e vede che contiene l'URL di dove si trova il film, viene contattato quindi il server multimediale al quale viene richiesto il film stesso. Il vantaggio di questo schema è che il lettore inizia velocemente, subito dopo il download del metafile. In questo esempio il server multimediale utilizza **RTSP** (*real time streaming protocol*) ed è specializzato nello streaming, quindi non userà mai HTTP; invece il lettore multimediale deve svolgere quattro operazioni fondamentali:

1. Gestire l'interfaccia utente.
2. Gestire gli errori di trasmissione: se la qualità si degrada troppo, il sistema di streaming può provare il recupero, attraverso trasmissioni ridondanti, richieste di ritrasmissione e interpolazione.
3. Decomprimere il contenuto.
4. Eliminare il jitter.

Per quanto riguarda il passo 5, bisogna trovare un compromesso tra affidabilità (ritrasmissione) e prestazioni; quindi si ritorna alla scelta fra TCP e UDP. Il punto chiave è che i vuoti devono essere molto brevi, quasi impercettibili. Con UDP si ha un buffering di 5sec per riprodurre il jitter, invece con TCP si ha una qualità migliore, ma il buffer può essere vuoto (starvation) e quindi riprodurre pause indesiderate durante la riproduzione.

RTSP (*real time streaming protocol*)

È un protocollo che permette al server che controlla la riproduzione e al media player di scambiare informazioni, e prevede anche l'interazione con la trasmissione del flusso multimediale.

7.4.5 Conferenze in tempo reale

Ora il traffico vocale è trasportato attraverso Internet; questa tecnologia è conosciuta come **VoIP** (*voice over IP*). L'approccio impiegato è utilizzare la tecnologia IP per costruire reti telefoniche a lunga distanza, e visto che queste apparecchiature risultano essere molto meno care rispetto a quelle

utilizzate per le telecomunicazioni, i servizi risultano essere più a buon mercato. A loro vantaggio hanno anche un minor consumo di banda, semplicità e estendibilità. Questi sistemi devono essere progettati usando tecniche che minimizzino la latenza, si parte da UDP; il ritardo può subire aumenti attraverso la distanza fisica e alla dimensione dei pacchetti. VoIP utilizza dei pacchetti piccoli per ridurre la latenza, al costo di ridurre l'efficienza di banda: avremo un ritardo di trasmissione minore, per via della dimensione ridotta dei pacchetti. Un altro punto focale è l'**overhead** introdotto dal software, in particolare per i video; per cui sia il codificatore che il decodificatore devono agire velocemente. Il buffer è ancora necessario per riprodurre i campioni multimediali, ma la quantità di buffer utilizzata deve essere mantenuta bassa: esiste un compromesso tra la dimensione del buffer usato per gestire il jitter e la quantità di contenuto perso, un buffer piccolo riduce la latenza ma genera più perdite dovute al jitter, e prima o poi queste perdite diventeranno evidenti per l'utente. Ai pacchetti VoIP viene assegnata l'etichetta a "basso ritardo", per farli saltare in cima alla coda, inoltre, sempre per prevenire il ritardo, dobbiamo assicurarci che ci sia banda sufficiente, poiché se essa varia o il tasso di trasmissione fluttua e a volte non c'è proprio banda sufficiente si creano code che portano a ritardi. Un altro problema è come aprire e chiudere le chiamate: ci sono due soluzioni H.323 e SIP.

SIP (*session initiation protocol*)

Questo protocollo a livello applicazione descrive come impostare le telefonate su internet e altre connessioni multimediali. SIP è un modulo singolo progettato per lavorare con applicazioni internet esistenti: Può stabilire sessioni tra 2 partecipanti, tra più partecipanti e sessioni multicast; tutte possono contenere audio, video o dati. Gestisce solo l'impostazione, la gestione e la terminazione delle sessioni (altri protocolli come RTP/RTCP sono usati per il trasporto dei dati), e può essere implementato su TCP o UDP. Questo protocollo prevede l'individuazione del chiamato e la determinazione della sua capacità, nonché i meccanismi di impostazione e terminazione della chiamata. In SIP i numeri telefonici sono rappresentati come URL, che possono anche contenere indirizzi IPv4, IPv6 o numeri telefonici veri e propri, anche perché usare gli IP è complicato ed inutile per via del DHCP. Una chiamata in SIP avviene nel seguente modo:

1. Supponiamo che Alice conosca l'IP di Bob
2. Viene mandato un messaggio di INVITE su UDP, porta 5060
3. Bob risponde sulla stessa porta attraverso un messaggio di risposta SIP
4. Gli utenti potrebbero codificare l'audio diversamente. La conversazione è simultanea.
5. È un protocollo out-of-band: i messaggi vengono inviati su diverse socket rispetto a quelle usate per audio e video.
6. I messaggi SIP sono in ASCII

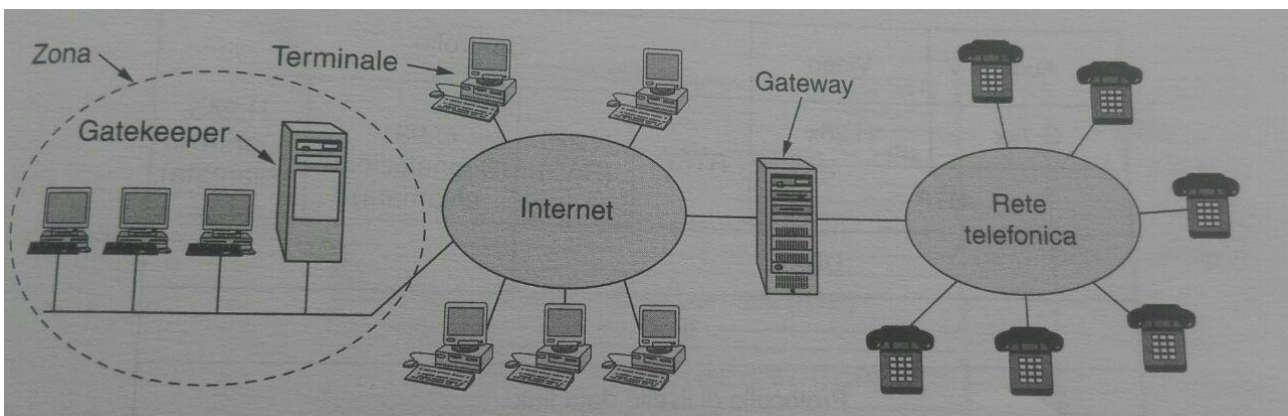
SIP si basa su un approccio client-server, dove si hanno un User Agent Client (UAC) e un User Agent Server (UAS): viene impiegato un *proxy server* SIP, che agisce per conto di un client, e inoltra le richieste ai server, possibilmente dopo averle tradotte, un *redirect server* che risponde alle richieste del client informandolo dell'indirizzo del server richiesto e infine un *location server* usato per gestire le richieste che arrivano dal proxy server o dal redirect server in merito alla possibile locazione del chiamato.

Server STUN

Un server **STUN** (*session traversal of UDP through NAT*) permette ai client NAT, cioè telefoni IP dietro firewall, di settare chiamate verso un provider VoIP fuori dalla rete locale. Permette ai client di scoprire il loro indirizzo pubblico, il tipo di NAT dietro il quale sono e la porta internet associata dal NAT ad una porta locale particolare: tutte queste informazioni vengono usate per settare una comunicazione UDP tra client e provider VoIP al fine di stabilire una chiamata. Il server STUN viene contattato sulla porta UDP 3478, ma a volte utilizza un secondo indirizzo IP.

H.323

H.323 non è un vero e proprio protocollo, ma rappresenta una panoramica architetturale della telefonia Internet: fa riferimento a diversi protocolli specializzati. Al centro del modello si trova il **gateway**, che connette internet al sistema telefonico: una LAN può avere in **gatekeeper** che controlla i punti finali sotto la sua giurisdizione fornendo la traduzione di indirizzi, il controllo di ammissione e il controllo della banda. All'interno di quest'area, detta zona, abbiamo anche i **Terminali**, dove usualmente un software a lato client viene usato per inizializzare la comunicazione a due vie con un altro terminale H.323, con un gateway o con un gatekeeper. Infine si ha l'**unità di controllo multipunto**, che sostanzialmente è un endpoint che manipola tre o più terminali o gateway che partecipano ad una conferenza multipunto. Può essere standalone o integrata nel gateway/gatekeeper.



Quando un terminale vuole connettersi ad un altro per una conversazione di voce, per prima cosa manda una richiesta al gatekeeper, se richiesto per l'ammissione della chiamata; se ammesso il chiamante manda una richiesta di connessione in cui specifica l'indirizzo di destinazione e la porta. Al terminale remoto, come ad esempio: `“://host@domain:port”`. dopo alcune negoziazioni di capacità viene costruito un canale di comunicazione per i 2 terminali. Questo protocollo può essere diviso in 2 piani:

1. **piano di controllo:** coordina i processi this setup e di chiusura di una sessione VoIP.
2. **piano dei dati:** si occupa della codifica e della trasmissione dei dati voice e multimediali.

Confronto fra H.323 e SIP

Entrambi i protocolli consentono chiamate a 2 o più partecipanti, utilizzando come punti finali computer e telefoni. H.323 È un tipico protocollo standard che definisce tutta la pila dei protocolli e specifica che cosa è consentito fare e cosa è negato: il prezzo da pagare è uno standard grande, rigido e complesso. al contrario SIP è un tipico protocollo di internet basato sullo scambio di brevi righe di testo ASCII, il che non è un di un protocollo leggero può essere adattato facilmente alle nuove applicazioni; lo svantaggio è rappresentato da problemi di interoperabilità.

Skype

Skype è una rete proprietaria di telefonia in internet basata su un'architettura peer to peer, che permette le chiamate a numeri di telefono regolari così come si fa fra computer attraverso l'indirizzo IP. Le chiamate effettuate attraverso Skype possono essere di 3 tipi:

- da Skype a Skype
- da Skype a rete telefonica/mobile
- da rete telefonica/mobile a Skype

I segnali vengono criptati usando RC4, mentre i dati voice con AES. In Skype abbiamo 3 diverse entità: **client Skype**, ossia un'applicazione Skype che può essere usata per chiamare e mandare messaggi di testo, **server di login**, che è l'unico server centralizzato responsabile della memorizzazione di user name e password e dell'autenticazione all'accesso, **supernodo**, ovvero un end point a cui si connettono i client Skype. Ogni nodo che ha un indirizzo IP pubblico con sufficienti valori di CPU, memoria e banda è candidato a diventare un super modo. La fase di login avviene in 3 passaggi:

1. il client autentica il suo user name con password al server di login, poi notifica la sua presenza agli altri peer.
2. viene determinato il tipo di NAT dietro il quale opera.
3. scopre i nodi Skype online con indirizzi IP pubblici per costruire un'Host Cache (HC), una tabella con gli IP e le porte dei supernodi.

Esistono diverse tipologie di chiamate Skype:

- **Skype2Skype: IP pubblici:** si ha quando il ricevente e il chiamante hanno IP pubblico, allora possono stabilire una chiamata attraverso una connessione TCP. I media invece vengono trasferiti attraverso UDP.
- **Skype2Skype: dietro NAT:** il NAT impedisce ad un peer esterno di iniziare una chiamata verso un peer interno, soluzione:
 1. ogni client è connesso al suo super nodo che non è dietro una NAT
 2. il client A informa il super nodo che vuole chiamare B
 3. Il super nodo di A informa il super nodo di B che ha sua volta in forma B
 4. se B Accetta la chiamata allora un terzo peer non NATtato viene scelto, che andrà a trasferire i dati tra A e B
- **Skype to PSTN:** per skype to PSTN (skype out) l'applicazione contatta inizialmente il super modo e poi il gateway PSTN alla porta 12340. I server gateway sono una parte separata dell'architettura e non una parte della rete ricoperta. Per PSTN to skype (skype in) si usa il processo opposto.

8.0 Sicurezza delle reti

Uno dei compiti fondamentali della sicurezza delle reti è quello di impedire che determinate persone possano accedere a servizi remoti che non sono autorizzate ad usare. I problemi di sicurezza delle reti si possono dividere in 4 categorie interconnesse fra loro:

- **Segretezza:** si occupa di mantenere le informazioni fuori dalla portata degli utenti non autorizzati
- **Autenticazione:** si occupa di stabilire l'identità del soggetto con cui stiamo comunicando
- **Non ripudio:** questo termine riguarda le firme, ovvero come si può dimostrare che il cliente ha veramente inviato un ordine elettronico
- **Controllo dell'integrità:** come si fa ad essere sicuri che il messaggio che abbiamo ricevuto è realmente quello spedito.

8.1 Crittografia

Cifrato: s'intende una trasformazione carattere per carattere senza considerare la struttura linguistica del messaggio.

Codice: rimpiazza ogni parola con un'altra parola o simbolo.

La crittografia può essere raggruppata in 4 aree:

- **Criptaggio simmetrico:** usato per nascondere i contenuti dei blocchi o dei flussi di dati di qualunque dimensione, inclusi i messaggi, i file, le chiavi e le password.
- **Criptaggio asimmetrico:** usato per nascondere piccoli blocchi di dati come le chiavi e i valori di funzione di hash.
- **Algoritmi di integrità dei dati:** usati per proteggere i blocchi di dati, come i messaggi, dall'alterazione.
- **Protocolli di autenticazione:** schemi basati sull'uso di algoritmi crittografici progettati per autenticare l'identità delle entità.

8.1.1 Introduzione alla crittografia

Un messaggio da cifrare, detto *testo in chiaro*, viene trasformato tramite una funzione parametrizzata da una *chiave*; l'output del processo di cifratura è noto come *testo cifrato*. Assumiamo che il nemico, o *intruso*, ascolti accuratamente e trascriva completamente il testo cifrato: al contrario del destinatario legittimo l'intruso non conosce la chiave di decifrazione e quindi sarà molto difficile se non impossibile decifrare il messaggio. Con il termine **decriptare** facciamo riferimento all'attività di decifrazione da parte dell'intruso, mentre **decifrare** è l'operazione legittimo di lettura di un messaggio cifrato.

Principio di Kerckhoff

“Tutti gli algoritmi devono essere pubblici, solo le chiavi sono segrete”. La mancanza di segretezza dell'algoritmo è fondamentale, poiché cercare di tenerlo segreto non funziona mai. Una caratteristica fondamentale è la lunghezza della chiave: più lunga la chiave è più alto è il fattore di lavoro del

criptoanalista. Visto che il lavoro per rompere un sistema con una ricerca completa nello spazio delle chiavi cresce esponenzialmente con la lunghezza della chiave stessa.

8.1.2 Cifrari a sostituzione

In un cifrario a sostituzione, ogni lettera o gruppo di lettere viene rimpiazzato da un'altra lettera o gruppo di lettere per mascherare il messaggio. Uno dei cifrati più antichi è il *cifrario di Cesare*. In questo metodo, *a* diventa *D*, *b* diventa *E*, *e* diventa *F*, e così via; una semplice generalizzazione del cifrario di Cesare consiste nello spostare l'alfabeto del testo cifrato di *k* lettere, dove *k* vale 3 per il cifrario di Cesare descritto sopra. In questo caso *k* chiama il generale; un miglioramento successivo consiste nel far sì che uno dei simboli del testo in chiaro corrisponda ad altre lettere.

Testo in chiaro: a b c d e f g h

Testo cifrato: Q W E R T Y U

Il sistema generale per la sostituzione simbolo a simbolo viene chiamato **sostituzione monoalfabetica**.

8.1.3 Cifrari a trasposizione

I cifrari a sostituzione conservano l'ordine dei simboli del testo in chiaro, limitandosi a mascherare la loro apparenza; i *cifrari a trasposizione* riordinano le lettere ma non le mascherano.

Trasposizione colonnare

Nella trasposizione colonnare il messaggio è scritto lungo le righe di una griglia di dimensioni prefissate e poi letto lungo le colonne, secondo un ordine particolare delle stesse. Sia la lunghezza delle righe che la permutazione delle colonne sono definite da una parola chiave. Ad esempio, la parola VETRINA è lunga 7 lettere, così le righe saranno anch'esse di lunghezza 7, e la permutazione è definita dall'ordine alfabetico delle lettere della parola chiave. In questo caso l'ordine sarebbe "7-2-6-5-3-4-1" perché l'ordine alfabetico delle lettere di VETRINA è A-E-I-N-R-T-V, e le loro posizioni sono quindi A=1, E=2, I=3, N=4, R=5, T=6 e V=7, per cui V-E-T-R-I-N-A corrisponde proprio a 7-2-6-5-3-4-1. Nei cifrari a trasposizione colonnare regolari le posizioni vuote alla fine dell'ultima riga vengono riempite con caratteri casuali; alla fine, il messaggio è letto sulle colonne secondo l'ordine specificato dalla parola chiave. Ad esempio, supponendo di usare la parola chiave VETRINA e il messaggio PIANTARE IL CAMPO DIETRO LA COLLINA, in una trasposizione colonnare regolare avremmo una griglia così composta.

V	E	T	R	I	N	A
-	-	-	-	-	-	-
7	2	6	5	3	4	1
-	-	-	-	-	-	-
P	I	A	N	T	A	R
E	I	L	C	A	M	P
O	D	I	E	T	R	O
L	A	C	O	L	L	I
N	A	D	V	Y	I	Q

8.2 Algoritmi a chiave simmetrica

Questi algoritmi utilizzano la stessa chiave per cifrare e decifrare i messaggi, seguendo sempre il principio di Kerckhoff. Il criptaggio simmetrico trasforma il testo in chiaro in un testo cifrato usando la stessa chiave segreta e un algoritmo di criptaggio: in questo modo il testo in chiaro verrà ricavato dal testo cifrato. Gli attacchi possibili a questo sistema sono 2:

- *Crittoanalisi*: basata su proprietà dell'algoritmo di criptaggio
- *Forza bruta*: che consiste nel provare tutte le chiavi possibili

I codici simmetrici tradizionali utilizzano delle tecniche di sostituzione e trasposizione (come abbiamo visto nel paragrafo precedente): dove le tecniche di sostituzione mappano gli elementi in chiaro in elementi cifrati, mentre le tecniche di trasposizione traspongono sistematicamente le posizioni degli elementi in chiaro. La particolarità di utilizzare la stessa chiave per entrambe le operazioni rende gli algoritmi a chiave simmetrica adatti ad un ampio uso.

One-time-pad: sicurezza teorica vs pratica

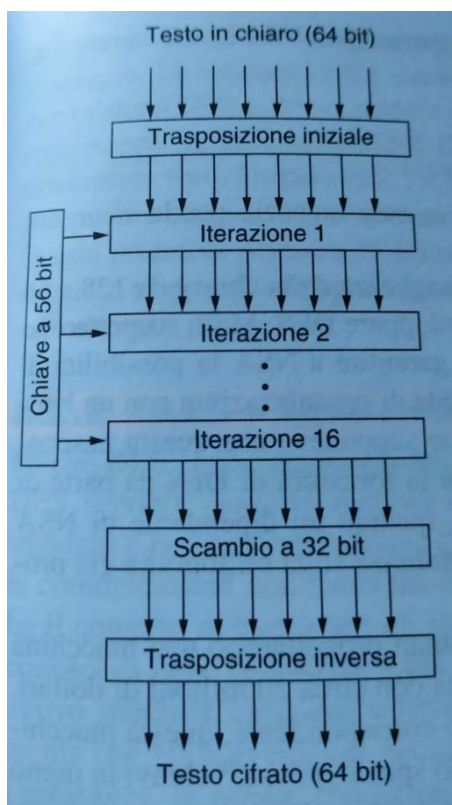
Un cifrato One-time-pad è un cifrato di flusso, in cui il flusso della chiave è un flusso casuale di bit; ha un elevatissimo livello di sicurezza (Shannon 1949). La chiave deve essere lunga quanto il testo in chiaro. I cifrati nella pratica usano chiavi molto più piccole e non sono incondizionatamente sicuri come i One-time-pad, ma computazionalmente non possono essere scalfiti. Provare questa caratteristica non è facile:

- non abbastanza da considerare solo gli attacchi forza bruta (dimensione della chiave)
- un cifrario potrebbe essere scalfito per via delle debolezze della sua struttura algebrica

Per queste ragioni non esiste una prova di sicurezza per molti cifrati nella pratica, e se esiste, di solito, si basa su assunzioni che sono credute vere (ad es. P diverso da NP).

8.2.1 DES – data encryption standard

È uno standard per la cifratura dei dati. Nella sua forma originale non è più sicuro, ma con qualche variante sì.



Il testo in chiaro viene cifrato in blocchi di 64 bit, che generano 64 bit di testo cifrato. L'algoritmo è parametrizzato da una chiave a 56bit e ha 19 stadi distinti: il primo è indipendente e corrisponde nella trasposizione dei 64 bit di testo in chiaro, l'ultimo è il suo inverso. Il penultimo consiste nello scambiare i 32 bit più a sx con i 32 a dx, mentre i rimanenti 16 sono tutti uguali, ma sono parametrizzati da diverse funzioni della chiave. L'algoritmo è stato progettato per permettere la decifrazione con la stessa chiave usata per la cifratura.

8.2.2 AES – advanced encryption standard

Questo algoritmo andò a sostituire il DES, e si basa sui seguenti principi:

- Doveva essere un cifrario simmetrico a blocchi
- Struttura interamente pubblica
- Supportare chiavi di lunghezza di 128, 192 e 256 bit
- Possibilità di implementazioni software e hardware
- Pubblico

Rijdael

Rijdael utilizza sostituzioni e permutazioni attuate in molteplici round: il numero dei round dipende dalla dimensione della chiave e del blocco. Si va da 10 per chiavi a 128bit, fino a 14 per la chiave e il blocco più lunghi. A differenza del DES tutte le operazioni coinvolgono interi byte, per avere un'implementazione efficiente sia hardware che software. L'algoritmo usato per il decriptaggio è diverso da quello usato per il criptaggio (ottimizzato per il criptaggio).

8.3 Algoritmi a chiave pubblica (asimmetrici)

Il problema di utilizzare chiavi simmetriche è che queste chiavi dovevano essere distribuite a tutti gli utenti, però nello stesso momento dovevano essere protetti dei furti. Venne ideato il sistema a chiave pubblica dove le chiavi di cifratura e decifrazione erano differenti: la chiave di decifrazione non era facilmente derivabile da quella di cifratura. I 2 algoritmi, di cifratura (E) e decifrazione (D), dovevano rispettare 3 proprietà:

1. $D(E(P)) = P$, dove P è il testo in chiaro originale
2. Risulta estremamente difficile dedurre D da E
3. E non può essere forzato da un attacco di tipo “testo in chiaro a scelta”

La crittografia a chiave pubblica richiede quindi che ciascun utente abbia 2 chiavi: una **chiave pubblica**, usata dal mondo intero per cifrare i messaggi da mandare a quell'utente, e una **chiave privata**, che l'utente usa per decifrare i messaggi. Questo tipo di criptaggio può essere impiegato per la confidenzialità e/o l'autenticazione; questi algoritmi a chiave pubblica sono basati su funzioni matematiche, piuttosto che su sostituzione e permutazione. Il procedimento è molto intuitivo e semplice:

1. il testo in chiaro viene trasformato in testo cifrato, usando una delle 2 chiavi e l'algoritmo di criptaggio.
2. usando la chiave appaiata e l'algoritmo di decriptaggio, viene ricavato il testo in chiaro da quello cifrato.

8.3.1 RSA

Non è facile trovare un sistema che rispetti le 3 proprietà descritti in precedenza, uno dei più usati è **RSA**, considerato un algoritmo molto robusto ma visto che richiede chiavi a 1024 bit per offrire un buon livello di sicurezza è abbastanza lento. Il codice RSA si basa su un procedimento che utilizza i numeri primi e funzioni matematiche che è quasi impossibile invertire. Dati due numeri primi, è molto

facile stabilire il loro prodotto, mentre è molto più difficile determinare, a partire da un determinato numero, quali numeri primi hanno prodotto quel risultato dopo essere stati moltiplicati tra loro. In questo modo si garantisce quel principio di sicurezza alla base della crittografia a chiave pubblica, infatti l'operazione di derivare la chiave segreta da quella pubblica è troppo complessa per venire eseguita in pratica. Vediamo in pratica l'algoritmo e le varie funzioni matematiche:

1. Calcolare il valore di n , prodotto di p e q , due numeri primi molto elevati (sono consigliati valori maggiori di 10^{100}). Negli esempi in seguito utilizzeremo numeri più piccoli per facilitare la lettura. Esempio: $n = p * q = 7 * 5 = 35$
2. Calcolare il valore di $z = (p-1) * (q-1)$. $z = (7-1) * (5-1) = 24$
3. Scegliere un intero D tale che D sia primo rispetto a z , il che significa che i due numeri non devono avere fattori primi in comune. Esempio: $z = 24$ $d=7$
4. Trovare un numero E tale che $E * D \bmod z = 1$, cioè che il resto della divisione tra $E * D$ e z sia 1. $E = 7 \rightarrow 49 \bmod 24 = 1$

Dopo aver calcolato questi parametri inizia la cifratura. Il testo in chiaro viene visto come una stringa di bit e viene diviso in blocchi costituiti da k bit, dove k è il più grande intero che soddisfa la disequazione $2^k < n$. A questo punto per ogni blocco M si procede con la cifratura calcolando $M_c = M^E \bmod n$. In ricezione, invece, per decifrare M_c si calcola $M_c^D \bmod n$. Come si può capire da quanto appena detto per la cifratura si devono conoscere E ed n che quindi costituiranno in qualche modo la chiave pubblica, mentre per decifrare è necessario conoscere D ed n che quindi faranno parte della chiave segreta.

8.3.2 Simmetrico vs Asimmetrico

Simmetrico

Per funzionare viene utilizzato lo stesso algoritmo con la stessa chiave per criptare e decriptare; Il mittente e destinatario condividono l'algoritmo e la chiave. Per quanto riguarda la sicurezza la chiave deve essere segreta e deve essere impossibile o almeno non pratico decifrare un messaggio se non ci sono informazioni disponibili. Se si conosce l'algoritmo utilizzato e alcuni campioni del testo cifrato deve comunque essere impossibile determinare la chiave.

Asimmetrico

Viene usato un singolo algoritmo per criptare e decriptare con l'ausilio di una coppia di chiavi, una per criptare e l'altra per decriptare; in questo caso il mittente e il destinatario devono avere ognuno una delle 2 chiavi ma non la stessa. Per ragioni di sicurezza una delle 2 chiavi deve essere segreta e deve essere impossibile o almeno non pratico decifrare un messaggio se non ci sono altre informazioni disponibili. Se si conosce un algoritmo più una delle chiavi e dei campioni del testo cifrato deve essere comunque impossibile determinare l'altra chiave.

8.4 Firme digitali e Autenticazione

Essenzialmente si ha bisogno di un sistema con il quale il mittente possa mandare un messaggio firmato al destinatario, in modo che valgano le seguenti condizioni:

1. il destinatario può verificare l'identità dichiarata dal mittente.
2. il mittente non può ripudiare in un secondo momento il contenuto del messaggio.

3. il destinatario non può falsificare i messaggi del mittente.

I concetti di firma digitale e autenticazione sono legati fra loro, ma al contempo leggermente diversi:

- **autenticazione:** processo attraverso il quale viene verificata l'identità di una persona.
- **firma digitale:** provare che un messaggio è stato effettivamente spedito da un mittente specifico.

8.4.1 Protocolli di autenticazione

Nella pratica vengono utilizzati differenti protocolli di autenticazione, i quali a loro volta hanno subito un'evoluzione nel tempo andandosi a migliorare di volta in volta.

Authentication protocol 1.0

In questo approccio non c'è nessuna informazione biometrica, solo messaggi scambiati.

Authentication protocol 2.0

in questo approccio viene utilizzato l'indirizzo IP: vengono costruiti datagrammi IP con IP artificiali si possiamo accedere al kernel del sistema operativo, questo approccio viene chiamato IP spoofing.

Authentication protocol 3.0

Questo protocollo utilizza l'approccio attraverso la password (PIN nei sistemi bancari o login nel sistema operativo). Il problema è la sicurezza visto che si può ascoltare la password, nel momento in cui vengono mandate in chiaro: se qualcuno in quel momento sniffasse la LAN potrebbe rubare le password di login.

Authentication protocol 3.1

Con questo approccio le password vengono criptate, ad esempio utilizzando una chiave simmetrica: questo evita il furto di password, ma purtroppo non risolve i problemi di autenticazione. Soffre dell'attacco *playback*: la chiave criptata può essere registrata e la si può usare per fingersi qualcun altro; ciò possibile perché in questo protocollo viene usata la stessa password più volte, infatti dovrebbe essere nuova ad ogni autenticazione. Alice e Bob potrebbero accordarsi su una sequenza di password oppure concordare un algoritmo di generazione.

Authentication protocol 4.0

E l'approccio più generale possibile, si utilizza il **Nonce**, ovvero un numero impiegato dal protocollo una sola volta. in questo caso Alice quando vuole iniziare la comunicazione dichiara di essere "Alice", Bob sceglie un R nonce e lo manda ad Alice; quest'ultima rimanderà a Bob R attraverso un sistema di crittografia simmetrica ed infine Bob andrà a decriptare il messaggio e se il nonce corrisponde a quello che aveva inviato all'inizio allora tutto è andato a buon fine.

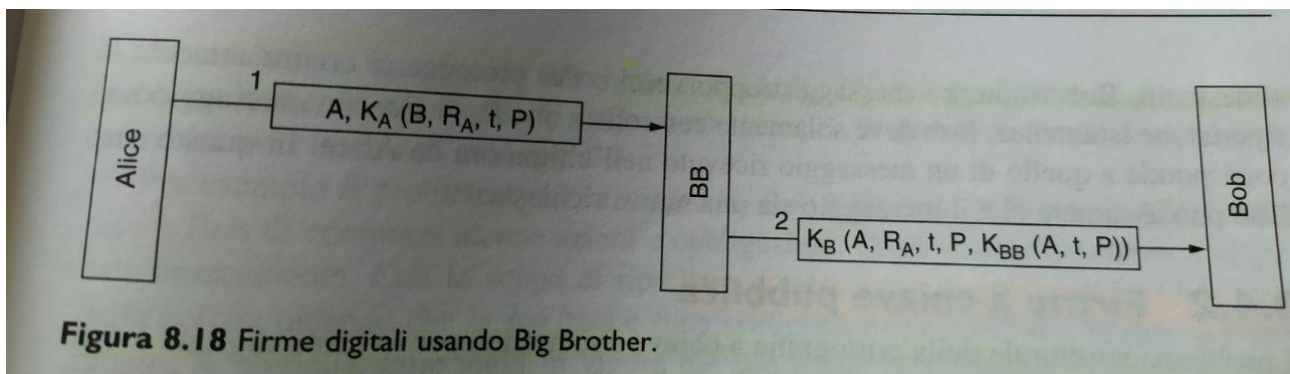
Authentication protocol 5.0

Questo protocollo ottiene lo stesso risultato del precedente, impiegando però le chiavi pubbliche. Alice invia "sono Alice", Bob sceglie R, Alice codifica R con la sua chiave privata che solo lei conosce. Il secondo momento Bob andrà a ricavarci R, chiaramente egli deve conoscere la chiave pubblica di Alice. Se Trudi mandasse un messaggio "sono Alice", Bob sceglierà un R e lo manderà ad Alice, ma Trudi intercetta il messaggio: ora lei può usare la chiave privata per criptare R. In questo caso Bob non può sapere se quella stringa di bit rappresenta R criptato attraverso la chiave privata di

Alice oppure di Trudi, per questo chiede ad Alice la sua chiave pubblica (che la può trovare tranquillamente nel suo sito web). Trudy riesce a intercettare e manda la sua chiave pubblica a Bob. Infine Bob che ha ricevuto la chiave pubblica di Trudi, credendo però che sia quella di Alice, erroneamente va a autenticare Trudi come Alice. In pratica questo piccolo esempio ci dice che questo protocollo è affidabile quando lo è la distribuzione della chiave pubblica.

8.4.2 Firme a chiave simmetrica

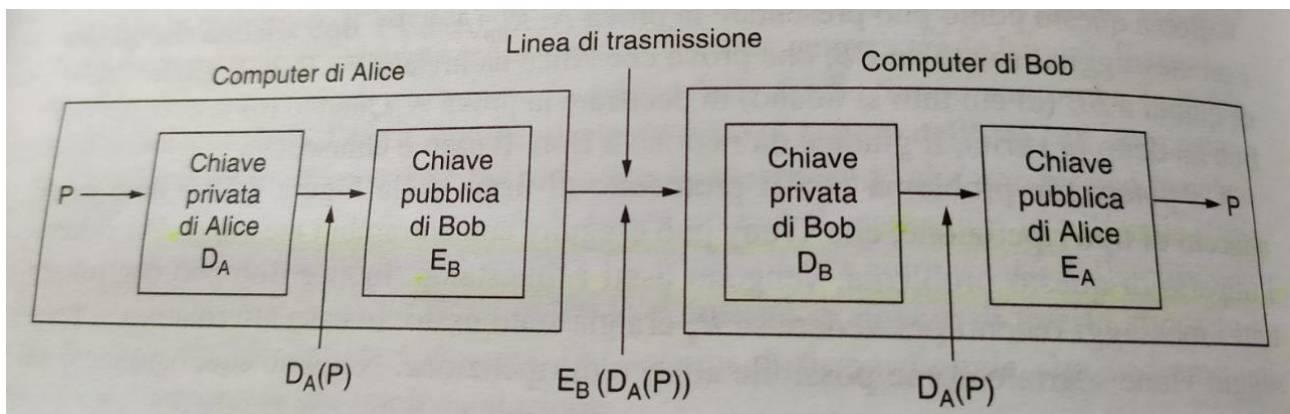
Un approccio alle firme digitali è la creazione di un'autorità centrale che le conosce tutte e di cui tutti hanno fiducia: chiamiamolo **BIG BROTHER**. In questo sistema ogni utente sceglie una chiave segreta e la porta a mano nell'ufficio di BB, quindi solamente l'utente specifico e BB conoscono la chiave segreta K .



Un potenziale problema di questo protocollo sono gli attacchi a ripetizione che ad esempio un terzo utente puoi eseguire con entrambi i messaggi; per ridurre il problema vengono usati i timestamp. Visto che basandosi su questi time stamp, Bob rifiuterà messaggi troppo vecchi.

8.4.3 Firme a chiave pubblica

Il problema strutturale della crittografia a chiave simmetrica per la firma digitale è dato dal fatto che tutti devono fidarsi di BB, il quale però allo stesso tempo può leggere i messaggi di tutti. Quindi è auspicabile avere un metodo per firmare i documenti che non richieda un'autorità fidata. La crittografia a chiave pubblica per la firma digitale è una scelta elegante, che però presenta alcuni problemi legati principalmente al contesto in cui viene utilizzata, piuttosto che agli algoritmi impiegati.



Come primo punto Bob può provare che un messaggio è stato realmente mandato ad Alice solo se la chiave D_A rimane segreta; poiché essa Alice rende pubblica la sua chiave segreta il discorso non vale.

più in quanto chiunque potrebbe avere inviato il messaggio. Un altro problema sorge quando Alice decide di cambiare la sua chiave: quest'operazione è comunque legale inoltre è buona idea farla periodicamente.

8.4.4 Message Digest

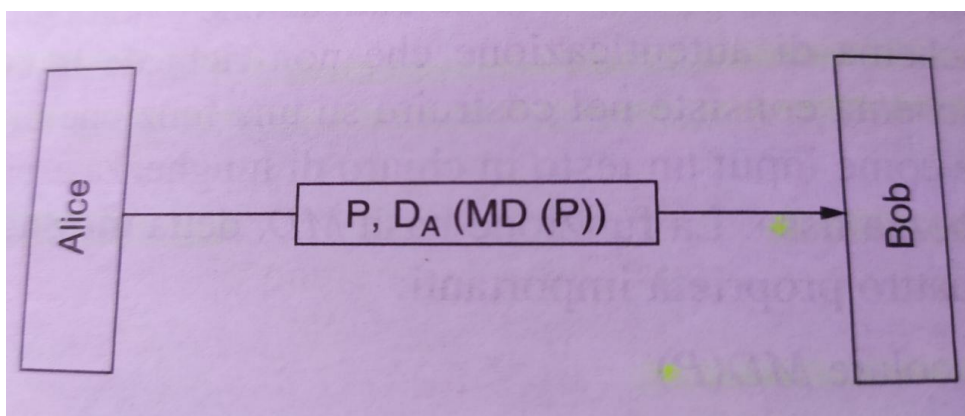
Spesso i metodi di firma digitale sono triplicati: nel loro funzionamento riuniscono 2 funzioni distinte, autenticazione e segretezza. Di seguito descriveremo uno schema di autenticazione che non richiede la cifratura dell'intero messaggio: questo schema consiste nel costruire su una funzione di tipo hash monodirezionale, che prende in input un testo in chiaro e calcola una stringa di bit a lunghezza fissa. La funzione di hash utilizzata è chiamata **message digest**, **MD** (riassunto del messaggio) e a 4 parametri importanti:

1. dato P , è facile ricavare $MD(P)$
2. dato $MD(P)$ è praticamente impossibile trovare P
3. dato P nessuno è in grado di trovare P' , tale che $MD(P)' = MD(P)$. In questo caso l'hash dovrebbe essere lungo almeno 128 bit.
4. se l'input varia anche di un bit, l'output diventa completamente diverso. Per soddisfare questo criterio l'hash deve modificare in modo radicale i bit in input.

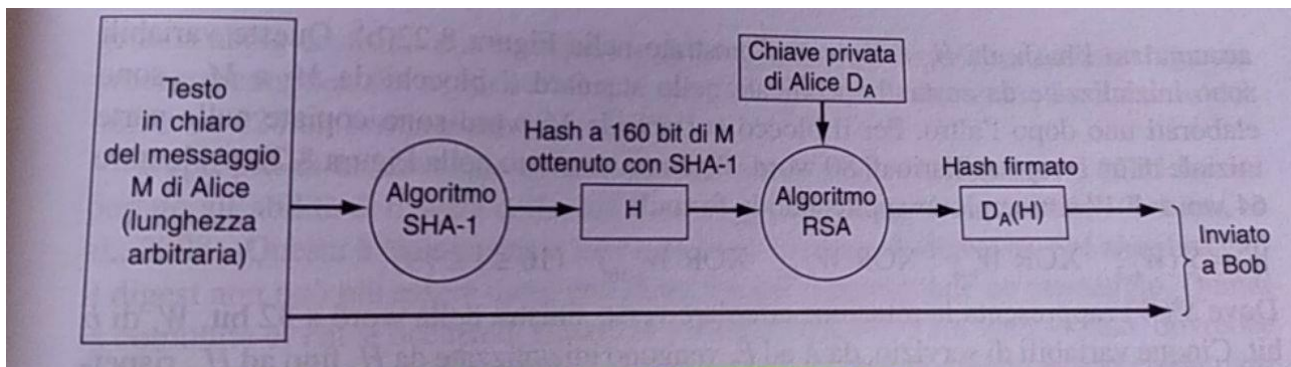
I message digest funzionano anche con sistemi a chiave pubblica.

SHA-1 e SHA-2

SHA-1 è una funzione usata per il message digest, andando a distruggere i bit in un modo sufficientemente complicato per cui ogni bit di output è influenzato da ogni bit di input. Il testo in chiaro viene utilizzato come input per l'algoritmo SHA-1 per ottenere l'hash a 160 bit; l'esempio mostrato nella figura seguente mostra come Alice può mandare un messaggio a Bob non segreto ma firmato. Per prima cosa Alice andrà a firmare l'hash con la sua chiave privata RSA per poi trasmettere Bob sia il testo in chiaro sia l'hash.



Dopo aver ricevuto il messaggio Bob calcola l'hash SHA-1 dal testo in chiaro e applica la chiave pubblica di Alice all'hash firmato per ottenere quello originale.



Lo schema precedente è ampiamente usato per trasmettere messaggi dove l'integrità è importante, ma i contenuti sono segreti; il tutto con un costo di calcolo relativamente basso. Sebbene sia ancora ampiamente utilizzato SHA-1 e ritenuto instabile e non sicuro, con l'andare del tempo verrà sostituito da nuove varianti. Nuove versioni di SHA-1 sono state sviluppate per produrre hash da 224, 256, 384 e 512 bit; non solo questi hash sono più lunghi del primo, ma la funzione è stata cambiata per combattere alcune debolezze.

8.4.5 Attacco del compleanno

L'attacco del compleanno è un tipo di attacco crittografico utilizzato per la crittoanalisi degli algoritmi di cifratura; è così chiamato perché sfrutta i principi matematici del paradosso del compleanno nella teoria della probabilità. L'attacco del compleanno si può riassumere come segue: data una funzione f , lo scopo dell'attacco è quello di trovare 2 numeri x_1 e x_2 , tali che $f(x_1) = f(x_2)$; Questa coppia di valori è chiamata collisione. Il metodo utilizzato per trovare una collisione è semplicemente quello di valutare la funzione f per differenti valori di input, che possono essere scelti casualmente fino a che non si ottiene lo stesso risultato.

Le firme digitali possono essere vulnerabili ad un attacco del compleanno. Un messaggio m è generalmente firmato prima calcolando $f(m)$, dove f è una funzione crittografica di hash, e poi usando una qualche chiave segreta per firmare $f(m)$. Supponiamo che Alice voglia imbrogliare Bob introducendolo a firmare un contratto fraudolento: Alice prepara un contratto m corretto e uno m' fraudolento, poi trova un certo numero di punti nel contratto per cui m può essere modificato senza cambiare il significato (ad esempio inserendo delle virgole, spazi vuoti). Combinando queste modifiche Alice potrà creare tante varianti del messaggio originale che sono in pratica tutti contratti corretti, in maniera simile crea anche un gran numero di varianti del contratto fraudolento m' . Alla fine Alice applica la funzione di hash tutte queste varianti finché non trova quella del contratto corretto e una variante di quello fraudolento che hanno lo stesso valore di hash, cioè $f(m) = f(m')$. A questo punto Alice presenta a Bob la versione corretta per la firma, dopo che Bob lo ha firmato Alice prende la firma e la attacca al contratto fraudolento: questo prova che Bob ha firmato un contratto fraudolento.

8.4.6 HMAC

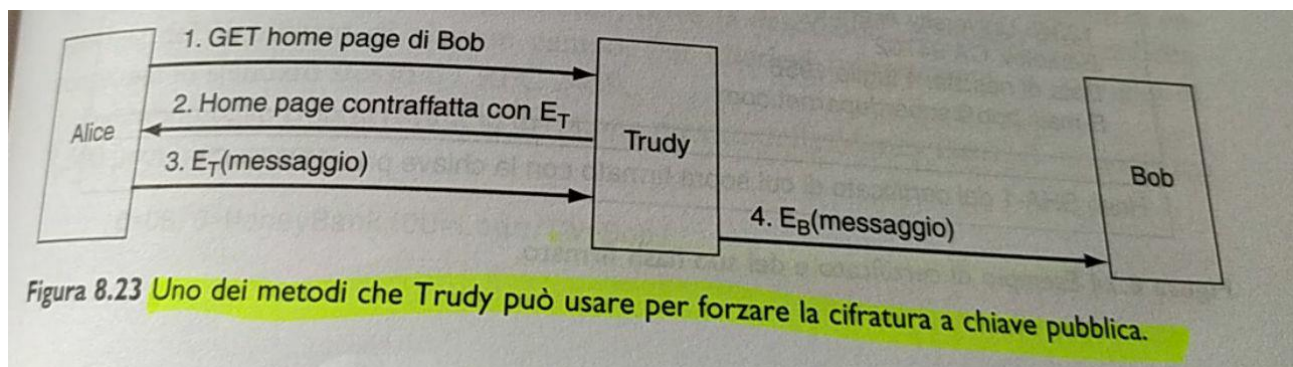
HMAC (*keyed-hash message authentication code*) è una modalità per l'autenticazione di messaggi (message authentication code) basata su una funzione di hash. Tramite HMAC è infatti possibile garantire sia l'integrità, sia l'autenticità di un messaggio; utilizza una combinazione del messaggio originale e una chiave segreta per la generazione del codice. Una caratteristica peculiare di HMAC è il non essere legato a nessuna funzione di hash in particolare, questo per rendere possibile una sostituzione della funzione nel caso non fosse abbastanza sicura. Nonostante ciò le funzioni più utilizzate sono MD5 e SHA-1, entrambe attualmente considerate poco sicure.

8.5 Distribuzione e certificazione delle chiavi

Uno svantaggio della crittografia simmetrica è che entrambe le parti devono accordarsi sulla chiave prima della comunicazione, per questo si fa affidamento a un'autorità chiamata **KDC** (key distribution center). Alice cifra un messaggio con K_{a-kdc} dicendo che desidera comunicare con Bob e lo manderà al KDC; il server decodifica il messaggio e genera un numero random R_1 inviando un messaggio criptato ad Alice con:

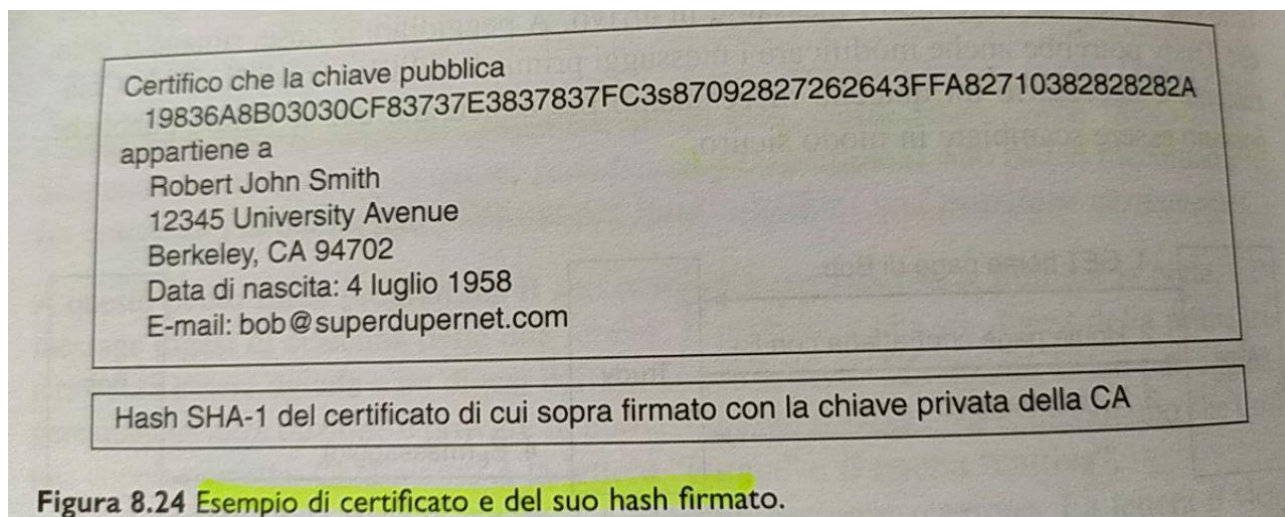
- R_1 , ovvero la chiave della sessione
- il nome di Alice ed R_1 criptati con la chiave di Bob, K_{b-kdc}
- Alice decodifica il messaggio ed estrae R_1 e $K_{b-kdc}(A, R_1)$ che viene inviato a Bob
- Bob decifra il messaggio utilizzando K_{b-kdc} : ora conosce la chiave della sessione R_1 e la persona che condivide con lui la chiave, ossia Alice
- ora Bob può autenticare Alice usando R_1 e comunicare

Nei sistemi a chiave pubblica invece gli utenti possono scambiarsi le chiavi tramite e-mail o usando pagine web. L'esempio più incalzante è e-commerce, dove in un primo momento Bob invia un messaggio in chiaro e lo firma digitalmente, successivamente Alice ottiene la chiave pubblica di Bob e controlla se Bob ha davvero ordinato. Potrebbe succedere però che Trudy dice di essere Bob e richiede una consegna, firma il messaggio con la sua chiave privata e allega la sua chiave pubblica dicendo che è quella di Bob: Alice applicherà la chiave pubblica di Trudy alla firma digitale e concluderà che il messaggio in chiaro è stato generato da Bob. Questo piccolo esempio ci fa notare che per usare la crittografia chiave pubblica, gli utenti, i browser e i router devono sapere che la chiave pubblica appartiene davvero all'entità corrispondente, quindi sarà necessario un qualche meccanismo per assicurarsi che le chiavi pubbliche possono essere scambiate in modo sicuro.



8.5.1 Certificati

L'**autorità di certificazione (CA)** va a certificare le chiavi pubbliche che appartengono alle persone, alle aziende e alle altre organizzazioni. Questa autorità va a garantire che un'entità sia davvero chi dice di essere, andando a rilasciare un certificato che autentica la corrispondenza *chiave pubblica* <-> *entità*. Questo certificato contiene: la chiave pubblica e le informazioni sul proprietario, tutto firmato digitalmente dalla CA.



Facendo riferimento all'esempio precedente dell' e-commerce, quando Alice riceve l'ordine di Bob cerca il suo certificato; viene usata la chiave pubblica della CA per controllare l'affidabilità della chiave del messaggio e si assumiamo che la chiave pubblica della CA sia "universalmente conosciuta", Alice si può fidare di Bob.

8.5.2 X.509

È uno standard largamente utilizzato su internet, che ha lo scopo di descrivere i certificati. I campi primari di un certificato di questo tipo sono elencati nella figura successiva.

Campo	Significato
Version	Numero della versione di X.509
Serial number	Il certificato è univocalmente identificato da questo numero più il nome della CA
Signature algorithm	Algoritmo usato per firmare il certificato
Issuer	Nome X.500 della CA
Validity period	Inizio e fine del periodo di validità
Subject name	L'entità proprietaria della chiave da certificare
Public key	La chiave pubblica del soggetto e l'ID dell'algoritmo che la usa
Issuer ID	Facoltativo: identificativo univoco di chi emette il certificato
Subject ID	Facoltativo: identificativo univoco del soggetto del certificato
Extensions	Sono state definite molte estensioni
Signature	La firma del certificato (firmato dalla chiave privata della CA)

Figura 8.25 I campi essenziali di un certificato X.509.

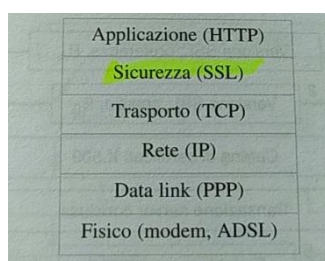
8.9 Sicurezza del web

La sicurezza del web può essere divisa in 3 categorie principali:

1. come si possono gestire in modo sicuro i nomi degli oggetti e delle risorse?
2. come si può stabilire una connessione autenticata?
3. che cosa succede quando un sito web invia ad un client un programma eseguibile?

8.9.1 SSL – secure socket layer

SSL instaura una connessione sicura fra due socket e si occupa anche di negoziare i parametri fra client e server, autenticazione del server presso il client, comunicazioni segrete e protezione dell'integrità dei dati.



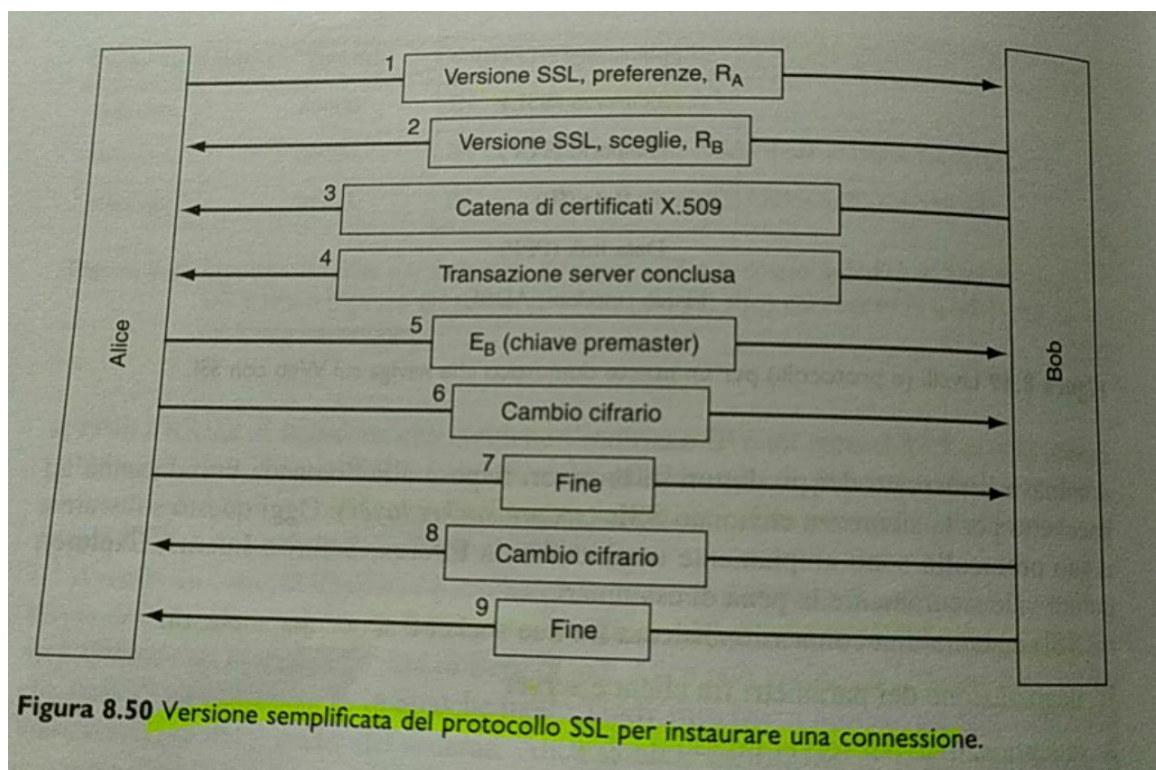
SSL è un nuovo livello interposto fra i livelli di applicazione e trasporto. Accetta e richiede dal browser e le manda a TCP perché siano trasmesse al server; dopo aver stabilito la connessione il compito principale di SSL è quello di gestire la compressione e la cifratura. HTTP usato sopra SSL prende il nome di HTTPS (*secure HTTP*), disponibile sulla porta 443. SSL supporta molti algoritmi e opzioni, che indicano per esempio, la presenza o assenza di compressione, gli algoritmi crittografici utilizzati e altri

dettagli legati alle restrizioni.

Stabilire una connessione SSL

SSL consiste in due subprotocolli, uno per stabilire le connessioni sicure e uno per usarle. Il subprotocollo usato per le connessioni sicure è mostrato nella figura seguente.

I primi 2 passi sono intuitivi: sia Bob che Alice vanno specificare la versione di SSL, le preferenze per quanto riguarda la compressione della crittografia ed infine il nonce (R). al terzo step Bob invia

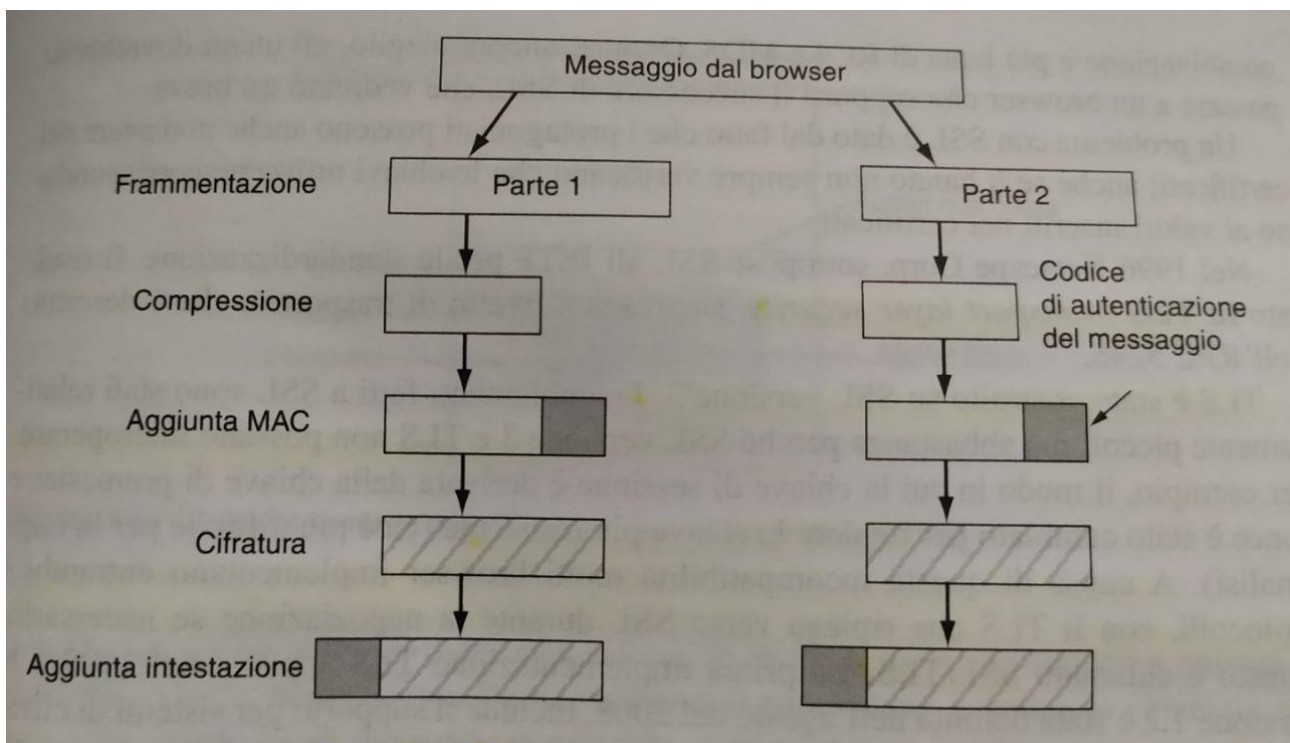


un certificato contenente la sua chiave pubblica; questo può diventare una catema se il certificato non è firmato da un'autorità nota. La chiave di premaster usata da Alice come risposta è di 384 bit, che verrà impiegata in combinazione con i 2 nonce per ricavare la chiave di sessione. con il messaggio sei Alice dice a Bob di iniziare a usare la chiave di sessione calcolata in precedenza, Ehi con il messaggio 7 stabilisce che il subprotocollo ha finito il suo compito. i messaggi 8 e 9 sono la conferma che Bob ricevuto i messaggi 6 e 7.

Trasmissione dei dati con SSL

Il trasporto dei dati inizia dopo il login, che è al di fuori di SSL. SSL Supporta diversi algoritmi crittografici, il più forte utilizza per la cifratura il triplo DES con 3 chiavi separate, oltre a SHA-1 per gestire l'integrità dei messaggi; ma visto che la combinazione di queste strategie è lenta, viene impiegata solo per operazioni di massima sicurezza. Di norma si usa RC4 con chiave a 128 bit per la cifratura e MD5 per l'autenticazione dei messaggi. Per il trasporto vero e proprio si usa il seguente sub protocollo:

1. i messaggi vengono suddivisi in unità di 16 KB, e se la compressione è abilitata ogni unità viene compresso singolarmente.
2. viene calcolata una chiave segreta a partire dai 2 nonce e dalla chiave di premaster.
3. questa nuova chiave viene concatenata il testo in chiaro e il tutto viene passato attraverso l'algoritmo di hash negoziati in precedenza. questo hash viene aggiunto ad ogni frammento come MAC.
4. il frammento compresso e il suo MAC vengono cifrati con l'algoritmo di crittografia simmetrica negoziato in precedenza.
5. Infine viene aggiunta un intestazione e si procede alla trasmissione.



È stato dimostrato che alcune chiavi deboli di questo algoritmo si possono facilmente analizzare con la crittoanalisi, quindi la sicurezza di SSL con RC4 è un pò traballante.

8.9.2 TLS – Transport Layer Security

TLS è stato costruito su SSL versione 3.1, i cambiamenti sono stati relativamente piccoli ma abbastanza perché SSL v3 e TLS non possano interoperare. Il modo in cui la chiave di sessione è derivata dalla chiave di premaster e nonce è stato cambiato per rendere la chiave più resistente. La causa di questa incompatibilità molti browser implementano entrambi i protocolli per cui sentiamo parlare di SSL/TLS.

Caratteristiche di TLS

Il protocollo TLS consente alle applicazioni client/server di comunicare attraverso una rete in modo tale da prevenire il 'tampering' (manomissione) dei dati, la falsificazione e l'intercettazione. Nell'utilizzo tipico di un browser da parte di utente finale, l'autenticazione TLS è unilaterale: è il solo server ad autenticarsi presso il client (il client, cioè, conosce l'identità del server, ma non viceversa cioè il client rimane anonimo e non autenticato sul server. Il protocollo TLS permette anche un'autenticazione *bilaterale*, tipicamente utilizzata in applicazioni aziendali, in cui entrambe le parti si autenticano in modo sicuro scambiandosi i relativi certificati. Questa autenticazione (definita mutua autenticazione) richiede che anche il client possieda un proprio certificato digitale cosa molto improbabile in un normale scenario.

Principio di funzionamento

Il funzionamento del protocollo TLS può essere suddiviso in tre fasi principali:

- Negoziazione fra le parti dell'algoritmo da utilizzare
- Scambio delle chiavi e autenticazione
- Cifratura simmetrica e autenticazione dei messaggi

Nella prima fase, il client e il server negoziano il protocollo di cifratura che sarà utilizzato nella comunicazione sicura, il protocollo per lo scambio delle chiavi e l'algoritmo di autenticazione nonché il Message authentication code (MAC). L'algoritmo per lo scambio delle chiavi e quello per l'autenticazione normalmente sono algoritmi a chiave pubblica o, come nel caso di TLS-PSK, fanno uso di una chiave precondivisa (Pre-Shared Key). L'integrità dei messaggi è garantita da una funzione di hash che usa un costrutto HMAC per il protocollo TLS o una funzione pseudorandom non standard per il protocollo SSL.

8.9.3 Autenticazione

Le password sono state il mezzo primario per verificare l'identità dell'utente sin da quando è emersa la necessità di proteggere i dati. Sempre gli utenti richiedono accesso da una serie di dispositivi mobili, e la quantità di informazioni memorizzate sui server e nel cloud continua a crescere. Da qui sorgono però più vulnerabilità, perché più hacker cercano di ricavare informazioni sensibili; c'è anche da considerare il fatto che sempre più utenti utilizzano password deboli per accedere a questi servizi.

8.9.4 Autenticazione multifattore

L'obiettivo dell'autenticazione multifattore (MFA) è di creare una difesa stratificata di 2 o più credenziali indipendenti:

- password - ciò che si sa
- gettone di sicurezza - ciò che si ha

- verifica biometrica - ciò che si è

Richiedere più fattori per autenticare l'utente rende più difficile per una persona non autorizzata ottenere l'accesso i dati sensibili.

8.9.4.1 Autenticazione a due fattori

L'autenticazione a 2 fattori, chiamata anche 2FA, va oltre il modello base di sicurezza tramite user name e password. Questo sistema è raccomandato per proteggere dati sensibili, ma a volte risulta di troppo quando gli utenti hanno password forti e uniche per ogni servizio visto che ci sarà un minor necessità di applicare un'autenticazione a 2 fattori; però nel caso reale gli utenti utilizzano sempre la stessa password per accedere dai vari servizi, semplificando il lavoro dell'hacker. Si utilizza 2FA per fare in modo che il fallimento di un fattore non garantisca accesso agli attaccanti: quindi se una password è un fattore, allora il secondo fattore può proteggere l'utente se quella password è debole. Questo fa sì che l'hacker dovrà penetrare 2 livelli di sicurezza. Ci sono molti modi per effettuare l'autenticazione a 2 fattori, oltre a user e password viene chiesto uno dei seguenti fattori:

1. *Qualcosa che si sa*: PIN, password uh qualunque altra informazione che sono l'utente dovrebbe conoscere.
2. *Qualcosa che si ha*: smart card, gettone, chiave, telefono o altri dispositivi elettronici. un elemento fisico trasportato dal dall'utente che è unico.
3. *Qualcosa che si è*: impronta, voce o scan della retina.

Esempi di 2FA: Usare una carta bancaria ed un pin per prelevare denaro da un ATM, usare un'app sullo smartphone per accedere all'home banking.

8.9.4.2 Autenticazione a due passi

La verifica a 2 passi combina un login, che include una password, con un accesso fisico ad uno smartphone un telefono fisso per verificare l'accesso autorizzato ad un account. Un utente quando effettua il login, oltre a fornire id e password, riceve un **OTC** (*one time code*) oppure una **OTP** (*one time password*) via SMS o chiamata telefonica, attraverso il numero di telefono associato all'account. Inserire queste credenziali aggiuntive costituisce un secondo passo di verifica, con l'idea che solo qualcuno che conosce la password dell'account e che possiede e fisicamente l'oggetto richiesto può tenerlo accesso. Questa versione viene spesso chiamata verifica 2 passi ed è molto confusa con l'autenticazione a 2 fattori: anche se simili ci sono 2 sistemi differenti, per via dell'infrastruttura e della metodologia di sicurezza applicati.

8.9.4.3 Sicurezza di 2FA

La sicurezza dipende dall'implementazione, dallo scenario nel quale viene impiegata e dalle risorse disponibili dell'attaccante.

8.9.4.4 Implementazione di 2FA

L'implementazione di questi sistemi prevede l'aggiunta di un fattore indipendente a username e password. Per Una protezione più sofisticata, possono essere combinati più di 2 di questi fattori per raggiungere l'autenticazione multifattore (MFU).

8.9.4.5 OTP

Generatori offline di OTP

Questi includono i gettoni OTP tradizionali: un pezzo di hardware o software usato per generare un codice più cifre, che provano il possesso di quel generatore di token. quest'ultimo semina sia il generatore sia il server con lo stesso secret simmetrico e l'OTP viene generato basandosi sull'ora corrente o su un contatore. Esistono degli elementi chiave dell'OTP:

- **sicurezza del seme:** se il generatore di OTP viene compromesso da un attaccante (oppure il server per validare la password) viene scalfito la sicurezza. in questo caso l'attaccante potrà generare l'OTP corretta in ogni momento e se possiede anche il primo fattore di autenticazione, sarà in grado di impersonare l'utente finale.
- **sicurezza del canale usato per inviare l'OTP:** come detto in precedenza il generatore è offline e se dispositivo usato viene compromesso tramite malware, oppure se l'utente invia questa informazione su un sito web fraudolento, l'attaccante potrà eseguire una singola autenticazione per conto dell'utente.
- **sicurezza del token hardware:** è importante assicurarsi che il token hardware venga consegnato il corretto utente finale, senza intercettazioni.

I gettoni OTP hanno dei difetti:

- **Usabilità:** gli utenti finali devono copiare l'OTP dal loro dispositivo in quello che richiede l'autenticazione, e la trascrizione corretta di 6/8 caratteri può essere difficile. Quando l'OTP viene diffusa usando un'app su un dispositivo mobile, il problema di disabilità diventa più grande, perché l'utente deve passare da un dispositivo ad un altro per generare inviare l'OTP.
- **Diffusione:** in caso di token OTP hardware, il costo per acquistare, configurare e distribuire può essere significativo. L'utilizzo di dispositivi mobili come generatori di OTP in qualche modo riduce i costi, ma aumenta la possibilità che il malware su questi dispositivi possono rubare il secret simmetrico insieme nell'algoritmo di generazione.
- **Manutenibilità:** i token OTP hanno costi nascosti e gentile. I token hardware richiedono batterie, e gli utenti potrebbero perderli, romperli o dimenticarli; inoltre alcuni token hardware hanno un tempo di vita preciso.

OTP SMS/Chiamata

Invece di utilizzare un token hardware dedicato, o un'applicazione, vengono inviati OTP generati da un server tramite un SMS o una chiamata telefonica, prendendo come riferimento il numero telefonico associato all'utente. chiaramente in questo sistema bisogna tenere in considerazione la sicurezza del canale utilizzato per consegnare l'OTP: il possesso del numero telefonico usato per ricevere l'OTP è fattore di sicurezza critico per questa soluzione, ok se il telefono dell'utente finale viene rubato e l'attaccante conosce l'username e la password, è in grado di impersonare l'utente (non c'è bisogno volte che rubi nemmeno il telefono, basta clonare la sim, oppure fare in modo che le chiamate gli SMS vengono ridirezionate al numero desiderato). Se un utente riceve l'OTP in modo sicuro, ma lui invia a un'applicazione o un web browser compromesso, un attaccante potrebbe penetrare attraverso un attacco real-time per ottenere la sessione valida con i service provider. I difetti di questo sistema sono:

- **variabilità del costo:** in alcuni casi il costo sarà la consegna di SMS o della chiamata telefonica, ma in certi casi può essere difficile prevedere il volume ed e transizioni e quindi i corsi saranno variabili.
- **affidabilità:** è necessario che gli utenti siano in possesso del loro telefono e abbiano ricezione per utilizzare questo servizio. Questo può essere problematico e nelle aree prive di rete mobile oppure su confini internazionali, visto che in quest'ultimo caso i costi del servizio aumenteranno.

8.9.4.6 Notifiche push

L'autenticazione basata su notifiche push usa un app mobile dedicata per ricevere richieste per approvare un tentativo di autenticazione. Questa applicazione potrebbe usare un criptaggio a chiave pubblica, permettendo al telefono di generare una risposta criptata ad una challenge consegnata dalla notifica push, oppure potrebbero essere usati dei secret simmetrici. In alcuni casi la notifica push funge da livello sopra la consueta OTP, con l'applicazione che essenzialmente è un modo per risolvere l'usabilità. La sicurezza di questo sistema dipenderà dalla sicurezza delle chiavi simmetriche, delle chiavi private o dei token di sessione dati al dispositivo. I difetti di questo sistema sono:

- **Affidabilità:** l'efficacia delle notifiche push dipende dall'affidabilità della connessione dati; di solito però le transazioni per le autenticazioni sono gratuite.
- **Sicurezza del dispositivo:** a volte possedere dispositivo dell'utente finale è tutto ciò che necessario per compromettere il suo account, visto che alcune soluzioni non richiedono nemmeno user name e password.

8.9.5 Sicurezza nei sistemi di controllo: Firewall, IPS e IDS

I sistemi di controllo industriali vengono spesso chiamati **SCADA**; una rete industriale è di solito composta da aree distinte fra loro:

1. Rete di lavoro o impresa
2. Operazioni business
3. Rete di supervisione
4. Reti di processo e controllo

Per poter mettere in sicurezza tutti questi sistemi si devono seguire delle procedure ben definite: per prima cosa si vanno a identificare quali sistemi vanno messi in sicurezza, per poi separarli in gruppi funzionali attraverso un metodo logico. Successivamente si va a implementare una strategia profonda di difesa attorno ad ogni sistema, in modo da prevedere il controllo all'accesso verso e tra ogni gruppo.

Sistemi critici

Per potere identificare i sistemi critici bisogna avere un inventario completo di tutti i dispositivi connessi, dove ognuno di loro viene valutato indipendentemente: se esegue una funzione critica deve essere classificato come critico, se non lo fa bisogna considerare che potrebbe andare a impattare su altri dispositivi critici o operazioni critiche.

Segmentazione della rete

La separazione delle risorse in gruppi funzionali, permette a servizi specifici di essere chiusi e controllati strettamente. Raggruppare le risorse per i servizi funzionali riduce la superficie esposta agli attacchi, detto ciò un approccio stratificato all'isolamento funzionale topologico può migliorare molto la difesa della rete.

Protocolli SCADA

Per mettere in sicurezza questi sistemi è fondamentale sapere quali protocolli vengono utilizzati: OPC, ICCO, Modbus, DNP3.

8.9.5.1 Vulnerabilità del centro di controllo

Un intruso elettronico può accedere al centro di controllo attraverso diverse interfacce:

- Collegamenti al sistema informativo aziendale: è uso comune nell'industria quello di collegare sistemi informativi per accedere ai dati necessari del centro di controllo.
- Collegamenti ad altri strumenti o ai power pool: i controlli a livello applicazione e i protocolli proprietari rendono questi collegamenti obiettivi difficili per un attacco elettronico.
- Collegamenti ai venditori di supporto: gli strumenti stanno usando sempre più software sviluppato commercialmente e l'accesso remoto viene di solito raggiunto attraverso una porta dial-in sul sistema, che però rappresentano un punto di accesso potenziale per un intruso.
- Manutenzione remota e porte di amministrazione: esistono strumenti che permettono alle operazioni al personale di accedere ai sistemi in modo remoto per il supporto fuori orario, chiaramente però anche questi rappresentano un punto di accesso potenziale per un intruso.

8.9.5.2 Vulnerabilità delle sottostazioni

Dispositivi digitali programmabili

Digitando (dial-in) su una porta di un interruttore digitale, un ingegnere degli strumenti può resettare il dispositivo o selezionare uno dei sei livelli di protezione; un intruso elettronico potrebbe digitare su una porta non protetta e resettare l'interruttore ad un più alto livello di tolleranza.

Unità terminali remote

Oltre a raccogliere dati per il centro di controllo, una RTU opera come cleaning house per controllare i segnali per l'attrezzatura di trasmissione e distribuzione. diversi strumenti hanno porte di manutenzione sulle RTU delle stazioni a cui si può accedere in maniera remota: un intruso può digitare su questa porta e mandare dei comandi attrezzatura della sottostazione O riportare dati spuri al centro di controllo.

8.9.5.3 Firewall

Il firewall forma una barriera attraverso la quale deve passare tutto il traffico di una rete; agisce come un filtro per i pacchetti, ispezionandoli sia entrata che in uscita. Vengono create delle politiche di sicurezza per stabilire quali pacchetti sono autorizzati a passare. Il firewall può essere progettato per operare come filtro a livello di rete (per i pacchetti IP) oppure operare ad un livello più alto per gli altri protocolli. In sostanza un firewall non è altro che una linea di difesa, per isolare i sistemi interni dalle reti esterne. Le sue caratteristiche sono le seguenti:

- **Controllo del servizio:** determina i tipi di servizi internet a cui si può accedere.
- **Controllo della direzione:** determina la direzione nella quale le richieste di un servizio particolare possono essere iniziate e lasciate passare.
- **Controllo dell'utente:** controlla l'accesso ad un servizio a seconda di quale utente sta cercando di accedere.
- **Controllo del comportamento:** controlla come vengono utilizzati i particolari servizi.

Un firewall definisce un singolo punto di strozzatura che tiene fuori dalla rete protetta gli utenti non autorizzati, vieta ai servizi potenzialmente vulnerabili di entrare o lasciare la rete e fornisce protezione dei vari tipi di IP spoofing e attacchi di routing. Fornisce anche un posto per monitorare gli eventi legati alla sicurezza. Detto ciò un firewall ha anche dei limiti: non può proteggere dagli attacchi studiati specificatamente per bypassare il suo sistema di sicurezza (i sistemi interni potrebbero avere ad esempio strumenti di dial-out per connettersi a un ISP, oppure semplicemente non è in grado di proteggersi da minacce interne).

Tipologie di firewall

- Packet Filtering Firewall: Applicano un set di regole ad ogni indirizzo IP entrante ed uscente.
- Stateful Inspection Firewall: tiene traccia dello stato delle connessioni di rete (flussi TCP). viene programmato per distinguere i pacchetti legittimi da diversi tipi di connessioni, sono i pacchetti che corrispondono ad una connessione attiva conosciuta saranno permessi dal firewall.
- Application-level Gateway: Chiamato anche application proxy, funge da tramite del traffico a livello applicazione. L'utente contatta il gateway usando un'applicazione TCP/IP e il gateway chiede all'utente il nome dell'host remoto a cui desidera accedere. quando l'utente risponde e fornisce un user ID valido, il gateway contatta l'applicazione sull'host remoto e passa i segmenti TCP che contengono i dati dell'applicazione.
- Circuit-Level Gateway: può essere un sistema stand alone che non permette connessioni TCP end-to-end. piuttosto va a settare 2 connessioni TCP, una tra di sé e un utente TCP su un host interno e una tra di sé e un utente TCP su un host esterno.

8.9.5.4 IDS e IPS

L'**Intrusion Detection** è l'arte di **rilevare attività anomale**: insieme ad altri strumenti, un **Intrusion Detection System** (IDS) può essere usato per determinare se una rete di computer o un server ha subito un'intrusione. Un **Intrusion Prevention System** (IPS) viene usato per **cestinare attivamente i pacchetti di dati** o disconnettere connessioni che contengono dati non autorizzati.

IDS

IDS è uno strumento utilizzato per catturare e fornire visibilità in una rete: nell'uso comune viene posto nel perimetro tra il router di confine e il firewall, per permettere a chi si difende di rilevare l'attaccante prima che provi ad entrare in una zona protetta del firewall. Installare un IDS fuori dal firewall e dal router di confine può permettere di vedere tutti i tentativi di attacchi alla rete.

Il sistema IDS viene diffuso in modalità **non promiscua**, vuol dire che il sensore viene piazzato dove **può ascoltare tutto il traffico della rete ma non collegato direttamente ad essa**. Questa soluzione porta diversi vantaggi:

- **Nessun impatto sulla rete** (jitter, latenza...)
- **Nessun impatto sulla rete se c'è failure del sensore**
- **Nessun impatto sulla rete se c'è un sensore sovraccarico**.

Comparato a IPS ha però diversi svantaggi:

- **Se i pacchetti triggerano un allarme**, tutto ciò che fa è **dirci che sta avvenendo un attacco** (senza agire).
- L'azienda deve avere una **policy di sicurezza ben pensata**.
- Le reti vulnerabili devono avere **tecniche di evasione**.
- Numero dei **falsi positivi che la tecnologia tende a rilevare**, ovvero parte del traffico legittimo viene riconosciuto come cattivo.

Svantaggi di IPS

- Poiché il sensore viene connesso direttamente nella rete (inline), **potrebbe influenzare il traffico di rete**.
- Se il sensore è sovraccarico potrebbe impattare la rete.
- Un'azienda deve avere una **policy di sicurezza ben pensata**.
- Soffre di **latenza, jitter...**

Gli attacchi alla rete **devono avvenire velocemente**, per cui nel momento in cui il tecnico di rete entra nell'attrezzatura, l'attaccante potrebbe già essersi defilato. IPS **è capace di caricare dei file di marcatura sul software anti-virus**, che cerca dei pattern basati sull'attacco, e può bloccare tali attacchi a seconda delle policy di sicurezza implementate (oltre che allertare l'utente). Un'altra funzionalità di questo sistema è che può rispondere ad una minaccia rilevata con la riconfigurazione di altri controlli di sicurezza, come firewall o router, per bloccare un attacco.

I dispositivi **IPS e IDS** possono essere usati **insieme**, visto che stanno in punti differenti della rete:

- **IPS, installato sul perimetro della rete** aiuterà a fermare gli attacchi zero-day, come worm o virus.
- **IDS installato nel firewall** monitorerà l'attività interna, proteggendo dalle minacce interne onnipresenti, e permetterà di vedere gli eventi di sicurezza passati e presenti.

8.9.5.5 Implementazione Open Source: SNORT

SNORT è uno **strumento di rilevazione open source molto popolare che monitora la rete e rileva possibili intrusioni**. Attraverso questo sistema è **possibile catturare i pacchetti sulla superficie della rete e leggerne la traccia**. SNORT può anche esaminare i pacchetti per vedere se **corrispondono alle regole di rilevazione**, che considerano alcuni valori negli header dei pacchetti e alcune firme nei pacchetti.

8.9.5 VPN – Reti private virtuali

Le VPN permettono di sovrapporre delle reti sopra le reti pubbliche, senza abbandonare le proprietà di sicurezza tipiche delle reti private virtuali. Un approccio comune è costruire le VPN direttamente su internet, attrezzando ogni ufficio con un firewall e creando dei tunnel attraverso internet tra tutte le coppie di uffici. Per un router su internet, un pacchetto che viaggia attraverso un tunnel VPN non è altro che un normale pacchetto; l'unica cosa strana è la presenza di un'intestazione IP. Un altro approccio che sta prendendo piede è quello di far creare la VPN all'ISP: i percorsi creati da quest'ultimo tengono il traffico VPN separato dal restante, garantendo una certa qualità di banda. Un vantaggio fondamentale delle VPN è che sono completamente trasparenti al software dell'utente.